

OI WIKI

Language Fundamentals

Complete English Translation

A faithful English edition of the chapter <https://oi-wiki.org/lang/>

Source work	OI Wiki, chapter “Language Fundamentals” (语言基础)
Source URL	https://oi-wiki.org/lang/ (41 wiki pages)
Contents	C++ from first steps to advanced features; the C++ standard library; <code>pb_ds</code> policy data structures; compilation and optimization; introductions to Pascal, Python, and Java
Source license	Content of OI Wiki is licensed under CC BY-SA 4.0; this translation is likewise released under CC BY-SA 4.0
Translation	Complete and faithful English translation, produced September 1, 2026; all code, bounds, examples, tables, and figures preserved; code comments translated; three flowcharts redrawn in English
Intended use	Printable study reference for competitive-programming contestants preparing for NOI, IOI, and similar contests

How to read this book

Cross-references to sections of this book are written as §*n* (for example, §3 for Variables). References that point outside the Language Fundamentals chapter are kept as textual pointers of the form “OI Wiki → chapter → page”, because those pages are outside the scope of this translation. External web addresses are given in footnotes on the page where they first appear.

About This Translation

OI Wiki (oi-wiki.org) is the largest open community-maintained knowledge base for competitive programming in Chinese, and its Language Fundamentals chapter is the standard on-ramp used by countless contestants training for NOI, provincial selections, and IOI-level contests. This book is a complete, expert English translation of that entire chapter — all 41 pages, from “Hello, World!” through advanced Java — so that an English reader can study exactly what a Chinese reader would study, without ever needing machine translation.

The translation follows the same fidelity protocol used for official contest statements: nothing is added, dropped, simplified, or reinterpreted. Every definition, example, table, complexity bound, footnote, and admonition of the source is preserved. Code listings were carried over mechanically, character for character; only the natural-language comments inside code have been translated, which is exactly the difference a bilingual reader would perceive. Mathematical notation follows the source. Where the original text refers to pages of OI Wiki that lie outside this chapter, the reference is preserved as a textual pointer (“OI Wiki → ...”) rather than silently deleted.

A small number of presentation-level adaptations were unavoidable in moving from a web page to a printed book, and they are all listed here:

- Wiki-page metadata that never renders as content for readers (per-page author credits and **disqus** markers) is omitted; the “author” credits of the original pages remain visible in the wiki itself.
- The collapsible “note / warning / tip / example / info” boxes of the wiki are rendered as titled boxes of the same five colors and the same wording.
- Hyperlinks within this chapter become numeric cross-references (§*n*); hyperlinks to other chapters and to external sites become textual pointers and footnote URLs.
- The three loop flowcharts of the Loops section, originally drawn with Chinese labels, have been redrawn with English labels, one-to-one in layout; all other figures are the original images of the wiki, embedded unchanged.
- Under each section heading, the original Chinese title and the source page path are printed in small gray type for cross-referencing against the online wiki.

Terminology has been kept consistent throughout, in line with official contest English: `is container`, `is iterator`, `is read-only object`, `is complexity`, and so on. Identifiers, standard-library names, and code are never translated.

Contents

About This Translation	2
Introduction to This Chapter	11
1 Hello, World! (helloworld)	11
1.1 Setting Up the Environment	11
1.2 Your First Program	12
2 C++ Syntax Basics (basic)	13
2.1 The Code Framework	13
2.2 Comments	14
2.3 Input and Output	15
2.4 Some Additional Material	17
3 Variables (var)	19
3.1 Data Types	19
3.2 Fixed-Width Integer Types	26
3.3 Type Conversions	28
3.4 Defining Variables	32
3.5 Variable Scope	32
3.6 Constants	32
3.7 References and Notes	33
4 Operators and Operations (op)	33
4.1 Arithmetic Operators	34
4.2 Bitwise Operators	35
4.3 Increment and Decrement Operators	36
4.4 Compound Assignment Operators	36
4.5 The Conditional Operator	36
4.6 Comparison Operators	37
4.7 Logical Operators	37
4.8 The Comma Operator	37
4.9 Member Access Operators	38
4.10 The Complete C++ Operator Precedence Table	38

5	Branching (branch)	41
5.1	The <code>if</code> Statement	41
5.2	The <code>switch</code> Statement	42
6	Loops (loop)	45
6.1	The <code>for</code> Statement	45
6.2	The <code>while</code> Statement	45
6.3	The <code>do...while</code> Statement	46
6.4	How the Three Statements Relate	47
6.5	The <code>break</code> and <code>continue</code> Statements	48
7	Arrays (array)	49
7.1	Defining Arrays	49
7.2	Accessing Array Elements	50
7.3	Multidimensional Arrays	51
8	Structures (struct)	51
8.1	Defining a Structure	52
8.2	Accessing / Modifying Members	52
8.3	Why Do We Need Structures?	52
8.4	More Operations?	53
8.5	Points of Caution	53
8.6	References	53
9	Unions (union)	53
9.1	Defining a Union	54
9.2	Accessing / Modifying Members	54
10	Pointers (pointer)	54
10.1	Addresses of Variables, and Pointers	54
10.2	Declaring and Using Pointers	55
10.3	Pointer Offset	55
10.4	The Null Pointer	56
10.5	Advanced Uses of Pointers	57
10.6	Pointers to Functions	61

11 Functions (func)	63
11.1 Declaring Functions	63
11.2 Implementing Functions: Writing the Definition	63
11.3 Calling Functions	64
11.4 The <code>main</code> Function	65
12 File Operations (file-op)	66
12.1 The Concept of Files	66
12.2 The Steps of File Operations	66
12.3 The <code>freopen</code> Function	66
12.4 The <code>fopen</code> Function (Optional Reading)	68
12.5 C++ <code>ifstream/ofstream</code> File Input/Output Streams	69
12.6 References	69
13 Introduction to the C++ Standard Library (csl)	70
13.1 The C++ Standards	70
13.2 The Standard Template Library (STL)	71
13.3 The Boost Libraries	71
13.4 References	72
14 Introduction to STL Containers (csl/container)	72
14.1 Classification	72
14.2 Common Properties	73
15 Iterators (csl/iterator)	74
15.1 Basic Usage	74
15.2 Categories	75
15.3 Related Functions	76
16 Sequence Containers (csl/sequence-container)	76
16.1 <code>vector</code>	76
16.2 <code>array</code> (C++11)	80
16.3 <code>deque</code>	82
16.4 <code>list</code>	83
16.5 <code>forward_list</code> (C++11)	84

17 Associative Containers (csl/associative-container)	84
17.1 set	84
17.2 map	86
17.3 Traversing the Containers	88
17.4 Custom Comparison	88
18 Unordered Associative Containers (csl/unordered-container)	89
18.1 Overview	89
18.2 Manufacturing Hash Collisions	90
18.3 Custom Hash Functions	90
19 Container Adapters (csl/container-adapter)	91
19.1 Stack	91
19.2 Queue	92
19.3 Priority Queue	93
20 STL Algorithms (csl/algorithm)	94
20.1 Usage Examples	96
21 bitset (csl/bitset)	97
21.1 Introduction	98
21.2 Usage	99
21.3 Applications	100
21.4 “LibreOJ β Round #2”: Greed Only Passes the Sample	100
21.5 CF1097F — Alex and a TV Show	103
22 string (csl/string)	112
22.1 What Is string	112
22.2 Why Use string	112
22.3 Usage	113
23 pair (csl/pair)	116
23.1 Usage	116
23.2 Application Examples	118
24 Classes (class)	120
24.1 Defining Classes	120

24.2 Accessing and Modifying Members	121
24.3 Member Functions	121
24.4 References	127
25 Namespaces (namespace)	127
25.1 Overview	128
25.2 Declaration	128
25.3 The <code>using</code> Directive	128
25.4 Unnamed Namespaces	130
25.5 Applications	130
25.6 References	131
26 Value Categories (value-category)	131
26.1 Unnecessary Copies	131
26.2 Value Categories in the C Language	132
26.3 Value Categories in C++98	132
26.4 Value Categories in C++11	133
26.5 Value Categories in C++17	135
26.6 References and Recommended Reading	136
27 Operator Overloading (op-overload)	136
27.1 Restrictions	137
27.2 Implementation	137
27.3 Basic Arithmetic Operators	137
27.4 Increment and Decrement Operators	138
27.5 The Function Call Operator	139
27.6 Comparison Operators	139
28 References (reference)	141
28.1 The Lvalue Reference <code>T&</code>	141
28.2 The Rvalue Reference <code>T&&</code> (C++11)	142
28.3 Dangling References	143
28.4 Optimization Techniques Related to References	144
28.5 Reference Material	145

29 Constants (<code>const</code>)	146
29.1 The <code>const</code> Type Qualifier	146
29.2 <code>const</code> Member Functions	147
29.3 Constant Expressions: <code>constexpr</code> (C++11)	147
30 New C++ Features (<code>new</code>)	150
30.1 The <code>auto</code> Type Specifier	150
30.2 The <code>decltype</code> Specifier	150
30.3 <code>constexpr</code>	150
30.4 The Range-Based <code>for</code> Loop	151
30.5 Structured Bindings (C++17)	152
30.6 The <code>std::tuple</code>	153
30.7 Function Objects	154
30.8 Lambda Expressions	155
30.9 <code>std::function</code>	155
30.10 Variadic Function Templates	156
30.11 The Ranges Library (C++20)	158
30.12 References	162
31 Lambda Expressions (<code>lambda</code>)	162
31.1 Lambda Expressions	162
31.2 Applications of Lambda Expressions	171
31.3 References	172
32 Introduction to <code>pb_ds</code> (<code>pb-ds</code>)	173
33 Priority Queues (<code>pb-ds/pq</code>)	174
33.1 <code>__gnu_pbds::priority_queue</code>	174
33.2 Template Parameters	174
33.3 Construction	175
33.4 Member Functions	175
33.5 Example	175
33.6 The Invalidation Guarantees of Iterators in <code>__gnu_pbds</code>	176
34 Balanced Trees (<code>pb-ds/tree</code>)	177
34.1 <code>__gnu_pbds::tree</code>	177

34.2	Template Parameters	178
34.3	Construction	178
34.4	Member Functions	179
34.5	Example	179
34.6	References	180
35	Compilation and Optimization (optimizations)	181
35.1	A Brief Introduction to Compiler Optimization	181
35.2	Common Compiler Optimizations	181
35.3	Common Language Misuses Related to Compiler Optimization	192
35.4	Undefined Behavior and Compiler Optimization	193
35.5	Sanitizers	195
35.6	Miscellaneous	196
35.7	Extended Reading	196
36	Differences Between C++ and Other Common Languages (cpp-other-langs)	198
36.1	Differences Between C and C++	198
36.2	Differences Between Python and C++	201
36.3	Differences Between Java and C++	201
37	An Emergency Guide: Pascal to C++ (pas-cpp)	201
37.1	Quick Installation and Environment Setup for C++	201
37.2	A Quick Syntax Digest of C++	202
37.3	Syntax	204
37.4	Important Differences	209
37.5	The C++ Standard Library and Reference Material	214
37.6	Afterword	215
37.7	Appendix: Comparison Table of Pascal and C++ Operators and Mathematical Functions	216
38	A Crash Course in Python (python)	217
38.1	About Python	217
38.2	Environment Setup	218
38.3	Installing Third-Party Libraries with pip	220
38.4	Basic Syntax	220
38.5	Creating Arrays	225

38.6 Input and Output	229
38.7 File Reading and Writing	231
38.8 Control Flow	232
38.9 Built-in Containers	233
38.10 Writing Functions	235
38.11 Decorators	237
38.12 Common Built-in Libraries	238
38.13 Comparing C++ and Python on an Example Problem	238
38.14 Reference Documents	244
38.15 References and Notes	244
39 A Crash Course in Java (java)	244
39.1 About Java	244
39.2 Environment Installation	244
39.3 Basic Syntax	244
39.4 Points of Caution	249
40 Advanced Java (java-pro)	250
40.1 Faster Input and Output	250
40.2 BigInteger and Number Theory	252
40.3 Primitive Types and Wrapper Types	258
40.4 Inheritance	260
40.5 Polymorphism	261
40.6 Generics	261
40.7 Interfaces	261
40.8 Lambda Expressions	262
40.9 Collection	265
40.10 Arrays	277
40.11 Collections	281
40.12 Miscellaneous	282
40.13 References	283

PART I

Getting Started

Introduction to This Chapter

Original title: 语言基础简介 · source page /lang/

This chapter introduces programming-related knowledge, including a C++ tutorial that goes from first steps to advanced material, together with brief introductions to several other languages.

Programs are the carriers of algorithms and data structures, and they are the tool with which OI problems are solved.

In OI, the most commonly used programming language is C++.

Learning to program is the most fundamental part of learning OI.

1 Hello, World! (helloworld)

Original title: Hello, World! · source page /lang/helloworld/

1.1 Setting Up the Environment

As the saying goes: to do good work, one must first sharpen one's tools.

1.1.1 Integrated Development Environments (IDEs)

IDEs are relatively simple to operate, so beginners generally choose an IDE for writing code. The most common one in contests is Dev-C++¹ (if the contest environment is Windows, this IDE is usually provided as well).

¹<https://sourceforge.net/projects/orwelldvcpp/> — see the original page link “OI Wiki → Tools → Editors → Dev-C++”.

1.1.2 Compilers

Windows. The GNU compiler is recommended. Go to the MinGW Distro² to download and install MinGW. On Windows you may alternatively choose the Microsoft Visual C++ compiler,³ which you can download and install from the Visual Studio page.⁴

macOS. Execute the following in a terminal:

```
1 xcode-select --install
```

Linux. Use `g++ -v` to check whether `g++` has already been installed. To install it, use:

```
1 sudo apt update && sudo apt install g++
```

Compiling Code from the Command Line. Once you are proficient, you may prefer to compile code from the more flexible command line; then you no longer depend on an IDE and can write code in whatever text editor you are comfortable with.

```
1 g++ test.cpp -o test -lm
```

Here `g++` is the compiler of the C++ language (the compiler of the C language is `gcc`); `-o` specifies the file name of the executable, and the compilation option `-lm` links the mathematical library `libm`, so that code using `math.h` can compile and run normally.

Note

C++ programs do not need `-lm` in order to compile and run. The C++ compilation options of NOI/NOIP tasks over the years have included `-lm`, so it is added here as well.

1.2 Your First Program

Let us set off on the C++ learning journey with the following example program.

Note

Before typing the code, make sure your input method is set to English.

In C++:

```
1 #include <iostream> // include the header file
2
3 int main() { // define the main function
4     std::cout << "Hello, world!"; // use cout from the standard namespace
5     return 0; // return 0, ending the main function. The compiler usually adds
6               // this line automatically, so it can generally be omitted
7 }
```

²<https://nuwen.net/mingw.html>

³<https://docs.microsoft.com/en-us/cpp/build/projects-and-build-systems-cpp>

⁴<https://visualstudio.microsoft.com/downloads/>

In C:

```
1 #include <stdio.h> // include the header file
2
3 int main() {          // define the main function
4     printf("Hello, world!"); // output Hello, world!
5     return 0;         // return 0, ending the main function
6 }
```

Warning

C is shown here only for reference. C++ is largely compatible with the C language and provides many new features that let a contestant get twice the result with half the effort in contest. See §36 for the differences between C++ and other common languages.

2 C++ Syntax Basics (basic)

Original title: C++语法基础 · source page /lang/basic/

2.1 The Code Framework

If you do not want to dig into the principles behind it right away, as a beginner you can simply memorize this “framework”:

```
1 #include <cstdio>
2 #include <iostream>
3
4 int main() {
5     // do something...
6     return 0;
7 }
```

Note — What is #include?

#include is in fact a preprocessing directive whose meaning is to “place” a file at the location of this statement; the file being “placed” there is called a *header file*. That is to say, at compile time the compiler “copies” the contents of the header file **iostream** and “pastes” them at the statement **#include <iostream>**. This way, you can use the objects provided by **iostream**, such as **std::cin**, **std::cout**, and **std::endl**.

If you have learned C, you will notice that the C++ header files we have encountered so far generally do not carry the **.h** suffix, while those header files of the C language **xx.h** have become **cxx**: for example, **stdio.h** became **cstdio**. Because C++ must stay compatible with C, it uses the C header files directly; to distinguish C++ header files from C header files, the **c** prefix is used.

Generally speaking, you should decide which header files to **#include** according to the needs of the C++ program you want to write. If you **#include** more headers than needed, it only increases compilation time and has almost no effect on running time. So far we have

only encountered the two header files `iostream` and `cstdio`; if you only need `scanf` and `printf`, you can skip `#include <iostream>`.

Can you `#include` a header file you wrote yourself? The answer is: yes.

You can write your own header file, for example `myheader.h`, put it in the same directory as your code, and then write `#include "myheader.h"`. Note that a self-written header file must be enclosed in quotation marks rather than angle brackets. You can of course also use the compilation option `-I <header_file_path>` to tell the compiler where to look for header files, in which case you do not need to place the header file in the same directory as the code.

Note — What is `main()`?

You can understand it as: when the program runs, the code inside `main()` is executed.

In fact, the `main` function is called by the system or by an external program. For example, when you invoke your program on the command line, you are calling the `main` function of your program (before that, the construction of the global [variables](#) has already been completed).

The final `return 0;` indicates that the program ran successfully. By default, returning 0 when the program ends means everything is normal; otherwise the return value represents an error code (under Windows, the hexadecimal form of this error code can be looked up on the Windows Error Codes site^a). To whom is this value returned? To the system or external program that called your program, which receives this return value when your program ends. If no `return` statement is written, a program that ends normally also returns 0 by default.

In C or C++, a program whose return value is not 0 causes a runtime error (RE).

^ahttps://docs.microsoft.com/en-us/openspecs/windows_protocols/ms-erref/

2.2 Comments

In C++ code, comments can be written in two ways:

1. Inline comments: starting with `//`, everything after `//` on that line is a comment.
2. Comment blocks: starting with `/*` and ending with `*/`; everything in between is a comment, and this may span several lines.

Comments have no effect on the execution of a program. They can be used to explain what a program means, and also to keep a piece of code from executing (while it still remains in the source file).

In software development, comments make later maintenance and reading by other people easier.

In OI, very few people write a lot of comments, but comments help you organize your thinking while writing code, and make later review easier. Moreover, if you want to write editorials or tutorials, a suitable number of comments helps readers follow the code and understand its intent. I hope every reader develops the good habit of writing comments.

2.3 Input and Output

2.3.1 cin and cout

```
1 #include <iostream>
2
3 int main() {
4     int x, y;                // declare variables
5     std::cin >> x >> y;      // read x and y
6     std::cout << y << std::endl << x; // output y, a newline, then x
7     return 0;               // end the main function
8 }
```

Note — What are variables?

See the page on [Variables](#).

Note — What is std?

`std` is the **namespace** used by the C++ standard library. Namespaces exist to avoid name collisions. For detailed knowledge about namespaces, see the page on [Namespaces](#).

2.3.2 scanf and printf

`scanf` and `printf` are in fact functions provided by the C language. In most cases they are faster than `cin` and `cout`, and they let you control the input/output format conveniently.

Note — Input/output optimization

For the precise differences between `cin/cout` and `scanf/printf`, and for input/output optimization, see the page on [Fast Input and Output \(OI Wiki → Contest → Fast I/O\)](#).

```
1 #include <cstdio>
2
3 int main() {
4     int x, y;
5     scanf("%d%d", &x, &y); // read x and y
6     printf("%d\n%d", y, x); // output y, a newline, then x
7     return 0;
8 }
```

Here `%d` means that the variable being read/printed is a variable of signed integer type (`int`).

Similarly:

1. `%s` denotes a string.
2. `%c` denotes a character.
3. `%lf` denotes a double-precision floating-point number (`double`).
4. `%lld` denotes a long integer (`long long`). Depending on the system, it may instead be `%I64d`.

5. `%u` denotes an unsigned integer (`unsigned int`).
6. `%llu` denotes an unsigned long integer (`unsigned long long`); it may instead be `%I64u`.

Besides the type specifiers, there are other ways to control the format. Many of them are rarely used; two common ones are listed here:

1. `%1d` denotes an integer of width 1. When reading, digits can be read one by one even if there are no spaces. When printing, if the specified width is greater than the number of digits, the number is padded in front with spaces; if the specified width is smaller than the number of digits, it has no effect.
2. `%.6lf`, used for output, keeps six decimal places.

The corresponding places in these two specifiers can be filled with other numbers; for example, `%.3lf` keeps three decimal places.

Note — What are “double-precision floating-point number” and “long integer”?

These denote the types of variables. As above, they will be explained together in the [Variables](#) page.

Note — Why does `scanf` have the `&` operator?

Here, `&` is in fact the address-of operator, which returns the address of a variable in memory, and the parameters `scanf` receives are precisely the addresses of variables. This can only be fully clarified in the [Pointers](#) section; for now, just remember it.

Note — What is `\n`?

`\n` is an **escape character** denoting a newline.

Escape characters are used to represent characters that cannot be typed directly: the newline character, which cannot be typed directly because a string literal cannot contain a line break; the quotation mark, which cannot be typed because of its special meaning; and the backslash, which cannot be typed because it is used to denote escape characters.

Commonly used escape characters are:

1. `\t` denotes the tab character.
2. `\\` denotes `\`.
3. `\"` denotes `"`.
4. `\0` denotes the null character, used to mark the end of a C-style string.
5. `\r` denotes the carriage return. In Linux the newline character is `\n`; in Windows it is `\r\n`. In OI, if your output needs a line break, using `\n` is enough. But when reading input, if you read character by character, the line-break characters may cause problems, and you need to be careful. For example, `gets` treats `\n` as the end of the string; if the line break is `\r\n`, the `\r` remains at the end of the string.

6. Specially, `%%` denotes `%`; it can only be used in `printf` or `scanf`. In other string literals, simply using `%` is fine.

Note — What is a literal?

A “literal” is a piece of program text that appears in code directly as a value: for example, `3` is an `int` literal, `'c'` is a `char` literal, and the `"hello world"` in the program we wrote above is also a string literal.

A literal that appears without any explanation or justification is also called a “magic number”; if the code is meant to be read by others, this is highly discouraged.

2.4 Some Additional Material

2.4.1 Whitespace Characters in C++

In C++, all whitespace characters (spaces, tabs, newlines), whether multiple or single, are treated identically. (Of course, those inside quotation marks, which are part of a string literal, do not count.)

Therefore, you are free to use any code style you like (except that inline comments, string literals, and preprocessing directives must stay on a single line), for example:

```
1 --8<-- "docs/lang/code/basic/basic_1.cpp:main"
```

Of course, doing this is not recommended.

A style that is also widely used but differs from the code style required by **OI Wiki**:

```
1 --8<-- "docs/lang/code/basic/basic_2.cpp:main"
```

2.4.2 The `#define` Directive

`#define` is a preprocessing directive used to define macros; in essence it performs textual replacement. For example:

```
1 #include <iostream>
2 #define n 233
3
4 // n is not a variable: the compiler replaces every occurrence of the text n
5 // with 233, except when n is part of a larger identifier (fn is not replaced
6 // by f233); occurrences inside strings are not replaced either
7
8 int main() {
9     std::cout << n; // output 233
10    return 0;
11 }
```

Note — What is an identifier?

An identifier is a sequence of characters that can be used as a variable name. For example, `abcd` and `abc1` are valid identifiers, while `1a` and `c+b` are not valid identifiers.

An identifier begins with an English letter or an underscore, and only English letters, underscores, and digits may appear after that. Note that keywords (such as `int`, `for`, `if`) cannot be used as identifiers.

Note — What is a preprocessing directive?

Preprocessing directives are the commands accepted by the preprocessor, which performs preliminary textual transformations on the code — for example the file-inclusion operation `#include` and macro processing `#define`. For GCC, the `.i` file produced by the preprocessing stage is not kept by default; you can keep the output file with the `-E` option.

Macros can take parameters, and macros with parameters can be used like functions:

```
1 #include <iostream>
2 #define sum(x, y) ((x) + (y))
3 #define square(x) ((x) * (x))
4
5 int main() {
6     std::cout << sum(1, 2) << ' ' << 2 * sum(3, 5) << std::endl; // output 3 16
7 }
```

However, macros with parameters are different from functions. Because macros are textual replacement, many problems can arise. For example:

```
1 #include <iostream>
2 #define sum(x, y) x + y
3 // this line should be #define sum(x, y) ((x) + (y))
4 #define square(x) ((x) * (x))
5
6 int main() {
7     std::cout << sum(1, 2) << ' ' << 2 * sum(3, 5) << std::endl;
8     // output is 3 11, because #define is textual replacement and the statement
9     // becomes 2 * 3 + 5
10    int i = 1;
11    std::cout << square(++i) << ' ' << i;
12    // output is undefined, because ++i is evaluated twice; modifying the same
13    // variable several times in one statement is undefined behavior (with
14    // exceptions)
15 }
```

Using `#define` is risky (since the scope of `#define` is the whole program, text may be replaced unexpectedly; you need `#undef` to cancel definitions in time), so use it with care. The recommended practice is: declare constants with the `const` qualifier, and use functions instead of macros.

That said, in OI, `#define` still has its uses (the following two usages are not recommended, because they reduce the standardization of your code):

1. `#define int long long` together with `signed main()`. Usually used to avoid errors

caused by forgetting to use `long long`, or to rule out that possibility when debugging. (It may also increase the constant factor or even cause TLE, or cause MLE by blowing up memory.)

2. `#define For(i, l, r)for (int i = (l);i <= (r);++i),#define pb push_back, #define mid ((l + r) / 2)`, used to shorten code.

Nevertheless, `#define` also has advantages: combined with preprocessing directives such as `#ifdef` it can work wonders, for example:

```
1 #ifdef LINUX
2 // code for linux
3 #else
4 // code for other OS
5 #endif
```

You can control the compiled code at compile time via `-DLINUX` without modifying the source file. This has one more advantage: the executable produced with `-DLINUX` contains no code for other operating systems at all, because that code was already removed during preprocessing.

`#define` can also use the `#` and `##` operators, which greatly facilitate debugging.

3 Variables (var)

Original title: 变量 · source page /lang/var/

3.1 Data Types

The type system of C++ consists of the following parts:

1. Basic types (representative keywords/types in parentheses)
 - (a) the untyped/`void` type (`void`);
 - (b) (since C++11) the null-pointer type (`std::nullptr_t`);
 - (c) arithmetic types
 - i. integer types (`int`);
 - ii. the Boolean/`bool` type (`bool`);
 - iii. character types (`char`);
 - iv. floating-point types (`float`, `double`).

2. Composite types⁵

⁵Including array types, reference types, pointer types, class types, function types, and so on. Because this article is aimed at beginners, they are not introduced in detail here; see Types on cppreference.com, <https://zh.cppreference.com/w/cpp/language/type>.

3.1.1 The Boolean Type

A variable of `bool` type can take only two possible values: `true` and `false`.

In general, a `bool` variable occupies 1 byte of space (in general, 1 byte = 8 bits).

Tip

You can obtain the number of bits in a byte through the macro constant `CHAR_BIT` in the header `<climits>` (C++) / `<limits.h>` (C).

Note — The Boolean type in the C language

See also §36 (C++ versus other common languages), subsection on `bool`.

The C language originally had no Boolean type; only at C99 was the `_Bool` keyword introduced as the Boolean type, and it is regarded as an unsigned integer type.

Note

From C23 on, the C language's `bool` type is no longer defined by the integer values zero and non-zero; it is defined as a type capable of storing the two constants `true` and `false`.

For convenience, `stdbool.h` provides the three macros `bool`, `true`, `false`, defined as follows:

```
1 #define bool _Bool
2 #define true 1
3 #define false 0
```

These macros were removed in C23, and C23 introduced `true`, `false`, and `bool` as keywords, while keeping `_Bool` as an alternative spelling (see <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3054.pdf>).

In addition, from C23 on, the bit width of the Boolean type can be obtained through the macro constant `BOOL_WIDTH` in `<limits.h>`.

3.1.2 Integer Types

Used to store integers. The most basic integer type is `int`.

Warning

For historical reasons, in C++ the Boolean type and the character types are regarded as special integer types.

In almost all situations you **should not** use character types other than `signed char` and `unsigned char` as integers.

Integer types generally come in 5 grades by bit width: `char`, `short`, `int`, `long`, `long long`.

The C++ standard guarantees $1 == \text{sizeof}(\text{char}) \leq \text{sizeof}(\text{short}) \leq \text{sizeof}(\text{int}) \leq \text{sizeof}(\text{long}) \leq \text{sizeof}(\text{long long})$.

For historical reasons there are several popular models for the bit widths of integer types; to solve this problem, C99/C++11 introduced [fixed-width integer types](#).

Note — The size of the `int` type

The C++ standard stipulates that `int` has **at least** 16 bits.

In fact, on the vast majority of current platforms, `int` has 32 bits.

The `int` keyword can be modified with the following modifier keywords.

Signedness:

- **signed**: signed integer (the default);
- **unsigned**: unsigned integer.

Size:

- **short**: an integer of **at least** 16 bits;
- **long**: an integer of **at least** 32 bits;
- (since C++11) **long long**: an integer of **at least** 64 bits.

The following table gives, **under ordinary circumstances**, the bit width and the size of the representable range of each integer type (on a few platforms the representable range of some types may differ from this table):

Type name	Equivalent type	Width (C++ std.)	Width (common)
<code>signed char</code>	<code>signed char</code>	8	–
<code>unsigned char</code>	<code>unsigned char</code>	8	–
<code>short</code> , <code>short int</code> , <code>signed short</code> , <code>signed short int</code>	<code>short int</code>	≥ 16	16
<code>unsigned short</code> , <code>unsigned short int</code>	<code>unsigned short int</code>	≥ 16	16
<code>int</code> , <code>signed</code> , <code>signed int</code>	<code>int</code>	≥ 16	32
<code>unsigned</code> , <code>unsigned int</code>	<code>unsigned int</code>	≥ 16	32
<code>long</code> , <code>long int</code> , <code>signed long</code> , <code>signed long int</code>	<code>long int</code>	≥ 32	32
<code>unsigned long</code> , <code>unsigned long int</code>	<code>unsigned long int</code>	≥ 32	32
<code>long long</code> , <code>long long int</code> , <code>signed long long</code> , <code>signed long long int</code>	<code>long long int</code>	≥ 64	64
<code>unsigned long long</code> , <code>unsigned long long int</code>	<code>unsigned long long int</code>	≥ 64	64

When the bit width is x , the representable range of a signed type is $-2^{x-1} \sim 2^{x-1} - 1$,⁶ and the representable range of an unsigned type is $0 \sim 2^x - 1$. Specifically:

⁶Before C++20, signed integers were required to cover at least the range of *ones'-complement* representation (i.e., $-2^{x-1} + 1 \sim 2^{x-1} - 1$), but in practice the vast majority of implementations use *two's complement*; from C++20 on, it is further stipulated that signed integers must be implemented as two's complement. See Range of values on cppreference, https://en.cppreference.com/w/cpp/language/types.html#Range_of_values.

Width	Representable range
8	signed: $-2^7 \sim 2^7 - 1$; unsigned: $0 \sim 2^8 - 1$
16	signed: $-2^{15} \sim 2^{15} - 1$; unsigned: $0 \sim 2^{16} - 1$
32	signed: $-2^{31} \sim 2^{31} - 1$; unsigned: $0 \sim 2^{32} - 1$
64	signed: $-2^{63} \sim 2^{63} - 1$; unsigned: $0 \sim 2^{64} - 1$

Note — Equivalent spellings of types

As long as no ambiguity arises, some modifier keywords may be omitted, or the order of the modifier keywords may be changed. This means that the same type can be spelled in several equivalent ways.

For example, `int`, `signed`, `int signed`, `signed int` denote the same type, and `unsigned long` and `unsigned long int` denote the same type.

In addition, some compilers implement extended integer types. For example, GCC implements 128-bit integers: the signed `__int128_t` and the unsigned `__uint128_t`. If you want to use these types in a contest, **read the contest rules carefully** to determine whether extended integer types are permitted or supported.

Warning

The STL does not necessarily support extended integer types well enough, so be especially careful when using them.

Example code

```
1 #include <cmath>
2 #include <iostream>
3
4 int f1(int n) {
5     return abs(n); // Good
6 }
7
8 int f2(int n) {
9     return std::abs(n); // Good
10 }
11
12 __int128_t f3(__int128_t n) {
13     return abs(n); // Bad
14 }
15
16 // Wrong
17 // __int128_t f4(__int128_t n) {
18 //     return std::abs(n);
19 // }
20
21 int main() {
22     std::cout << "f1: " << f1(-42) << std::endl;
23     std::cout << "f2: " << f2(-42) << std::endl;
24     // std::cout << "f3: " << f3(-42) << std::endl; // Wrong
25     // std::cout << "f4: " << f4(-42) << std::endl; // Wrong
26     return 0;
27 }
```

The example code above has the following problems:

1. In `__int128_t f3(__int128_t)`, the C-style absolute-value function is used, whose signature is `int abs(int)`; therefore `n` is first forcibly converted to `int`, and only then is the `abs` function called.
2. In `__int128_t f4(__int128_t)`, the C++-style absolute-value function is used, and there is no overload with the signature `__int128_t std::abs(__int128_t)`, so the code fails to compile.
3. C++ stream output does not support `__int128_t` and `__uint128_t`.

One solution is the following:

Corrected code

```

1  #include <cmath>
2  #include <iostream>
3
4  __int128_t abs(__int128_t n) { return n < 0 ? -n : n; }
5
6  std::ostream &operator<<(std::ostream &os, __uint128_t n) {
7      if (n > 9) os << n / 10;
8      os << (int)(n % 10);
9      return os;
10 }
11
12 std::ostream &operator<<(std::ostream &os, __int128_t n) {
13     if (n < 0) {
14         os << '-';
15         n = -n;
16     }
17     return os << (__uint128_t)n;
18 }
19
20 int f1(int n) { return abs(n); }
21
22 int f2(int n) { return std::abs(n); }
23
24 __int128_t f3(__int128_t n) { return abs(n); }
25
26 int main() {
27     std::cout << "f1: " << f1(-42) << std::endl;
28     std::cout << "f2: " << f2(-42) << std::endl;
29     std::cout << "f3: " << f3(-42) << std::endl;
30 }

```

3.1.3 Character Types

Character types divide into “narrow character types” and “wide character types”. Since algorithm contests almost never use wide character types, only narrow character types are introduced here.

Narrow character types are generally 8 bits wide; under the hood they are still stored as integers, and a one-to-one correspondence between characters and integers is usually realized through ASCII encoding⁷. There are the following three:

- **signed char**: the type denoting signed characters, with a representable range of $-128 \sim 127$.
- **unsigned char**: the type denoting unsigned characters, with a representable range of $0 \sim 255$.

⁷<http://www.asciitable.com/>

- `char` has the same representation and alignment as one of `signed char` or `unsigned char`, but it is always a separate type.

The signedness of `char` depends on the compiler and the target platform: the defaults on ARM and PowerPC are usually unsigned, while the defaults on x86 and x64 are usually signed.

GCC lets you add `-fsigned-char` or `-funsigned-char` to the compilation options to treat `char` as `signed char` or `unsigned char`; for other compilers consult their documentation. Note that specifying a signedness different from the architecture default may break the ABI and stop the program from working correctly.

Warning

Unlike the other integer types, `char`, `signed char`, and `unsigned char` are **three different types**.

Generally speaking, `signed char` and `unsigned char` should not be used to store characters; in the vast majority of cases, these two types are regarded as integer types.

3.1.4 Floating-Point Types

Used to store “real numbers” (note: not real numbers in the strict mathematical sense, but approximations of real numbers under certain rules). They include the following three:

- `float`: the single-precision floating-point type. If supported, it matches the IEEE-754 binary32 format.
- `double`: the double-precision floating-point type. If supported, it matches the IEEE-754 binary64 format.
- `long double`: the extended-precision floating-point type. If supported, it matches the IEEE-754 binary128 format; otherwise, if supported, it matches the IEEE-754 binary64-extended format; otherwise it matches some non-IEEE-754 extended floating-point format whose precision is better than binary64 and whose value range is at least as good as binary64; otherwise it matches the IEEE-754 binary64 format.

Floating-point format	Width	Largest positive number	Precision (digits)
IEEE-754 binary32 format	32	3.4×10^{38}	6 ~ 9
IEEE-754 binary64 format	64	1.8×10^{308}	15 ~ 17
IEEE-754 binary64-extended	≥ 80	$\geq 1.2 \times 10^{4932}$	$\geq 18 \sim 21$
IEEE-754 binary128 format	128	1.2×10^{4932}	33 ~ 36

The most negative number of an IEEE-754 floating-point format is the opposite of its largest positive number.

Because the `float` type has a small representable range and modest precision, in practice the `double` type is commonly used for floating-point numbers.

In addition, floating-point types support some special values:

- Infinity (positive or negative): `INFINITY`.
- Negative zero: `-0.0`; for example, `1.0 / 0.0 == INFINITY`, `1.0 / -0.0 == -INFINITY`.
- Not-a-Number (NaN): `std::nan`, `NAN`; generally produced by operations such as `0.0 / 0.0`. It compares unequal to every value (including itself); from C++11 on, `std::isnan` can be used to test whether a floating-point number is NaN.

3.1.5 The Void Type

The `void` type is the untyped type. Unlike the types above, you cannot declare a variable of type `void`. However, the return value of a function is allowed to be of `void` type, meaning that the function has no return value.

3.1.6 The Null-Pointer Type

See the corresponding part of the [Pointers](#) section.

3.2 Fixed-Width Integer Types

Since C++11, fixed-width integers are supported, as follows:

- `<cstdint>`: provides several fixed-width integer types together with macro constants for the maximum, minimum, and so on of each fixed-width integer type.
- `<inttypes>`: provides format macro constants for the fixed-width integer types, for use with the `std::fprintf` family and the `std::fscanf` family of functions.

The fixed-width integers are as follows:

- `intN_t`: a signed integer type whose width is **exactly** N bits, e.g. `int32_t`.
- `int_fastN_t`: the **fastest** signed integer type whose width is **at least** N bits, e.g. `int_fast32_t`.
- `int_leastN_t`: the **smallest** signed integer type whose width is **at least** N bits, e.g. `int_least32_t`.

For the unsigned versions, simply add the letter `u` in front of the signed names, e.g. `uint32_t`, `uint_least8_t`.

The standard requires the following 16 types to be implemented:

```
int_fast8_t, int_fast16_t, int_fast32_t, int_fast64_t,
int_least8_t, int_least16_t, int_least32_t, int_least64_t,
uint_fast8_t, uint_fast16_t, uint_fast32_t, uint_fast64_t,
uint_least8_t, uint_least16_t, uint_least32_t, uint_least64_t.
```

On top of these, the vast majority of compilers implement the following 8 types:

`int8_t`, `int16_t`, `int32_t`, `int64_t`,
`uint8_t`, `uint16_t`, `uint32_t`, `uint64_t`.

Where a corresponding type is implemented, the C++ standard requires implementing macro constants for the maximum, minimum, and bit width of that type, named by removing the trailing `_t` of the type name, converting to upper case, and adding a suffix:

- `_MAX` for the maximum, e.g. `INT32_MAX` is the maximum of `int32_t`.
- `_MIN` for the minimum, e.g. `INT32_MIN` is the minimum of `int32_t`.

Warning

Fixed-width integer types are essentially type aliases of ordinary integer types, so mixing fixed-width integer types with ordinary integer types may affect cross-platform compilation. For example:

Example code

```

1  #include <algorithm>
2  #include <cstdint>
3  #include <iostream>
4
5  int main() {
6      long long a;
7      int64_t b;
8      std::cin >> a >> b;
9      std::cout << std::max(a, b) << std::endl;
10     return 0;
11 }
```

Under 64-bit Windows, `int64_t` is generally `long long int`, while under 64-bit Linux it is generally `long int`. Hence this code fails to compile with GCC on 64-bit Linux but compiles with MSVC on 64-bit Windows, because `std::max` requires its two arguments to have the same type.

In addition, since C++17 the class template `std::numeric_limits` is provided in `<limits>` for querying the properties of the various arithmetic types, such as the maximum, the minimum, whether it is integral, whether it is signed, and so on.

```

1  #include <cstdint>
2  #include <limits>
3
4  std::numeric_limits<int32_t>::max(); // max value of int32_t: 2'147'483'647
5  std::numeric_limits<int32_t>::min(); // min value of int32_t: -2'147'483'648
6
7  std::numeric_limits<double>::min(); // min value of double, about 2.22507e-308
8  std::numeric_limits<double>::epsilon(); // difference between 1.0 and the
9                                           // next representable double value,
10                                           // about 2.22045e-16
```

3.3 Type Conversions

Sometimes (for example, a function takes an `int` parameter but a `double` variable is passed in), we need to convert a value of one type into another type.

The conversion mechanism of types in C++ is fairly complicated; here we mainly introduce two conversions of the basic data types: *numeric promotions* and *numeric conversions*.

3.3.1 Numeric Promotions

During a numeric promotion, the value itself does not change.

Note

C-style variadic argument regions perform the default argument promotions when values are passed. For example:

Example code

```

1  #include <stdarg.h>
2  #include <stdio.h>
3
4  void test(int tot, ...) {
5      va_list valist;
6      int i;
7
8      // initialize the variable-argument list
9      va_start(valist, tot);
10
11     for (i = 0; i < tot; ++i) {
12         // get the value of the i-th variable
13         double xx = va_arg(valist, double); // Correct
14         // float xx = va_arg(valist, float); // Wrong
15
16         // print the underlying storage of the i-th variable
17         printf("i = %d, value = 0x%016llx\n", i, *(long long *)&xx);
18     }
19
20     // free the memory of the variable-argument list
21     va_end(valist);
22 }
23
24 int main() {
25     float f;
26     double fd, d;
27     f = 123.; // 0x42f60000
28     fd = 123.; // 0x405ec00000000000
29     d = 456.; // 0x407c800000000000
30     test(3, f, fd, d);
31 }
```

When `test` is called, `f` is promoted to `double`, so its underlying storage becomes identical to that of `fd`, and the output is

```

1 i = 0, value = 0x405ec00000000000
2 i = 1, value = 0x405ec00000000000
3 i = 2, value = 0x407c800000000000

```

If `double xx = va_arg(valist, double);` is changed to `float xx = va_arg(valist, float);`, GCC should issue a warning similar to the following:

```

1 In file included from test.c:2:
2 test.c: In function 'test':
3 test.c:14:35: warning: 'float' is promoted to 'double' when passed through '...'
4   14 |         float xx = va_arg(valist, float);
5      |         ^
6 test.c:14:35: note: (so you should pass 'double' not 'float' to 'va_arg')
7 test.c:14:35: note: if this code is reached, the program will abort

```

The program will then abort before producing any output.

This also explains why the `%f` of `printf` matches both `float` and `double`.

Integral promotions. A prvalue of a small integer type (such as `char`) can be converted to a prvalue of a larger integer type (such as `int`).

Specifically, the arithmetic operators do not accept types smaller than `int` as their operands, and after the lvalue-to-rvalue conversion, the integral promotion is applied automatically if applicable.

Concretely, the rules are:

- If the source type is `signed char` or `signed short/short`, it can be promoted to `int`.
- If the source type is `unsigned char` or `unsigned short`, it can be promoted to `int` if `int` can hold the entire value range of the source type; otherwise it can be promoted to `unsigned int`. (Since C++20, `char8_t` also follows this rule.)
- The promotion rules for `char` depend on whether its underlying type is `signed char` or `unsigned char`.
- The `bool` type can be converted to `int`: `false` becomes `0` and `true` becomes `1`.
- If the value range of the destination type contains that of the source type, and the value range of the source type cannot be contained by `int` and `unsigned int`, then the source type can be promoted to the destination type.⁸

Warning

`char` $\rightarrow$ `short` is *not* a numeric promotion, because `char` is first promoted to `int/unsigned int` in preference to anything else, and only then `int/unsigned int` $\rightarrow$ `short`; this does not satisfy the conditions of a numeric promotion.

For example (in the following, assume `int` is 32 bits, `unsigned short` is 16 bits, `signed char` and `unsigned char` are 8 bits, and `bool` is 1 bit):

⁸wide character types, bit-fields, and enumeration types are excluded; see Integer conversions on cppreference, https://zh.cppreference.com/w/cpp/language/implicit_conversion.

- `(signed char)'\0' - (signed char)'\xff'` first promotes `(signed char)'\0'` to `(int)0` and `(signed char)'\xff'` to `(int)-1`, then performs the operation between ints; the final result is `(int)1`.
- `(unsigned char)'\0' - (unsigned char)'\xff'` first promotes `(unsigned char)'\0'` to `(int)0` and `(unsigned char)'\xff'` to `(int)255`, then performs the operation between ints; the final result is `(int)-255`.
- `false - (unsigned short)12` first promotes `false` to `(int)0` and `(unsigned short)12` to `(int)12`, then performs the operation between ints; the final result is `(int)-12`.

Floating-point promotions. A floating-point value of smaller width can be promoted to one of larger width (for example, when a `float` variable and a `double` variable undergo arithmetic operations, the `float` variable is promoted to `double`); its value is unchanged.

3.3.2 Numeric Conversions

During a numeric conversion, the value may change.

Warning

Numeric promotions take priority over numeric conversions. For example, `bool → int` is a numeric promotion, not a numeric conversion.

Integer conversions.

- If the destination type is an unsigned integer type of width x , the conversion result is the original value taken mod 2^x .
 - If the destination width is greater than the source width:
 - * If the source type is signed, then in general sign extension is performed first, and then the conversion.
For example, converting `(short)-1` (`(short)0b1111'1111'1111'1111`) to `unsigned int`: first perform sign extension, obtaining `0b1111'1111'1111'1111'1111'1111'1111'1111`, then perform the integer conversion; the result is `(unsigned int)4'294'967'295` (`(unsigned int)0b1111'1111'1111'1111'1111'1111'1111'1111`).
Converting `(short)32'767` (`(short)0b0111'1111'1111'1111`) to `unsigned int`: first perform sign extension, obtaining `0b0000'0000'0000'0000'0111'1111'1111'1111`, then perform the integer conversion; the result is `(unsigned int)32'767` (`(unsigned int)0b0000'0000'0000'0000'0111'1111'1111'1111`).
 - * If the source type is unsigned, zero extension is performed first, and then the conversion.
For example, converting `(unsigned short)65'535` (`(unsigned short)0b1111'1111'1111'1111`) to `unsigned int`: first perform zero extension, obtaining

`0b0000'0000'0000'0000'1111'1111'1111'1111`, then perform the integer conversion; the result is `(unsigned int)65'535 ((unsigned int)0b0000'0000'0000'0000'1111'1111'1111'1111)`.

- If the destination width is not greater than the source width, truncation is performed first, and then the conversion.

For example, converting `(unsigned int)4'294'967'295 ((unsigned int)0b1111'1111'1111'1111'1111'1111'1111'1111)` to `unsigned short`: first truncate, obtaining `0b1111'1111'1111'1111`, then perform the integer conversion; the result is `(unsigned short)65'535 ((unsigned short)0b1111'1111'1111'1111)`.

- If the destination type is a signed integer type of width x , then **in general** the conversion result can be regarded as the original value taken mod 2^x .⁹

For example, converting `(unsigned int)4'294'967'295 ((unsigned int)0b1111'1111'1111'1111'1111'1111'1111'1111)` to `short` gives `(short)-1 ((short)0b1111'1111'1111'1111)`.

- If the destination type is `bool`, it is a [Boolean conversion](#).
- If the source type is `bool`, then `false` converts to the 0 of the destination type and `true` converts to the 1 of the destination type.

Floating-point conversions. Converting a floating-point value of larger width to one of smaller width rounds the value to the closest value representable in the destination type.

Floating-integer conversions.

- When a floating-point value is converted to an integer, the entire fractional part is discarded. If the destination type is `bool`, it is a [Boolean conversion](#).

- When an integer is converted to a floating-point value, it is rounded to the closest value representable in the destination type.

If the value cannot fit in the destination type, the behavior is undefined.

If the source type is `bool`, then `false` converts to zero and `true` converts to one.

Boolean conversions. When other types are converted to `bool`, a zero value converts to `false` and a non-zero value converts to `true`.

⁹Effective since C++20. Before C++20 the result was implementation-defined; see Integer conversions on cp-preference, https://zh.cppreference.com/w/cpp/language/implicit_conversion.

3.4 Defining Variables

Simply put,¹⁰ defining a variable requires a type specifier (which states the type of the variable) and the name of the variable to be defined.

For example, each of the following statements is a variable definition statement:

```
1 int oi;
2 double wiki;
3 char org = 'c';
```

In the program fragments we have seen so far, a variable defined in a place wrapped in curly braces is a *local variable*, while a variable defined in a place not wrapped in curly braces is a *global variable*. There are in fact exceptions, but you need not understand them now.

A global variable defined without an initial value is initialized to 0. Local variables do not have this property: you must assign an initial value by hand, otherwise hard-to-find bugs may arise.

3.5 Variable Scope

A scope is the block of code in which a variable can take effect.

The scope of a global variable starts at the place where it is defined¹¹ and extends to the end of the file.

The scope of a local variable starts at the place where it is defined and extends to the end of the code block.

Several statements wrapped in a pair of curly braces form a code block.

```
1 int g = 20; // define a global variable
2
3 int main() {
4     int g = 10; // define a local variable
5     printf("%d\n", g); // output g
6     return 0;
7 }
```

If a variable with the same name is defined in a block nested inside another block, then in the inner block you cannot access the variable of the same name in the outer block.

For example, in the code above, the value of *g* printed will be 10. Therefore, to avoid unexpected errors, try to avoid giving local variables the same names as global variables.

3.6 Constants

A constant is a fixed value that does not change during program execution.

The value of a constant cannot be modified after it is defined. Just add the `const` keyword when defining it.

¹⁰When defining a variable, other specifiers may also appear besides the type specifier; see Declarations on cppreference, <https://zh.cppreference.com/w/cpp/language/declarations>.

¹¹More precisely, at its *point of declaration*; see <https://zh.cppreference.com/w/cpp/language/scope>.


```
1 const int a = 2;  
2 a = 3;
```

If you modify the value of a constant, an error is reported at compile time: `error: assignment of read-only variable 'a'`.

3.7 References and Notes

1. Working Draft, Standard for Programming Language C++, <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/n4917.pdf>
2. Types — cppreference.com, <https://zh.cppreference.com/w/cpp/language/type>
3. Arithmetic types (C language) — cppreference.com, https://zh.cppreference.com/w/c/language/arithmetic_types
4. Fundamental types — cppreference.com, <https://zh.cppreference.com/w/cpp/language/types>
5. Fixed-width integer types (since C++11) — cppreference.com, <https://zh.cppreference.com/w/cpp/types/integer>
6. William Kahan (1 October 1997), “Lecture Notes on the Status of IEEE Standard 754 for Binary Floating-Point Arithmetic”, <https://people.eecs.berkeley.edu/~wkahan/ieee754status/IEEE754.PDF>
7. Implicit conversions — cppreference.com, https://zh.cppreference.com/w/cpp/language/implicit_conversion
8. Declarations — cppreference, <https://zh.cppreference.com/w/cpp/language/declarations>
9. Scope — cppreference.com, <https://zh.cppreference.com/w/cpp/language/scope>

4 Operators and Operations (op)

Original title: 运算 · source page /lang/op/

4.1 Arithmetic Operators

Operator	Function
<code>+</code> (unary)	unary plus
<code>-</code> (unary)	unary minus
<code>*</code> (binary)	multiplication
<code>/</code>	division
<code>%</code>	modulo
<code>+</code> (binary)	addition
<code>-</code> (binary)	subtraction

Note — Unary and binary operators

A *unary operator* (also called a one-place operator) is an operator with only one operand, while a *binary operator* (also called a two-place operator) has two operands. For example, in `1 + 2` the plus sign is a binary operator, and it has two operands, `1` and `2`. In addition, C++ has exactly one ternary operator, `? :`.

Among the arithmetic operators there are two unary operators (unary plus, unary minus) and five binary operators (multiplication, division, modulo, addition, subtraction); the unary operators have the highest precedence.

The modulo operator `%` computes the remainder obtained by dividing one integer by another, i.e., it computes the remainder.

The binary `-` acts as the subtraction operator, as in `2-1`; the unary `-` acts as the negation operator, as in `-1`.

It is used as

$$\text{op} = x - y * z$$

The computed value of `op` follows the usual precedence rules of arithmetic (addition, subtraction, multiplication, division): operations with higher precedence are performed first, operations of equal precedence are performed according to their associativity, and parentheses raise precedence.

4.1.1 Type Conversions in Arithmetic

For a binary arithmetic operator, when the two operands have the same type, no [type conversion](#) happens, and the result is stored in the type of the operands; otherwise a type conversion happens so that the two operands have the same type. For the conversion rules, see §3.3.

For example, for an `int` variable `x` and a `double` variable `y`:

- `x/3` has an integer (`int`) result;
- `x/3.0` has a double-precision floating-point result;
- `x/y` has a double-precision floating-point result;

- `x*1/3` has an integer result;
- `x*1.0/3` has a double-precision floating-point result.

4.2 Bitwise Operators

See also: Bit Operations (OI Wiki → Mathematics → Bit Operations).

Operator	Function
<code>~</code>	bitwise NOT
<code>&</code> (binary)	bitwise AND
<code> </code>	bitwise OR
<code>^</code>	bitwise XOR
<code><<</code>	bitwise left shift
<code>>></code>	bitwise right shift

For the meaning of the bitwise operations, see the page on Bit Operations (OI Wiki → Mathematics). Note that bitwise operators have *lower* precedence than the arithmetic operators (except bitwise NOT), and bitwise AND, OR, and XOR rank below the comparison operators (see the complete C++ operator precedence table later in this section), so be careful when using them and add parentheses when necessary.

In a shift operation, the behavior is undefined if either of the following occurs:

1. the right operand (the shift count) is negative;
2. the right operand is greater than or equal to the bit width of the left operand.

For example, for an `int32_t` variable `a`, both `a<<-1` and `a<<32` are undefined.

For a left shift of a signed non-negative value, you must make sure the shifted result fits in the original type; otherwise the behavior is also undefined.¹² A left shift applied to a negative value is also undefined.¹³

For the right shift, the extra bits on the right are discarded, and the left side is more complicated: for unsigned values, 0 bits are shifted in on the left¹⁴; for signed values, copies of the highest bit (in fact the sign bit: 0 for non-negative values, 1 for negative values) are shifted in.¹⁵

¹²Before C++20: if the original value had a signed type and the shifted result fits in the unsigned version of that type, the result was *converted* back to the corresponding signed value; otherwise the behavior was undefined; for a left shift of an unsigned value, the bits shifted out of the result type were discarded. From C++20 on, `a << b` is stipulated to be the value of $a \cdot 2^b$ modulo 2^N (where N is the bit width of the result type) — that is, for both signed and unsigned values, a left shift simply discards the bits shifted out of the result type (the arithmetic/logical left shift; see OI Wiki → Mathematics → Bit Operations, “Shifts”).

¹³Before C++20. For the behavior from C++20 on, see the previous footnote.

¹⁴That is, the logical right shift (OI Wiki → Mathematics → Bit Operations, “Shifts”).

¹⁵That is, the arithmetic right shift (OI Wiki → Mathematics → Bit Operations, “Shifts”). Before C++20, the right shift of signed values was implementation-defined, and most implementations used the arithmetic right shift. From C++20 on, `a >> b` is stipulated to be $\lfloor a/2^b \rfloor$, so the right shift of signed values is the arithmetic right shift.

4.3 Increment and Decrement Operators

Sometimes we need a variable to increase by 1 (increment) or decrease by 1 (decrement); that is where the increment operator `++` and the decrement operator `--` come in.

The increment/decrement operators may be placed before or after the variable: before the variable is called the prefix form, after it the postfix form. Used alone as statements, prefix and postfix need not be distinguished; but if the value of the expression is used, you must pay attention. See the example below. For details see the examples in the [References](#) section.

```
1 i = 100;
2
3 op1 = i++; // op1 = 100: first op1 = i, then i = i + 1
4
5 i = 100;
6
7 op2 = ++i; // op2 = 101: first i = i + 1, then assign to op2
8
9 i = 100;
10
11 op3 = i--; // op3 = 100: first assign to op3, then i = i - 1
12
13 i = 100;
14
15 op4 = --i; // op4 = 99: first i = i - 1, then assign to op4
```

4.4 Compound Assignment Operators

The compound assignment operators are in fact shorthand forms of expressions. They divide into the compound arithmetic operators `+=`, `-=`, `*=`, `/=`, `%=` and the compound bitwise operators `&=`, `|=`, `^=`, `<<=`, `>>=`.

For example, `op = op + 2` can be written `op += 2`; `op = op - 2` can be written `op -= 2`; and `op = op * 2` can be written `op *= 2`.

4.5 The Conditional Operator

The conditional operator can be regarded as a shorthand form of the `if` statement: in `a ? b : c`, if the expression `a` holds, the result of the conditional expression is `b`; otherwise the result is `c`.

4.6 Comparison Operators

Operator	Function
>	greater than
>=	greater than or equal to
<	less than
<=	less than or equal to
==	equal to
!=	not equal to

In particular, be careful to distinguish the equality operator `==` from the assignment operator `=`; this matters most inside conditionals.

`if (op=1)` and `if (op==1)` look similar but do very different things. The first statement assigns to `op`; when the assigned value is non-zero the condition is true, so the condition is always satisfied and no actual test is achieved. Only the second statement actually tests the value of `op`.

4.7 Logical Operators

Operator	Function
<code>&&</code>	logical AND
<code> </code>	logical OR
<code>!</code>	logical NOT

```

1 Result = op1 && op2; // Result is true when both op1 and op2 are true
2
3 Result = op1 || op2; // Result is true when either op1 or op2 is true
4
5 Result = !op1; // Result is true when op1 is false

```

The **built-in** operators `&&` and `||` perform short-circuit evaluation (if the result is already known after evaluating the first operand, the second is not evaluated); overloaded operators do not have this property and always evaluate both operands.

4.8 The Comma Operator

The comma operator separates several expressions; the separated expressions are evaluated in order from left to right, and the value of the whole expression is the value of the last expression. Among all operators, the comma expression has the **lowest** precedence.

```

1 exp1, exp2, exp3; // the final value is the result of evaluating exp3.
2
3 Result = 1 + 2, 3 + 4, 5 + 6;
4 // Result gets the value 3, not 11, because the assignment operator "="
5 // has higher precedence than the comma operator: the assignment is performed

```

```

6 // first, and only then the comma operation.
7
8 Result = (1 + 2, 3 + 4, 5 + 6);
9
10 // If you want Result to get the result of the comma operation, you must
11 // raise the precedence of the whole expression with parentheses; only then
12 // does Result get the value 11.

```

4.9 Member Access Operators

Operator	Function
[]	array subscripting
.	member of an object
& (unary)	address-of / obtaining a reference
* (unary)	indirection / dereferencing
->	member via pointer

These operators are used to access the members of an object or memory. All of them can be overloaded except the last ones mentioned below. For content related to `&`, `*`, and `->`, read the [Pointers](#) and [References](#) tutorials. Two rarely used operators, `.*` and `->*`, are omitted here; for their usage see the C++ language manual on member access.¹⁶

```

1 auto result1 = v[1]; // get the object at index 2 in v
2 auto result2 = p.q;  // get member q of object p
3 auto result3 = p -> q; // get member q of the object pointed to by pointer p;
4                      // equivalent to (*p).q
5 auto result4 = &v;   // get a pointer to v
6 auto result5 = *v;   // get the object pointed to by pointer v

```

4.10 The Complete C++ Operator Precedence Table

From “C++ operator precedence” on cppreference,¹⁷ with modifications.

Operator	Description	Example	Overloadable
Level 1			
::	scope resolution	Class::age = 2;	no
Level 2			
++	postfix increment	for (int i = 0; i < 10; i++) cout << i;	yes
--	postfix decrement	for (int i = 10; i > 0; i--) cout << i;	yes

¹⁶https://zh.cppreference.com/w/cpp/language/operator_member_access

¹⁷https://zh.cppreference.com/w/cpp/language/operator_precedence

Operator	Description	Example	Overloadable
<code>type() type{}</code>	functional-form conversion	<code>type unsigned int a = unsigned(3.14);</code>	yes
<code>()</code>	function call	<code>isdigit('1')</code>	yes
<code>[]</code>	array subscripting	<code>array[4] = 2;</code>	yes
<code>.</code>	member of an object	<code>obj.age = 34;</code>	no
<code>-></code>	member via pointer	<code>ptr->age = 34;</code>	yes
Level 3 (right-to-left associativity)			
<code>++</code>	prefix increment	<code>for (i = 0; i < 10; ++i) cout << i;</code>	yes
<code>--</code>	prefix decrement	<code>for (i = 10; i > 0; --i) cout << i;</code>	yes
<code>+</code>	unary plus	<code>int i = +1;</code>	yes
<code>-</code>	unary minus	<code>int i = -1;</code>	yes
<code>!</code>	logical NOT	<code>if (!done) ...</code>	yes
<code>~</code>	bitwise NOT	<code>flags = ~flags;</code>	yes
<code>(type)</code>	C-style cast	<code>int i = (int) floatNum;</code>	yes
<code>*</code>	dereference	<code>int data = *intPtr;</code>	yes
<code>&</code>	address-of	<code>int *intPtr = &data;</code>	yes
<code>sizeof</code>	size in memory of a type	<code>int size = sizeof floatNum; int size = sizeof(float);</code>	no
<code>new</code>	dynamic allocation of a single object	<code>long *pVar = new long; MyClass *ptr = new MyClass(args);</code>	yes
<code>new []</code>	dynamic allocation of an array	<code>long *array = new long[n];</code>	yes
<code>delete</code>	dynamic destruction of a single object	<code>delete pVar;</code>	yes
<code>delete []</code>	dynamic destruction of an array	<code>delete [] array;</code>	yes
Level 4			
<code>.*</code>	member via pointer-to-member (object)	<code>obj.*var = 24;</code>	no
<code>->*</code>	member via pointer-to-member (pointer)	<code>ptr->*var = 24;</code>	yes
Level 5			
<code>*</code>	multiplication	<code>int i = 2 * 4;</code>	yes
<code>/</code>	division	<code>float f = 10.0 / 3.0;</code>	yes
<code>%</code>	remainder (modulo)	<code>int rem = 4 % 3;</code>	yes
Level 6			
<code>+</code>	addition	<code>int i = 2 + 3;</code>	yes
<code>-</code>	subtraction	<code>int i = 5 - 1;</code>	yes
Level 7			

Operator	Description	Example	Overloadable
<<	bitwise left shift	<code>int flags = 33 << 1;</code>	yes
>>	bitwise right shift	<code>int flags = 33 >> 1;</code>	yes
Level 8			
<=>	three-way comparison operator	<code>if ((i <=> 42) < 0) ...</code>	yes
Level 9			
<	less than	<code>if (i < 42) ...</code>	yes
<=	less than or equal	<code>if (i <= 42) ...</code>	yes
>	greater than	<code>if (i > 42) ...</code>	yes
>=	greater than or equal	<code>if (i >= 42) ...</code>	yes
Level 10			
==	equal to	<code>if (i == 42) ...</code>	yes
!=	not equal to	<code>if (i != 42) ...</code>	yes
Level 11			
&	bitwise AND	<code>flags = flags & 42;</code>	yes
Level 12			
^	bitwise XOR	<code>flags = flags ^ 42;</code>	yes
Level 13			
	bitwise OR	<code>flags = flags 42;</code>	yes
Level 14			
&&	logical AND	<code>if (conditionA && conditionB) ...</code>	yes
Level 15			
	logical OR	<code>if (conditionA conditionB) ...</code>	yes
Level 16 (right-to-left associativity)			
? :	conditional operator	<code>int i = a > b ? a : b;</code>	no
throw	throw expression	<code>throw EClass("Message");</code>	no
=	assignment	<code>int a = b;</code>	yes
+=	add-assign	<code>a += 3;</code>	yes
-=	subtract-assign	<code>b -= 4;</code>	yes
*=	multiply-assign	<code>a *= 5;</code>	yes
/=	divide-assign	<code>a /= 2;</code>	yes
%=	modulo-assign	<code>a %= 3;</code>	yes
<<=	bitwise left-shift assign	<code>flags <<= 2;</code>	yes
>>=	bitwise right-shift assign	<code>flags >>= 2;</code>	yes
&=	bitwise AND assign	<code>flags &= new_flags;</code>	yes
^=	bitwise XOR assign	<code>flags ^= new_flags;</code>	yes

Operator	Description	Example	Overloadable
<code> =</code>	bitwise OR assign	<code>flags = new_flags;</code>	yes
Level 17			
<code>,</code>	comma separator	<code>for (i = 0, j = 0; i < 10; i++, j++) ...</code>	yes

Note that the table does not list operators such as `const_cast`, `static_cast`, `dynamic_cast`, `reinterpret_cast`, `typeid`, `sizeof...`, `noexcept`, and `alignof`, because they are used in the same form as function calls and can cause no ambiguity.

5 Branching (branch)

Original title: 分支 · source page </lang/branch/>

By default a program executes in the order the code is written; sometimes we need to execute certain statements selectively, which is what branching is for. Choosing suitable branching statements can improve the efficiency of a program.

5.1 The `if` Statement

5.1.1 The Basic `if` Statement

The following is the structure of the basic `if` statement.

```

1 if (condition) {
2     body;
3 }
```

The `if` statement evaluates the condition; if the result is true (non-zero), the statements are executed, otherwise they are not.

If the body contains only a single statement, the curly braces may be omitted.

5.1.2 The `if...else` Statement

```

1 if (condition) {
2     body1;
3 } else {
4     body2;
5 }
```

The `if...else` statement is similar to the `if` statement; `else` does not need a condition. When the condition of the `if` statement is satisfied, the statements inside `if` are executed; when it is not satisfied, the statements inside `else` are executed. Likewise, when the body has only one statement, the curly braces may be omitted.

5.1.3 The else if Statement

```
1  if (condition1) {  
2      body1;  
3  } else if (condition2) {  
4      body2;  
5  } else if (condition3) {  
6      body3;  
7  } else {  
8      body4;  
9  }
```

The **else if** statement is a combination of **if** and **else**, testing several conditions and selecting different branches of statements. The final **else** needs no condition. For example, if condition 1 is true, body 1 is executed; if condition 3 is true while conditions 1 and 2 are both false, body 3 is executed; only when all conditions are false is body 4 executed.

In fact, this statement is equivalent to the **else** clause of the first **if** holding only a single **if** statement, with the curly braces omitted and everything placed together. If the conditions are mutually parallel, writing it this way makes the logic of the code clearer.

Logically, it is roughly equivalent to the following passage:

When solving a quadratic equation, the relation between the roots and the discriminant:

- if ($\Delta < 0$) the equation has no real solution;
- otherwise, if ($\Delta = 0$) the equation has two identical real solutions;
- otherwise the equation has two distinct real solutions.

5.2 The switch Statement

```
1  switch (selector) {  
2      case label1:  
3          body1;  
4      case label2:  
5          body2;  
6      default:  
7          body3;  
8  }
```

When a **switch** statement executes, the value of the selector is computed first, then the corresponding label is chosen according to that value, and execution starts from that label. Here the selector must be an expression of integer type, and the labels must all be constants of integer type. For example:

```
1  int i = 1; // here the data type of i is integer, which satisfies the  
2              // requirement that the selector be an integer-type expression  
3  
4  switch (i) {  
5      case 1:  
6          cout << "OI WIKI" << endl;
```

```
7 }
```

```
1 char i = 'A';
2
3 // here the data type of i is character, but char also belongs to the
4 // integer types, so the requirement that the selector be an integer-type
5 // expression is satisfied
6 switch (i) {
7     case 'A':
8         cout << "OI WIKI" << endl;
9 }
```

A **switch** statement also needs **break** statements inserted where required; otherwise, after the matching **case** has been selected, the statements of all the following **cases** and of **default** are executed as well. See the example below.

```
1 char i = 'B';
2
3 switch (i) {
4     case 'A':
5         cout << "OI" << endl;
6         break;
7
8     case 'B':
9         cout << "WIKI" << endl;
10
11     default:
12         cout << "Hello World" << endl;
13 }
```

After the code above runs, the output is **WIKI** and **Hello World**. If you do not want the statements of the following branches to run, you need **break**; see the example below.

```
1 char i = 'B';
2
3 switch (i) {
4     case 'A':
5         cout << "OI" << endl;
6         break;
7
8     case 'B':
9         cout << "WIKI" << endl;
10        break;
11
12     default:
13         cout << "Hello World" << endl;
14 }
```

After the code above runs, the output is **WIKI** only; because of the **break**, the following statements are not executed. The last statement does not need a **break**, because there is nothing below it.

The case labels must not repeat, but they may be out of order. In other words, the order of the labels does not matter; the **cases** (including **default**) may appear in any order. For example:

```
1 char i = 'B';
2
3 switch (i) {
4     case 'B':
5         cout << "WIKI" << endl;
6         break;
7
8     default:
9         cout << "Hello World" << endl;
10        break;
11
12    case 'A':
13        cout << "OI" << endl;
14 }
```

The **case** clauses of a **switch** may also optionally be wrapped in curly braces. Note, however, that if you need to define variables inside the **switch** statement, the curly braces are mandatory. For example:

```
1 char i = 'B';
2
3 switch (i) {
4     case 'A': {
5         int i = 1, j = 2;
6         cout << "OI" << endl;
7         ans = i + j;
8         break;
9     }
10
11    case 'B': {
12        int qwq = 3;
13        cout << "WIKI" << endl;
14        ans = qwq * qwq;
15        break;
16    }
17
18    default: {
19        cout << "Hello World" << endl;
20    }
21 }
```

Note — How to understand switch

Above, terms such as “**case** clause” were used a great deal. In fact, at the level of the underlying implementation, a **switch** is equivalent to a group of jump statements. That is also why tricks such as Duff’s Device exist; interested readers may study them on their own.

6 Loops (loop)

Original title: 循环 · source page /lang/loop/

Sometimes we need to do the same thing many times; to avoid writing too much repetitive code, we need loops.

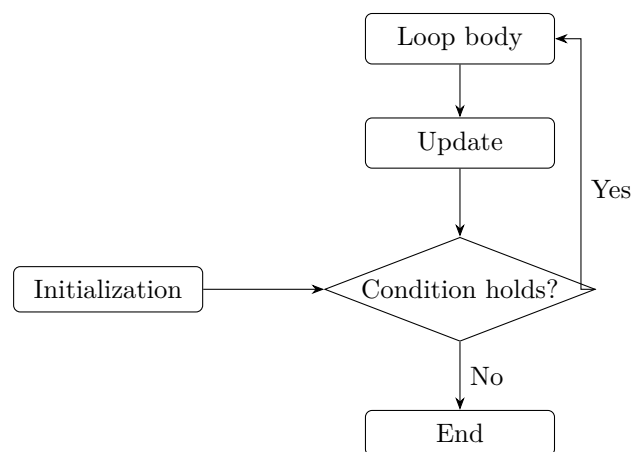
Sometimes the number of iterations is not a constant; then we cannot duplicate the code many times and must use a loop.

6.1 The for Statement

The following is the structure of the **for** statement:

```
1 for (init; condition; update) {  
2     loop body;  
3 }
```

Execution order:



e.g. reading n numbers:

```
1 for (int i = 1; i <= n; ++i) {  
2     cin >> a[i];  
3 }
```

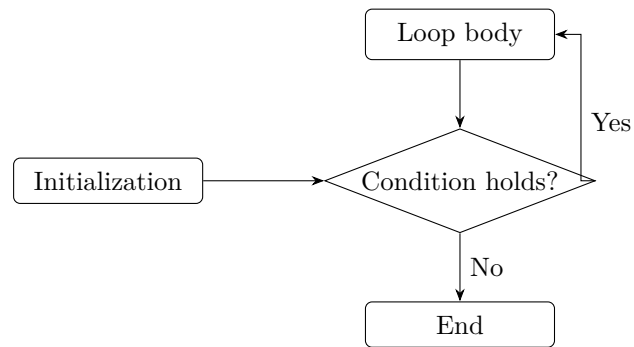
Of the three parts of the **for** statement, any part may be omitted. In particular, if the condition is omitted, it is equivalent to the condition always being true.

6.2 The while Statement

The following is the structure of the **while** statement:

```
1 while (condition) {  
2     loop body;  
3 }
```

Execution order:



e.g. verifying the $3x + 1$ conjecture:

```

1 while (x > 1) {
2     if (x % 2 == 1) {
3         x = 3 * x + 1;
4     } else {
5         x = x / 2;
6     }
7 }
  
```

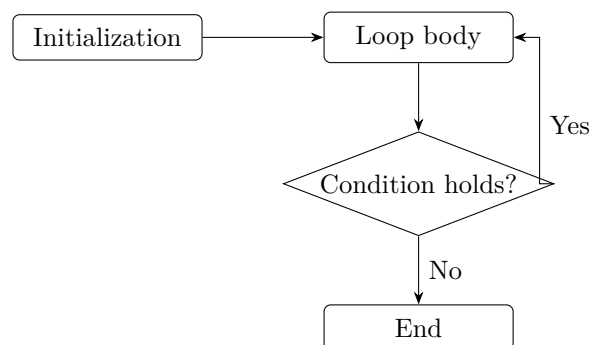
6.3 The do...while Statement

The following is the structure of the **do...while** statement:

```

1 do {
2     loop body;
3 } while (condition);
  
```

Execution order:



The difference from the **while** statement is that **do...while** executes the loop body first and tests afterwards.

e.g. enumerating permutations:

```

1 do {
2     // do something...
3 } while (next_permutation(a + 1, a + n + 1));
  
```

6.4 How the Three Statements Relate

```
1 // for statement
2
3 for (statement1; statement2; statement3) {
4     statement4;
5 }
6
7 // while statement
8
9 statement1;
10 while (statement2) {
11     statement4;
12     statement3;
13 }
```

These are equivalent when `statement4` contains no `continue` statement (see below), but the second form is rarely used.

```
1 // while statement
2
3 statement1;
4 while (statement2) {
5     statement1;
6 }
7
8 // do...while statement
9
10 do {
11     statement1;
12 } while (statement2);
```

When `statement1` contains no `continue` statement, these two forms are also equivalent.

```
1 while (1) {
2     // do something...
3 }
4
5 for (;;) {
6     // do something...
7 }
```

Both of these loop forever. (You can use `break` (see below) to exit.)

As you can see, the three statements can replace one another, but in general their selection follows these principles:

1. When the loop has a fixed increment step (most commonly enumeration), use a `for` statement;
2. when only the termination condition of the loop is known, use a `while` statement;

- when using a **while** statement, if the loop body must run once before testing, use a **do...while** statement. This is rarely used; a common scenario is user input.

6.5 The **break** and **continue** Statements

The **break** statement exits the loop.

The **continue** statement skips the remainder of the loop body. Taking the use of **continue** inside a **do...while** statement as an example:

```

1 do {
2     // do something...
3     continue; // equivalent to goto END;
4     // do something...
5     END;;
6 } while (statement);

```

Both **break** and **continue** can be used in the loop bodies of all three loop statements.

Generally speaking, **break** and **continue** are used to make the logic of the code clearer, for example:

```

1 // less clear logic, deeply nested braces
2
3 for (int i = 1; i <= n; ++i) {
4     if (i != x) {
5         for (int j = 1; j <= n; ++j) {
6             if (j != x) {
7                 // do something...
8             }
9         }
10    }
11 }
12
13 // clearer logic, simple and plain brace levels
14
15 for (int i = 1; i <= n; ++i) {
16     if (i == x) continue;
17     for (int j = 1; j <= n; ++j) {
18         if (j == x) continue;
19         // do something...
20     }
21 }

```

```

1 // the for statement's condition is complicated and does not express the
2 // essence of "enumeration"
3
4 for (int i = l; i <= r && i % 10 != 0; ++i) {
5     // do something...
6 }
7
8 // the for statement enumerates, and break expresses "when to stop"
9

```



```

10 for (int i = l; i <= r; ++i) {
11     if (i % 10 == 0) break;
12     // do something...
13 }

```

```

1 // statements repeated, order unnatural
2
3 statement1;
4 while (statement3) {
5     statement2;
6     statement1;
7 }
8
9 // no repeated statements, order natural
10
11 while (1) {
12     statement1;
13     if (!statement3) break;
14     statement2;
15 }

```

7 Arrays (array)

Original title: 数组 · source page </lang/array/>

An array is a container that holds objects of the same type. The objects stored in an array have no names; they are accessed through their positions. The size of an array is fixed: the length of an array cannot be changed at will.

7.1 Defining Arrays

The declaration of an array has the form `a[d]`, where `a` is the name of the array and `d` is the number of elements in the array. At compile time, `d` must be known; that is to say, `d` must be a constant expression of integer type.

```

1 unsigned int d1 = 42;
2 const int d2 = 42;
3 int arr1[d1]; // error: d1 is not a constant expression
4 int arr2[d2]; // correct: arr2 is an array of length 42

```

One array cannot be assigned directly to another array:

```

1 int arr1[3];
2 int arr2 = arr1; // error
3 arr2 = arr1;     // error

```

Try to define large arrays as global variables. Local variables are created on the stack, and an over-large array (larger than the stack size) will blow the stack and cause an RE. If an array is declared in the global scope, the array is created in the static storage area instead.

7.2 Accessing Array Elements

You can access the elements of an array with the subscript operator `[]`; the indices of an array (the values in square brackets) start from 0. Take an array of 10 elements as an example: its indices are 0 through 9, not 1 through 10. In OI, however, for convenience we usually make the array a bit larger and do not use the first element of the array, accessing array elements starting from index 1.

Example 1: read an integer n from standard input, then read n numbers and store them into an array, where $n \leq 1000$.

```
1 #include <iostream>
2 using namespace std;
3
4 int arr[1001]; // the index range of the array arr is [0, 1001)
5
6 int main() {
7     int n;
8     cin >> n;
9     for (int i = 1; i <= n; ++i) {
10         cin >> arr[i];
11     }
12 }
```

Example 2: (continuing example 1) sum the elements of the array `arr` and output the sum, where the sum of all elements of the array is at most $2^{31} - 1$.

```
1 #include <iostream>
2 using namespace std;
3
4 int arr[1001];
5
6 int main() {
7     int n;
8     cin >> n;
9     for (int i = 1; i <= n; ++i) {
10         cin >> arr[i];
11     }
12
13     int sum = 0;
14     for (int i = 1; i <= n; ++i) {
15         sum += arr[i];
16     }
17
18     printf("%d\n", sum);
19     return 0;
20 }
```

7.2.1 Out-of-Bounds Subscripts

The subscript idx of an array should satisfy $0 \leq idx < size$. If the subscript is outside this range, it is undefined behavior and can produce unpredictable consequences, such as a segmentation fault, or the modification of variables other than the intended one.

7.3 Multidimensional Arrays

The essence of a multidimensional array is an “array of arrays”: the elements of the outer array are themselves arrays. A two-dimensional array is defined by two dimensions: the length of the array and the length of the elements inside the array. When accessing a two-dimensional array you must write out two indices:

```
1 int arr[3][4]; // an array of length 3 whose elements are "arrays of
2               // length 4 with elements of type int"
3 arr[2][1] = 1; // accessing the 2-D array
```

We often use nested `for` loops to process a two-dimensional array.

Example: read two numbers n and m from standard input, the height and the width of a black-and-white picture, with $n, m \leq 1000$. For each of the next n lines, the line contains m numbers separated by spaces, representing the brightness values of the corresponding positions. Now we read this picture and store it in a two-dimensional array.

```
1 const int MAXN = 1001;
2 int pic[MAXN][MAXN];
3 int n, m;
4
5 cin >> n >> m;
6 for (int i = 1; i <= n; ++i)
7     for (int j = 1; j <= m; ++j) cin >> pic[i][j];
```

Similarly, you can define three-dimensional, four-dimensional, and higher-dimensional arrays.

8 Structures (`struct`)

Original title: 结构体 · source page [/lang/struct/](#)

A **structure** (`struct`) can be seen as a combination of a series of items called *members*.

It can be seen as a user-defined data type.

Note

The `struct` described on this page is different from the `struct` of C: in C++, `struct` has been extended into a class specifier similar to a [class](#).

8.1 Defining a Structure

```
1 struct Object {  
2     int weight;  
3     int value;  
4 } e[array_length];  
5  
6 const Object a;  
7 Object b, B[array_length], tmp;  
8 Object *c;
```

The example above defines a structure named `Object` with two members, `value` and `weight`, both of type `int`.

After the `}`, the example defines the constant `a` of data type `Object`, the variables `b` and `tmp`, the array `B`, and the pointer `c`. For any type that already exists, the methods shown here can be used to define constants, variables, pointers, arrays, and so on.

About pointers: you need not force yourself to master them yet.

8.1.1 Defining Pointers

For pointers to built-in types, the definition is the same as the ordinary definition of pointers.

For pointers to structures, use `StructName*` in the definition.

```
1 struct Edge {  
2     /*  
3     ...  
4     */  
5     Edge* nxt;  
6 };
```

The example above is only for illustration; do not worry about its actual meaning.

8.2 Accessing / Modifying Members

You can use `variable_name.member_name` to access a member. For example, `cout << var.v` outputs the member `v` of `var`.

You can also use `pointer_name->member_name` or `(*pointer_name).member_name` to access it. For example, `(*ptr).v = tmp` or `ptr->v = tmp` assigns `tmp` to the member `v` of the structure that the structure pointer `ptr` points to.

8.3 Why Do We Need Structures?

First of all, all roads lead to Rome: you could achieve the same effect without structures. But a structure can explicitly bundle its members together (in algorithm contests these are usually variables): the `Object` structure in this example, for instance, puts `value` and `weight` together (the actual meaning of this structure is to represent the weight and the value of an item). The benefit is that it constrains the use of the members.

Imagine that without structures you had two arrays `value[]` and `Value[]`; it would be easy to mix them up. With structures, the chance of using the wrong variable is reduced.

Moreover, different structures (structure types, such as the `Object` structure) or different structure variables (instances of a structure, such as the array `e` above) may have members with the same name (such as `tmp.value` and `b.value`), and same-named members are independent of one another (they have their own memory: modifying `tmp.value` does not affect the value of `b.value`).

The benefit is that you can use identical or near-identical names to describe an item. For example, `Object` has the member `value`; we can also define a `Car` structure that also has a member `value`. Without structures, we might need to define differently named arrays such as `valueOfObject[]` and `valueOfCar[]` to tell them apart.

If you want to describe a thing in more detail, you can also define member functions; see §24 for details.

8.4 More Operations?

See §24.

8.5 Points of Caution

To make memory access more efficient, when the compiler handles the actual storage of members inside a structure it may align the members at certain byte positions, which means the structure has some empty places. As a result, the space occupied by the structure can be larger than the sum of the space occupied by all its members.

8.6 References

1. Class — zh.cppreference.com, <https://zh.cppreference.com/w/cpp/language/class>
2. Data structures — cplusplus.com, <http://www.cplusplus.com/doc/tutorial/structures/>
3. Alignment — Microsoft Docs, <https://docs.microsoft.com/zh-cn/cpp/cpp/alignment-cpp-declarations>

9 Unions (union)

Original title: 联合体 · source page /lang/union/

A **union** is a special class type that can hold only one of its non-static data members at a time. Unions were officially added to the entry level of the NOI syllabus in 2023.

9.1 Defining a Union

The class specifier of a union declaration is similar to that of a class or a [structure](#) declaration:

```
1 union MyUnion {  
2     int x;  
3     long long y;  
4 } x;
```

The definition of a union is similar to that of a structure. Under the definition above, `MyUnion` can likewise be used as a user-defined type. The name `MyUnion` may be omitted.

9.2 Accessing / Modifying Members

As with structures, you can use `variable_name.member_name` to access a member.

The memory occupied by a union is **no smaller than** the size of its largest member, and all members **share the same memory space and address**. When one member is assigned a value, because the memory is shared, the other members of the union are overwritten. That is, at any moment a union can hold the value of only one member.

For more uses of unions, see Union declarations on cppreference: <https://zh.cppreference.com/w/cpp/language/union>.

10 Pointers (pointer)

Original title: 指针 · source page /lang/pointer/

10.1 Addresses of Variables, and Pointers

In a program, all of our data has an address where it is stored. In each actual run of the program, the physical locations where variables are stored are not necessarily the same. Nevertheless, while programming we can, through certain statements, obtain the address of data in memory.

An address is also data. The variable type used to store addresses has a special name: the “pointer variable”, sometimes shortened to “pointer”.

Note — The size of a pointer variable

The size of a pointer variable differs across environments. On a 32-bit machine, addresses are represented by 32-bit binary integers, so a pointer has a size of 4 bytes. On a 64-bit machine, addresses are represented by 64-bit binary integers, so a pointer has a size of 8 bytes.

An address is just a mark-like piece of data. To work with data of different types, “pointer variables” also come in different types: there can be, for example, a pointer variable of `int` type, whose stored address (that is, the numeric value stored in the pointer variable) corresponds to the starting address of a 32-bit block of space; and a pointer variable of `char` type, whose stored address corresponds to the starting address of an 8-bit block.

In fact, the user can also declare a pointer variable that points to a pointer variable.

Suppose the user has defined a structure:

```
1 struct ThreeInt {
2     int a;
3     int b;
4     int c;
5 };
```

Then a pointer variable of type `ThreeInt` corresponds to a block of $3 \times 32 = 96$ bits.

10.2 Declaring and Using Pointers

In C/C++, the type of a pointer variable is the type name followed by a star `*`. For example, the type name of a pointer variable for `int` is `int*`.

We can use the `&` symbol to obtain the address of a variable.

To access the space corresponding to the address held by a pointer variable (also called the space the pointer **points to**), you must **dereference** the pointer variable using the `*` symbol.

```
1 int main() {
2     int a = 123; // a: 123
3     int* pa = &a;
4     *pa = 321; // a: 321
5 }
```

The same works for structure variables. To access a member of the structure a pointer points to, you must first dereference the pointer and then use the `.` member operator. However, the more convenient “arrow” operator `->` is recommended instead.

```
1 struct ThreeInt {
2     int a;
3     int b;
4     int c;
5 };
6
7 int main() {
8     ThreeInt x{1, 2, 3}, y{6, 7, 8};
9     ThreeInt* px = &x;
10    (*px) = y; // x: {6,7,8}
11    (*px).a = 4; // x: {4,7,8}
12    px->b = 5; // x: {4,5,8}
13 }
```

10.3 Pointer Offset

Pointer variables can also be incremented and decremented **with integers**. For an `int` pointer, adding 1 (incrementing by 1) shifts the address it points to by 32 bits (i.e., 4 bytes); adding 2 shifts the address by $2 \times 32 = 64$ bits. Likewise, for a `char` pointer, each increment shifts the address it points to by 8 bits (i.e., 1 byte).

10.3.1 Using Pointer Offsets to Access Arrays

We said earlier that an array is a contiguous block of storage. In C/C++, using the array name directly yields the starting address of the array.

```

1 int main() {
2     int a[3] = {1, 2, 3};
3     int* p = a; // p points to a[0]
4     *p = 4;      // a: [4, 2, 3]
5     p = p + 1;   // p points to a[1]
6     *p = 5;      // a: [4, 5, 3]
7     p++;         // p points to a[2]
8     *p = 6;      // a: [4, 5, 6]
9 }
```

When elements of an array are accessed through a pointer, “pointer offsets” are usually involved; in other words, access happens via a base address (the starting address of the array) plus an offset.

We commonly use the `[]` operator to access the element at a given offset in an array, e.g. `a[3]` or `p[4]`. This way of writing it is equivalent to performing pointer arithmetic and then dereferencing: `p[4]` and `*(p + 4)` are two equivalent spellings.

10.4 The Null Pointer

Before C++11, C++ used the `NULL` macro for the null-pointer constant, just as C does. The implementation of `NULL` in C++ was generally:

```

1 // before C++11
2 #define NULL 0
```

Note — The definition of `NULL` in the C language

Before C23, the C language had two definitions of `NULL` differing only in type: one is an integer constant expression, the other a constant expression converted to type `void *`; both have value 0, and the compiler may choose either implementation.

Mixing the null pointer with the integer `0` causes many problems in C++. For example:

```

1 int f(int x);
2 int f(int* p);
```

When `f(NULL)` is called, the function actually called is of type `int(int)` and not `int(int *)`.

Note — Problems `NULL` causes in the C language

More so than in C++: because there are two definitions, the problems `NULL` causes are more serious in C. If the writer of a variadic function intends it to accept a pointer, but the caller passes a `NULL` defined as an integer, undefined behavior results — when the function uses the passed variadic arguments, type conversions must occur, and the conversion from an integer type to a pointer type is undefined behavior (see *Introduce the `nullptr` constant*, <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3042.htm>).

To solve these problems, C++11 introduced the `nullptr` keyword as the null-pointer constant.

C++ stipulates that `nullptr` converts implicitly to any pointer type, and the result of such a conversion is the null-pointer value of that type.

The type of `nullptr` is `std::nullptr_t`, called the null-pointer type. A possible implementation is:

```
1 namespace std {
2     typedef decltype(nullptr) nullptr_t;
3 }
```

In addition, from C++11 on, the implementation of the `NULL` macro was changed to:

```
1 // since C++11
2 #define NULL nullptr
```

Note — The C language's improvement of the null-pointer constant

For similar reasons, C23 also introduced `nullptr` as the null-pointer constant, together with `nullptr_t` as its type (see *Introduce the `nullptr` constant*, <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3042.htm>).

10.5 Advanced Uses of Pointers

Using pointers lets the programmer manipulate data anywhere in the running program, not limited to a single scope.

10.5.1 Using Pointer-Type Parameters

In C/C++, the arguments used when calling a function (procedure) are all passed into the sub-routine as copies (references are the exception; they are introduced later). By default, a function can return results to the caller only through its return value. But if a function wants to modify data outside itself, or a structure/class is too large to be copied comfortably, one can pass in the address of the external data, which then lets the function access and even modify the external data.

The following `my_swap` routine receives two `int` pointers and, using a temporary variable inside the function, swaps the values of two `int` variables.

```
1 void my_swap(int *a, int *b) {
2     int t;
3     t = *a;
4     *a = *b;
5     *b = t;
6 }
7
8 int main() {
9     int a = 6, b = 10;
10    my_swap(&a, &b);
11    // after the call, in main the variable a has become 10 and b has become 6
```

```
12 }
```

C++ introduced the concept of references, which are easier to use and safer than pointers. See §28 (C++: References) and §36 (C versus C++: Pointers and References).

10.5.2 Dynamic Instantiation

Beyond that, writing programs often involves dynamic memory allocation: at run time, the program dynamically requests memory from the operating system, or returns it. When the program requests memory by calling an operating-system interface, the operating system returns the address of the requested space. To use this space, we must store the address of the space in a pointer variable.

In C++, we use the **new** operator to obtain a block of memory and the **delete** operator to free the space a pointer points to.

```
1 int* p = new int(1234);
2 /* ... */
3 delete p;
```

The statement above uses the **new** operator to request from the operating system a block of space the size of an **int**, initializes the value inside to 1234, and declares an **int** pointer **p** pointing to this space.

Likewise, **new** can open up new objects:

```
1 class A {
2     int a;
3
4     public:
5     A(int a_) : a(a_) {}
6 };
7
8 int main() {
9     A* p = new A(1234);
10    /* ... */
11    delete p;
12 }
```

As above, a “**new** expression” will try to open up a block of space of the corresponding size, try to construct the object on this space, and return the address of this space.

```
1 struct ThreeInt {
2     int a;
3     int b;
4     int c;
5 };
6
7 int main() {
8     ThreeInt* p = new ThreeInt{1, 2, 3};
9     /* ... */
10    delete p;
```

```
11 }
```

Note — List initialization

The `{}` operator can initialize structures without constructors. In addition, using the `{}` operator makes the initialization form of variables uniform. See “list initialization (since C++11)”: https://en.cppreference.com/w/cpp/language/list_initialization.

Note that when memory obtained with `new` is no longer used, `delete` must be used to free the space. A block of memory must not be freed two or more times. Applying `delete` to the null pointer `nullptr`, however, is legal.

10.5.3 Dynamically Creating Arrays

The `new[]` operator can also create arrays; it returns the first address of the array, i.e., the address of the array’s first element, and we can store this address in a pointer of the corresponding type. To free it, use the `delete[]` operator.

```
1 size_t element_cnt = 5;
2 int *p = new int[element_cnt];
3 delete[] p;
```

The elements of the array are stored contiguously: `p + 1` points to the successor of `p`.

10.5.4 Two-Dimensional Arrays

When storing data in matrix form, a data type like the “two-dimensional array” may be used. Semantically, a two-dimensional array is an array of arrays, while computer memory can be viewed as one very long one-dimensional array. Storing a two-dimensional array in computer memory therefore raises the question of whether it is “contiguous”.

“Contiguous” means that the end of any row of the two-dimensional array and the start of the next row are adjacent in physical address; in other words, the whole two-dimensional array can be viewed as a one-dimensional array. Otherwise, the two are not necessarily adjacent physically.

For a “contiguous” two-dimensional array, a single loop, with the help of an ever-increasing pointer, suffices to traverse all the data. For a non-contiguous two-dimensional array, since each row is not contiguous, you must first obtain the address of the head of a row, and then access the elements of that row.

Note — How two-dimensional arrays are stored

This way of storing data row by row is called *row-major* storage; correspondingly, data may also be stored by column (*column-major*). Because of how computer memory access works, accessing contiguous data is generally more efficient. Therefore, choose “row-major” or “column-major” storage according to how the data will be used.

10.5.5 Dynamically Creating Two-Dimensional Arrays

In C/C++, we can declare an N -row by M -column two-dimensional array whose space is physically contiguous with a statement like the following.

Note — Describing the dimensions of an array

A more general way is to speak of the n -th dimension. For “row-major” storage, the first dimension of the array has length N and the second has length M .

```
1 int a[N][M];
```

This form of declaration requires that N and M be constant expressions determinable at compile time.

In C/C++ the first element of an array has subscript 0, so the expression `a[r][c]` denotes the $(c + 1)$ -th element of the $(r + 1)$ -th row of the two-dimensional array `a`; we also say this element has subscript (r, c) .

In practice, however, the size of a (two-dimensional) array may not be fixed, and dynamic memory allocation is needed.

A common way is to declare a **one-dimensional array** of length $N \times M$ and access the element with subscript (r, c) of the two-dimensional array through the subscript `r * M + c`.

```
1 int* a = new int[N * M];
```

This method guarantees that the two-dimensional array is **contiguous**.

Note — Linear storage of arrays at the physical level

In fact, data in memory can always be regarded as stored linearly; under suitable rules, dynamically opening up the space of a one-dimensional array suffices to store an n -dimensional array on top of it.

In addition, memory can be obtained and used according to the concept of an “array of arrays”. An array holding several arrays is in fact an array holding the first addresses of several arrays, that is, an array holding several pointer variables.

We need a variable to hold the first address of this “array of arrays” — that is, the address of a pointer. Such a variable is a “pointer to a pointer”, sometimes called a “double pointer”, e.g.:

```
1 int** a = new int*[5];
```

Then we need to request space for each array:

```
1 for (int i = 0; i < 5; i++) {
2     a[i] = new int[5];
3 }
```

At this point we have obtained the memory. To free memory obtained this way, a reverse operation is required: first free each array, then free the array that stored the first addresses of those arrays, e.g.:

```

1 for (int i = 0; i < 5; i++) {
2     delete[] a[i];
3 }
4 delete[] a;

```

Note that the contiguity of the space of a two-dimensional array obtained this way is *not* guaranteed.

There is still another way, which uses a “pointer to an array”.

Note — The difference between an array name and the address of the array’s first element

We said before that in C/C++, using the array name directly yields a value equal to the address of the array’s first element. But the type of the variable denoted by the array name is in fact the entire array, not a single element.

```

1 int main() { int a[5] = {1, 2, 3, 4, 5}; }

```

Conceptually, the type of the identifier `a` in the code is `int[5]`; practically, the address pointed to by `a + 1` is offset from the address pointed to by `a` by the length of 5 `int` variables.

```

1 int main() {
2     int(*a)[5] = new int[5][5];
3     int* p = a[2];
4     a[2][1] = 1;
5     delete[] a;
6 }

```

What is obtained this way is also contiguous memory, and the first address of the $(n + 1)$ -th row of the array can be obtained directly in the form `a[n]`; therefore the element with subscript (r, c) can be accessed in the form `a[r][c]`.

Since a pointer to an array is also a definite data type, all dimensions except the first must be constants determinable at compile time. Otherwise the compiler cannot translate expressions like `a[n]` (where `a` is a pointer to an array).

10.6 Pointers to Functions

For the introduction to functions, see the [Functions](#) section.

Simply put, to call a function you must know the parameter types, the number of parameters, and the return-value type of the function; these are together called the *interface type* of the function.

Functions can be called through function pointers. Sometimes several functions share the same interface type; using a function pointer, the function to call can be selected **dynamically** as the program runs. In other words, without modifying a function, you can change its behavior merely by changing the argument passed to it (the function pointer).

Suppose we have several binary-operation functions on `int`: their parameters are 2 `ints` and the return value is also an `int`. Below is an example using a function pointer:

```

1 #include <iostream>

```

```

2
3 int (*binary_int_op)(int, int);
4
5 int foo1(int a, int b) { return a * b + b; }
6
7 int foo2(int a, int b) { return (a + b) * b; }
8
9 int main() {
10     int choice;
11     std::cin >> choice;
12     if (choice == 1) {
13         binary_int_op = foo1;
14     } else {
15         binary_int_op = foo2;
16     }
17
18     int m, n;
19     std::cin >> m >> n;
20     std::cout << binary_int_op(m, n);
21 }

```

Note — &, *, and function pointers

In the C language, spellings such as `void (*p)() = foo;`, `void (*p)() = &foo;`, `void (*p)() = *foo;`, `void (*p)() = ***foo` all have the same effect.

Because a function (such as `foo`) can be implicitly converted to a pointer to the function, the spelling `void (*p)() = foo;` works.

Using the `&` operator obtains the address of an object, and this also holds for functions, so the spelling `void (*p)() = &foo;` still works.

Applying the `*` operator to a function pointer obtains the function the pointer points to; for a spelling like `**foo`, `*foo` yields the function `foo`, which is immediately implicitly converted back to a pointer to `foo`. And so on: `**foo` still ends up being a function pointer to `foo`; the user may use any number of `*`s and the result is the same.

Likewise, in a call, statements like `(*p)()` and `p()` are the same; the `*` operator can be omitted.

Reference: “Why do function pointer definitions work with any number of ampersands ‘&’ or asterisks ‘*’” on stackoverflow.com, <https://stackoverflow.com/questions/6893285/why-do-function-pointer-definitions-work-with-any-number-of-ampersands-or-as>

The `typedef` keyword can declare the type of a function pointer:

```

1 typedef int (*p_bi_int_op)(int, int);

```

This way we can later use the type `p_bi_int_op`, i.e., a pointer to a function “whose parameters are 2 `ints` and whose return value is also an `int`”.

Functions can be referenced more conveniently via `std::function`. (To be continued.)

Using function pointers, “callback functions” can be implemented. (To be continued.)

11 Functions (func)

Original title: 函数 · source page /lang/func/

11.1 Declaring Functions

A function in programming is generally a collection of several statements; we can also call it a “**subroutine**”. In programming, if there are repeated procedures, we can extract them into a function. A function can receive several values, called the parameters of the function. A function can also return a value, called the return value of the function.

To declare a function, we need the return-value type, the name of the function, and the parameter list.

```
1 // return-value type: int
2 // name of the function: some_function
3 // parameter list: int, int
4 int some_function(int, int);
```

As above, we declared a function named `some_function` that must receive two parameters of type `int` and whose return-value type is also `int`. One may imagine that this function will perform some operations on the two passed integers and return a result of the same type.

11.2 Implementing Functions: Writing the Definition

A declaration (declaration) alone is not enough: it only lets callers know the **interface** type of the function (what data it receives, what data it returns), but it lacks the concrete internal implementation, which is the **definition** (definition) of the function. We can write code to *implement* this function **somewhere after the declaration** (implementation may also live in another file, but the separately compiled files must be given together at link time).

If the function has a return value, the value must be returned to the caller through a **return** statement. Once execution reaches a **return** statement, the current function ends directly, and the statements after it are no longer executed.

```
1 int some_function(int, int); // declaration
2
3 /* some other code here... */
4
5 int some_function(int x, int y) { // definition
6     int result = 2 * x + y;
7     return result;
8     result = 3; // this statement is never executed
9 }
```

At the definition, we gave names to the variables in the parameter list. This way we can use these variables inside the function definition.

Within the same file, we can also **merge the declaration and the definition** directly; in other words, complete the definition at the point of declaration.

```
1 int some_function(int x, int y) { return 2 * x + y; }
```

If a function needs no return value, mark its return-value type as `void`; if it needs no parameters, the parameter list can be left empty. Likewise, a function with no return value also ends its execution when it reaches a `return;` statement.

```
1 void say_hello() {
2     cout << "hello!\n";
3     cout << "hello!\n";
4     cout << "hello!\n";
5     return;
6     cout << "hello!\n"; // this statement is never executed
7 }
```

11.3 Calling Functions

Like variables, functions must be declared before they can be used. The act of using a function is called a “call”. We can call other functions inside any function, including the function itself. A function calling itself is called **recursion**.

In most languages, the way to call a function is **the function name followed by a pair of parentheses** `()`, as in `foo()`. If the function needs parameters, we write the parameters it needs inside the parentheses, in order, separated by commas, as in `foo(1, 2)`. A function call is itself an expression, and **the return value of the function is the value of the expression**.

The parameters written at declaration can be understood as variables usable inside **the current invocation** of the function; their values are initialized from the values passed at the call site. Consider this example:

```
1 void foo(int, int);
2
3 /* ... */
4
5 void foo(int x, int y) {
6     x = x * 2;
7     y = y + 3;
8 }
9
10 /* ... */
11
12 a = 1;
13 b = 1;
14 // before the call: a = 1, b = 1
15 foo(a, b); // call foo
16           // after the call: a = 1, b = 1
```

In the example above, `foo(a, b)` is one call to `foo`. At the call, the variables `x` and `y` inside `foo` are initialized respectively from the values of `a` and `b` at the call site. Therefore, modifications of the variables `x` and `y` inside `foo` **do not affect the values of the variables at the call site**.

If we need to modify the values of variables inside a function (subroutine), we must use “pass by reference”.

```
1 void foo(int& x, int& y) {
2     x = x * 2;
3     y = y + 3;
4 }
5
6 /* ... */
7
8 a = 1;
9 b = 1;
10 // before the call: a = 1, b = 1
11 foo(a, b); // call foo
12           // after the call: a = 2, b = 4
```

In the code above, we see that after the “`int`” in the function’s parameter list an “`&` (ampersand)” was added, which denotes a **reference** to the `int` type. When `foo` is called, the variables `a` and `b` at the call site initialize the two references-to-`int`, `x` and `y`, inside `foo`. The `x` and `y` in `foo` can be understood as “aliases” of the call-site variables `a` and `b`: operations on `x` and `y` in `foo` are operations on `a` and `b` at the call site.

11.4 The main Function

Specially, every C/C++ program needs a function named `main`. Every program starts running from the `main` function.

The `main` function may also have parameters: through the parameters of `main` we can obtain the instructions the outside world passes to the program (the “command-line arguments”) and react differently accordingly.

Below is a piece of code that calls a function (subroutine):

```
1 // hello_subroutine.cpp
2
3 #include <iostream>
4
5 void say_hello() {
6     std::cout << "hello!\n";
7     std::cout << "hello!\n";
8     std::cout << "hello!\n";
9 }
10
11 int main() {
12     say_hello();
13     say_hello();
14 }
```

12 File Operations (file-op)

Original title: 文件操作 · source page /lang/file-op/

12.1 The Concept of Files

A file is a collection of related data gathered together for a specific purpose. C/C++ regards every file as an ordered byte stream; every file ends with an **end-of-file marker** (EOF). To operate on a file, the program should first open it. Whenever a file is opened (remember to close opened files), the file is associated with a stream, where the stream is in fact a sequence of bytes.

C/C++ divides files into text files and binary files. A text file is a simple text file (the focus here); a binary file is a file of a special format or an executable code file, and so on.

12.2 The Steps of File Operations

1. Open the file, point the file pointer at the file, and decide the file-open type; 2. Read from or write to the file (the main operations used in contests; other operations are omitted for now); 3. After using the file, close it.

12.3 The freopen Function

12.3.1 Introduction

This function redirects a specified input/output stream to a file in a specified way; it is contained in the header `stdio.h` (`cstdio`). It can change the input/output environment without changing the appearance of the code, but you should ensure the stream is reliable when using it.

The function has three main modes: reading, writing, and appending.

12.3.2 Command Format

```
1 FILE* freopen(const char* filename, const char* mode, FILE* stream);
```

12.3.3 Parameter Descriptions

- **filename**: the name of the file to open;
- **mode**: the file-open mode, expressing the file access permissions;
- **stream**: the file pointer; usually the standard file streams (`stdin/stdout`) or the standard error output stream (`stderr`);
- **return value**: a file pointer, pointing to the opened file.

12.3.4 File-Open Modes (Optional Reading)

- **r**: open the file for reading only; the file must exist; only reading is allowed (**commonly used**)
- **r+**: open the file for reading/writing; the file must exist; reading/writing allowed
- **rb**: open a binary file for reading only; the file must exist; only reading allowed
- **rb+**: open a binary file for reading/writing; the file must exist; reading/writing allowed
- **rt+**: open a text file for reading/writing; reading/writing allowed
- **w**: open the file for writing only; if the file does not exist a new file is created, otherwise its contents are cleared; only writing allowed (**commonly used**)
- **w+**: open the file for reading/writing; if the file does not exist a new file is created, otherwise its contents are cleared; reading/writing allowed
- **wb**: open a binary file for writing only; if it does not exist a new file is created, otherwise its contents are cleared; only writing allowed
- **wb+**: open a binary file for reading/writing; if it does not exist a new file is created, otherwise its contents are cleared; reading/writing allowed
- **a**: open the file for writing only; if it does not exist a new file is created; written data is appended to the end of the file (the EOF marker is preserved)
- **a+**: open the file for reading/writing; if it does not exist a new file is created; written data is appended to the end of the file (the EOF marker is not preserved)
- **at+**: open a text file for reading/writing; written data is appended to the end of the file
- **ab+**: open a binary file for reading/writing; written data is appended to the end of the file

12.3.5 Usage

Reading file contents:

```
1 freopen("data.in", "r", stdin);
2 // data.in is the name of the file to read; it must be in the same
3 // directory as the executable
```

Outputting to a file:

```
1 freopen("data.out", "w", stdout);
2 // data.out is the name of the output file, in the same directory as
3 // the executable
```

Closing the standard input/output streams:

```
1 fclose(stdin);
2 fclose(stdout);
```

Note

Functions such as `printf/scanf/cin/cout` use `stdin/stdout` by default; after `stdin/stdout` are redirected, these functions input from/output to the redirected files.

12.3.6 Template

```
1 #include <cstdio>
2 #include <iostream>
3
4 int main(void) {
5     freopen("data.in", "r", stdin);
6     freopen("data.out", "w", stdout);
7     /*
8     The code in between needs no changes: just use cin and cout as usual.
9     */
10    fclose(stdin);
11    fclose(stdout);
12    return 0;
13 }
```

12.4 The `fopen` Function (Optional Reading)

This function is roughly the same as `freopen`; it opens the specified file and returns a pointer to the opened file.

12.4.1 Prototype

```
1 FILE* fopen(const char* path, const char* mode)
```

The meanings of the parameters are the same as for `freopen`.

12.4.2 Available Reading/Writing Functions (Basic)

`fread/fwrite`; `fgetc/fputc`; `fscanf/fprintf`; `fgets/fputs`

12.4.3 Usage

```
1 FILE *in, *out; // define file pointers
2 in = fopen("data.in", "r");
3 out = fopen("data.out", "w");
4 /*
5 do what you want to do
6 */
7 fclose(in);
8 fclose(out);
```

12.5 C++ ifstream/ofstream File Input/Output Streams

12.5.1 Usage

Reading file contents:

```
1 ifstream fin("data.in");
2 // data.in is the relative or absolute location of the file to read
```

Outputting to a file:

```
1 ofstream fout("data.out");
2 // data.out is the relative or absolute location of the output file
```

Closing the input/output streams:

```
1 fin.close();
2 fout.close();
```

12.5.2 Template

```
1 #include <fstream>
2 using namespace std; // both types live in the std namespace
3
4 ifstream fin("data.in");
5 ofstream fout("data.out");
6
7 int main(void) {
8     /*
9     In the code in between, simply change cin to fin and cout to fout.
10    */
11    fin.close();
12    fout.close();
13    return 0;
14 }
```

12.6 References

1. *Information Science Olympiad: A Complete Guide* (信息学奥赛一本通)

PART III

The C++ Standard Library

13 Introduction to the C++ Standard Library (csl)

Original title: C++标准库简介 · source page [/lang/csl/](#)

13.1 The C++ Standards

The first thing to introduce is the versions of C++ itself. Since C++ is only a language, and different compilers implement it in different ways, standardization is needed to constrain the implementations of compilers so that C++ code behaves consistently under different compilers. Since its birth in 1985, C++ has seen 7 official C++ standards published by the International Organization for Standardization (ISO), in order: C++98, C++03, C++11 (also known as C++0x), C++14 (also known as C++1y), C++17 (also known as C++1z), C++20 (also known as C++2a), and C++23 (also known as C++2b). The draft C++ standards live on the open-std website,¹⁸ and the progress of the latest standard can be followed at Current Status: Standard C++.¹⁹ There are also some supplementary standards, such as C++ TR1.

Each version of the C++ standard regulates not only the syntax and language features of C++, but also a specification of the implementation of a built-in library of C++, and this library is the *C++ standard library*. The C++ standard library contains implementations of a great deal of commonly used code: input/output, basic data structures, memory management, multi-threading support, and so on. Mastering the C++ standard library is a necessary step towards writing more modern C++ code. The detailed documentation of the C++ standard library lives on the cppreference website;²⁰ it covers the usage, efficiency, and caveats of the types and functions of the standard library — make good use of it.

¹⁸<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/>

¹⁹<https://isocpp.org/std/status>

²⁰<https://zh.cppreference.com/>

It should be pointed out that different OJ platforms differ in their C++ versions; for example, the latest ICPC contest rules²¹ support the C++20 standard. By decision of the NOI Scientific Committee, from September 1, 2021, NOI Linux 2.0²² is used as the standard environment for NOI-series contests and events such as CSP-J/S. The g++ 9.3.0 specified in NOI Linux 2.0 defaults to the C++14 standard²³ and also supports the C++17 standard, which satisfies the needs of the vast majority of contestants. So when learning C++, pay attention to the standard supported by your contest, and avoid compilation errors in the contest room.

13.2 The Standard Template Library (STL)

STL stands for Standard Template Library; it is part of the C++ standard library and contains templated, general-purpose data structures and algorithms. Because of its templated nature, it is compatible with user-defined data types and avoids a great deal of wheel reinvention. Both NOI and ICPC events support the use of the STL, so using the STL sensibly avoids writing useless algorithms and exploits the compiler's optimizations of the template library to increase efficiency. For a detailed introduction to the STL, see the corresponding pages: [STL Containers](#) and [STL Algorithms](#).

Note — What is “reinventing the wheel”?

Reinventing the wheel^a means re-inventing algorithms that already exist, or re-writing code that is already optimized. Reinventing the wheel usually costs much time and effort, and the result is still not as good as what others have. But if the purpose is learning or practice, then reinventing the wheel is necessary.

^ahttps://en.wikipedia.org/wiki/Reinventing_the_wheel

13.3 The Boost Libraries

Boost²⁴ is, besides the standard library, another long-renowned open-source C++ utility library whose code is portable, high-quality, high-performance, and highly reliable. Boost contains a very large number of modules with comprehensive functionality and complete cross-platform support, so it is regarded as a quasi-standard library of C++. Many features of the C++ standard also originated in Boost, such as smart pointers, metaprogramming, and dates and times. Although Boost cannot be used in OI, many of its wheels can be used to verify algorithms or to generate brute-force test data: Boost.Geometry has an R-tree implementation, Boost.Graph has graph algorithms, and Boost.Intrusive provides a set of intrusive containers whose usage resembles the STL containers. Interested readers may search the web for tutorials.

²¹<https://docs.icpc.global/worldfinals-programming-environment/>

²²<https://www.noi.cn/gynoi/jsgz/2021-07-16/732450.shtml>

²³<https://gcc.gnu.org/projects/cxx-status.html>

²⁴<https://www.boost.org/>

13.4 References

1. C++ reference, <https://en.cppreference.com/>
2. C++ reference manual (Chinese), <https://zh.cppreference.com/>
3. Wikipedia — C++, <https://zh.wikipedia.org/wiki/C%2B%2B>
4. Boost official website, <https://www.boost.org/>
5. Boost tutorials, <https://theboostcplibraries.com/>

14 Introduction to STL Containers (csl/container)

Original title: STL容器简介 · source page /lang/container/

14.1 Classification

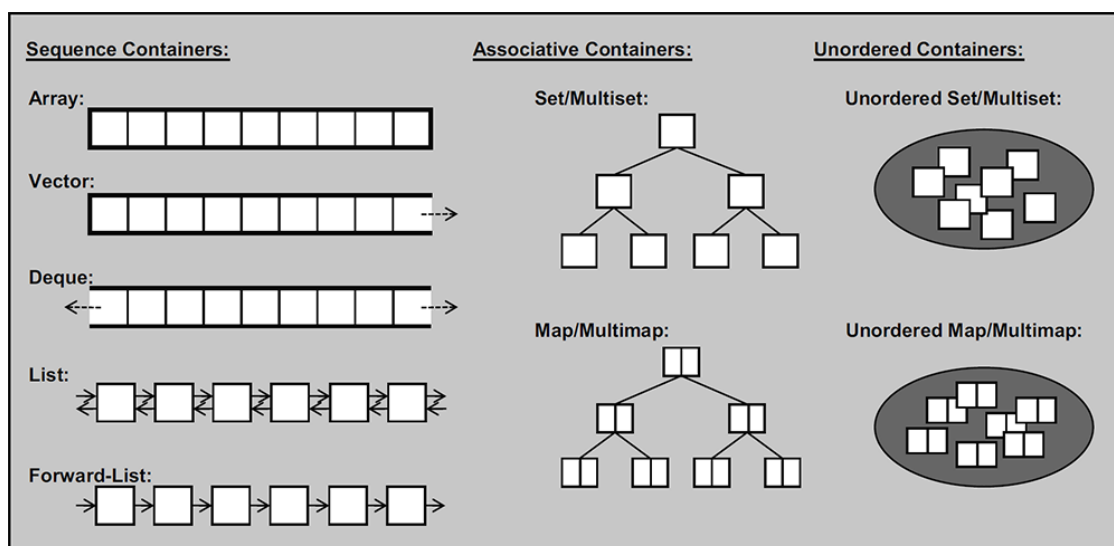


Figure 1: The containers of the STL (from the original page).

14.1.1 Sequence Containers

- **Vector** (`vector`): a sequential container whose elements can be appended efficiently at the back.
- **Array** (`array`) (**C++11**): a fixed-length sequential container, a thin wrapper over C-style arrays.
- **Deque** (`deque`): a sequential container to which elements can be appended efficiently at both ends.
- **List** (`list`): a linked list that can be traversed in both directions.
- **Forward list** (`forward_list`): a linked list that can be traversed in only one direction.

14.1.2 Associative Containers

- **Set** (`set`): a container that stores **distinct** elements in order. Its implementation is a red–black tree of nodes, each node containing an element, the nodes arranged by some predicate comparing element sizes.
- **Multiset** (`multiset`): a container that stores elements in order; equal elements are allowed.
- **Map** (`map`): a collection of {key, value} pairs arranged by some predicate comparing the keys.
- **Multimap** (`multimap`): a multiset of {key, value} pairs, i.e., a mapping that allows equal keys.

Note — What is a predicate (`Predicate`)?

A predicate is a function whose return value is true or false. Predicates are used very often in STL containers, as template parameters.

14.1.3 Unordered (Associative) Containers

- **Unordered (multi)set** (`unordered_set/unordered_multiset`) (**C++11**): differs from `set/multiset` in that the elements are unordered — only “whether an element exists” matters; implemented with hashing.
- **Unordered (multi)map** (`unordered_map/unordered_multimap`) (**C++11**): differs from `map/multimap` in that the keys are unordered — only the “correspondence between keys and values” matters; implemented with hashing.

14.1.4 Container Adapters

Container adapters are in fact not containers. They lack some properties of containers (such as having iterators, having a `clear()` function, ...).

“An adapter is a mechanism that makes the behavior of one thing resemble the behavior of another”; adapters wrap a container so that it exhibits a different behavior.

- **Stack** (`stack`): a last-in-first-out (LIFO) container; by default a wrapper over the deque (`deque`).
- **Queue** (`queue`): a first-in-first-out (FIFO) container; by default a wrapper over the deque (`deque`).
- **Priority queue** (`priority_queue`): a queue whose element order is decided by some predicate applied to the stored values; by default a wrapper over the vector (`vector`).

14.2 Common Properties

14.2.1 Container Declarations

All containers are declared in the form `containerName<typeName,...> name`, but the number and form of the template arguments (the arguments inside `<>`) vary with the concrete container.

The essential reason: STL is the “Standard *Template* Library”, so containers are all template classes.

14.2.2 Iterators

See [Iterators](#).

14.2.3 Common Functions

`=:` there is an assignment operator and a copy constructor.

`begin()`: returns an iterator to the first element.

`end()`: returns an iterator to the placeholder after the end. `end()` does not point to an element, but it is the successor of the last element.

`size()`: returns the number of elements in the container.

`max_size()`: returns the **theoretical** maximum number of elements the container can store; depends on the container type and the stored variable type.

`empty()`: returns whether the container is empty.

`swap()`: swaps two containers.

`clear()`: empties the container.

`==/!=/</>/<=>=:` compare the sizes of two containers **lexicographically**. (When comparing elements, each element of `map` acts like an element of `set<pair<key, value>>`; unordered containers do not support `</>/<=>=`.)

15 Iterators (csl/iterator)

Original title: 迭代器 · source page </lang/iterator/>

In the STL, an iterator is an object used to access and inspect the elements of an STL container; its mode of behavior resembles a pointer, but it encapsulates some validity checks and provides a uniform access format. Similar concepts exist in many other high-level languages, such as Python’s `__iter__` function and C#’s `IEnumerator`.

15.1 Basic Usage

Iterators sound obscure, but an iterator itself can be viewed as a data pointer. Iterators mainly support two operators: increment (`++`) and dereference (the unary `*` operator); increment moves the iterator, and dereferencing reads or modifies the element it points to.

The type of an iterator pointing to an element of an [STL container](#) `container` is generally `container::iterator`.

Iterators can be used to traverse containers; for example, the following two `for` loops have the same effect:

```

1 vector<int> data(10);
2
3 for (int i = 0; i < data.size(); i++)
4     cout << data[i] << endl; // access elements by subscript
5
6 for (vector<int>::iterator iter = data.begin(); iter != data.end(); iter++)
7     cout << *iter << endl; // access elements with an iterator
8 // from C++11 on, auto iter = data.begin() can simplify the code above

```

Tip — Using `auto` in contests

Most contestants like to use `auto` to replace verbose iterator declarations. According to the “Supplementary Remarks on the Restrictions on Programming Languages in NOI-Series Events” published in September 2021,^a NOI-series contests (including CSP J/S) grade with C++14, a version that already supports the `auto` keyword.

^a<https://www.noi.cn/xw/2021-09-01/735729.shtml>

15.2 Categories

In the STL definition, iterators are classified by the operations they support, as follows:

- **InputIterator** (input iterator): only copy, increment, and dereference-for-reading are required.
- **OutputIterator** (output iterator): only copy, increment, and dereference-for-writing are required.
- **ForwardIterator** (forward iterator): on top of **InputIterator**, supports multiple traversals with dereference-for-reading, and guarantees that multiple accesses give consistent results.
- **BidirectionalIterator** (bidirectional iterator): on top of **ForwardIterator**, supports decrement (i.e., reverse access).
- **RandomAccessIterator** (random-access iterator): on top of **BidirectionalIterator**, supports addition/subtraction and comparisons (i.e., random access).
- **ContiguousIterator** (contiguous iterator): on top of **RandomAccessIterator**, requires that for a dereferenceable iterator `a + n`, `*(a + n)` is equivalent to `*(std::address_of(*a) + n)` (i.e., contiguous storage, where `a` is a contiguous iterator and `n` an integer value).

ContiguousIterator was formally introduced in C++17.

Tip — Why is the input iterator called an “input” iterator?

“Input” means “one can take input from the iterator”, and “output” means “one can output to the iterator”.

The agent of the “input” and “output” is the rest of the program, not the iterator itself.

These iterator categories are not mutually exclusive. In fact, except for the output iterator, each category listed earlier contains all the categories listed after it. For example, wherever a forward iterator is required, a bidirectional iterator may be used. From the forward iterator on, if these iterators also implement the output iterator’s functionality (i.e., write operations are allowed), they are called *mutable* iterators. From this one can derive categories such as “mutable random-access iterator”.

Different [STL containers](#) support different iterator types; pay attention when using them.

Array pointers satisfy all requirements of the contiguous iterator (or random-access iterator, for C++14 and earlier) and can be used as contiguous iterators.

15.3 Related Functions

Many [STL functions](#) take iterators as parameters.

`std::advance(it, n)` moves the iterator `it` forward by `n` steps; if `n` is negative it moves backward, in which case the iterator must satisfy the bidirectional-iterator requirements, otherwise the behavior is undefined.

From C++11 on, `std::next(it)` obtains the successor of the forward iterator `it` (the iterator `it` itself is unchanged), and `std::next(it, n)` obtains the `n`-th successor of `it`.

From C++11 on, `std::prev(it)` obtains the predecessor of the bidirectional iterator `it` (the iterator `it` itself is unchanged), and `std::prev(it, n)` obtains the `n`-th predecessor of `it`.

[STL containers](#) generally support access from one end or both ends, as well as `const` qualifiers. For example, a container’s `begin()` obtains an iterator to the container’s first element, `rbegin()` a reverse iterator to the container’s last element, `cbegin()` a `const` iterator to the container’s first element, and `end()` an iterator to the container’s end sentinel (the “end sentinel” is not the last element; regard it as the successor of the last element: its predecessor is the container’s last element, and it itself points to no element).

See Iterator library — cppreference.com²⁵ for more usage.

16 Sequence Containers (csl/sequence-container)

Original title: 序列式容器 · source page [/lang/sequence-container/](#)

16.1 vector

`std::vector` is the **memory-contiguous**, **variable-length** array (also called list) data structure provided by the STL. It offers linear-complexity insertion and deletion, and constant-complexity random access.

²⁵<https://en.cppreference.com/w/cpp/iterator>

16.1.1 Why Use `vector`

As an OIer, the pursuit of program efficiency far exceeds the pursuit of engineering-grade stability; because `vector` manages memory dynamically, its time efficiency is in some cases lower than a static array, and the situation is worse when OJ servers do not enable full optimization. So for ordinary data storage, `vector` is usually not the choice. Below are several excellent properties of `vector`; in situations that need these properties, `vector` helps a great deal.

`vector` can allocate memory dynamically. Often we cannot open up that much space in advance (e.g., precomputing the divisors of all numbers in $1 \sim n$). Although we may know the total data volume is within a space-allowable scale, a single datum may still be very large; then we need `vector` to keep memory usage in a suitable range. `vector` also supports dynamic growth — a property useful when memory is extremely tight.

`vector` overloads the comparison and assignment operators. `vector` overloads the six comparison operators, implemented lexicographically, which makes it convenient to test two containers for equality (complexity linear in the container sizes). For example, `vector<char>` can implement string comparison (though `std::string` is faster and more convenient). `vector` also overloads the assignment operator, making array copying more convenient.

`vector` has convenient initialization. Because `vector` overloads `=`, whole-vector assignment is easy. In addition, from C++11 on `vector` supports list initialization,²⁶ e.g. `vector<int> data {1, 2, 3};`.

16.1.2 Using `vector`

The common usages are introduced below; for details see the C++ documentation on `vector`.²⁷

Constructors. See the following code (assuming you have already brought in the relevant types of the `std` namespace with `using`):

```

1 // 1. create an empty vector; constant complexity
2 vector<int> v0;
3 // 1+. this line guarantees constant time complexity when inserting the
4 // first 3 elements into the vector
5 v0.reserve(3);
6 // 2. create a vector with initial space 3 and default element value 0; linear
   ↳ complexity
7 vector<int> v1(3);
8 // 3. create a vector with initial space 3 and default element value 2; linear
   ↳ complexity
9 vector<int> v2(3, 2);
10 // 4. create a vector with initial space 3 and default element value 1,
11 // using v2's allocator; linear complexity
12 vector<int> v3(3, 1, v2.get_allocator());

```

²⁶https://zh.cppreference.com/w/cpp/language/list_initialization

²⁷<https://zh.cppreference.com/w/cpp/container/vector>

```

13 // 5. create a copy v4 of v2, whose elements are the same as v2's; linear complexity
14 vector<int> v4(v2);
15 // 6. create a copy v5 of v4 whose contents are {v4[1], v4[2]}; linear complexity
16 vector<int> v5(v4.begin() + 1, v4.begin() + 3);
17 // 7. move v2 into the newly created vector v6 without copying; constant
18 // complexity; requires C++11
19 vector<int> v6(std::move(v2)); // or v6 = std::move(v2);

```

Note — Test code

```

1 // The following is test code; interested readers may compile and run it themselves
  ↪ .
2 cout << "v1 = ";
3 copy(v1.begin(), v1.end(), ostream_iterator<int>(cout, " "));
4 cout << endl;
5 cout << "v2 = ";
6 copy(v2.begin(), v2.end(), ostream_iterator<int>(cout, " "));
7 cout << endl;
8 cout << "v3 = ";
9 copy(v3.begin(), v3.end(), ostream_iterator<int>(cout, " "));
10 cout << endl;
11 cout << "v4 = ";
12 copy(v4.begin(), v4.end(), ostream_iterator<int>(cout, " "));
13 cout << endl;
14 cout << "v5 = ";
15 copy(v5.begin(), v5.end(), ostream_iterator<int>(cout, " "));
16 cout << endl;
17 cout << "v6 = ";
18 copy(v6.begin(), v6.end(), ostream_iterator<int>(cout, " "));
19 cout << endl;

```

With the methods above we can construct a **vector**, which is enough for our use.

Element access. **vector** provides the following ways of element access:

1. **at()**: **v.at(pos)** returns a reference to the element with subscript **pos**. If the array index is out of bounds, an exception of type **std::out_of_range** is thrown.
2. **operator[]**: **v[pos]** returns a reference to the element with subscript **pos**. No bounds check is performed.
3. **front()**: **v.front()** returns a reference to the first element.
4. **back()**: **v.back()** returns a reference to the last element.
5. **data()**: **v.data()** returns a pointer to the first element of the contiguous memory **v** uses internally to store its data.

Iterators. **vector** provides the following [iterators](#):

1. **begin()/cbegin()**: returns an iterator to the first element, where ***begin = front**.

2. `end()/cend()`: returns an iterator to the container's end placeholder; note that there is no element there.
3. `rbegin()/crbegin()`: returns a reverse iterator to the first element of the reversed array, which can be understood as the last element of the forward container.
4. `rend()/crend()`: returns an iterator to the position after the last element of the reversed array, corresponding to the position before the container's head; there is no element there.

Among the iterators listed above, those containing the letter **c** are read-only iterators: you cannot modify the elements of the **vector** through them. If a **vector** is itself read-only, its ordinary and read-only iterators are fully equivalent. Read-only iterators are supported from C++11 on.

Length and capacity. **vector** has the following functions related to the container's length and capacity. Note that the *length* (size) of a **vector** is the number of valid elements, while the *capacity* is the length of memory it has actually allocated; see the implementation details below.

Length-related:

- `empty()` returns a **bool**: `v.begin() == v.end()`; **true** means empty, **false** non-empty.
- `size()` returns the container's length (number of elements), i.e. `std::distance(v.begin(), v.end())`.
- `resize(n)` changes the **vector**'s length to **n**. If **n** is greater than the current length, elements are appended — using the value given in the parameter if provided, otherwise the default value; if **n** is smaller, the first **n** elements are kept and the rest are removed.
- `max_size()` returns the maximum possible length of the container.

Capacity-related:

- `reserve()` makes the **vector** reserve a certain amount of memory, avoiding unnecessary allocations and copies.
- `capacity()` returns the container's capacity, i.e., for how many elements the current **vector** has allocated space.
- `shrink_to_fit()` makes the **vector**'s capacity equal to its length, removing capacity the **vector** does not use.

16.1.3 Adding, Removing, and Modifying Elements

- `clear()` clears all elements.
- `insert()` inserts one or more elements at an iterator position. **The complexity is linear in the distance from pos to the end, not constant.**
- `erase()` removes the elements at an iterator or in a range, returning the last removed iterator. Same complexity as `insert`.

- `push_back()` inserts an element at the end; amortized complexity **constant**, worst case linear.
- `pop_back()` removes the last element; constant complexity.
- `swap()` swaps with another container; this operation is **constant complexity**, not linear.

16.1.4 Implementation Details of `vector`

Under the hood, `vector` is still a fixed-length array; it achieves dynamic growth by adding operations that avoid overflow. First note that the number of elements in a `vector` (its length) n and the number of elements its allocated memory can hold (its capacity) N are different, and `vector` stores the two quantities separately. When an element is added, if the container finds $n > N$, it allocates an array of size $2N$, copies the old data from the old place into the new array, and then frees the original memory. Although this operation has asymptotic complexity $O(n)$, it can be proved that its amortized complexity is $O(1)$. Deleting an element at the end and accessing elements remain $O(1)$ costs.

Therefore, as long as you estimate the size of the `vector` well and make good use of `resize()` and `reserve()`, the efficiency of `vector` will not differ much from a fixed-length array.

16.1.5 `vector<bool>`

The standard library specially provides a `vector` specialization for `bool`: each “`bool`” occupies only 1 bit, and dynamic growth is supported. But the return type of its `operator[]` is not `bool&` but `vector<bool>::reference`. Therefore use `vector<bool>` with care; consider `deque<bool>` or `vector<char>` as replacements. And if you need to save space, use `bitset` directly.

16.2 `array` (C++11)

`std::array` is the **memory-contiguous, fixed-length** array data structure provided by the STL. Its essence is a direct wrapper over native arrays.

16.2.1 Why Use `array`

`array` is in fact the STL’s wrapper over arrays. Compared with `vector` it gives up dynamic growth, but in exchange gains performance almost identical to a native array (given full optimization). So where C++11 features are available, almost every place a native array could be used can directly use `array`, and dynamically allocated arrays can be replaced with `vector`.

16.2.2 Member Functions

Implicitly defined member functions.

Function	Effect
<code>operator=</code>	overwrites the corresponding elements of the <code>array</code> with each element of another <code>array</code>

Element access.

Function	Effect
<code>at</code>	accesses the specified element, with bounds checking
<code>operator[]</code>	accesses the specified element, no bounds checking
<code>front</code>	accesses the first element
<code>back</code>	accesses the last element
<code>data</code>	returns a pointer to the array's first element in memory

`at` throws `std::out_of_range` if it encounters `pos >= size()`.

Capacity.

Function	Effect
<code>empty</code>	checks whether the container is empty
<code>size</code>	returns the number of elements it holds
<code>max_size</code>	returns the maximum number of elements it can hold

Because every `array` is a fixed-size container, the value returned by `size()` equals the value returned by `max_size()`.

16.2.3 Operations

Function	Effect
<code>fill</code>	fills the container with a specified value
<code>swap</code>	swaps the contents

Note that swapping two arrays is $\Theta(\text{size})$, not $O(1)$ as with ordinary STL containers.

16.2.4 Non-Member Functions

Function	Effect
<code>operator==</code> etc.	compares the values of <code>arrays</code> lexicographically
<code>std::get</code>	accesses an element of the <code>array</code>
<code>std::swap</code>	the specialized <code>std::swap</code> algorithm

Below is a usage example of `array`:

```

1 // 1. create an empty array of length 3; constant complexity
2 std::array<int, 3> v0;
3 // 2. create an array with a specified constant; constant complexity
4 std::array<int, 3> v1{1, 2, 3};

```

```

5
6 v0.fill(1); // fill the array
7
8 // access the array
9 for (int i = 0; i != arr.size(); ++i) cout << arr[i] << " ";

```

16.3 deque

`std::deque` is the double-ended queue data structure provided by the STL (see OI Wiki → Data Structures → Queue, “Double-ended queue”). It offers linear-complexity insertion and deletion, and constant-complexity random access.

16.3.1 Using deque

The common usages are introduced below; for details see the C++ documentation on deque.²⁸ The iterator functions of **deque** are the same as **vector**’s, so they are not introduced in detail.

Constructors. See the following code (assuming you have already brought in the relevant types of the `std` namespace with `using`):

```

1 // 1. define an empty deque v0 of int
2 deque<int> v0;
3 // 2. define a deque v1 of int with initial size 10; linear complexity
4 deque<int> v1(10);
5 // 3. define a deque v2 of int initialized to ten 1s; linear complexity
6 deque<int> v2(10, 1);
7 // 4. copy the existing deque v1; linear complexity
8 deque<int> v3(v1);
9 // 5. create a copy v4 of v2 whose contents are v4[0] through v4[2]; linear complexity
10 deque<int> v4(v2.begin(), v2.begin() + 3);
11 // 6. move v2 into the newly created deque v5 without copying; constant
12 // complexity; requires C++11
13 deque<int> v5(std::move(v2));

```

Element access. Same as **vector**, but the underlying memory cannot be accessed. Its fast element access can be understood from the implementation details.

- `at()` returns a reference to the element at the specified position, with bounds checking; **constant complexity**.
- `operator[]` returns a reference to the element at the specified position; no bounds checking; **constant complexity**.
- `front()` returns a reference to the first element.
- `back()` returns a reference to the last element.

²⁸<https://zh.cppreference.com/w/cpp/container/deque>

Iterators. Same as `vector`.

Length. Same as `vector`, but there are no `reserve()` and `capacity()` functions. (The `shrink_to_fit()` function still exists.)

Adding, removing, and modifying elements. Same as `vector`, plus functions that add elements at the head of the queue.

- `clear()` clears all elements.
- `insert()` inserts one or more elements at an iterator position. **The complexity is linear in the smaller of the distances from `pos` to the two ends.**
- `erase()` removes the elements at an iterator or in a range, returning the last removed iterator. Same complexity as `insert`.
- `push_front()` inserts an element at the head; **constant complexity.**
- `pop_front()` removes the head element; **constant complexity.**
- `push_back()` inserts an element at the end; **constant complexity.**
- `pop_back()` removes the last element; **constant complexity.**
- `swap()` swaps with another container; this operation is **constant complexity**, not linear.

16.3.2 Implementation Details of `deque`

`deque` is usually implemented on top of several non-contiguous buffers, where the memory inside each buffer is contiguous. Each buffer also records a head pointer and a tail pointer marking the region of valid data. When a buffer fills up, a new buffer is allocated before or after it to store more data. For a more detailed explanation, see “The deque implementation principle in *STL Source Analysis*”²⁹.

16.4 `list`

`std::list` is the doubly linked list data structure provided by the STL (OI Wiki → Data Structures → Linked list). It offers linear-complexity random access, and constant-complexity insertion and deletion.

16.4.1 Using `list`

The usage of `list` is basically the same as `deque`, but the complexities of add/remove and access differ. For details see the C++ documentation on `list`.³⁰ The iterator, length, and element add/remove/modify functions of `list` are the same as `deque`’s, so they are not introduced in detail.

²⁹<https://www.cnblogs.com/q1076452761/p/16903229.html>

³⁰<https://zh.cppreference.com/w/cpp/container/list>

Element access. Because `list` is implemented as a linked list, it provides no random-access interface. To access middle elements you must use iterators.

- `front()` returns a reference to the first element.
- `back()` returns a reference to the last element.

Operations. The `list` type also provides some STL algorithm functions implemented specially for its properties. Because these algorithms require [random-access iterators](#), `list` provides special implementations for ease of use. These algorithms include `splice()`, `remove()`, `sort()`, `unique()`, `merge()`, and so on.

16.5 forward_list (C++11)

`std::forward_list` is the singly linked list data structure provided by the STL (OI Wiki → Data Structures → Linked list); compared with `std::list` it reduces the space overhead.

16.5.1 Using forward_list

The usage of `forward_list` is almost identical to `list`, but the iterator is one-directional only, so its concrete usage is not introduced in detail. For details see the C++ documentation on `forward_list`.³¹

17 Associative Containers (csl/associative-container)

Original title: 关联式容器 · source page /lang/associative-container/

17.1 set

`set` is an associative container holding a sorted set of objects of the key type; search, removal, and insertion have logarithmic complexity. `set` is usually implemented internally as a red-black tree (OI Wiki → Data Structures → Red-black tree). The properties of balanced binary trees (OI Wiki → Data Structures → BST) make `set` very suitable for situations that must balance search, insertion, and deletion at the same time.

Like sets in mathematics, elements of equal value never both appear in a `set`. If a set with equal elements is needed, use `multiset`, whose usage is basically the same as `set`'s.

17.1.1 Insertion and Deletion

- `insert(x)` inserts element `x` into the `set` when no equivalent element exists in the container.
- `erase(x)` deletes **all** elements of value `x`, returning the number of removed elements.
- `erase(pos)` deletes the element whose iterator is `pos`; the iterator must be valid.
- `erase(first, last)` deletes all elements whose iterators lie in the range $[first, last)$.

³¹https://zh.cppreference.com/w/cpp/container/forward_list

- `clear()` empties the `set`.

Note — The return value of the `insert` function

The return type of `insert` is `pair<iterator, bool>`, where `iterator` is an iterator pointing to the inserted element (or to the element already in the container that equals the inserted value), and `bool` indicates whether the insertion succeeded. Since the elements of a `set` are unique, if an equal element is already present the insertion fails and `false` is returned; otherwise the insertion succeeds and `true` is returned. The `insert` of `map` behaves the same way.

17.1.2 Iterators

`set` provides the following iterators:

1. `begin()/cbegin()`: returns an iterator to the first element, where `*begin = front`.
2. `end()/cend()`: returns an iterator to the array's end placeholder; note there is no element there.
3. `rbegin()/crbegin()`: returns a reverse iterator to the first element of the reversed array, i.e., the last element of the forward container.
4. `rend()/crend()`: returns an iterator to the position after the last element of the reversed array, corresponding to the position before the container's head; there is no element there.

Among the iterators listed above, those containing the letter `c` are read-only iterators: you cannot modify the elements of the `set` through them. If a `set` is itself read-only, its ordinary and read-only iterators are fully equivalent. Read-only iterators are supported from C++11 on.

17.1.3 Search Operations

- `count(x)` returns the number of elements with key `x` in the `set`.
- `find(x)` returns an iterator to the element with key `x` if one exists in the `set`, otherwise `end()`.
- `lower_bound(x)` returns an iterator to the first element whose key is not smaller than the given key; if no such element exists, returns `end()`.
- `upper_bound(x)` returns an iterator to the first element whose key is greater than the given key; if no such element exists, returns `end()`.
- `empty()` returns whether the container is empty.
- `size()` returns the number of elements in the container.

Warning — The time complexity of `lower_bound` and `upper_bound`

The `lower_bound` and `upper_bound` member functions of `set` have time complexity $O(\log n)$.

But using the `lower_bound` and `upper_bound` functions of the `algorithm` library to query elements of a `set` has time complexity $O(n)$.

Warning — The time complexity of `nth_element`

`set` provides no built-in `nth_element`. Using the `algorithm` library's `nth_element` to find the k -th largest element has time complexity $O(n)$.

If you need the balanced-BST ability of finding the k -th largest element in $O(\log n)$, you must write a balanced BST or a weight segment tree by hand, or use the balanced tree of the `pb_ds` library (§34).

17.1.4 Usage Example

Using `set` in greedy algorithms. Greedy algorithms often need an operation like **find and delete the smallest element greater than or equal to some value**. This operation is easy with a `set`.

```

1 // the elements currently available
2 set<int> available;
3 // the value we need to be greater than or equal to
4 int x;
5
6 // find the smallest element that is >= x
7 set<int>::iterator it = available.lower_bound(x);
8 if (it == available.end()) {
9     // no such element exists; perform the corresponding operation ...
10 } else {
11     // such an element was found; remove it from the currently available elements
12     available.erase(it);
13     // perform the corresponding operation ...
14 }
```

17.2 map

`map` is an ordered key–value container whose elements have unique keys. Search, removal, and insertion have logarithmic complexity. `map` is usually implemented as a red–black tree (OI Wiki → Data Structures → Red–black tree).

Imagine the following scenario: we need to store some key–value pairs, e.g., the scores of student names: Tom 0, Bob 100, Alan 100. But since array subscripts can only be non-negative integers, names cannot serve as subscripts; the simplest solution at this point is the STL's `map`.

`map` overloads `operator[]`, so any type that defines `operator <` can be used as the subscript (called the **key** in a `map`, i.e., the index):

```
1 map<Key, T> yourMap;
```

where `Key` is the key type and `T` the value type. Here is an example of using a `map`:

```
1 map<string, int> mp;
```

A `map` never contains elements with equal keys, while `multimap` allows several elements to share one key. The usage of `multimap` is basically the same as that of `map`.

Warning

Precisely because `multimap` allows several elements to share one key, `multimap` provides no way to access the corresponding value given a key.

17.2.1 Insertion and Deletion

- You can query or insert directly by subscript access, e.g. `mp["Alan"]=100`.
- Inserting a value of type `pair<Key, T>` into the `map` also inserts an element, e.g. `mp.insert(pair<string,int>("Alan",100));`
- `erase(key)` deletes **all** elements with key `key`; the return value is the number of removed elements.
- `erase(pos)`: deletes the element whose iterator is `pos`; the iterator must be valid.
- `erase(first,last)`: deletes all elements whose iterators lie in the range `[first,last)`.
- `clear()` empties the entire container.

Note — Caveats of subscript access

When using subscript access on an element of a `map`, if no element with the corresponding key exists, a new element is automatically inserted into the `map` with its value set to the default value (zero for integers; for types with a default constructor, the default constructor is called to initialize it).

When subscript accesses are too frequent, a large number of meaningless elements appear in the container and hurt the `map`'s efficiency. Therefore it is generally recommended to use the `find()` function to look up elements of a specific key.

17.2.2 Query Operations

- `count(x)`: returns the number of elements with key `x` in the container. Complexity $O(\log(\text{size}) + \text{ans})$ (logarithmic in the container size, plus the number of matches).
- `find(x)`: if the container contains an element with key `x`, returns an iterator to it; otherwise returns `end()`.
- `lower_bound(x)`: returns an iterator to the first element whose key is not smaller than the given key.
- `upper_bound(x)`: returns an iterator to the first element whose key is greater than the given key; if all elements of the container are $\leq$ the given key, returns `end()`.
- `empty()`: returns whether the container is empty.
- `size()`: returns the number of elements in the container.

17.2.3 Usage Example

Using map to store complex states. In search, we sometimes need to store rather complex states (coordinates, values that cannot be discretized, strings, etc.) together with answers related to them (e.g., the minimum number of steps to reach the state). A `map` can implement this: the key is the state, the value the related answer. The example below shows how to store states represented as `string` in a `map`.

```
1 // stores states together with the corresponding answers
2 map<string, int> record;
3
4 // the newly found state and its answer
5 string status;
6 int ans;
7 // check whether the corresponding state has been seen
8 map<string, int>::iterator it = record.find(status);
9 if (it == record.end()) {
10     // this state has not been searched yet; add it to the state record
11     record[status] = ans;
12     // perform the corresponding operation ...
13 } else {
14     // this state has already been searched; perform the corresponding operation ...
15 }
```

17.3 Traversing the Containers

You can use iterators to traverse all elements of an associative container:

```
1 set<int> s;
2 using si = set<int>::iterator;
3 for (si it = s.begin(); it != s.end(); it++) cout << *it << endl;
```

Note that dereferencing a `map`'s iterator yields a key-value pair of type `pair<Key, T>`.

In C++11, range-based `for` loops make the code much more concise:

```
1 set<int> s;
2 for (auto x : s) cout << x << endl;
```

For any associative container, traversing the container with an iterator has time complexity $O(n)$.

17.4 Custom Comparison

By default the comparison function of `set` is `<` (for non-built-in types you need to [overload the < operator](#)). In certain special cases, however, we may want to customize the comparison inside a `set`.

This can be solved by passing a custom comparator.

Concretely, we need to define a class and [overload the \(\) operator](#) in it.

For example, to maintain a `set` of integers with larger values first:


```

1 struct cmp {
2     bool operator()(int a, int b) const { return a > b; }
3 };
4
5 set<int, cmp> s;

```

Other associative containers can be given custom comparison in a similar way; the details are not repeated here.

18 Unordered Associative Containers (csl/unordered-container)

Original title: 无序关联式容器 · source page /lang/unordered-container/

18.1 Overview

From the C++11 standard on, four hash-based unordered associative containers were formally brought into the C++ standard template library: `unordered_set`, `unordered_multiset`, `unordered_map`, and `unordered_multimap`.

Note — Usage when the compiler does not support C++11

Before C++11, the unordered associative containers belonged to the TR1 extension of C++. So if the compiler does not support C++11, you need to add the `tr1/` prefix to the header names and use the `std::tr1` namespace: `#include <unordered_map>` must become `#include <tr1/unordered_map>`; `std::unordered_map` must become `std::tr1::unordered_map` (or `tr1::unordered_map` if you use `using namespace std`).

They have much in common with the corresponding associative containers in functionality and functions. The biggest difference: ordinary associative containers are generally implemented as red-black trees with elements sorted in a particular order, whereas these unordered associative containers store elements by hashing, with elements in no particular order; so when visiting the elements of an unordered associative container, the visiting order is entirely unspecified.

Hash storage means that **on average**, most operations of unordered associative containers (search, insertion, deletion) finish in constant time — better than the logarithmic-in-container-size complexity of associative containers.

Warning

In the worst case, insertion, deletion, and search on an unordered associative container take time **linear in the container size**! This usually happens when large numbers of hash collisions occur.

Also, because the operations of unordered associative containers usually carry a large constant factor, their efficiency is sometimes not much better than ordinary associative containers.

Therefore use unordered associative containers with care and try to avoid abusing them (for example, out of laziness for discretization, using an `unordered_map<int, int>` as an ordi-

nary array of unlimited space).

Since unordered associative containers share many purposes and operations with the corresponding associative containers, their various operations are not introduced here; see §17.

18.2 Manufacturing Hash Collisions

As mentioned above, in the worst case some operations on unordered associative containers take time linear in the container size.

Given a fixed hash function, one can construct data that creates massive hash collisions in the container, pushing the complexity to its upper bound.

In the standard library implementation, the hash value of each element is the value modulo a prime; more precisely, one of the primes in this list³² (for g++ 6 and earlier the prime is generally 126271; for g++ 7 and later, generally 107897).

Therefore, inserting multiples of these moduli into the container achieves the goal of creating massive hash collisions.

18.3 Custom Hash Functions

Using a custom hash function effectively avoids the massive hash collisions created by adversarial data.

To use a custom hash function, define a struct and overload its `()` operator, like this:

```
1 struct my_hash {
2     size_t operator()(int x) const { return x; }
3 };
```

Of course, to make sure the hash function cannot be cracked quickly (e.g., hacks against unordered-container submissions on Codeforces), try adding some randomization (such as the time) to the hash function to raise the difficulty of cracking.

For example, this hash function is given in this blog post:³³

```
1 struct my_hash {
2     static uint64_t splitmix64(uint64_t x) {
3         x += 0x9e3779b97f4a7c15;
4         x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
5         x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
6         return x ^ (x >> 31);
7     }
8
9     size_t operator()(uint64_t x) const {
10         static const uint64_t FIXED_RANDOM =
11             chrono::steady_clock::now().time_since_epoch().count();
12         return splitmix64(x + FIXED_RANDOM);
13     }
14 };
```

³²<https://github.com/gcc-mirror/gcc/blob/releases/gcc-8.1.0/libstdc++-v3/src/shared/hashtable-aux.cc>

³³<https://codeforces.com/blog/entry/62393>

```

13 }
14
15 // a hash function for the key type std::pair<int, int>
16 size_t operator()(pair<uint64_t, uint64_t> x) const {
17     static const uint64_t FIXED_RANDOM =
18         chrono::steady_clock::now().time_since_epoch().count();
19     return splitmix64(x.first + FIXED_RANDOM) ^
20         (splitmix64(x.second + FIXED_RANDOM) >> 1);
21 }
22 };

```

After writing the custom hash function, pass it into the container via `unordered_map<int, int, my_hash> my_map`; or `unordered_map<pair<int, int>, int, my_hash> my_pair_map`;

19 Container Adapters (csl/container-adapter)

Original title: 容器适配器 · source page </lang/container-adapter/>

19.1 Stack

The STL stack (`std::stack`, OI Wiki → Data Structures → Stack) is a Last In, First Out container adapter that supports only querying or removing the last inserted element (the top of the stack), supports no random access, and — to guarantee strict ordering of the data — supports no iterators.

19.1.1 Header

```
1 #include <stack>
```

19.1.2 Definition

```

1 std::stack<TypeName> s; // default underlying container deque, data type TypeName
2 std::stack<TypeName, Container> s; // use Container as the underlying container
3 std::stack<TypeName> s2(s1); // copy s1 to construct s2

```

19.1.3 Member Functions

All of the following functions have constant complexity

- `top()` accesses the top element of the stack (an error occurs if the stack is empty);
- `push(x)` inserts element `x` into the stack;
- `pop()` removes the top element of the stack;
- `size()` queries the number of elements in the container;
- `empty()` asks whether the container is empty.

19.1.4 Simple Example

```

1  std::stack<int> s1;
2  s1.push(2);
3  s1.push(1);
4  std::stack<int> s2(s1);
5  s1.pop();
6  std::cout << s1.size() << " " << s2.size() << std::endl;  // 1 2
7  std::cout << s1.top() << " " << s2.top() << std::endl;    // 2 1
8  s1.pop();
9  std::cout << s1.empty() << " " << s2.empty() << std::endl; // 1 0

```

19.2 Queue

The STL queue (`std::queue`, OI Wiki → Data Structures → Queue) is a First In, First Out container adapter that supports only querying or removing the first inserted element (the head of the queue), supports no random access, and — to guarantee strict ordering of the data — supports no iterators.

19.2.1 Header

```

1  #include <queue>

```

19.2.2 Definition

```

1  std::queue<TypeName> q;  // default underlying container deque, data type TypeName
2  std::queue<TypeName, Container> q;  // use Container as the underlying container
3
4  std::queue<TypeName> q2(q1);  // copy q1 to construct q2

```

19.2.3 Member Functions

All of the following functions have constant complexity

- `front()` accesses the head element of the queue (an error occurs if the queue is empty);
- `push(x)` inserts element `x` into the queue;
- `pop()` removes the head element of the queue;
- `size()` queries the number of elements in the container;
- `empty()` asks whether the container is empty.

19.2.4 Simple Example

```

1  std::queue<int> q1;
2  q1.push(2);
3  q1.push(1);
4  std::queue<int> q2(q1);

```

```

5 q1.pop();
6 std::cout << q1.size() << " " << q2.size() << std::endl;    // 1 2
7 std::cout << q1.front() << " " << q2.front() << std::endl;  // 1 2
8 q1.pop();
9 std::cout << q1.empty() << " " << q2.empty() << std::endl; // 1 0

```

19.3 Priority Queue

The priority queue `std::priority_queue` is a heap (OI Wiki → Data Structures → Heap), generally a binary heap (OI Wiki → Data Structures → Binary heap).

19.3.1 Header

```

1 #include <queue>

```

19.3.2 Definition

```

1 std::priority_queue<TypeName> q;           // data type TypeName
2 std::priority_queue<TypeName, Container> q; // use Container as the underlying
   ↪ container
3 std::priority_queue<TypeName, Container, Compare> q;
4 // use Container as the underlying container and Compare as the comparison type
5
6 // by default the underlying container is vector
7 // the comparison type defaults to less<TypeName> (so top() returns the maximum)
8 // if you want top() to return the minimum, set the comparison type to greater<TypeName>
   ↪ >
9 // note: you cannot skip Container and pass Compare directly
10
11 // from C++11 on, if you use a lambda function to customize Compare,
12 // you must also pass it as an argument of the constructor, e.g.:
13 auto cmp = [](const std::pair<int, int> &l, const std::pair<int, int> &r) {
14     return l.second < r.second;
15 };
16 std::priority_queue<std::pair<int, int>, std::vector<std::pair<int, int>>,
17                   decltype(cmp)>
18     pq(cmp);

```

19.3.3 Member Functions

All of the following functions have constant complexity

- `top()` accesses the top element of the heap (the priority queue must be non-empty at this point);
- `empty()` asks whether the container is empty;
- `size()` queries the number of elements in the container.

All of the following functions have logarithmic complexity

- `push(x)` inserts an element and re-sorts the underlying container;
- `pop()` removes the top element of the heap (the priority queue must be non-empty at this point).

19.3.4 Simple Example

```

1  std::priority_queue<int> q1;
2  std::priority_queue<int, std::vector<int>> q2;
3  // since C++11 the space may be omitted
4  std::priority_queue<int, std::deque<int>, std::greater<int>> q3;
5  // q3 is a min-heap
6  for (int i = 1; i <= 5; i++) q1.push(i);
7  // elements of q1 : [1, 2, 3, 4, 5]
8  std::cout << q1.top() << std::endl;
9  // output : 5
10 q1.pop();
11 // elements in the heap : [1, 2, 3, 4]
12 std::cout << q1.size() << std::endl;
13 // output : 4
14 for (int i = 1; i <= 5; i++) q3.push(i);
15 // elements of q3 : [1, 2, 3, 4, 5]
16 std::cout << q3.top() << std::endl;
17 // output : 1

```

20 STL Algorithms (csl/algorithm)

Original title: STL算法 · source page /lang/algorithm/

The STL provides about 100 template functions implementing algorithms, essentially all contained in `<algorithm>`, with some others in `<numeric>` and `<functional>`. For the complete function list see the reference manual;³⁴ for sorting-related content see the corresponding page on sorting (OI Wiki → Basics → STL sorting functions).

- **find**: sequential search. `find(v.begin(), v.end(), value)`, where `value` is the value to search for.
- **reverse**: reverses an array or a string. `reverse(v.begin(), v.end())` or `reverse(a + begin, a + end)`.
- **unique**: removes adjacent duplicate elements in a container. `unique(ForwardIterator first, ForwardIterator last)` returns an iterator pointing to the end of the **deduplicated** container; the original container size is unchanged. Combined with `sort` it can fully deduplicate a container.
- **random_shuffle**: randomly shuffles an array. `random_shuffle(v.begin(), v.end())` or `random_shuffle(v + begin, v + end)`.

³⁴<https://zh.cppreference.com/w/cpp/algorithm>

Warning — `random_shuffle` has been removed from the newest C++ standard

`random_shuffle` was deprecated in C++14 and removed in C++17.

Under C++11 and newer standards, you can use the `shuffle` function in place of the old `random_shuffle`: `shuffle(v.begin(), v.end(), rng)` (the last argument is the random-number generator to use; normally a Mersenne-Twister pseudorandom generator `mt19937`^a seeded by the true random generator `random_device`^b).

```
1 // #include <random>
2 std::mt19937 rng(std::random_device{}());
3 std::shuffle(v.begin(), v.end(), rng);
```

^ahttps://zh.cppreference.com/w/cpp/numeric/random/mersenne_twister_engine

^bhttps://zh.cppreference.com/w/cpp/numeric/random/random_device

- **sort**: sorts. `sort(v.begin(), v.end(), cmp)` or `sort(a + begin, a + end, cmp)`, where `end` is one past the last element of the array to sort, and `cmp` is a custom comparison function.
- **stable_sort**: stable sort; same usage as `sort()`.
- **nth_element**: partitions within the specified range, i.e., it finds the n -th largest element of the sequence so that everything to its left is smaller and everything to its right is greater. `nth_element(v.begin(), v.begin() + n, v.end(), cmp)` or `nth_element(a + begin, a + begin + n, a + end, cmp)`.
- **binary_search**: binary search. `binary_search(v.begin(), v.end(), value)`, where `value` is the value to search for.
- **merge**: **merges in order** two (already sorted) sequences onto an **insert iterator** of a third sequence. `merge(v1.begin(), v1.end(), v2.begin(), v2.end(), back_inserter(v3))`.
- **inplace_merge**: **merges in place into one sorted sequence** two (already sorted by `<`) ranges `[first, middle)` and `[middle, last)`. `inplace_merge(v.begin(), v.begin() + middle, v.end())`.
- **lower_bound**: binary search in an ordered sequence, returning an iterator to the position of the first element **greater than or equal to** x . If no such element exists, the end iterator is returned. `lower_bound(v.begin(), v.end(), x)`.
- **upper_bound**: binary search in an ordered sequence, returning an iterator to the position of the first element **greater than** x . If no such element exists, the end iterator is returned. `upper_bound(v.begin(), v.end(), x)`.

Warning — The time complexity of `lower_bound` and `upper_bound`

In ordinary arrays, both functions have time complexity $O(\log n)$; but in associative containers such as `set`, calling `lower_bound(s.begin(), s.end(), val)` directly costs $O(n)$.

Associative containers such as `set` already encapsulate functions like `lower_bound` (as in `s.lower_bound(val)`), and calling them that way costs $O(\log n)$.

- **next_permutation**: changes the current permutation into **the next permutation in the full-permutation order**. If the current permutation is already **the last permutation** (elements fully in descending order), the function returns **false** and changes the permutation into **the first permutation** (elements fully in ascending order); otherwise the function returns **true**. `next_permutation(v.begin(), v.end())` or `next_permutation(v + begin, v + end)`.
- **prev_permutation**: changes the current permutation into **the previous permutation in the full-permutation order**; same usage as `next_permutation`.
- **partial_sum**: computes prefix sums. With source container x and destination container y , it sets $y[i] = x[0] + x[1] + \dots + x[i]$. `partial_sum(src.begin(), src.end(), back_inserter(dst))`.

20.1 Usage Examples

- Using `next_permutation` to generate all permutations of 1 through 9. Example problem: Luogu P1706, “Permutation Problem”³⁵

Implementation

```
1 int N = 9, a[] = {1, 2, 3, 4, 5, 6, 7, 8, 9};
2 do {
3     for (int i = 0; i < N; i++) cout << a[i] << " ";
4     cout << endl;
5 } while (next_permutation(a, a + N));
```

- Using `lower_bound` and `upper_bound` to find, in the ordered array a , the boundaries of the elements smaller than, equal to, and greater than x .

Implementation

```
1 int N = 10, a[] = {1, 1, 2, 4, 5, 5, 7, 7, 9, 9}, x = 5;
2 int i = lower_bound(a, a + N, x) - a, j = upper_bound(a, a + N, x) - a;
3 // a[0] ~ a[i - 1] holds the elements smaller than x,
4 // a[i] ~ a[j - 1] the elements equal to x, and
5 // a[j] ~ a[N - 1] the elements greater than x
6 cout << i << " " << j << endl;
```

- Using `partial_sum` to compute the prefix sums of the elements of src and store them in dst .

Implementation

```
1 vector<int> src = {1, 2, 3, 4, 5}, dst;
2 // compute the prefix sums of the elements of src: dst[i] = src[0] + ... + src
  ↪ [i]
```

³⁵<https://www.luogu.com.cn/problem/P1706>


```

3 // the back_inserter function acts on the container dst, providing an iterator
4 partial_sum(src.begin(), src.end(), back_inserter(dst));
5 for (unsigned int i = 0; i < dst.size(); i++) cout << dst[i] << " ";

```

- Using `lower_bound` to find, in the ordered array a , the element closest to x . Example problem: UVa10487, “Closest Sums”³⁶

Implementation

```

1 int N = 10, a[] = {1, 1, 2, 4, 5, 5, 8, 8, 9, 9}, x = 6;
2 // lower_bound returns the address of the first element of a that is >= x;
3 // the computed i is its index
4 int i = lower_bound(a, a + N, x) - a;
5 // In the following two cases, a[i] (the first element of a that is >= x) is
6 //   ↪ the answer:
7 // 1. all elements of a are >= x;
8 // 2. a contains elements >= x, and the first such element (a[i])
9 //   is closer to x than the first element < x (a[i - 1]);
10 // otherwise, a[i - 1] (the first element of a that is < x) is the answer
11 if (i == 0 || (i < N && a[i] - x < x - a[i - 1]))
12     cout << a[i];
13 else
14     cout << a[i - 1];

```

- Using `sort` and `unique` to find the k -th smallest value in the array a (note: values that occur repeatedly count only once, so this is not finding the k -th smallest element). Example problem: Luogu P1138, “The k -th Smallest Integer”³⁷

Implementation

```

1 int N = 10, a[] = {1, 3, 3, 7, 2, 5, 1, 2, 4, 6}, k = 3;
2 sort(a, a + N);
3 // unique returns the address just past the last element of the deduplicated
4 // array; the computed cnt is the length of the array after deduplication
5 int cnt = unique(a, a + N) - a;
6 cout << a[k - 1];

```

21 bitset (csl/bitset)

Original title: bitset · source page /lang/bitset/

³⁶https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=8&category=16&page=show_problem&problem=1428

³⁷<https://www.luogu.com.cn/problem/P1138>

21.1 Introduction

`std::bitset` is a container in the standard library that stores 0/1 values and whose size is immutable. Strictly speaking, it does not belong to the STL.

Note — bitset and the STL

The C++ standard library provides some special container classes, the so-called container adapters (stack, queue, priority queue). In addition, a few classes provide a container-like interface (for example, strings, bitsets, and valarrays). All these classes are covered separately.^a Container adapters and bitsets are covered in Chapter 12.

The C++ standard library provides not only the containers for the STL framework but also some containers that fit some special needs and provide simple, almost self-explanatory, interfaces. You can group these containers into either the so-called container adapters, which adapt standard STL containers to fit special needs, or a bitset, which is a containers for bits or Boolean values. There are three standard container adapters: stacks, queues, and priority queues. In priority queues, the elements are sorted automatically according to a sorting criterion. Thus, the “next” element of a priority queue is the element with the “highest” value. A bitset is a bitfield with an arbitrary but fixed number of bits. Note that the C++ standard library also provides a special container with a variable size for Boolean values: vector.

^a1

— quoted from *The C++ Standard Library, 2nd Edition*

From this it appears that `bitset` does not belong to the STL but is a “Special Container” of the standard library. In fact, as a container it also fails to satisfy the STL container requirements; and calling it an adapter is wrong too, since it does not rely on other STL containers as its underlying implementation.

Because memory addresses are addressed by the byte (**byte**), not the bit (**bit**), a `bool` variable, although it can represent only 0/1, still occupies 1 byte of memory.

`bitset` uses fixed optimizations so that the eight bits of one byte store 8 digits of 0/1 separately.

For a 4-byte `int` variable, in the sense of storing only 0/1, a `bitset` occupies only $\frac{1}{32}$ of its space, and computing some kinds of information takes $\frac{1}{32}$ of its time.

In certain situations `bitset` can optimize a program’s running efficiency. Whether it optimizes the complexity or the constant depends on the angle from which the complexity is computed. The complexity of `bitset` is generally written in the following ways (let the original complexity be $O(n)$):

1. $O(n)$: this notation regards `bitset` as optimizing the complexity not at all.
2. $O(\frac{n}{32})$: this notation is not quite rigorous (constants should not appear in complexity), but it reflects that `bitset` can reduce the required time to $\frac{1}{32}$.
3. $O(\frac{n}{w})$, where $w = 32$ (the machine word size); this notation is more generally accepted.
4. $O(\frac{n}{\log w})$, where w is the size of one integer variable of the computer.

In addition, the specialization `vector<bool>` of `vector` stores data the same way as `bitset`; the difference is that it supports dynamic allocation, whereas `bitset`, like our ordinary static arrays, is allocated at compile time. However, `bitset` has some very useful library functions — not only convenient, but sometimes enabling SIMD and thereby reducing constants. Also, some behavior of `vector<bool>` is inconsistent with `vector` (e.g., for `std::vector<bool> vec, &vec[0] + i` does not equal `&vec[i]`). For these reasons, `vector<bool>` is generally avoided.

21.2 Usage

See `std::bitset` on [cppreference.com](https://en.cppreference.com).³⁸

21.2.1 Header

```
1 #include <bitset>
```

21.2.2 Specifying the Size

```
1 std::bitset<1000> bs; // a bitset with 1000 bits
```

21.2.3 Constructors

- `bitset()`: every bit is `false`.
- `bitset(unsigned long val)`: set to the binary form of `val`.
- `bitset(const string& str)`: set to the 01-string `str`.

21.2.4 Operators

- `operator []`: accesses one specific bit.
- `operator ==/operator !=`: compares whether two `bitsets` are entirely identical.
- `operator &/operator &=/operator |/operator |=/operator ^/operator ^=/operator ~`: bitwise AND/OR/XOR/NOT.

Note: a `bitset` can only be bit-operated with another `bitset`; to bit-operate with an integer, first convert the integer into a `bitset`.

- `operator <</operator >>/operator <<=/operator >>=`: binary left/right shift.

In addition, `bitset` supports C++ stream IO, which means you can input/output it through `cin/cout`.

³⁸<https://en.cppreference.com/w/cpp/utility/bitset>

21.2.5 Member Functions

- `count()`: returns the number of `true` bits.
- `size()`: returns the size of the `bitset`.
- `test(pos)`: does the same thing as `at()` in `vector`; the difference from the `[]` operator is the bounds check.
- `any()`: returns `true` if some bit is `true`, otherwise `false`.
- `none()`: returns `true` if all bits are `false`, otherwise `false`.
- `all()`: returns `true` if all bits are `true`, otherwise `false`.
- `set()`: sets the whole `bitset` to `true`. `set(pos, val = true)`: sets one bit to `true/false`.
- `reset()`: sets the whole `bitset` to `false`. `reset(pos)`: sets one bit to `false`; equivalent to `set(pos, false)`.
- `flip()`: flips every bit ($0 \leftrightarrow 1$, equivalent to XOR with an all-ones `bitset`). `flip(pos)`: flips one bit.
- `to_string()`: returns the string representation after conversion.
- `to_ulong()`: returns the converted `unsigned long` value (`long` equals `int` on NT and 32-bit POSIX systems, and equals `long long` on 64-bit POSIX).
- `to_ullong()`: (since C++11) returns the converted `unsigned long long` value.

In addition, `libstdc++` has some rather useful internal member functions:³⁹

- `_Find_first()`: returns the index of the first `true` bit of the `bitset`; if there is no `true` bit, returns the size of the `bitset`.
- `_Find_next(pos)`: returns the index of the first `true` bit after `pos` (at a position strictly greater than `pos`); if there is no `true` bit after `pos`, returns the size of the `bitset`.

21.3 Applications

21.4 “LibreOJ β Round #2”: Greed Only Passes the Sample

This problem can be done with DP; the transition is very simple:

Let $f(i, j)$ denote whether the sum of squares of the first i numbers can be j ; then $f(i, j) = \bigvee_{k=a}^b f(i-1, j-k^2)$ (OR-ed together).

But doing it directly is $O(n^5)$, which (apparently) does not pass.

One finds that `bitset` optimization works: shift left and OR together:

³⁹<https://gcc.gnu.org/onlinedocs/libstdc++/libstdc++-html-USERS-4.4/a00994.html>

Note — Submission record: std::bitset^a^a<https://loj.ac/submission/395274>

```

1  #include <bitset>
2  #include <cstdio>
3  #include <iostream>
4
5  using namespace std;
6
7  constexpr int N = 101;
8
9  int n, a[N], b[N];
10 bitset<N * N * N> f[N];
11
12 int main() {
13     int i, j;
14
15     cin >> n;
16
17     for (i = 1; i <= n; ++i) cin >> a[i] >> b[i];
18
19     f[0][0] = 1;
20
21     for (i = 1; i <= n; ++i) {
22         for (j = a[i]; j <= b[i]; ++j) {
23             f[i] |= (f[i - 1] << (j * j));
24         }
25     }
26
27     cout << f[n].count();
28
29     return 0;
30 }

```

Since libstdc++ packs `__CHAR_BIT__ * sizeof(unsigned long)` bits,⁴⁰ which on some platforms is 32, one can hand-write a `bitset` (only the shift-left-then-OR operation is needed) packing 64 bits (`__CHAR_BIT__ * sizeof(unsigned long long)`) for further optimization:

Note — Submission record: hand-written `bitset`^a^a<https://loj.ac/submission/395619>

```

1  #include <cstdio>
2  #include <iostream>
3
4  using namespace std;
5
6  constexpr int N = 101;
7  constexpr int W = 64;
8

```

⁴⁰<https://gcc.gnu.org/onlinedocs/libstdc++/libstdc++-html-USERS-4.4/a00219.html>

```

9 struct Bitset {
10     unsigned long long a[N * N * N >> 6];
11
12     void shiftoor(const Bitset &y, int p, int l, int r) {
13         int t = p - p / W * W;
14         int tt = (t == 0 ? 0 : W - t);
15         int to = (r + p) / W;
16         int qaq = (p + W - 1) / W;
17
18         for (int i = (l + p) / W; i <= to; ++i) {
19             if (i - qaq >= 0) a[i] |= y.a[i - qaq] >> tt;
20
21             a[i] |= ((y.a[i - qaq + 1] & ((1ull << tt) - 1)) << t);
22         }
23     }
24 } f[N];
25
26 int main() {
27     int n, a, b, l = 0, r = 0, ans = 0;
28
29     scanf("%d", &n);
30
31     f[0].a[0] = 1;
32
33     for (int i = 1; i <= n; ++i) {
34         scanf("%d%d", &a, &b);
35
36         for (int j = a; j <= b; ++j) f[i].shiftoor(f[i - 1], j * j, l, r);
37
38         l += a * a;
39         r += b * b;
40     }
41
42     for (int i = l / W; i <= r / W; ++i)
43         ans += __builtin_popcount(f[n].a[i] & 0xffffffffu) +
44             __builtin_popcount(f[n].a[i] >> 32);
45
46     printf("%d", ans);
47
48     return 0;
49 }

```

Also, brute force with a few prunings passes too:

Note — Submission record: brute force with a few prunings^a

^a<https://loj.ac/submission/395673>

```

1 #include <cstdio>
2 #include <iostream>
3
4 using namespace std;
5

```

```

6  constexpr int N = 101;
7  constexpr int W = 64;
8
9  bool f[N * N * N];
10
11 int main() {
12     int n, i, j, k, a, b, l = 0, r = 0, ans = 0;
13
14     scanf("%d", &n);
15
16     f[0] = true;
17
18     for (i = 1; i <= n; ++i) {
19         scanf("%d%d", &a, &b);
20         l += a * a;
21         r += b * b;
22
23         for (j = r; j >= l; --j) {
24             f[j] = false;
25
26             for (k = a; k <= b; ++k) {
27                 if (j - k * k < l - a * a) break;
28
29                 if (f[j - k * k]) {
30                     f[j] = true;
31                     break;
32                 }
33             }
34         }
35     }
36
37     for (i = l; i <= r; ++i) ans += f[i];
38
39     printf("%d", ans);
40
41     return 0;
42 }

```

21.5 CF1097F — Alex and a TV Show

Problem. You are given n multisets and four kinds of operations:

1. set one multiset to a single number;
2. set one multiset to the sum of two other multisets;
3. set one multiset to the gcds obtained by choosing one number from each of two other multisets, i.e., $A = \{\gcd(x, y) \mid x \in B, y \in C\}$;
4. query the count of a certain number in a certain multiset, **modulo 2**.

There are 10^5 multisets, 10^6 operations, and the value range is 7000.

Solution. Seeing “modulo 2”, one may think of maintaining each multiset with a **bitset**.

Then operation 1 is a direct assignment, operation 2 is XOR (because of modulo 2), and operation 4 is a direct query — but what about operation 3?

We can try maintaining, for each multiset, the multiset of *all divisors* of its elements; then operation 3 becomes a direct bitwise AND.

We can precompute, for every number in the value range, the **bitset** formed by its divisors, and operation 1 is then solved. Operation 2 is still XOR.

The remaining question: how to get the count of a number x in a multiset from the multiset of divisors of that multiset?

Let the original multiset be A and its divisor multiset be A' . To find the count of x in A , apply *Möbius inversion* (OI Wiki → Mathematics → Number theory → Möbius):

$$\begin{aligned} & \sum_{i \in A} [\frac{i}{x} = 1] \\ &= \sum_{i \in A} \sum_{d | \frac{i}{x}} \mu(d) \\ &= \sum_{d \in A', x | d} \mu(\frac{d}{x}) \end{aligned}$$

Since we work modulo 2, -1 and 1 are the same; we only need to know whether $\frac{d}{x}$ has a square factor. So, for every number in the value range, precompute the **bitset** of the positions of its multiples that, divided by it, are squarefree; then the answer is a bitwise AND followed by `count()`.

The complexity per query is then $O(\frac{v}{w})$ ($v = 7000$, $w = 32$).

As for the precomputation, $O(v\sqrt{v})$ or $O(v^2)$ is simple; the log-style precomputation shown in the code below has complexity given by the harmonic series, hence $O(v \log v)$.

Note — Reference code

```

1  #include <bitset>
2  #include <cctype>
3  #include <cmath>
4  #include <cstdio>
5  #include <iostream>
6
7  using namespace std;
8
9  int read() {
10     int out = 0;
11     char c;
12     while (!isdigit(c = getchar()));
13     for (; isdigit(c); c = getchar()) out = out * 10 + c - '0';
14     return out;
15 }
16
17 constexpr int N = 100005;
18 constexpr int M = 1000005;
```



```
19 constexpr int V = 7005;
20
21 bitset<V> pre[V], pre2[V], a[N], mu;
22 int n, m, tot;
23 char ans[M];
24
25 int main() {
26     int i, j, x, y, z;
27
28     n = read();
29     m = read();
30
31     mu.set();
32     for (i = 2; i * i < V; ++i) {
33         for (j = 1; i * i * j < V; ++j) {
34             mu[i * i * j] = 0;
35         }
36     }
37     for (i = 1; i < V; ++i) {
38         for (j = 1; i * j < V; ++j) {
39             pre[i * j][i] = 1;
40             pre2[i][i * j] = mu[j];
41         }
42     }
43
44     while (m--) {
45         switch (read()) {
46             case 1:
47                 x = read();
48                 y = read();
49                 a[x] = pre[y];
50                 break;
51             case 2:
52                 x = read();
53                 y = read();
54                 z = read();
55                 a[x] = a[y] ^ a[z];
56                 break;
57             case 3:
58                 x = read();
59                 y = read();
60                 z = read();
61                 a[x] = a[y] & a[z];
62                 break;
63             case 4:
64                 x = read();
65                 y = read();
66                 ans[tot++] = ((a[x] & pre2[y]).count() & 1) + '0';
67                 break;
68         }
69     }
70
71     printf("%s", ans);
```

```

72
73     return 0;
74 }

```

21.5.1 Combined with the Sieve of Eratosthenes

Because of `bitset`'s fast sequential read/write efficiency, it is very suitable for building prime tables in combination with the Sieve of Eratosthenes (OI Wiki → Mathematics → Number theory → Sieve).

The usage is also very simple: just replace the Boolean array in the sieve with a `bitset`.

Note — Speed tests

Tested with Quick C++ Benchmarks,^a compiler GCC 13.2, options `-std=c++20 -O2`.

Algorithm	Function name
Eratosthenes + C-style bool array, primes not stored	<code>Eratosthenes_CArray</code>
Eratosthenes + <code>vector<bool></code> , primes not stored	<code>Eratosthenes_vector</code>
Eratosthenes + <code>bitset</code> , primes not stored	<code>Eratosthenes_bitset</code>
Eratosthenes + C-style bool array, primes stored	<code>Eratosthenes_CArray_sp</code>
Eratosthenes + <code>vector<bool></code> , primes stored	<code>Eratosthenes_vector_sp</code>
Eratosthenes + <code>bitset</code> , primes stored	<code>Eratosthenes_bitset_sp</code>
Euler sieve + C-style bool array	<code>Euler_CArray</code>
Euler sieve + <code>vector<bool></code>	<code>Euler_vector</code>
Euler sieve + <code>bitset</code>	<code>Euler_bitset</code>

- When the sieve of Eratosthenes **stores** the sieved primes:

– test results for $N = 5 \times 10^7 + 1$ ^b

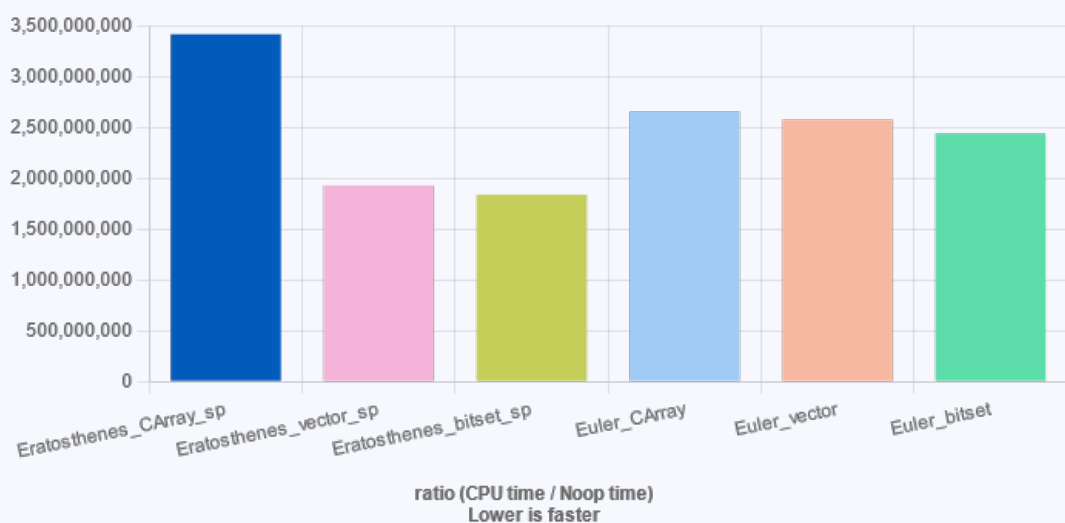


Figure 2: Benchmark: Eratosthenes (primes stored), $N = 5 \times 10^7 + 1$. Ratio of CPU time to no-op time; lower is faster.

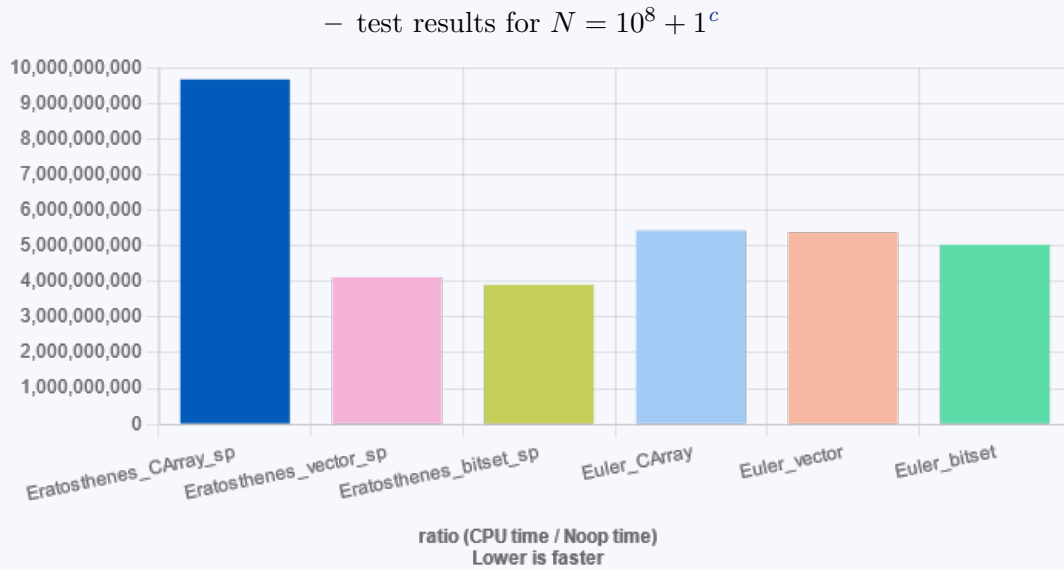


Figure 3: Benchmark: Eratosthenes (primes stored), $N = 10^8 + 1$.

- When the sieve of Eratosthenes does **not** store the sieved primes:

– test results for $N = 5 \times 10^7 + 1^d$

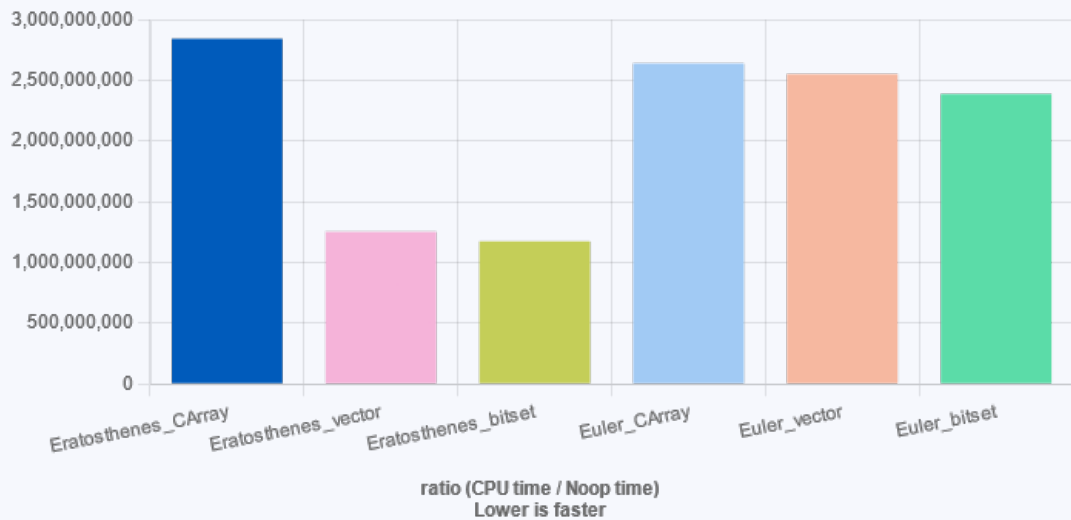


Figure 4: Benchmark: Eratosthenes (primes not stored), $N = 5 \times 10^7 + 1$.

– test results for $N = 10^8 + 1^e$

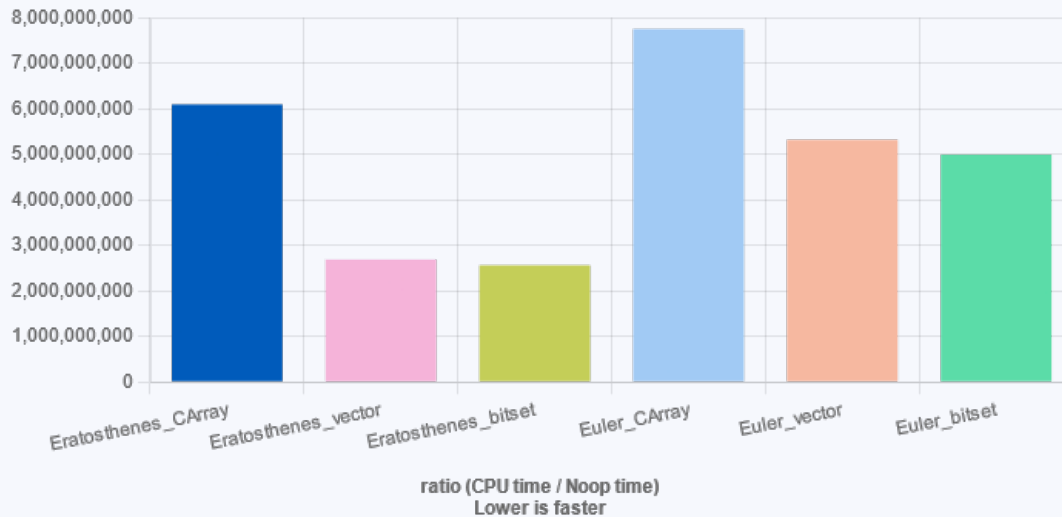


Figure 5: Benchmark: Eratosthenes (primes not stored), $N = 10^8 + 1$.

From the test results:

1. the Eratosthenes sieve, of time complexity $O(n \log \log n)$, when optimized with `bitset` or `vector<bool>`, even outperforms the Euler sieve, of time complexity $O(n)$;
2. the optimization effect of using `bitset` or `vector<bool>` in the Euler sieve is not obvious in most cases;
3. the optimization effect of `bitset` is slightly stronger than that of `vector<bool>`.

^a<https://quick-bench.com>

^b<https://quick-bench.com/q/iQL9FhsZ6PVV81HKABsidRw8hB8>

^c<https://quick-bench.com/q/pwEamEFUW-6nXeXEALRsYPd8FWI>

^d<https://quick-bench.com/q/rg2mCUxT02a44w9fWvHtZoNTJyU>

^e<https://quick-bench.com/q/lusNWxWsR0VXoRBoF7uBtqfvJuY>

Note — Reference code (requires `google/benchmark`^a)

^a<https://github.com/google/benchmark>

```

1  #include <benchmark/benchmark.h>
2  #include <bits/stdc++.h>
3  using namespace std;
4  using u32 = uint32_t;
5  using u64 = uint64_t;
6
7  #define ERATOSTHENES_STORAGE_PRIME
8  #define ENABLE_EULER
9  constexpr u32 N = 5e7 + 1;
10
11 #ifndef ERATOSTHENES_STORAGE_PRIME
12
13 void Eratosthenes_CArray(benchmark::State &state) {
14     static bool is_prime[N];

```

```

15     for (auto _ : state) {
16         fill(is_prime, is_prime + N, true);
17         is_prime[0] = is_prime[1] = false;
18         for (u32 i = 2; (u64)i * i < N; ++i)
19             if (is_prime[i])
20                 for (u32 j = i * i; j < N; j += i) is_prime[j] = false;
21         benchmark::DoNotOptimize(0);
22     }
23 }
24
25 BENCHMARK(Eratosthenes_CArray);
26
27 void Eratosthenes_vector(benchmark::State &state) {
28     static vector<bool> is_prime(N);
29     for (auto _ : state) {
30         fill(is_prime.begin(), is_prime.end(), true);
31         is_prime[0] = is_prime[1] = false;
32         for (u32 i = 2; (u64)i * i < N; ++i)
33             if (is_prime[i])
34                 for (u32 j = i * i; j < N; j += i) is_prime[j] = false;
35         benchmark::DoNotOptimize(0);
36     }
37 }
38
39 BENCHMARK(Eratosthenes_vector);
40
41 void Eratosthenes_bitset(benchmark::State &state) {
42     static bitset<N> is_prime;
43     for (auto _ : state) {
44         is_prime.set();
45         is_prime.reset(0);
46         is_prime.reset(1);
47         for (u32 i = 2; (u64)i * i < N; ++i)
48             if (is_prime[i])
49                 for (u32 j = i * i; j < N; j += i) is_prime.reset(j);
50         benchmark::DoNotOptimize(0);
51     }
52 }
53
54 BENCHMARK(Eratosthenes_bitset);
55
56 #else
57
58 void Eratosthenes_CArray_sp(benchmark::State &state) {
59     static bool is_prime[N];
60     for (auto _ : state) {
61         vector<u32> prime;
62         fill(is_prime, is_prime + N, true);
63         is_prime[0] = is_prime[1] = false;
64         for (u32 i = 2; (u64)i * i < N; ++i)
65             if (is_prime[i])
66                 for (u32 j = i * i; j < N; j += i) is_prime[j] = false;
67         for (u32 i = 2; i < N; ++i)

```

```

68     if (is_prime[i]) prime.push_back(i);
69     benchmark::DoNotOptimize(prime);
70 }
71 }
72
73 BENCHMARK(Eratosthenes_CArray_sp);
74
75 void Eratosthenes_vector_sp(benchmark::State &state) {
76     static vector<bool> is_prime(N);
77     for (auto _ : state) {
78         vector<u32> prime;
79         fill(is_prime.begin(), is_prime.end(), true);
80         is_prime[0] = is_prime[1] = false;
81         for (u32 i = 2; (u64)i * i < N; ++i)
82             if (is_prime[i])
83                 for (u32 j = i * i; j < N; j += i) is_prime[j] = false;
84         for (u32 i = 2; i < N; ++i)
85             if (is_prime[i]) prime.push_back(i);
86         benchmark::DoNotOptimize(prime);
87     }
88 }
89
90 BENCHMARK(Eratosthenes_vector_sp);
91
92 void Eratosthenes_bitset_sp(benchmark::State &state) {
93     static bitset<N> is_prime;
94     for (auto _ : state) {
95         vector<u32> prime;
96         is_prime.set();
97         is_prime.reset(0);
98         is_prime.reset(1);
99         for (u32 i = 2; (u64)i * i < N; ++i)
100             if (is_prime[i])
101                 for (u32 j = i * i; j < N; j += i) is_prime.reset(j);
102         for (u32 i = 2; i < N; ++i)
103             if (is_prime[i]) prime.push_back(i);
104         benchmark::DoNotOptimize(prime);
105     }
106 }
107
108 BENCHMARK(Eratosthenes_bitset_sp);
109
110 #endif
111
112 #ifdef ENABLE_EULER
113
114 void Euler_CArray(benchmark::State &state) {
115     static bool not_prime[N];
116     for (auto _ : state) {
117         vector<u32> prime;
118         fill(not_prime, not_prime + N, false);
119         not_prime[0] = not_prime[1] = true;
120         for (u32 i = 2; i < N; ++i) {

```

```

121     if (!not_prime[i]) prime.push_back(i);
122     for (u32 pri_j : prime) {
123         if (i * pri_j >= N) break;
124         not_prime[i * pri_j] = true;
125         if (i % pri_j == 0) break;
126     }
127 }
128 benchmark::DoNotOptimize(prime);
129 }
130 }
131
132 BENCHMARK(Euler_CArray);
133
134 void Euler_vector(benchmark::State &state) {
135     static vector<bool> not_prime(N);
136     for (auto _ : state) {
137         vector<u32> prime;
138         fill(not_prime.begin(), not_prime.end(), false);
139         not_prime[0] = not_prime[1] = true;
140         for (u32 i = 2; i < N; ++i) {
141             if (!not_prime[i]) prime.push_back(i);
142             for (u32 pri_j : prime) {
143                 if (i * pri_j >= N) break;
144                 not_prime[i * pri_j] = true;
145                 if (i % pri_j == 0) break;
146             }
147         }
148         benchmark::DoNotOptimize(prime);
149     }
150 }
151
152 BENCHMARK(Euler_vector);
153
154 void Euler_bitset(benchmark::State &state) {
155     static bitset<N> not_prime;
156     for (auto _ : state) {
157         vector<u32> prime;
158         not_prime.reset();
159         not_prime.set(0);
160         not_prime.set(1);
161         for (u32 i = 2; i < N; ++i) {
162             if (!not_prime[i]) prime.push_back(i);
163             for (u32 pri_j : prime) {
164                 if (i * pri_j >= N) break;
165                 not_prime.set(i * pri_j);
166                 if (i % pri_j == 0) break;
167             }
168         }
169         benchmark::DoNotOptimize(prime);
170     }
171 }
172
173 BENCHMARK(Euler_bitset);

```

```

174
175 #endif
176
177 static void Noop(benchmark::State &state) {
178     for (auto _ : state) benchmark::DoNotOptimize(0);
179 }
180
181 BENCHMARK(Noop);
182 BENCHMARK_MAIN();

```

21.5.2 Combined with Tree Blocking

Combining `bitset` with tree blocking solves a class of problems about computing the union of information over multiple tree paths; see OI Wiki → Data structures → Tree blocking.

21.5.3 Combined with Mo's Algorithm

See OI Wiki → Miscellaneous → Mo's algorithm with `bitset`.

21.5.4 Computing High-Dimensional Partial Orders

See the FHR slides:⁴¹ “FHR — blocked `bitset` for high-dimensional partial orders” (in Chinese).

22 string (csl/string)

Original title: `string` · source page `/lang/string/`

22.1 What Is `string`

`std::string` is a class provided in the standard library header `<string>` (note: not the C language's `<string.h>` library); it is essentially an alias of `std::basic_string<char>`.

22.2 Why Use `string`

The C language provides string operations, but strings can only be realized through arrays of characters. `string` is a simple class, easy to use, and widely used in OI contests. Moreover, compared with other STL containers, the constants of `string` can be regarded as excellent, roughly on a par with character arrays.

22.2.1 `string` Can Allocate Space Dynamically

Like many STL containers, `string` allocates space dynamically, which lets us input directly via `std::cin` (though this is likewise rather slow). This also spares us from worrying about memory.

⁴¹<https://github.com/OI-wiki/libs/blob/master/lang/csl/FHR-%E6%96%8E%E6%96%8E%E6%96%8E%E6%96%8E.pdf>

22.2.2 string Overloads the Addition and Comparison Operators

The addition operator of `string` concatenates two strings, or a string and a character. Like `std::vector`, `string` overloads the comparison operators, also comparing lexicographically, so we can call `std::sort` directly on several strings.

22.3 Usage

Below are the basic operations of `string`; see the C++ documentation⁴² for details.

22.3.1 Declaration

```
1 std::string s;
```

22.3.2 Converting to a char Array

The C language has many string functions, but their parameters are all of type char pointer. For ease of use, `string` has two member functions that convert itself to a char pointer — `data()/c_str()` (before C++11, `c_str()` guaranteed a trailing null character while `data()` did not; from C++11 on the two behave identically⁴³):

```
1 printf("%s", s);           // not guaranteed to compile; undefined behavior
2 printf("%s", s.data());    // undefined behavior before C++11; correct after
3 printf("%s", s.c_str());   // always outputs correctly
```

22.3.3 Getting the Length

Many functions can return the length of a string:

```
1 printf("the length of s is %zu", s.size());
2 printf("the length of s is %zu", s.length());
3 printf("the length of s is %zu", strlen(s.c_str()));
```

Note — The complexity of these functions

The complexity of `strlen()` must be linear in the string length.

The complexity of `size()` and `length()` is unspecified in C++98 and specified as constant in C++11. But on common compilers, even in C++98, the complexity of these two functions is constant.

Warning

The return types of these three functions (and of the `find` function mentioned below) are all `size_t` (unsigned long). Therefore these return values do not support direct comparison or arithmetic with negative numbers; cast explicitly when needed.

⁴²https://zh.cppreference.com/w/cpp/string/basic_string

⁴³https://en.cppreference.com/w/cpp/string/basic_string/data

22.3.4 Finding the First Occurrence of a Character (String)

The `find(str, pos)` function finds the position of the first occurrence of a character/string at or after `pos` (if no `pos` is passed, it defaults to `0`). If there is no occurrence, it returns `string::npos` (defined as `-1`, but still of type `size_t/unsigned long`).

Example:

```
1 string s = "OI Wiki", t = "OI", u = "i";
2 int pos = 5;
3 printf("character I first occurs in s at position %lu\n", s.find('I'));
4 printf("character a first occurs in s at position %lu\n", s.find('a'));
5 printf("character a first occurs in s at position %d\n", s.find('a'));
6 printf("string t first occurs in s at position %lu\n", s.find(t));
7 printf("searching s from position pos, string u first occurs at %lu", s.find(u, pos));
```

Output:

```
1 character I first occurs in s at position 1
2 character a first occurs in s at position 18446744073709551615 // i.e. size_t(-1); the
   ↳ concrete value depends on the platform.
3 character a first occurs in s at position -1 // after forcing a conversion to int, -1
   ↳ is printed normally
4 string t first occurs in s at position 0
5 searching s from position pos, string u first occurs at position 6
```

22.3.5 Extracting Substrings

The `substr(pos, len)` function returns the string formed by at most `len` characters extracted starting from position `pos` (if the suffix from `pos` is shorter than `len`, the whole suffix is extracted).

Example:

```
1 string s = "OI Wiki", t = "OI";
2 printf("the substring of s of at most three characters starting from the\nfourth
   ↳ position is %s\n",
3       s.substr(3, 3).c_str());
4 printf("the substring of t of at most three characters starting from the\nsecond
   ↳ position is %s",
5       t.substr(1, 3).c_str());
```

Output:

```
1 the substring of s of at most three characters starting from the fourth position is Wik
2 the substring of t of at most three characters starting from the second position is I
```

22.3.6 Inserting / Deleting Characters (Strings)

`insert(index, count, ch)` and `insert(index, str)` are the more common insertion functions. They respectively insert the character `ch` `count` times in a row at `index`, and insert the string `str` at `index`.

The `erase(index, count)` function deletes `count` characters starting from (and including) position `index` (if no `count` is passed, all characters from `index` onward are deleted).

Example:

```

1 string s = "OI Wiki", t = " Wiki";
2 char u = '!';
3 s.erase(2);
4 printf("after erasing all characters of s from the third position, the\nstring is %s\n"
   ↪ , s.c_str());
5 s.insert(2, t);
6 printf("after inserting string t into s at the third position, the string\nis %s\n", s.
   ↪ c_str());
7 s.insert(7, 3, u);
8 printf("after inserting string u into s at the eighth position 3 times in\na row, the
   ↪ string is %s",
9       s.c_str());

```

Output:

```

1 after erasing all characters of s from the third position, the string is OI
2 after inserting string t into s at the third position, the string is OI Wiki
3 after inserting string u into s at the eighth position 3 times in a row, the string is
   ↪ OI Wiki!!!

```

22.3.7 Replacing Characters (Strings)

Example:

```

1 string s = "OI Wiki";
2 s.replace(2, 5, "");
3 printf("after replacing positions 3~7 of s with an empty string, the result\nis %s\n",
   ↪ s.c_str());
4 s.replace(s.begin(), s.begin() + 2, "NOI");
5 printf("after replacing the first two positions of s with NOI, the result is %s", s.
   ↪ c_str());

```

Output:

```

1 after replacing positions 3~7 of s with an empty string, the result is OI
2 after replacing the first two positions of s with NOI, the result is NOI

```

23 pair (csl/pair)

Original title: pair · source page /lang/pair/

`std::pair` is a class template defined in the standard library. It associates two variables into a “pair”, and the data types of the two variables may differ.

Note — Class template

A class template is itself not a class but a “template” that can produce **different classes** according to **different data types**.

When used, the compiler generates the corresponding class from the passed data types and then creates the corresponding instances.

Templates are a relatively advanced C++ language feature and almost never appear in informatics contests. If you are interested, read *C++ Primer* for deeper C++ knowledge.

By using `pair` flexibly, you can easily handle scenarios **where related data must be bundled, stored, and processed**.

Note — Struct

Compared with a custom `struct`, `pair` requires no extra structure definition or operator overloading, so it is more convenient.

However, the variable naming of a custom `struct` is often clearer (`pair` can only access its two contained variables via `first` and `second`). Also, if more than two variables must be associated, a custom `struct` is more suitable.

23.1 Usage

23.1.1 Initialization

You can initialize a `pair` directly at definition:

```
1 pair<int, double> p0(1, 2.0);
```

You can also define first and assign afterwards:

```
1 pair<int, double> p1;  
2 p1.first = 1;  
3 p1.second = 2.0;
```

You can also use the `std::make_pair` function, which takes two variables and returns the `pair` formed by them:

```
1 pair<int, double> p2 = make_pair(1, 2.0);
```

A common trick is the macro definition `#define mp make_pair`, shortening the rather verbose `make_pair` to `mp`.

In C++11 and later, `make_pair` can be combined with `auto` to avoid declaring data types explicitly:

```
1 auto p3 = make_pair(1, 2.0);
```

For the use of `auto` in informatics contests, see the remarks in the [Iterators](#) section.

23.1.2 Access

Through the member functions `first` and `second`, you can access the two variables contained in a `pair`:

```
1 int i = p0.first;
2 double d = p0.second;
```

You can also modify them:

```
1 p1.first++;
```

23.1.3 Comparison

`pair` predefines all comparison operators: `<`, `>`, `<=`, `>=`, `==`, `!=`. Of course, this requires the data types of the two contained variables to define `==` and/or `<`.

Among them, the four operators `<`, `>`, `<=`, `>=` first compare the first variables of the two `pairs`, and compare the second variables only when the first variables are equal.

```
1 if (p2 >= p3) {
2     cout << "do something here" << endl;
3 }
```

Because `pair` defines `<` and `==`, which the STL uses often, it cooperates well with other STL functions or data structures. For instance, `pair` can serve as the data type of a `priority_queue`:

```
1 priority_queue<pair<int, double>> q;
```

23.1.4 Assignment and Swap

You can assign the value of a `pair` to another `pair` of consistent type:

```
1 p0 = p1;
```

You can also use the `swap` function to exchange the values of `pairs`:

```
1 swap(p0, p1);
2 p2.swap(p3);
```

23.2 Application Examples

23.2.1 Discretization

`pair` implements discretization easily.

We can create a `pair` array, using the values of the original data as the first variable of each `pair` and the positions of the original data as the second variable. After sorting, assign the rank of each original data value (its position after sorting) back to the position where that value originally was.

```

1 // a is the original data
2 pair<int, int> a[MAXN];
3 // ai is the data after discretization
4 int ai[MAXN];
5 for (int i = 0; i < n; i++) {
6     // first is the value of the original data, second its position
7     scanf("%d", &a[i].first);
8     a[i].second = i;
9 }
10 // sort
11 sort(a, a + n);
12 for (int i = 0; i < n; i++) {
13     // assign the rank of this value back to its original position
14     ai[a[i].second] = i;
15 }
```

23.2.2 Dijkstra

As mentioned above, `pair` can be the data type of a `priority_queue`.

In the heap optimization of Dijkstra's algorithm, we can maintain nodes with a `pair` and a `priority_queue`, using the node's current distance to the source as the first variable and the node index as the second variable.

```

1 priority_queue<pair<int, int>, std::vector<pair<int, int>>,
2               std::greater<pair<int, int>>>
3               > q;
4 ... while (!q.empty()) {
5     // dis is the node's distance to the source when pushed into the heap;
6     // i is the node index
7     int dis = q.top().first, i = q.top().second;
8     q.pop();
9     ...
10 }
```

23.2.3 pair and map

`map` is the data structure in C++ that stores key-value pairs. In many cases, the key-value pairs stored in a `map` are exposed to the outside through `pair`.

```

1 map<int, double> m;
2 m.insert(make_pair(1, 2.0));
```

For more on `map`, see the relevant parts of §17 and §18.

PART IV

Classes and Modern C++

24 Classes (`class`)

Original title: 类 · source page </lang/class/>

A class is an extension of a structure: besides member elements, it can also have member functions.

In object-oriented programming (OOP), an object is an instance of a class, i.e., a variable.

The **`struct`** keyword in C++ also defines classes; the definition of **`structure`** given earlier comes from C. For certain historical reasons, C++ kept and extended **`struct`**.

24.1 Defining Classes

Classes are defined with the keyword **`class`** or **`struct`**; below we use **`class`** for the examples.

```
1 class ClassName {  
2     ...  
3 };  
4  
5 // Example:  
6 class Object {  
7     public:  
8         int weight;  
9         int value;  
10 } e[array_length];  
11  
12 const Object a;  
13 Object b, B[array_length];  
14 Object *c;
```

It is much the same as using **`struct`**. This example defines a class named **`Object`**. The class has two members, **`weight`** and **`value`**; after the **`}`**, an array **`e`** of that type is defined.

Defining a pointer to a class is the same as for a `struct`.

24.1.1 Access Specifiers

Unlike the `struct` example, the word `public` appears in this example; it belongs to the access specifiers.

- **public**: the members after this access specifier can be accessed publicly; simply put, they can be accessed both **inside and outside the class**.
- **protected**: the members after this access specifier can be accessed **inside the class**, by derived classes, or by friends, but **cannot be accessed** from outside the class.
- **private**: the members after this access specifier can **only** be accessed by members **inside the class** or by friends; they **cannot** be accessed from outside the class or from derived classes.

For `struct`, all members default to `public`. For `class`, all members default to `private`.

Note — Basic concepts of friends and derived classes

A friend (**friend**): the `friend` keyword decorates a function or a class. It lets the **decorated** thing access the private (`private`) or protected (`protected`) members of the class without becoming a member function or member class. Simply put, anything carrying the `friend` mark of this class can access its private or protected members.

A derived class (**derived class**): C++ allows one class to serve as a **base class**, from which **derived classes** are **derived**. A derived class inherits the member variables and member functions of the base class (according to specific rules), which improves code reuse.

A derived class is an “is” relationship: a cat (derived class) “is” a mammal (base class).

For the difference between `private` and `protected` above: a derived class can access the `protected` members of the base class (same for `public`) but cannot access its `private` members.

24.2 Accessing and Modifying Members

The methods are the same as for a `struct`:

- for variables, use the `.` symbol;
- for pointers, use the `->` symbol.

24.3 Member Functions

Member functions are, as the name suggests, the functions contained in a class.

Note — Examples of common member functions

```
1 vector.push_back();
2 set.insert();
```

```

3 queue.empty();

1 class Class_Name {
2     ... type Function_Name(...) { ... }
3 };
4
5 // Example:
6 class Object {
7     public:
8         int weight;
9         int value;
10
11        void print() {
12            cout << weight << endl;
13            return;
14        }
15
16        void change_w(int);
17 };
18
19 void Object::change_w(int _weight) { weight = _weight; }
20
21 Object var;

```

This class has a function that prints the members of an `Object`, and a function that changes the member `weight`.

Like functions, member functions can be declared first and defined later, as on line 14 (the declaration) and after line 17 (the definition) above.

To call the `print` member function of `var`, use `var.print()`.

24.3.1 Operator Overloading

Note — What is overloading

C++ allows the programmer to give different definitions to functions or operators with the same name. This is called **overloading**.

If same-named functions differ pairwise in the kinds and/or number of their parameters, they are regarded as different functions.

Note: if two same-named functions differ only in their return-value types, they cannot be overloaded — the compiler will refuse to compile!

If no confusion arises at a call site (meaning: when calling some same-named functions, the arguments given cannot uniquely determine which function is called; this often happens with functions having default arguments), the compiler decides which function to call from the arguments given at the call.

The above process is called overload resolution.

Overloading operators can partly replace functions and simplify code.

Here is an example of overloaded operators.

```

1 class Vector {
2     public:
3         int x, y;
4
5         Vector() : x(0), y(0) {}
6
7         Vector(int _x, int _y) : x(_x), y(_y) {}
8
9         int operator*(const Vector& other) const { return x * other.x + y * other.y; }
10
11         Vector operator+(const Vector& other) const;
12         Vector operator-(const Vector& other) const;
13 };
14
15 Vector Vector::operator+(const Vector& other) const {
16     return Vector(x + other.x, y + other.y);
17 }
18
19 Vector Vector::operator-(const Vector& other) const {
20     return Vector(x - other.x, y - other.y);
21 }
22
23 // Lines 4-5 above assign to x and y; see the explanation later for the
24 // concrete mechanism.

```

This example defines a vector class and overloads the `*` `+` `-` operators, representing the inner product, vector addition, and vector subtraction.

The template of operator overloading falls roughly into the following parts.

```

1 /* overloaded inside the class definition */ return-type operator@(parameters) {...}
2
3 /* declared inside the class definition, defined outside */
4 return-type ClassName::operator@(parameters) {...}

```

For your own classes, once certain operators are overloaded (generally it suffices to overload the comparison operator `<`), the corresponding STL containers or algorithms can be used, e.g. `sort` (OI Wiki → Basics → STL sorting functions).

To learn more, see item 4 of the references.

Note — Operators that can be overloaded

1	+	-	*	/	%	^	&
2		~	!	=	<	>	+=
3	--	*=	/=	%=	^=	&=	=
4	<<	>>	>>=	<<=	==	!=	<=
5	>=	&&		++	--	,	->*
6	->	()	[]	new	new []	delete	delete []

24.3.2 Setting Initial Values at Instantiation

To do this, you need to define a **default constructor** (Default constructor).

```
1 class ClassName {  
2     ... ClassName(...)... { ... }  
3 };  
4  
5 // Example:  
6 class Object {  
7     public:  
8         int weight;  
9         int value;  
10  
11     Object() {  
12         weight = 0;  
13         value = 0;  
14     }  
15 };
```

This example defines the default constructor of `Object`, which can initialize all members to 0 when we instantiate a variable of type `Object`.

If there is no explicit constructor, the compiler assumes the class has an implicit default constructor. In other words, if no constructor is defined at all, the compiler automatically generates a default constructor and initializes according to the types of the members (the same as defining built-type variables).

In that case, the members are all uninitialized, and accessing an uninitialized variable has undefined results (that is, there is no knowing what value is returned).

If you need custom initial values, you can define (or overload) constructors.

Note — On defining (or overloading) constructors

Generally speaking, a default constructor takes no parameters; this distinguishes it from ordinary constructors. The definitions of constructors and default constructors are much the same, differing only in the number of parameters.

Constructors can be overloaded (the first one is of course called a definition). Note that once a constructor has been defined, the compiler will no longer generate a parameterless default constructor. This may make attempts to construct variables the default way (i.e., without passing initialization arguments) fail to compile.

With C++11 or above, you can use `{}` to initialize variables.

Note — About `{}`

Initialization with `{}` uses the lightweight proxy object `std::initializer_list`.

The initialization steps are roughly:

1. try to find a default constructor whose parameters include `std::initializer_list`; if found, call it (after the call the following steps are skipped; likewise below);

2. try to fill the elements of {} into other constructor parameters; if the parameters can be filled in order (default parameters included), call that constructor;
3. if there are no **private** members, try to assign values **outside the class** in the order of element definition or by subscript order.

The above is a simplified version of the full process; see reference 9 for the details.

```

1  class Object {
2      public:
3          int weight;
4          int value;
5
6          Object() {
7              weight = 0;
8              value = 0;
9          }
10
11         Object(int _weight = 0, int _value = 0) {
12             weight = _weight;
13             value = _value;
14         }
15
16         // the same as
17         // Object(int _weight,int _value):weight(_weight),value(_value) {}
18     };
19
20     // the same as
21     // Object::Object(int _weight,int _value){
22     //     weight = _weight;
23     //     value = _value;
24     // }
25     //}
26
27     Object A;           // ok
28     Object B(1, 2);     // ok
29     Object C{1, 2};     // ok,(C++11)

```

Note — About implicit type conversion

Sometimes one writes code like this:

```

1  class Node {
2      public:
3          int var;
4
5          Node(int _var) : var(_var) {}
6      };
7
8      Node a = 1;

```

It looks quite illogical: an `int` cannot possibly convert into a `Node`. But the compiler emits no error.

The reason: during the assignment, `1` is first used as an argument to call `Node::Node(int)`, and then the default copy function is called to perform the assignment.

But in most cases the author would prefer the compiler to report an error. In that case, add the `explicit` keyword before the constructor. This tells the compiler the constructor must be called explicitly.

```
1 class Node {
2     public:
3         int var;
4
5         explicit Node(int _var) : var(_var) {}
6     };
```

That is, `Node a=1` will report an error, but `Node a=Node(1)` will not, because the latter calls the constructor explicitly. Most people would never write the latter, but this example suffices to show what `explicit` does.

In algorithm contests, the usual way to avoid this situation is “tightening code discipline”, avoiding it at the source.

24.3.3 Destruction

This is an unavoidable problem. Every variable is destroyed when its scope ends.

But for a pointer that already points to dynamically allocated memory, the pointed-to memory is not released automatically when the pointer is destroyed; the dynamic memory must be freed by hand.

If the members of a structure contain pointers, the same problem arises. A destructor is needed to free the dynamic memory by hand.

The **destructor** (Destructor) is called when the variable is destroyed. It is overloaded in the same way as the constructor, except a `~` is added in front.

The default destructor is usually enough for algorithm contests; we usually overload the destructor only when members contain pointers.

```
1 class Object {
2     public:
3         int weight;
4         int value;
5         int* ned;
6
7         Object() {
8             weight = 0;
9             value = 0;
10        }
11
12        ~Object() { delete ned; }
13    };
```

24.3.4 Assigning to Class Variables

By default, assignment proceeds by the rule of assigning the corresponding members. You can also use `ClassName()` or `ClassName{}` as a temporary variable for the assignment.

The former only calls the copy constructor; the latter calls the default constructor before calling the copy constructor.

Also, by default all assignments perform a **shallow copy** between corresponding elements: if a member contains a pointer, after the assignment the member pointers of the two variables have the same address.

```
1 // A, tmp1, tmp2, tmp3 are of type Object
2 tmp1 = A;
3 tmp2 = Object(...);
4 tmp3 = {...};
```

To solve the pointer problem or do more, overload the corresponding constructors.

For more on constructors, see item 6 of the references.

24.4 References

1. cppreference class, <https://zh.cppreference.com/w/cpp/language/class>
2. cppreference access, <https://zh.cppreference.com/w/cpp/language/access>
3. cppreference default constructor, https://zh.cppreference.com/w/cpp/language/default_constructor
4. cppreference operator, <https://zh.cppreference.com/w/cpp/language/operators>
5. cplusplus Data structures, <http://www.cplusplus.com/doc/tutorial/structures/>
6. cplusplus Special members, <http://www.cplusplus.com/doc/tutorial/classes2/>
7. C++11 FAQ, <http://www.stroustrup.com/C++11FAQ.html>
8. cppreference Friendship and inheritance, <http://www.cplusplus.com/doc/tutorial/inheritance/>
9. cppreference value initialization, https://zh.cppreference.com/w/cpp/language/value_initialization

25 Namespaces (namespace)

Original title: 命名空间 · source page /lang/namespace/

25.1 Overview

The **namespace** mechanism of C++ can be used to solve the problem of name collisions in complex projects.

For example: everything in the C++ standard library is defined in the `std` namespace. If you define a variable named `cin`, you can access your `cin` via `cin` and the standard library's `cin` object via `std::cin`, without worrying about a collision.

25.2 Declaration

The following code declares a namespace named `A`:

```
1 namespace A {  
2   int cnt;  
3  
4   void f(int x) { cnt = x; }  
5 } // namespace A
```

After the declaration, outside the namespace you can access the function `f` inside namespace `A` via `A::f(x)`, and the variable `cnt` inside namespace `A` via `A::cnt`.

Namespace declarations can be nested, so the following code is also allowed:

```
1 namespace A {  
2   namespace B {  
3     void f() { ... }  
4   } // namespace B  
5  
6   void f() {  
7     B::f(); // what is actually accessed is A::B::f(); since we are  
8             // currently inside namespace A, the leading A:: may be omitted  
9   }  
10 } // namespace A  
11  
12 void f() // this defines the f function of the global namespace, which  
13          // conflicts with neither A::f nor A::B::f  
14 {  
15   A::f();  
16   A::B::f();  
17 }
```

25.3 The using Directive

After a namespace is declared, accessing members of the namespace from outside requires prefixing the member name with `namespace::`.

Is there a more convenient way to access members of a namespace directly by their member names? The answer is yes: the **using** directive.

The **using** directive has the following two forms:

1. `using namespace::memberName;` this directive lets us omit the namespace before a certain member name and access the member directly by name, as if importing this member into the current scope.
2. `using namespace namespaceName;` this directive lets us access **any** member of the namespace directly by member name, as if importing all members of the namespace into the current scope.

Thus, executing `using namespace std;` introduces all names in `std` into the global namespace within the current scope. Then we can use `cin` instead of `std::cin` and `cout` instead of `std::cout`.

Warning — The `using` directive may cause naming conflicts!

Because `using namespace std;` introduces **all names** in `std`, declaring a variable or function with the same name as one in `std` may cause compilation errors due to naming conflicts. Therefore in engineering, the `using namespace namespaceName;` directive is not recommended.

With the `using` directive, the code from the C++ Syntax Basics section can be written in these two equivalent ways:

```
1 #include <iostream>
2
3 using std::cin;
4 using std::cout;
5 using std::endl;
6
7 int main() {
8     int x, y;
9     cin >> x >> y;
10    cout << y << endl << x;
11    return 0;
12 }
```

```
1 #include <iostream>
2
3 using namespace std;
4
5 int main() {
6     int x, y;
7     cin >> x >> y;
8     cout << y << endl << x;
9     return 0;
10 }
```

25.4 Unnamed Namespaces

When, within one scope, we only want a namespace to prevent name collisions, its definition and use can become very concise: the unnamed namespace.

A namespace defined in the form `namespace { /* something ... */ }` (omitting the namespace's name) is called an unnamed namespace. An unnamed namespace in a file is regarded as having a unique name, different from all other namespaces, but several unnamed namespaces in the same scope are regarded as the same namespace. After an unnamed namespace is defined, the names inside it can be found when used in the enclosing scope, as if a `using namespace` directive were added after the unnamed namespace's definition.

25.5 Applications

25.5.1 Preventing Name Collisions between Subtasks

In problems with multiple subtasks, we can define a namespace for each subtask, holding the variables and functions needed to solve that subtask. Then even if two subtasks' implementations declare identical names, they do not collide, so the subtasks do not interfere with one another; this eases debugging to some extent and improves the readability of the program.

25.5.2 Preventing Collisions with Names Introduced by the Standard Library and the Environment

Also, using namespaces can prevent names commonly used in algorithm contests from colliding with the standard library or the environment, as in the following example:

```

1  #include <math.h>
2
3  #include <vector>
4
5  using namespace std;
6
7  namespace Sol {
8  int end; // std::end was brought in by using namespace std;
9
10 int y1; // y1 is the second-kind Bessel function defined by POSIX
11
12 // so usually there is a conflict under Linux but not under Windows
13
14 void solve() {
15     // Using our declared end and y1 unqualified (without ::) inside
16     // Sol::solve() causes no name conflicts. But if the code above lived in the
17     // global namespace, conflicts would arise: end would conflict with std::end
18     // only at name lookup (i.e., when compiling code that uses it), while y1
19     // already conflicts at the point of declaration; moreover, since y1's
20     // conflict depends on the environment it may not even be noticed under
21     // Windows, yet it causes a compilation error in a Linux judging
22     // environment.
23 }
24 } // namespace Sol
25

```

```
26 int main() { Sol::solve(); }
```

25.6 References

- Namespaces — cppreference.com, <https://en.cppreference.com/w/cpp/language/namespaces>

26 Value Categories (value-category)

Original title: 值类别 · source page [/lang/value-category/](#)

Value categories are a very important concept in C++. Although they may be of little direct use in algorithm contests, understanding them helps us find and avoid unnecessary copies, thereby improving the efficiency and performance of our code.

The concept of value categories went through several rounds of development in the C language, C++98, C++11, and C++17, gradually becoming a rather complex concept.

26.1 Unnecessary Copies

Consider the process of putting strings into a vector:

```
1 int main() {
2     std::vector<std::string> vec;
3     vec.reserve(3);
4     for (int i = 0; i < 3; ++i) {
5         std::string str;
6         std::cin >> str;
7         vec.push_back(str);
8     }
9     return 0;
10 }
```

We can see that during the transfer, the string is saved once in `str` and once in `vec`: memory usage is doubled.

If we insist on saving this memory, we can implement a crude move operation: define our own `MyString` struct with a pointer inside pointing at our string; we only need to copy the pointer over and carefully clear the original object's pointer, to prevent a wrong destruction.

```
1 struct MyString {
2     char *beg, *end;
3     // ...
4 };
5
6 void move_to(MyString &src, MyString &dst) {
7     dst.beg = src.beg;
8     dst.end = src.end;
9     src.beg = src.end = nullptr;
10 }
```

Because this kind of efficient transfer of objects is a common need, and because it interacts awkwardly with C++'s construction, destruction, and similar operations, C++11 brought move semantics into the core of the language.

26.2 Value Categories in the C Language

In the C language standard, an object is a concept more general than a variable: it denotes a region of memory with a memory address. The main attributes of an object include: size, effective type, value, and identifier. The identifier is the variable name; the value is the meaning of that memory when interpreted as its type. For example, `int` and `float` both occupy 4 bytes, but for the same block of memory we would interpret different meanings.

In C, every expression has a type and a value category. Value categories fall mainly into three classes:

- **lvalue**: an expression that implicitly denotes an object; that is, we can take the address of the expression.
- **rvalue**: an expression that does not denote an object, i.e., it denotes a value with no storage location; we cannot take the address of such a value.
- **function designator**: an expression of function type.

Therefore, only modifiable lvalues (lvalues without `const` and not arrays) may stand on the left side of an assignment expression.

For an operator that requires an rvalue as its operand, whenever an lvalue is used as the operand, the lvalue-to-rvalue, array-to-pointer, or function-to-pointer standard conversion is applied to the expression to convert it into an rvalue.

Common misunderstandings:

- Further operation on an rvalue expression may produce an lvalue. For example, with `int *a`, the expression `a + 1` is an rvalue, but `*(a + 1)` is an lvalue.
- Expressions have value categories; variables do not. For example, with `int *a`, you cannot say the variable `a` is an lvalue; you can say that in the expression `a` it acts as an lvalue.

26.3 Value Categories in C++98

C++98 is almost identical to C in value categories, but adds some new rules:

- Functions are lvalues, because their addresses can be taken.
- An lvalue reference (`T&`) is an lvalue, because its address can be taken.
- Only `const T&` can bind to rvalues.

26.3.1 Copy Elision

C++ allows the compiler to perform copy elision, reducing the creation and destruction of temporary objects.

For example, the code below triggers the Return Value Optimization (RVO) of copy elision: you will see only one construction and one copy construction, even though construction and destruction have side effects.

```

1 struct X {
2     X() { std::puts("X::X()"); }
3
4     X(const X &) { std::puts("X::X(const X &)"); }
5
6     ~X() { std::puts("X::~X()"); }
7 };
8
9 X get() {
10     X x;
11     return x;
12 }
13
14 int main() {
15     X x = get();
16     X y = X(X(X(x)));
17     return 0;
18 }

```

26.4 Value Categories in C++11

C++11 introduced move semantics and rvalue references (T&&), including move construction and move assignment. This gives us a way to exploit temporary objects.

Our `move_to` above can be rewritten as:

```

1 struct MyString {
2     // ...
3     MyString(MyString&& other) {
4         beg = other.beg;
5         end = other.end;
6         other.beg = other.end = nullptr;
7     }
8 };

```

The expression properties we now care about have grown by one:

- whether it has identity: whether it denotes an object, i.e., whether it has an address;
- whether it can be moved: whether it has functions such as move construction and move assignment, giving us a way to exploit such temporary objects.

Hence we have three value categories:

- has identity, cannot be moved: lvalue (lvalue).
- has identity, can be moved: xvalue (xvalue).
- no identity, can be moved: prvalue (prvalue).
- no identity, cannot be moved: expressions of this kind are unusable.

In addition, C++11 introduced two composite categories:

- has identity: glvalue (glvalue), i.e., lvalues and xvalues.
- can be moved: rvalue (rvalue), i.e., prvalues and xvalues.

26.4.1 `std::move`

To go with move semantics, C++11 also introduced the utility function `std::move`, which casts an lvalue into an rvalue so as to trigger move semantics.

```
1 int main() {
2     std::vector<int> a = {1, 2, 3};
3     std::cout << "a: " << a.data() << std::endl;
4     std::vector<int> b = a;
5     std::cout << "b: " << b.data() << std::endl;
6     std::vector<int> c = std::move(b);
7     std::cout << "c: " << c.data() << std::endl;
8 }
```

So we only need to change `push_back(str)` to `push_back(std::move(str))` to avoid the copy.

```
1 int main() {
2     std::vector<std::string> vec;
3     vec.reserve(3);
4     for (int i = 0; i < 3; ++i) {
5         std::string str;
6         std::cin >> str;
7         vec.push_back(std::move(str));
8         // another clever spelling; requires C++17
9         // std::cin >> vec.emplace_back();
10    }
11    return 0;
12 }
```

Because `std::string` has the Small String Optimization (SSO), short strings are stored directly inside the structure; you may have to input rather long strings to observe that the `data` pointer stays the same.

26.5 Value Categories in C++17

C++17 further simplified the value categories:

- lvalue (lvalue): has identity, cannot be moved.
- xvalue (xvalue): has identity, can be moved.
- prvalue (prvalue): the initialization of an object.

C++11 extended copy elision to moves: in the code below, with RVO enabled `urvo` involves no move.

C++17 requires that a prvalue need not be materialized; it is constructed directly into the storage of its final destination, and the object does not exist before the construction. Therefore in C++17 we no longer have the return step, and need not rely on RVO. One can also understand it as making URVO (Unnamed RVO) mandatory, while NRVO (Named RVO) remains optional.

```

1  std::string urvo() { return std::string("123"); }
2
3  std::string nrvo() {
4      std::string s;
5      s = "123";
6      std::cout << s;
7      return s;
8  }
9
10 int main() {
11     std::string str = urvo(); // constructed directly
12     std::string str = nrvo(); // not necessarily constructed directly;
13                             // depends on optimization
14 }
```

Meanwhile C++17 introduced the mechanism of temporary materialization: when we need to access member variables, call member functions, and other situations requiring a glvalue, an implicit conversion to an xvalue may occur.

26.5.1 Common Misunderstandings

In the following example:

- returning `std::move(x)` in `f1` is redundant: it brings no performance gain and instead interferes with the compiler's NRVO optimization.
- returning `std::move(x)` in `f2` is dangerous: the function returns an rvalue reference pointing to the already-destroyed local variable `s`, producing a dangling reference.

```

1  std::string f1() {
2      std::string s = "123";
3      // equivalent to return std::string(std::move(s))
4      return std::move(s);
5  }
```

```

6
7 std::string&& f2() {
8     std::string s = "123";
9     return std::move(s);
10 }

```

26.6 References and Recommended Reading

1. Value categories, https://en.cppreference.com/w/cpp/language/value_category
2. Wording for guaranteed copy elision through simplified value categories, <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0135r1.html>
3. Value categories in C++ (Chinese), <https://paul.pub/cpp-value-category/>
4. Everything you need about C++ rvalue references, moves, and the value category system (Chinese), https://zclll.com/index.php/cpp/value_category.html
5. Copy elision, https://en.cppreference.com/w/cpp/language/copy_elision

27 Operator Overloading (op-overload)

Original title: 重载运算符 · source page /lang/op-overload/

Operator overloading redefines operators so that they support operations on specific data types. Operator overloading is a special case of function overloading.

When an operator appears in an expression, and at least one of the operator’s operands has a class or enumeration type, overload resolution is used to determine which user-defined function satisfying the corresponding declaration should be called.^a

^aOperator overloading — cppreference, <https://zh.cppreference.com/w/cpp/language/operators>

Colloquially: if using an “operator” is viewed as calling a special function (e.g., viewing `1+2` as calling `add(1, 2)`), and at least one of this function’s arguments (operands) has type `class`, `struct`, or `enum`, the compiler needs to decide, from the operands’ types, which user-defined function to call.

In C++, we can overload almost all usable operators.

Note — A list of some overloadable operators

Unary operations: `+` (unary plus); `-` (unary minus); `~` (bitwise NOT); `++`; `--`; `!` (logical NOT); `*` (pointer dereference); `&` (address-of); `->` (class member access); etc.

Binary operations: `+`; `-`; `&` (bitwise AND); `[]` (subscript); `=`; `=` (assignment); etc.

Others: `()` (function call); `"` (literal suffix,^a since C++11); `new` (memory allocation); `,` (comma operator); `<=>` (three-way comparison,^b since C++20); etc.

^aUser-defined literals — cppreference, https://zh.cppreference.com/w/cpp/language/user_literal

^bThree-way comparison — cppreference, https://zh.cppreference.com/w/cpp/language/operator_comparison

27.1 Restrictions

Operator overloading has the following restrictions:

- Only existing operators can be overloaded; you cannot define new operators yourself.
- The following operators cannot be overloaded: `::` (scope resolution), `.` (member access), `.*` (member access through a pointer-to-member), `?:` (ternary operator).
- After overloading, an operator's precedence, number of operands, and associativity must not change.
- Overloaded `&&` (logical AND) and `||` (logical OR) lose short-circuit evaluation.

27.2 Implementation

There are two cases of operator overloading: overloading as a member function or as a non-member function.

When overloaded as a member function, because an implicit `this` pointer to the current member is one parameter, the function takes one fewer parameter than there are operands.

When overloaded as a non-member function, the function takes exactly as many parameters as there are operands.

The basic format is (assume the operator to be overloaded is `@`):

```

1 class Example {
2     // example as a member function
3     return-value operator@(parameters other than the object itself) { /* ... */ }
4 };
5
6 // example as a non-member function
7 return-value operator@(all operands taking part in the operation) { /* ... */ }
```

Below are several examples of overloaded operators.

27.3 Basic Arithmetic Operators

The following defines a 2-D vector struct `Vector2D` and implements the corresponding overloads of addition and inner product.

Note — Example of overloading arithmetic operators

```

1 struct Vector2D {
2     double x, y;
3
4     Vector2D(double a = 0, double b = 0) : x(a), y(b) {}
5
6     Vector2D operator+(Vector2D v) const { return Vector2D(x + v.x, y + v.y); }
7
8     // note that the return type need not be this class
9     double operator*(Vector2D v) const { return x * v.x + y * v.y; }
```

```
10 };
```

27.4 Increment and Decrement Operators

The increment/decrement operators come in two classes: prefix (`++a`) and postfix (`a++`). To distinguish them, overloading the postfix operator requires adding an empty dummy parameter of type `int`.

You can understand prefix increment as calling `operator++(a)` or `a.operator++()`, and postfix increment as calling `operator++(a, 0)` or `a.operator++(0)`.

Note — Example of overloading prefix and postfix increment separately

```
1 struct MyInt {
2     int x;
3
4     // prefix, corresponding to ++a
5     MyInt &operator++() {
6         x++;
7         return *this;
8     }
9
10    // postfix, corresponding to a++
11    MyInt operator++(int) {
12        MyInt tmp;
13        tmp.x = x;
14        x++;
15        return tmp;
16    }
17 };
```

One more point: among the built-in increment/decrement operators, the prefix operator returns a reference and the postfix operator returns a value. Overloaded operators need not follow this restriction, but semantically the overloaded operators are still expected to keep the same return-value types as the built-in ones.

For a type `T`, the typical overloads of the increment operator are:

Overload (using <code>++</code> as the example)	Member function	Non-member function
prefix	<code>T& T::operator++();</code>	<code>T& operator++(T& a);</code>
postfix	<code>T T::operator++(int);</code>	<code>T operator++(T& a, int);</code>

27.5 The Function Call Operator

The function call operator `()` can only be overloaded as a member function. By overloading `()` for a class, objects of that class can be called like functions.

A common application of overloading `()`: passing a struct that overloads `()` as a custom comparison function into STL containers such as priority queues.

Here is an example: given the names and scores of n students, sort by score in descending order, breaking ties by name in ascending dictionary order, and output the name and score of the top-ranked person.

The following defines a comparison struct implementing the custom ordering of a priority queue.

Note — Example of overloading the function call operator

```

1 struct student {
2     string name;
3     int score;
4 };
5
6 struct cmp {
7     bool operator()(const student& a, const student& b) const {
8         return a.score < b.score || (a.score == b.score && a.name > b.name);
9     }
10 };
11
12 // note: the template argument passed is the struct name, not an instance
13 priority_queue<student, vector<student>, cmp> pq;
```

27.6 Comparison Operators

In `std::sort` and some STL containers, the `<` operator is needed. With custom types, we must overload it by hand.

Below is an example implementing the same functionality as the previous subsection.

Note — Example of overloading comparison operators

```

1 struct student {
2     string name;
3     int score;
4
5     // overload the < operator
6     bool operator<(const student& a) const {
7         return score < a.score || (score == a.score && name > a.name);
8         // the this pointer was omitted above; the full expression is:
9         // this->score < a.score || (this->score == a.score && this->name > a.name);
10    }
11 };
12
13 priority_queue<student> pq;
```

The code above overloaded the less-than sign as a member function; of course, overloading as a non-member function also works.

Note — Overloading as a non-member function

```

1 struct student {
2     string name;
3     int score;
4 };
5
6 bool operator<(const student& a, const student& b) {
7     return a.score < b.score || (a.score == b.score && a.name > b.name);
8 }
9
10 priority_queue<student> pq;
```

In fact, once the < operator exists, the other five comparison operators can easily be implemented as well.

```

1 /* clang-format off */
2
3 // the implementations below all overload the less-than operator as a
4 // non-member function
5
6 bool operator<(const T& lhs, const T& rhs) { /* overload the < operator here */ }
7 bool operator>(const T& lhs, const T& rhs) { return rhs < lhs; }
8 bool operator<=(const T& lhs, const T& rhs) { return !(lhs > rhs); }
9 bool operator>=(const T& lhs, const T& rhs) { return !(lhs < rhs); }
10 bool operator==(const T& lhs, const T& rhs) { return !(lhs < rhs) && !(lhs > rhs); }
11 bool operator!=(const T& lhs, const T& rhs) { return !(lhs == rhs); }
```

Note — The three-way comparison operator under C++20

With C++20 or later, we can use the defaulted three-way comparison operator to simplify the code.^a

```

1 auto operator<=>(const T &lhs, const T &rhs) = default;
```

Defaulted comparison compares members one by one in the order of their declaration.^b

A custom three-way comparison can also be used. It must select an embodied ordering relation (`std::strong_ordering`, `std::weak_ordering`, or `std::partial_ordering`), or return an object such that:

- if $a < b$, then $(a <=> b) < 0$;
- if $a > b$, then $(a <=> b) > 0$;
- if a and b are equal or equivalent, then $(a <=> b) == 0$.

For implementation details, see Comparison operators #Three-way comparison on cppreference: https://zh.cppreference.com/w/cpp/language/operator_comparison.

^aComparison operators #Three-way comparison — cppreference, https://zh.cppreference.com/w/cpp/language/operator_comparison

^bDefault comparisons — cppreference, https://zh.cppreference.com/w/cpp/language/default_comparisons

References and notes:

- Operator overloading — cppreference, <https://zh.cppreference.com/w/cpp/language/operators>
- User-defined literals — cppreference, https://zh.cppreference.com/w/cpp/language/user_literal
- Comparison operators #Three-way comparison — cppreference, https://zh.cppreference.com/w/cpp/language/operator_comparison
- Default comparisons — cppreference, https://zh.cppreference.com/w/cpp/language/default_comparisons

28 References (reference)

Original title: 引用 · source page /lang/reference/

| Declares a named variable as a reference, i.e., an alias of an already-existing object or function. A reference can be seen as a non-null pointer wrapped by C++, usable to pass the object it points to; at declaration it must be bound to an object.

A reference is not an object; therefore arrays of references do not exist, pointers to references cannot be obtained, and references to references do not exist.

Note — The reference type is not an object type

If you want references to support ordinary copy, assignment, and other operations — e.g., as container elements — you need `reference_wrapper`,^a usually implemented by maintaining a non-null pointer.

^ahttps://zh.cppreference.com/w/cpp/utility/functional/reference_wrapper

References come mainly in two kinds: lvalue references and rvalue references.

Note — Lvalues and rvalues

For lvalues and rvalues, see §26.

28.1 The Lvalue Reference T&

The references we usually encounter are lvalue references, i.e., references bound to lvalues; also, a `const`-qualified lvalue reference can bind to rvalues. The following example code comes from the reference manual.⁴⁴

⁴⁴<https://zh.cppreference.com/w/cpp/language/reference>

```

1 #include <iostream>
2 #include <string>
3
4 int main() {
5     std::string s = "Ex";
6     std::string& r1 = s;
7     const std::string& r2 = s;
8
9     r1 += "ample"; // modifying r1 modifies s
10    // r2 += "!"; // error: cannot modify through a reference to const
11    std::cout << r2 << '\n'; // printing r2 accesses s, output "Example"
12 }

```

The most common use of lvalue references is function parameters, to avoid unneeded copies.

```

1 #include <iostream>
2 #include <string>
3
4 // s in the parameter list is a reference, so no copy happens at the call
5 char& char_number(std::string& s, std::size_t n) {
6     s += s; // 's' and 'str' in main() are the same object; this also
7           // shows an lvalue can stand on the right-hand side too
8     return s.at(n); // string::at() returns a reference to char
9 }
10
11 int main() {
12     std::string str = "Test";
13     char_number(str, 1) = 'a'; // the function's return is an lvalue and can be assigned
14                               // to
15     std::cout << str << '\n'; // this outputs "TastTest"
16 }

```

28.2 The Rvalue Reference T&& (C++11)

An rvalue reference is a reference bound to an rvalue, used to move objects, and it can also **extend the lifetime of a temporary object**.

```

1 #include <iostream>
2 #include <string>
3
4 using namespace std;
5
6 int main() {
7     string s1 = "Test";
8     // string&& r1 = s1; // error: cannot bind to an lvalue; std::move or
9                       // static_cast is needed
10
11     const string& r2 = s1 + s1; // OK: an lvalue reference to const extends the lifetime
12     // r2 += "Test"; // error: cannot modify through a reference to const
13     cout << r2 << '\n';
14
15     string&& r3 = s1 + s1; // OK: an rvalue reference extends the lifetime

```

```
16  r3 += "Test";
17  cout << r3 << '\n';
18
19  const string& r4 = r3; // an rvalue reference can convert to a const-qualified
    ↪ lvalue
20  cout << r4 << '\n';
21
22  string& r5 = r3; // an rvalue reference can convert to an lvalue
23  cout << r5 << '\n';
24 }
```

28.3 Dangling References

When the object a reference denotes has been destroyed, the reference becomes a dangling reference; accessing a dangling reference is undefined behavior and may crash the program.

Here are common examples of dangling references:

- A reference to a local variable

```
1  #include <iostream>
2
3  int& foo() {
4      int a = 1;
5      return a;
6  }
7
8  int main() {
9      int& b = foo();
10     std::cout << b << std::endl; // undefined behavior
11 }
```

- A dangling reference caused by deallocation

```
1  #include <iostream>
2
3  int main() {
4      int* ptr = new int(10);
5      int& ref = *ptr;
6      delete ptr;
7
8      std::cout << ref << std::endl; // undefined behavior
9  }
```

- A dangling reference caused by reallocation

```
1  #include <iostream>
2
3  int main() {
4      std::string str = "hello";
5
6      const char& ref = str.front();
```

```
7 |
8 |     str.append("world"); // may reallocate memory, freeing what ref points to
9 |
10 |     std::cout << ref << std::endl; // undefined behavior
11 | }
```

Insertion operations of containers such as `std::vector` and `std::unordered_map` may all cause memory reallocation.

When using references, always keep an eye on the lifetime of the object the reference points to, and avoid dangling references.

In general, static checking tools and good coding habits let us avoid the problem of dangling references.

28.4 Optimization Techniques Related to References

28.4.1 Removing the Copy Cost of Non-Lightweight Arguments

Common **non-lightweight objects** are:

- containers `vector`, `array`, `map`, etc.;
- `string`;
- other types that implement or inherit custom copy construction, move construction, and other special functions.

Using references for **lightweight objects** brings no benefit at all: the space taken by a reference-type parameter may even be larger than the type itself.

This may bring some performance burden, and may also block compiler optimizations.

The following are **lightweight objects**:

- basic types `int`, `float`, etc.;
- smaller aggregate types;⁴⁵
- iterators of the standard library containers.

28.4.2 Converting an Lvalue into an Rvalue

Use `std::move` to **transfer** ownership of an object. This is usually seen between local variables, or between a parameter and a local variable:

```
1 | #include <iostream>
2 | #include <string>
3 | #include <vector>
4 |
```

⁴⁵https://zh.cppreference.com/w/cpp/language/aggregate_initialization


```
5 using namespace std;
6
7 string world(string str) { return std::move(str) += " world!"; }
8
9 int main() {
10     // 1
11     cout << world("hello") << '\n';
12
13     vector<string> vec0;
14
15     // 2
16     {
17         string&& size = to_string(vec0.size());
18
19         size += ", " + to_string(size.size());
20
21         vec0.emplace_back(std::move(size));
22     }
23
24     cout << vec0.front();
25 }
```

But this is not always needed; see [function return value optimization](#).

28.4.3 Rvalues Extend the Lifetime of a Temporary

Semantically, temporaries can bring extra copies or moves; although in most cases the compiler optimizes them away via [copy elision](#), a reference can force the compiler not to perform these extra operations, avoiding uncertainty.

28.5 Reference Material

1. C++ language documentation — Reference declaration, <https://zh.cppreference.com/w/cpp/language/reference>
2. C++ language documentation — Value category, https://zh.cppreference.com/w/cpp/language/value_category
3. Does const ref lvalue to non-const func return value specifically reduce copies?, <https://stackoverflow.com/questions/38909228/does-const-ref-lvalue-to-non-const-func-return-value-specifically-reduce-copies>

29 Constants (`const`)

Original title: 常量 · source page </lang/const/>

C++ defines a complete system of read-only object definitions: variables modified by `const` are read-only objects, the compiler checks for conflicts at compile time to avoid modifying read-only objects, and it may also perform some optimizations.

Under ordinary circumstances, use `const` on variables and parameters as much as possible to improve code robustness.

29.1 The `const` Type Qualifier

29.1.1 Constants

A variable qualified by `const` cannot change its value after initialization.

```
1 const int a = 0; // the type of a is const int
2
3 // a = 1; // a constant cannot be modified
```

29.1.2 Constant References and Constant Pointers

Constant references and constant pointers both restrict modification of the pointed-to value.

```
1 int a = 0;
2 const int b = 0;
3
4 int *p1 = &a;
5 *p1 = 1;
6 const int *p2 = &a;
7 // *p2 = 2; // the variable cannot be modified through the const pointer
8 // int *p3 = &b; // an int* cannot point to a const int variable
9 const int *p4 = &b;
10
11 int &r1 = a;
12 r1 = 1;
13 const int &r2 = a;
14 // r2 = 2; // the variable cannot be modified through the const reference
15 // int &p3 = b; // an int& cannot reference a const int variable
16 const int &r4 = b;
```

In addition, one must distinguish the pointer to constant (`const t*`) from the constant pointer (`t* const`); for example, in the following declarations:

```
1 int* const p1; // constant pointer: the pointed-to address is fixed after
2 // initialization, but the pointed-to value can change
3 const int* p2; // pointer to const: the dereferenced value cannot change,
4 // but it can point to other int variables
5 const int* const p3; // const pointer to const: neither the value nor the
6 // pointed-to address can change
7
```

```

8 // using aliases improves readability
9 using const_int = const int;
10 using ptr_to_const_int = const_int*;
11 using const_ptr_to_const_int = const ptr_to_const_int;

```

Using `const` in function parameters to qualify parameter types avoids the accidental modification of variables and improves readability:

```

1 void sum(const std::vector<int> &data, int &total) {
2     for (auto iter = data.begin(); iter != data.end(); ++iter)
3         total += *iter; // iter is an iterator; after dereferencing its type is
4                         // const int
5 }

```

29.2 const Member Functions

A `const`-qualified member function of a type can be used to restrict modification of members.

```

1 #include <iostream>
2
3 struct ConstMember {
4     int s = 0;
5
6     void func() { std::cout << "General Function" << std::endl; }
7
8     void constFunc1() const { std::cout << "Const Function 1" << std::endl; }
9
10    void constFunc2(int ss) const {
11        // func(); // a const member function cannot call a non-const member function
12        constFunc1();
13
14        // s = ss; // a const member function cannot modify member variables
15    }
16 };
17
18 int main() {
19     int b = 1;
20     ConstMember c{};
21     const ConstMember d = c;
22     // d.func(); // a constant cannot call a non-const member function
23     d.constFunc2(b);
24     return 0;
25 }

```

29.3 Constant Expressions: constexpr (C++11)

A constant expression is an expression whose result can be computed at compile time; `constexpr` requires the compiler to be able to compute the value of a function or variable at compile time.

Compile-time computation allows better optimizations, e.g., hard-coding results into assembly and eliminating runtime computation costs. Unlike the optimizations that come with `const`, when a

variable qualified by `constexpr` satisfies the conditions of a constant expression, the compiler is *forced* to compute the result at compile time rather than at runtime.

Note — A more intuitive understanding: read `const` as “read-only” and `constexpr` as “immutable”

```
1 constexpr int a = 10; // define a constant directly
2
3 constexpr int FivePlus(int x) { return 5 + x; }
4
5 void test(const int x) {
6     std::array<int, x> c1;           // error: x is not known at compile time
7     std::array<int, FivePlus(6)> c2; // OK: FivePlus is known at compile time
8 }
```

The following example shows the difference between `const` and `constexpr` well: the code computes Fibonacci numbers recursively and prints them with control flow.

Note — Implementation

```
1 #include <iostream>
2
3 using namespace std;
4
5 constexpr unsigned fib0(unsigned n) {
6     return n <= 1 ? 1 : (fib0(n - 1) + fib0(n - 2));
7 }
8
9 unsigned fib1(unsigned n) { return n <= 1 ? 1 : (fib1(n - 1) + fib1(n - 2)); }
10
11 int main() {
12     constexpr auto v0 = fib0(9);
13     const auto v1 = fib1(9);
14
15     cout << v0;
16     cout << ' ';
17     cout << v1;
18 }
```

Note — Possible assembly after compilation (Compiler Explorer, Clang 19)

```
1 fib1(unsigned int):
2     push    r14
3     push    rbx
4     push    rax
5     mov     ebx, 1
6     cmp     edi, 2
7     jb     .LBB0_4
8     mov     r14d, edi
9     xor     ebx, ebx
10 .LBB0_2:
```

```

11     lea     edi, [r14 - 1]
12     call    fib1(unsigned int)
13     add     r14d, -2
14     add     ebx, eax
15     cmp     r14d, 1
16     ja      .LBB0_2
17     inc     ebx
18 .LBB0_4:
19     mov     eax, ebx
20     add     rsp, 8
21     pop     rbx
22     pop     r14
23     ret
24
25 main:
26     push    r14
27     push    rbx
28     push    rax
29     mov     edi, 9
30     call    fib1(unsigned int) # the initialization of `v1` made a function
    ↪ call
31     mov     ebx, eax
32     mov     r14, qword ptr [rip + std::__1::cout@GOTPCREL]
33     mov     rdi, r14
34     mov     esi, 55 # `v0` was replaced by the final computed result
35     call    std::__1::basic_ostream<char, std::__1::char_traits<char>>::
    ↪ operator<<(unsigned int)&PLT
36     mov     byte ptr [rsp + 7], 32
37     lea     rsi, [rsp + 7]
38     mov     edx, 1
39     mov     rdi, r14
40     call    std::__1::basic_ostream<char, std::__1::char_traits<char>>& std::__
    ↪ __1::__put_character_sequence[abi:ne2000000]<char, std::__1::char_traits<char
    ↪ >>(std::__1::basic_ostream<char, std::__1::char_traits<char>>&, char const*,
    ↪ unsigned long)
41     mov     rdi, r14
42     mov     esi, ebx # the variable's value was read
43     call    std::__1::basic_ostream<char, std::__1::char_traits<char>>::
    ↪ operator<<(unsigned int)&PLT
44     xor     eax, eax
45     add     rsp, 8
46     pop     rbx
47     pop     r14
48     ret

```

30 New C++ Features (new)

Original title: 新版C++特性 · source page </lang/new/>

Note: in view of the actual situation of algorithm contests, this article will not study the syntax exhaustively; it only covers the parts that may be applied in algorithm contests.

The syntax of this article follows the **C++11** standard. Where the semantics differ, **C++11** is taken as the standard; C++14, C++17, and other syntaxes are mentioned as appropriate and specially marked.

30.1 The auto Type Specifier

The **auto** type specifier automatically deduces the type of variables and so on. For example:

```
1 auto a = 1;           // a is of type int
2 auto b = a + 0.1;    // b is of type double
```

Note that **auto** drops references; if you do not want copy costs, specify it manually:

```
1 int a = 1;
2 int& b = a;
3 auto c = b;    // c is of type int, with a copying cost
4 auto& e = a;   // e is of type int&, with no copying cost
```

30.2 The decltype Specifier

decltype can deduce a type from an **entity** or an **expression**. Note that the two deduce types differently, and wrong use can create dangling references. It is uncommon in contests; only a rough introduction is given here.

```
1 #include <iostream>
2 #include <vector>
3
4 int main() {
5     int a = 1926;
6     decltype(a) b;           // deduced from an entity: b is of type int
7     decltype(1 + 1) c;       // deduced from an expression: c is of type int
8     decltype((a)) d = a;     // deduced from an expression: d is of type int&!
9     std::vector<decltype(b)> vec; // deduced from an entity: vec is of type
10                                // std::vector<int>
11     return 0;
12 }
```

30.3 constexpr

See also §29, Constant expressions: **constexpr** (C++11).

30.4 The Range-Based `for` Loop

Traversing an iterable object with a range-based `for` has the same efficiency as traversing with iterators. Both are generally faster than index traversal, because no addressing via an index is needed.

The following is a simple syntax of the range-based `for` loop:

```
1 for (item_declaration : range_initializer) statement
```

For example:

```
1 std::array<int, 4> arr = {1, 2, 3, 4};
2 for (int x : arr) {
3     std::cout << x << std::endl;
4 }
```

The code produced by the syntax above is equivalent to:

```
1 std::array<int, 4> arr = {1, 2, 3, 4};
2 for (auto px = arr.begin(), ed = arr.end(); px != ed; ++px) {
3     std::cout << *px << std::endl;
4 }
```

30.4.1 The item-declaration

Declares a variable that receives the elements of the container on the right; the variable's type must match the type of the container's elements. `auto` can deduce the type automatically; for complex types `auto&` is commonly used to prevent copy costs.

30.4.2 The range-initializer

The range-initializer can be any iterable object (e.g., an array, or a class object that defines `begin` and `end` member functions). If an expression is placed there, the expression is evaluated only once.

Example:

```
1 int a[] = {1, 1, 4, 5, 1, 4};
2 std::vector<int> b{1, 1, 4, 5, 1, 4};
3 std::map<std::string, int> c{{"114", 114}, {"514", 514}};
4 for (int i : a) std::cout << i;
5 for (auto i : b) std::cout << i;
6 // below, the type of i is std::pair<const std::string, int>&
7 for (auto& i : c) std::cout << i.first << i.second;
8 for (auto i : {1, 1, 4, 5, 1, 4}) std::cout << i;
```

30.4.3 Supporting Range-Based `for` for Custom Types

Just provide `begin` and `end` member functions whose return types support comparison, increment, and dereference (the `*` operator).

Here is an example:

```

1  #include <iostream>
2
3  struct C {
4      int a[4];
5
6      int* begin() { return a; }
7
8      int* end() { return a + 4; }
9  };
10
11 int main() {
12     C c = {1, 9, 2, 6};
13     for (auto i : c) std::cout << i << " ";
14     std::cout << std::endl;
15     // output: 1 9 2 6
16     return 0;
17 }

```

30.4.4 The init-statement (C++20)

In C++20 the init-statement can also be used to implement some features, e.g., a loop counter:

```

1  #include <iostream>
2  #include <vector>
3
4  int main() {
5      std::vector<int> v = {0, 1, 2, 3, 4, 5};
6
7      for (int counter = 0; auto i : v) // the init-statement (C++20)
8          std::cout << counter++ << ' ' << i << std::endl;
9  }

```

30.5 Structured Bindings (C++17)

Structured binding is a piece of syntactic sugar provided by C++17 that conveniently extracts subobjects or references to subobjects, like this:

```

1  struct C {
2      int x{1}, y{2};
3  };
4
5  int arr[] {4, 5, 6};
6
7  auto [c1, c2] = C{}; // c1=1, c2=2; of type int
8  auto& [a1, a2, a3] = arr; // a1=arr[0], a2=arr[1], a3=arr[2]; of type int&

```

Note the following:

- the number of variables declared on the left must equal the number of the right object's subobjects;

- the type declaration must use `auto`;
- `&` may be added to obtain references.

When traversing a `map` container you can write:

```
1 std::map<std::string, int> m = {"k1", 1}, {"k2", 2};
2
3 // using "auto&": no copying cost
4 for (auto& [k, v] : m) {
5     // the type of k is const std::string&, because the key carries const
6     // the type of v is int&
7     std::cout << k << ' ' << v << std::endl;
8 }
```

30.6 The `std::tuple`

A tuple⁴⁶ is defined in the header `<tuple>`; it is a generalization of `std::pair` and can store several values of different types. Consider an example:

```
1 #include <iostream>
2 #include <tuple>
3 #include <vector>
4
5 constexpr auto expr = 4 - 1; // expr = 3
6
7 int main() {
8     std::vector<int> vec = {1, 9, 2, 6, 0};
9     std::tuple<int, int, std::string, std::vector<int>> tup =
10         std::make_tuple(817, 114, "514", vec);
11
12     // use get<> to fetch subobjects; the angle brackets must contain an
13     // integral constant expression
14     for (auto i : std::get<expr>(tup)) std::cout << i << " ";
15     // the first element is numbered 0, so std::get<3> yields the
16     // std::vector<int>
17     return 0;
18 }
```

From C++17 on, structured bindings can extract the values, like this:

```
1 std::vector<int> vec = {1, 9, 2, 6, 0};
2 std::tuple<int, int, std::string, std::vector<int>> tup =
3     std::make_tuple(817, 114, "514", vec);
4
5 auto& [a, b, c, d] = tup; // C++17 Structured binding
6 std::cout << a << ' ' << b << c << std::endl;
7 std::cout << d.size() << ' ' << d[2] << std::endl;
```

⁴⁶<https://zh.cppreference.com/w/cpp/utility/tuple>

30.6.1 Member Functions

Function	Effect
<code>operator=</code>	assigns the contents of one <code>tuple</code> to another
<code>swap</code>	swaps the contents of two <code>tuples</code>

Example:

```

1 constexpr std::tuple<int, int> tup = {1, 2};
2 std::tuple<int, int> tupA = {2, 3}, tupB;
3 tupB = tup;
4 tupB.swap(tupA);

```

30.6.2 Non-Member Functions

Function	Effect
<code>make_tuple</code>	creates a <code>tuple</code> object whose types are defined by the argument types
<code>std::get</code>	tuple-style access of the specified element
<code>std::tie</code>	assigns the values in the tuple to existing variables
<code>operator==</code> etc.	compares the values in the <code>tuple</code> lexicographically
<code>std::swap</code>	the specialized <code>std::swap</code> algorithm

Example:

```

1 std::tuple<int, int> tupA = {2, 3}, tupB;
2 tupB = std::make_tuple(1, 2);
3 std::swap(tupA, tupB);
4 std::cout << std::get<1>(tupA) << std::endl;
5 int x;
6 std::tie(x, std::ignore) = tupB;
7 std::cout << x << std::endl;

```

`std::tie` assigns tuple elements to existing variables; `std::ignore` can skip unneeded elements. Structured bindings declare new variables directly (value/reference binding supported) and must accept all elements.

30.7 Function Objects

An object on which the function call operator `operator()` can be used is called a function object (FunctionObject).

It is not a language feature but a concept/requirement,⁴⁷ widely used in the standard library.

Function objects fall roughly into two classes:

1. function pointers;

⁴⁷https://zh.cppreference.com/w/cpp/named_req/FunctionObject

2. class objects that overload the `operator()` operator.

A `lambda` is a typical function object of the second class: it stores the captured content in member variables and overloads the function call operator.

30.8 Lambda Expressions

See the [Lambda Expressions](#) page.

30.9 `std::function`

Warning — Mind the performance cost

`std::function` introduces a certain performance cost. Benchmarks (see the recursion in [lambdas](#) section of §31) usually show a performance loss of 2 to 3 times or more.

Because it uses type erasure, typically implemented via the virtual-function mechanism, calling a virtual function brings extra overhead.^a

Consider using [lambda expressions](#) or [function objects](#) (§30) instead.

^a<https://stackoverflow.com/questions/5057382/what-is-the-performance-overhead-of-std-function>

`std::function` is a general-purpose function wrapper, defined in the header `<functional>`.

An instance of `std::function` can store, copy, and call any [Callable](#)⁴⁸ object, including [lambda expressions](#), member function pointers, or other [function objects](#).

If a `std::function` holds no callable object (e.g., default-constructed), calling it throws the `std::bad_function_call`⁴⁹ exception.

```

1  #include <functional>
2  #include <iostream>
3
4  struct Foo {
5      Foo(int num) : num_(num) {}
6
7      void print_add(int i) const { std::cout << num_ + i << '\n'; }
8
9      int num_;
10 };
11
12 void print_num(int i) { std::cout << i << '\n'; }
13
14 struct PrintNum {
15     void operator()(int i) const { std::cout << i << '\n'; }
16 };
17
18 int main() {
19     // store a free function
20     std::function<void(int)> f_display = print_num;
```

⁴⁸https://zh.cppreference.com/w/cpp/named_req/Callable

⁴⁹https://zh.cppreference.com/w/cpp/utility/functional/bad_function_call

```

21 f_display(-9);
22
23 // store a lambda
24 std::function<void()> f_display_42 = []() { print_num(42); };
25 f_display_42();
26
27 // store a call to a member function
28 std::function<void(const Foo&, int)> f_add_display = &Foo::print_add;
29 const Foo foo(314159);
30 f_add_display(foo, 1);
31 f_add_display(314159, 1);
32
33 // store a call to a data member accessor
34 std::function<int(Foo const&)> f_num = &Foo::num_;
35 std::cout << "num_: " << f_num(foo) << '\n';
36
37 // store a call to a function object
38 std::function<void(int)> f_display_obj = PrintNum();
39 f_display_obj(18);
40 }

```

30.10 Variadic Function Templates

Before C++11, class templates and function templates could only accept a fixed number of template arguments. C++11 allows **any number, of any types**.

Here we briefly introduce variadic **function** templates.

The function template **fun** declared below can accept any number of template arguments of any types as its template parameters:

```

1 template <typename...Clazz>
2 void fun(Clazz... paras) {}

```

paras is a function parameter pack, accepting 0 or more function arguments. **Clazz** is a template parameter pack, accepting 0 or more template arguments (non-type, type, or template); marked with **typename**, it only accepts types.

A simple way to understand this:

- a template parameter pack is usually a list of type names (though compile-time constants or template names may also be used);
- a function parameter pack is usually a list of variable names.

Now the **fun** function can be called like this:

```

1 fun();
2 fun(1);
3 fun(1, 2, 3);
4 fun(1, 0.0, "abc");

```

30.10.1 Parameter Pack Expansion

Expansion syntax. Pack expansion is very simple: use `...`, and the items will be separated by `,` automatically. For example:

```
1 template <class A, class... C>
2 void func(A arg1, C... arg2) {
3     // C is a template parameter pack
4     tuple<A, C...>(); // expands to tuple<int, int, double, bool>();
5
6     // arg2 is a function parameter pack
7     func(arg2...); // expands to func( 2, 1.1, true );
8 }
9
10 func(1, 2, 1.1, true);
```

The expansion can also carry operations, e.g.:

```
1 template <class A, class... C>
2 void func(A arg1, C... arg2) {
3     func((arg2 + 1)...);
4     // expands to func( (2+1) , (1.1+1), (2.1f+1) );
5 }
6
7 func(1, 2, 1.1, 2.1f);
```

The terminating function. The function above cannot run, because the number of parameters keeps shrinking until an empty call errors out.

We need to specify a terminating condition; we can provide an ordinary function, like this:

```
1 void func() {}
2
3 template <class A, class... C>
4 void func(A arg1, C... arg2) {
5     std::cout << arg1 << std::endl;
6     func((arg2 + 1)...);
7 }
8
9 func(1, 2, 1.1, 2.1f);
```

This way, a non-zero parameter count calls the template and an empty call calls the ordinary function, and everything runs.

30.10.2 Fold Expressions (C++17)

C++17 provides a convenient syntax for handling **function parameter packs**; its syntax is as follows (must be wrapped in parentheses):

1. `( pack op ... )` becomes `(E1 op (... op (EN-1 op EN)))`
2. `( ... op pack )` becomes `((E1 op E2) op ...) op EN`

3. (pack op ... op init) becomes (E1 op (... op (EN-1 op (EN op I))))
4. (init op ... op pack) becomes (((I op E1) op E2) op ...) op EN)

A simple demonstration makes it clear:

```

1  template <class... C>
2  void func(C... args) {
3      (std::cout << ... << args) << std::endl;
4      // syntax 4, equivalent to ↓
5      // ( ( ( std::cout << 1 ) << 2.1 ) << true ) << std::endl;
6      // output: 12.11  note that true is printed as 1, because boolalpha was
7      // not specified
8
9      std::cout << (args && ...) << std::endl;
10     // syntax 1, equivalent to ↓
11     // std::cout << ( 1 && ( 2.1 && true ) ) << std::endl;
12     // output: 1
13 }
14
15 func(1, 2.1, true);

```

30.10.3 Abbreviated Function Templates (C++20)

From C++20 on, `auto ...` can be used directly as parameter types to abbreviate a function template:

```

1  void func(auto... args) { (std::cout << ... << args) << std::endl; }

```

Note that it is still essentially a function template, equivalent to:

```

1  template <class... T>
2  void func(T... args) {
3      (std::cout << ... << args) << std::endl;
4  }

```

30.11 The Ranges Library (C++20)

The ranges library is an extension of the iterator and generic-algorithm libraries, making iterators and algorithms more powerful through composition, and reducing errors.

A range is a traversable sequence: arrays, containers, views, and so on.

When complex operations on containers and other ranges are needed, the ranges library⁵⁰ makes writing algorithms easier and clearer.

⁵⁰<https://zh.cppreference.com/w/cpp/ranges>

30.11.1 Views

A view is a lightweight object that implements certain algorithms through specific mechanisms (such as custom iterators), giving ranges more ways of traversal to satisfy needs.

The ranges library implements some common views, roughly of two kinds:

1. **Range factories**, used to construct special ranges; using such a factory skips the step of manually constructing a container, lowering the cost and directly generating a range.
2. **Range adapters**, providing rich traversal support; they can be called like functions or chained through the pipe operator `|`.

A **range adapter**, being a **range adaptor closure object**,⁵¹ is also a **function object**; they overload **operator|** so that they can be assembled like pipelines.

Note — The pipe operator

The `|` here should be understood as a pipe operator, not bitwise OR; this usage comes from the pipes of Linux.^a

^a[https://zh.wikipedia.org/wiki/Pipeline_\(Unix\)](https://zh.wikipedia.org/wiki/Pipeline_(Unix))

Under complex operations good readability is maintained, with the following property:

If A, B, C are some range adaptor closure objects, R is some range, and the other letters are possibly valid arguments, the expression

$$R \mid A(a) \mid B(b) \mid C(c, d)$$

is equivalent to

$$C(B(A(R, a), b), c, d)$$

Take `ranges::take_view` and `ranges::iota_view` as an example:

```
1 #include <iostream>
2 #include <ranges>
3
4 int main() {
5     const auto even = [](int i) { return 0 == i % 2; };
6
7     for (int i : std::views::iota(0, 6) | std::views::filter(even))
8         std::cout << i << ' ';
9 }
```

1. the range factory `std::views::iota(0, 6)` generates the range of the integer sequence from 0 to 5;
2. the range adapter `std::views::filter(even)` filters the previous range, generating a range with only even numbers left;

⁵¹https://zh.cppreference.com/w/cpp/named_req/RangeAdaptorClosureObject

- the two operations are chained with the pipe operator.

The code above needs no extra heap allocation to store the range generated at each step; the actual generating and filtering happen during traversal (more concretely, the internal iterator's construction, increment, and dereference), i.e., it is zero overhead.

Meanwhile, the lifetime of the externally input range equals the lifetime of the internal elements of the **range adapter**. If the external range (e.g., a container or range factory) has been destroyed, traversing such a view again has the same effect as dereferencing a dangling pointer — undefined behavior.

To avoid the above, strictly require that the adapter's lifetime lies within the lifetimes of all ranges it uses.

Note — When the range is destroyed, all elements in the view dangle

```

1  #include <iostream>
2  #include <ranges>
3  #include <vector>
4
5  using namespace std;
6
7  int main() {
8      auto view = [] {
9          vector<int> vec{1, 2, 3, 4, 5};
10         return vec | std::views::filter([](int i) { return 0 == i % 2; });
11     }();
12
13     for (int i : view) cout << i << ' '; // runtime undefined behavior
14
15     return 0;
16 }
```

30.11.2 Constrained Algorithms

C++20 provides constrained versions of most algorithms in the namespace `std::ranges`; ranges can be specified by an iterator–sentinel pair or a single range argument, and projections and invocables pointing to members are supported. Also, the return types of most algorithms were changed to return all potentially useful information computed during the algorithm's execution. These algorithms can be understood as improved versions of the old standard-library algorithms; they are all function objects, with friendlier overloads and argument type checking (based on concepts⁵²). Let us start with a comparison of `std::sort` and `ranges::sort`:

```

1  #include <algorithm>
2  #include <iostream>
3  #include <vector>
4
5  using namespace std;
6
```

⁵²<https://zh.cppreference.com/w/cpp/language/constraints>


```

7 int main() {
8     vector<int> vec{4, 2, 5, 3, 1};
9
10    sort(vec.begin(), vec.end()); // {1, 2, 3, 4, 5}
11
12    for (const int i : vec) cout << i << ", ";
13    cout << '\n';
14
15    ranges::sort(vec, ranges::greater{}); // {5, 4, 3, 2, 1}
16
17    for (const int i : vec) cout << i << ", ";
18
19    return 0;
20 }

```

`ranges::sort` implements the same algorithm as `sort` but provides range-based overloads, making argument passing more concise. Most other algorithms of the `std` namespace also have corresponding range-overloaded versions in the `ranges` namespace.

Using these range arguments, combined with the views of the previous section, lets us perform complex operations while keeping the code readable. Consider an example:

```

1 #include <algorithm>
2 #include <array>
3 #include <iostream>
4 #include <ranges>
5
6 using namespace std;
7
8 int main() {
9     const auto& inputs = views::iota(0u, 9u); // produce the integer sequence 0 to 8
10    const auto& chunks = inputs | views::chunk(3); // split the sequence into
11                                                    // chunks of 3 elements
12
13    const auto& cartesian_product =
14        views::cartesian_product(chunks, chunks); // take the Cartesian product
15                                                    // of the chunks with themselves
16
17    for (const auto [l_chunk, r_chunk] : cartesian_product)
18        // compute the sum of the two chunk integers under the Cartesian product
19        cout << ranges::fold_left(l_chunk, 0u, plus{}) +
20              ranges::fold_left(r_chunk, 0u, plus{})
21              << ' ';
22 }

```

Note — Output

6 15 24 15 24 33 24 33 42

30.12 References

- C++ reference manual (Chinese), <https://zh.cppreference.com/>

31 Lambda Expressions (lambda)

Original title: Lambda表达式 · source page /lang/lambda/

Note: in view of the actual situation of algorithm contests, this article will not study the syntax exhaustively; it only covers the parts that may be applied in algorithm contests.

The syntax of this article follows the **C++11** standard; syntax from higher standards is mentioned as appropriate and specially marked.

31.1 Lambda Expressions

Lambda expressions are named after the λ calculus in mathematics, corresponding directly to the lambda abstraction there. At compile time the compiler generates an anonymous **function object** (§30) according to the syntax, with the captured variables as its members and the parameters and function body implementing the **operator()** overload.

Note — Function objects

A function object is a class object generally implemented by overloading **operator()**, so it can be called like a function. Compared with ordinary functions, function objects have many advantages: they can hold state, can be passed as arguments to other functions, and so on.

One syntax of a lambda is:

```
1 [capture] (parameters) mutable -> return-type {statement}
```

A lambda expression is itself a class; expanded, it takes the following form:

```
1 class Lambda_1 {
2     private:
3         Lambda_1() : capture-list(init-value) { }
4
5     public:
6         return-type operator()(parameters) const { statement }
7
8     private:
9         mutable capture-list
10 };
```

A lambda with an empty capture list converts implicitly to a function pointer, e.g.:

```
1 void (*f)(int, int) = [](int, int) -> void {};
```

Below we introduce each part of the syntax.

31.1.1 The statement (Function Body)

The body of a lambda is similar to the body of an ordinary function: besides accessing parameters and global variables, it can also access the [captured](#) variables.

31.1.2 The capture Clause

A lambda begins with the capture clause, which specifies which variables are captured. The capture list may be empty or specify the capture method: variables with a `&` prefix are accessed by [reference](#), and those without it are accessed by value.

We can also use the default capture mode, capturing all variables mentioned in the lambda: `&` means all captured variables are accessed by reference, `=` means all captured variables are accessed by value.

After a default capture, the capture mode of specific variables can still be specified **explicitly**.

If we need to access the external variable `a` by reference and the external variable `b` by value, all of the following capture clauses do it:

- `[&a, b]`
- `[b, &a]`
- `[&, b]`
- `[b, &]`
- `[=, &a]`

The capture list can also be used to declare variables whose types are deduced from the initializers, similar to declaring variables with `auto`.

Some common examples:

```

1  int a = 0;
2  auto f0 = []() { return a * 9; }; // Error: 'a' is inaccessible
3  auto f1 = [a]() { return a * 9; }; // OK: 'a' is captured by value
4  auto f2 = [&a]() { return a++; }; // OK: 'a' is captured by reference
5  auto f3 = [v = a + 1]() {
6      return v + 1;
7  }; // OK: declares variable v with an initializer, of the same type as a
8
9  // note: when capturing by reference, make sure a is not destroyed by the
10 // time the lambda is called
11 auto b = f2(); // f2 gets a's value from its capture list; no need to pass
12              // a as a parameter

```

Generalized capture with initializers (C++14). From C++14 on, the capture list can not only capture external variables but also declare and initialize new variables, e.g.:

```

1  auto f1 = [val = 520]() {
2      return val;

```

```

3 }; // OK: val is of type int, initialized to 520; return type int
4
5 auto f2 = [val = 520LL]() {
6     return val;
7 }; // OK: val is of type long long, initialized to 520; return type
8     // long long
9
10 auto f3 = [val = "520"]() {
11     return val;
12 }; // OK: val is of type const char*, initialized to "520"; return type
13     // const char*
14
15 auto f4 = [val = "520"s]() {
16     return val;
17 }; // OK: since C++14 (requires using namespace std; or
18     // using namespace std::literals;); val is of type std::string,
19     // initialized to std::string("520"); return type
20     // std::string
21
22 auto f5 = [val = std::string("520")]() {
23     return val;
24 }; // OK: val is of type std::string, initialized to std::string("520");
25     // return type
26     // std::string
27
28 auto f6 = [val = std::vector<int>(3, 6)]() {
29     return val;
30 }; // OK: val is of type std::vector<int>, of size 3, filled with 6;
31     // return type
32     // std::vector<int>
33
34 auto f7 = [val = 520]() -> int {
35     return val;
36 }; // OK: val is of type int, initialized to 520; return type int
37
38 auto f8 = [val = 520]() -> long long {
39     return val;
40 }; // OK: val is of type int, initialized to 520; return type long long

```

When defining a new variable the initial value cannot be omitted; the variable's type is determined by the initial value's type, equivalent to:

```
1 auto val = init-value;
```

The following is a wrong spelling:

```

1 auto f = [val]() { return val; }; // Error: 'val' was not declared in this
2                                     // scope, identifier "val" is undefined

```

The initial value may also be an external variable, e.g.:

```

1 int value = 520;
2 auto f = [val = value]() { return val; };
3 std::cout << f(); // Output: 520

```

`val` may also be a reference type, referencing an external variable; this way one can give an alias to an external variable captured by reference, e.g.:

```

1  int value = 520;
2
3  auto f = [&val = value]() {
4      return val;
5  }; // OK: val is of type int&, return type int; equivalent to
6      // int& val = value;
7
8  std::cout << f() << '\n'; // Output: 520
9
10 value = 1314;
11
12 std::cout << f() << '\n'; // Output: 1314

```

Capturing external variables and defining new variables can be used at the same time.

If you want to modify a new variable defined in the capture inside the lambda body, you need the `mutable` keyword; for references it is not needed, e.g.:

```

1  int value = 520;
2
3  {
4      auto f = [val = value]() mutable -> int {
5          return val = 1314;
6      }; // mutable is needed
7      auto val_f = f();
8      std::cout << value << ' ' << val_f << std::endl; // Output: 520 1314
9  }
10
11 {
12     auto f = [&val = value]() -> int { return val = 1314; }; // mutable is not
13                                                             // needed
14     auto val_f = f();
15     std::cout << value << ' ' << val_f << std::endl; // Output: 1314 1314
16 }

```

See the mutable specifier below for details.

The lifetime of a variable defined in the capture follows the receiver of the lambda expression — in the examples above, the variable `f` — because the lambda is really a class, and everything in the capture is a `private` member variable of this class; e.g.:

```

1  int main() {
2      auto f = [val = 0]() mutable -> int { return ++val; }; // val is constructed
3                                                             // and initialized
4
5      std::cout << f() << '\n'; // Output: 1
6      std::cout << f() << '\n'; // Output: 2
7      std::cout << f() << '\n'; // Output: 3
8  } // val is destroyed together with f

```

31.1.3 The parameters (Parameter List)

In most cases it is like a function's parameter list, e.g.:

```
1 int x[] = {5, 1, 7, 6, 1, 4, 2};
2 std::sort(x, x + 7, [](int a, int b) { return (a > b); });
3 for (auto i : x) std::cout << i << " ";
```

This prints the array `x` sorted from large to small.

Since the **parameter list** is optional: if no parameters are passed to the lambda, and its declaration contains neither **mutable** nor a trailing return type, the empty parentheses may be omitted.

Note — Parameters declared with auto

From C++14 on, if a parameter's type is declared with **auto**, a **generic lambda** is constructed.

Explicit object parameter (C++23). From C++23 on, explicit object parameters⁵³ may be used in a lambda's parameters.

```
1 auto nth_fibonacci = [](this auto self, unsigned n) -> unsigned {
2     return n < 2 ? n : self(n - 1) + self(n - 2);
3 };
4
5 cout << nth_fibonacci(10u);
```

Note — The difference between `auto self`, `auto& self`, and `auto&& self`

In theory both `auto& self` and `auto&& self` use only 8 bytes (the pointer size) for the argument, with no other copies; the concrete behavior depends on how the compiler implements the lambda and the corresponding optimizations.

Using `auto self` causes a copy of the object, whose size depends on the elements in the capture list, because they are all private member variables of this lambda class.

31.1.4 The mutable Specifier

Lets the function body modify the variables captured by value.

```
1 int a = 0;
2 auto by_value = [a]() mutable { ++a; };
3 auto by_ref = [&a] { ++a; };
4
5 by_value();
6 by_ref();
```

After `by_value()` runs, `by_value`'s captured member `a` is 1, but the external variable `a` is still 0. After `by_ref()` runs, the external `a` becomes 1.

⁵³<https://zh.cppreference.com/w/cpp/language/function>

31.1.5 The return-type

Specifies the return type of the lambda. If omitted, the return type is deduced automatically (the same behavior as a function whose return value is declared with `auto`).

Multiple `return` statements with inconsistent deduced types produce a compilation error.

```
1 auto lam = [](int a, int b) -> int { return 0; };
2
3 auto x1 = [](int i) { return i; };
4
5 auto x2 = [](bool condition) {
6     if (condition) return 1;
7     return 1.0;
8 }; // Error: the deduced types are inconsistent
```

31.1.6 Generic Lambdas (C++14)

Using `auto` as a parameter type, one can construct a generic lambda.

```
1 auto add = [](auto a, auto b) { return a + b; };
```

In cpp insights⁵⁴ you can observe the lambda class definition the compiler generates:

```
1 class add_lambda {
2 public:
3     template <class T, class U>
4     auto operator()(T a, U b) const {
5         return a + b;
6     }
7 };
8
9 add_lambda add{};
```

Both parameter declarations of `add` use `auto`, corresponding to the two template parameters `T` and `U` of the `operator()` function template of the `add_lambda` class.

31.1.7 Recursion in Lambdas

First, a failing example:

```
1 int n = 10;
2
3 auto dfs = [&](int i) -> void {
4     if (i == n)
5         return;
6     else
7         dfs(i + 1); // Error: a variable declared with an auto type specifier
8                     // cannot appear in its own initializer
9 };
```

⁵⁴<https://cppinsights.io>

Here we try to capture *dfs* in the capture list, but there is a problem: the type of *dfs* is `auto`, and it is deduced only after the right-hand side's type deduction completes; for the lambda to capture *dfs*, the type of *dfs* must be fixed before its reference variable can be created — a nested-doll process.

How to solve it?

1. Specify the type of *dfs* explicitly; `std::function` can be used.

Example — Change the code above to

```
1  int n = 10;
2
3  std::function<void(int)> dfs = [&](int i) -> void {
4      if (i == n)
5          return;
6      else
7          dfs(i + 1); // OK
8  };
9
10 dfs(1);
```

Warning — Recursion via `std::function` is not recommended

The type erasure of `std::function` usually requires extra memory allocation, and the indirection of the call lowers performance further.

In a benchmark,^{[a](#)} with Clang 17 and libc++, the `std::function` implementation was about 2.5 times slower than the lambda implementation of recursion.

Note — Test code

```

1  #include <algorithm>
2  #include <functional>
3  #include <numeric>
4  #include <random>
5
6  using namespace std;
7
8  const auto& nums = [] {
9      random_device rd;
10     mt19937 gen{rd()};
11     array<unsigned, 32> arr{};
12
13     std::iota(arr.begin(), arr.end(), 0u);
14     ranges::shuffle(arr, gen);
15
16     return arr;
17 }();
18
19 static void std_function_fib(benchmark::State& state) {
20     std::function<int(int)> fib;
21
22     fib = [&](int n) { return n <= 2 ? 1 : fib(n - 1) + fib(n - 2); };
23
24     unsigned i = 0;
25
26     for (auto _ : state) {
27         auto res = fib(nums[i]);
28         benchmark::DoNotOptimize(res);
29
30         ++i;
31
32         if (i == nums.size()) i = 0;
33     }
34 }
35
36 BENCHMARK(std_function_fib);
37
38 static void template_lambda_fib(benchmark::State& state) {
39     auto n_fibonacci = [](const auto& self, int n) -> int {
40         return n <= 2 ? 1 : self(self, n - 1) + self(self, n - 2);
41     };
42
43     unsigned i = 0;
44
45     for (auto _ : state) {
46         auto res = n_fibonacci(n_fibonacci, nums[i]);
47         benchmark::DoNotOptimize(res);
48
49         ++i;
50
51         if (i == nums.size()) i = 0;
52     }
53 }
54
55 BENCHMARK(template_lambda_fib);

```

^ahttps://quick-bench.com/q/U5qf_dHHKsSyVU83jmt0p_U541c

- Do not obtain *dfs* via capture; pass it as a function argument instead.

Example — Change the code above to

```

1  int n = 10;
2
3  // if a parameter in the parameter list has type auto, the operator()
4  // function of this lambda class will be defined as a template function; a
5  // template function can be instantiated later, when it is called
6  auto dfs = [&](auto& self,
7              int i) -> void // [&] captures only used variables, so the
8                          ↪ auto dfs                                // parameter is not captured
9  {
10     if (i == n)
11         return;
12     else
13         self(self, i + 1); // OK
14 };
15
16 dfs(dfs, 1);

```

Note — The difference between `auto self`, `auto& self`, and `auto&& self`

In theory both `auto& self` and `auto&& self` use only 8 bytes (the pointer size) for the argument, with no other copies; the concrete behavior depends on how the compiler implements the lambda and on the corresponding optimizations. Using `auto self` causes a copy of the object, whose size depends on the elements of the capture list, because they are all private member variables of this lambda class.

- You can also hand-expand the lambda class, or use a similar spelling, so that the type of *dfs* can be declared directly.

Example — Change the code above to

```

1  int n = 10;
2
3  class Lambda_1 {
4  public:
5      auto operator()(int i) const -> void {
6          if (i == n)
7              return;
8          else
9              (*this)(i + 1); // OK
10     }
11
12     explicit Lambda_1(int& __n) : n(__n) {}
13

```

```

14 private:
15     int& n;
16 } dfs(n);
17
18 dfs(1);

```

4. If the lambda captures nothing, we can also use a function pointer.

If a lambda captures nothing, it converts implicitly to a function pointer. The lambda can also be declared `static`, and the function pointer type can be declared `static` as well. In that way, the lambda can access the function pointer without capturing anything, achieving recursion.

Example

```

1 static unsigned (*fptr)(unsigned);
2
3 static const auto lambda = [](const unsigned a) {
4     return a < 2 ? a : (*fptr)(a - 2) + (*fptr)(a - 1);
5 };
6
7 static auto init = [] {
8     fptr = +lambda;
9     // Or
10    // fptr = static_cast<unsigned (*)>(unsigned)>(lambda);
11    return 0;
12 }();
13
14 cout << lambda(10);

```

31.2 Applications of Lambda Expressions

31.2.1 As a Predicate for Standard-Library Algorithms

Sorting from large to small:

```

1 std::vector<int> v = {1, 2, 3, 4, 5};
2 std::sort(v.begin(), v.end(), [](int a, int b) { return a > b; });

```

Using `std::find_if`⁵⁵ to find the first element greater than 3:

```

1 std::vector<int> v = {1, 2, 3, 4, 5};
2 auto it = std::find_if(v.begin(), v.end(), [](int a) { return a > 3; });

```

⁵⁵<https://zh.cppreference.com/w/cpp/algorithm/find>

31.2.2 Controlling the Lifetime of Intermediate Variables

In algorithm contests we encounter this scenario: initializing a variable requires previously declared variables, and the initialization process generates intermediate variables that take up much space.

We want to destruct these intermediate variables as soon as possible to lower memory consumption. At such times, a lambda can control the lifetime of these intermediate variables.

```
1 void solution(const vector<int>& input) {  
2     int b = [8] {  
3         vector<int> large_objects(input.size());  
4         int c = 0;  
5  
6         for (int i = 0; i < large_objects.size(); ++i)  
7             large_objects[i] = i + input[i];  
8  
9         for (int i = 0; i < input.size(); ++i) c += large_objects[input[i]];  
10  
11     return c;  
12 }();  
13  
14 // ...  
15 }
```

Compared with using a block scope, a lambda lets us use a return value, making the code more concise; compared with a function, we need no extra naming or declaration of the various captured parameters, making the code more compact.

31.3 References

- cplusplus — lambda, <https://en.cppreference.com/w/cpp/language/lambda>
- Stackoverflow: Overhead with `std::function`, <https://stackoverflow.com/a/33881130/11120338>

PART V

Policy-Based Data Structures (pb_ds)

32 Introduction to pb_ds (pb-ds)

Original title: pb_ds简介 · source page /lang/pb-ds/

The **pb_ds** library is short for Policy-Based Data Structures.

The **pb_ds** library wraps many data structures, such as hash tables, balanced binary trees, tries, and heaps (priority queues).

Just like **vector**, **set**, and **map**, its components all conform to the relevant STL interface specifications. Some of them (e.g., the priority queue) include all the functionality of the corresponding STL component, but with more features than the STL's.

pb_ds is only available with compilers that use libstdc++ as the standard library.

You can use **begin()** and **end()** to obtain iterators and traverse the structures.

You can use **increase_key**, **decrease_key**, and delete single elements.

Because the main content of the **pb_ds** library lives in the underscore-initial namespace **__gnu_pbds**, its compliance in NOI-series events was long undetermined. On September 1, 2021, according to the “Supplementary Remarks on the Restrictions on Programming Languages in NOI-Series Events”,⁵⁶ library functions or macros starting with an underscore are allowed (except for those with explicitly forbidden operations), giving documentary grounds for using the **pb_ds** library in NOI-series events.

Reference: “Applications of the C++ pb_ds library in OI”⁵⁷

⁵⁶<https://www.noi.cn/xw/2021-09-01/735729.shtml>

⁵⁷<https://github.com/OI-Wiki/libs/tree/master/lang/pb-ds> — the PDF (C++的pb_ds库在OI中的应用.pdf, in Chinese) lives in this directory.

33 Priority Queues (pb-ds/pq)

Original title: 堆 · source page /lang/pb-ds-pq/

33.1 `__gnu_pbds::priority_queue`

Appendix: official documentation — complexity and constant tests.⁵⁸

```
1 #include <ext/pb_ds/priority_queue.hpp>
2 using namespace __gnu_pbds;
3 __gnu_pbds::priority_queue<T, Compare, Tag, Allocator>
```

33.2 Template Parameters

- **T**: the type of the stored elements.
- **Compare**: a type providing strict weak ordering comparisons.
- **Tag**: selects among the five different heaps `__gnu_pbds` provides; the Tag parameter defaults to `pairing_heap_tag`. The five are:
 - **pairing_heap_tag**: the pairing heap. The official documentation holds that among non-native elements (such as custom structs/`std::string/pair`), the pairing heap performs best.
 - **binary_heap_tag**: the binary heap. The official documentation holds that the binary heap performs best on native elements, though the author's tests were not that good.
 - **binomial_heap_tag**: the binomial heap. The binomial heap beats the binary heap at merging, but its pop-the-top operation has higher complexity than the binary heap's.
 - **rc_binomial_heap_tag**: the redundancy-counter binomial heap.
 - **thin_heap_tag**: a tag whose complexities, except for merging, are all the same as the Fibonacci heap's.
- **Allocator**: the allocator; it rarely appears in OI, so it is not explained here.

Since this article is aimed at students studying algorithm contests, only the complexities of the last four tags are briefly introduced; the first one gets a full introduction to its member functions and usage.

Based on the author's local tests of basic heap operations (Core i5 @3.1 GHz on macOS), combined with GNU's official complexity tests and Dijkstra tests: at least for OIers, the four tags other than the pairing heap are of little use — either useless, or with constants so large they lose to the `std` ones, possibly even causing MLE. So only the default pairing heap is recommended here. Likewise, the pairing heap also beats `make_heap()` of the `algorithm` library.

⁵⁸https://gcc.gnu.org/onlinedocs/libstdc++/ext/pb_ds/pq_performance_tests.html#std_mod1

33.3 Construction

The namespace must be given because the class name clashes with `std`'s.

```
1 // __gnu_pbds::priority_queue<int>;
2 // __gnu_pbds::priority_queue<int, greater<int>>;
3 // __gnu_pbds::priority_queue<int, greater<int>, pairing_heap_tag>;
4 __gnu_pbds::priority_queue<int>::point_iterator id; // a point-type iterator
5 // both modify and push return a point_iterator; its usage is explained in
6 // detail below
7 id = q.push(1);
```

33.4 Member Functions

- `push()`: pushes an element into the heap, returning an iterator to that element's position.
- `pop()`: pops the top element of the heap.
- `top()`: returns the top element of the heap.
- `size()`: returns the number of elements.
- `empty()`: returns whether the heap is non-empty.
- `modify(point_iterator, const key)`: changes the `key` at the iterator's position to the given `key` and re-sorts the underlying storage structure.
- `erase(point_iterator)`: erases the key at the iterator's position from the heap.
- `join(__gnu_pbds::priority_queue &other)`: merges `other` into `*this` and empties `other`.

The tag used determines the time complexity of each operation:

	push	pop	modify
<code>pairing_heap_tag</code>	$O(1)$	worst $\Theta(n)$; amortized $\Theta(\log n)$	worst $\Theta(n)$; amortized $\Theta(\log n)$
<code>binary_heap_tag</code>	worst $\Theta(n)$; amortized $\Theta(\log n)$	worst $\Theta(n)$; amortized $\Theta(\log n)$	$\Theta(n)$
<code>binomial_heap_tag</code>	worst $\Theta(\log n)$; amortized $O(1)$	$\Theta(\log n)$	$\Theta(\log n)$
<code>rc_binomial_heap_tag</code>	$O(1)$	$\Theta(\log n)$	$\Theta(\log n)$
<code>thin_heap_tag</code>	$O(1)$	worst $\Theta(n)$; amortized $\Theta(\log n)$	worst $\Theta(\log n)$; amortized $\Theta(\log n)$

33.5 Example

```
1 #include <algorithm>
2 #include <cstdio>
3 #include <ext/pb_ds/priority_queue.hpp>
4 #include <iostream>
5 using namespace __gnu_pbds;
6 // aimed at OIers, this article uses the common heap, pairing_heap_tag, as
7 // the example
8 // for better readability, define the following aliases:
```

```

9  using pair_heap = __gnu_pbds::priority_queue<int>;
10 pair_heap q1; // a max-heap; pairing heap
11 pair_heap q2;
12 pair_heap::point_iterator id; // an iterator
13
14 int main() {
15     id = q1.push(1);
16     // elements in the heap : [1];
17     for (int i = 2; i <= 5; i++) q1.push(i);
18     // elements in the heap : [1, 2, 3, 4, 5];
19     std::cout << q1.top() << std::endl;
20     // output : 5;
21     q1.pop();
22     // elements in the heap : [1, 2, 3, 4];
23     id = q1.push(10);
24     // elements in the heap : [1, 2, 3, 4, 10];
25     q1.modify(id, 1);
26     // elements in the heap : [1, 1, 2, 3, 4];
27     std::cout << q1.top() << std::endl;
28     // output : 4;
29     q1.pop();
30     // elements in the heap : [1, 1, 2, 3];
31     id = q1.push(7);
32     // elements in the heap : [1, 1, 2, 3, 7];
33     q1.erase(id);
34     // elements in the heap : [1, 1, 2, 3];
35     q2.push(1), q2.push(3), q2.push(5);
36     // elements of q1 : [1, 1, 2, 3]; elements of q2 : [1, 3, 5];
37     q2.join(q1);
38     // q1 has no elements; elements of q2 : [1, 1, 1, 2, 3, 3, 5];
39 }

```

33.6 The Invalidation Guarantees of Iterators in __gnu_pbds

In the example above, and in some practice (e.g., using the pb_ds heap of this chapter to write single-source shortest path and similar algorithms), one often needs to save and use heap iterators (such as `__gnu_pbds::priority_queue<int>::point_iterator`).

But for the different Tag parameters of `__gnu_pbds::priority_queue`, the underlying implementations differ, and so do the iterator invalidation conditions. According to the design of the `__gnu_pbds` library, the following three cases derive from top to bottom:

1. Basic invalidation guarantee (`basic_invalidation_guarantee`): as long as the container is not modified, point-type iterators (`point_iterator`), pointers, and references (key/value) **remain** valid.
2. Point invalidation guarantee (`point_invalidation_guarantee`): after the container is **modified**, point-type iterators (`point_iterator`), pointers, and references (key/value) **remain** valid as long as the corresponding element has not been removed from the container.
3. Range invalidation guarantee (`range_invalidation_guarantee`): after the container is **modified**, beyond the property of (2), iterators of any range type (including the return

values of `begin()` and `end()` are correct. Tags with the range invalidation guarantee are `rb_tree_tag`, `splay_tree_tag` for `__gnu_pbds::tree`, and `pat_trie_tag` for `__gnu_pbds::trie`.

Running the code below shows that apart from `binary_heap_tag`, which is `basic_invalidation_guarantee` and whose iterators invalidate after modification, all the others are `point_invalidation_guarantee`, satisfying the need for point-type iterators (`point_iterator`) to survive modifications.

```

1  #include <iostream>
2  using namespace std;
3  #include <ext/pb_ds/assoc_container.hpp>
4  #include <ext/pb_ds/priority_queue.hpp>
5  using namespace __gnu_pbds;
6  #include <cxxabi.h>
7
8  template <typename T>
9  void print_invalidation_guarantee() {
10     using gute = __gnu_pbds::container_traits<T>::invalidation_guarantee;
11     cout << abi::__cxa_demangle(typeid(gute).name(), 0, 0, 0) << endl;
12 }
13
14 int main() {
15     using pairing =
16         __gnu_pbds::priority_queue<int, greater<int>, pairing_heap_tag>;
17     using binary = __gnu_pbds::priority_queue<int, greater<int>, binary_heap_tag>;
18     using binomial =
19         __gnu_pbds::priority_queue<int, greater<int>, binomial_heap_tag>;
20     using rc_binomial =
21         __gnu_pbds::priority_queue<int, greater<int>, rc_binomial_heap_tag>;
22     using thin = __gnu_pbds::priority_queue<int, greater<int>, thin_heap_tag>;
23     print_invalidation_guarantee<pairing>();
24     print_invalidation_guarantee<binary>();
25     print_invalidation_guarantee<binomial>();
26     print_invalidation_guarantee<rc_binomial>();
27     print_invalidation_guarantee<thin>();
28     return 0;
29 }

```

34 Balanced Trees (pb-ds/tree)

Original title: 平衡树 · source page /lang/pbds-tree/

34.1 __gnu_pbds::tree

Appendix: official documentation.⁵⁹

```

1  #include <ext/pb_ds/assoc_container.hpp> // tree is defined here, so this
2                                          // header must be included

```

⁵⁹https://gcc.gnu.org/onlinedocs/libstdc++/ext/pb_ds/tree_based_containers.html

```

3 #include <ext/pb_ds/tree_policy.hpp>
4 using namespace __gnu_pbds;
5 __gnu_pbds::tree<Key, Mapped, Cmp_Fn = std::less<Key>, Tag = rb_tree_tag,
6             Node_Update = null_tree_node_update,
7             Allocator = std::allocator<char>>

```

34.2 Template Parameters

- **Key**: the type of the stored elements. If you want to store several identical **Key** elements, you need something like `std::pair` or a `struct`, combined with the `lower_bound` and `upper_bound` member functions for searching.
- **Mapped**: the mapped-policy type. To make the associative container a **set** (like storing elements in `std::set`), put `null_type` here (in older g++ versions this was `null_mapped_type`); to make it a **set with values** (like storing elements in `std::map`), put here a **Value** type like that of `std::map<Key, Value>`.
- **Cmp_Fn**: the key comparison functor, e.g. `std::less<Key>`.
- **Tag**: selects the underlying data structure; the default is `rb_tree_tag`. `__gnu_pbds` provides three different balanced trees:
 - `rb_tree_tag`: the red-black tree; generally use this one — the performance of the other two is usually inferior.
 - `splay_tree_tag`: the splay tree.
 - `ov_tree_tag`: the ordered-vector tree, just an ordered structure built on `vector`, implementing a balanced tree like a sorted `vector`; its performance depends on whether the data wants to hack you.
- **Node_Update**: the policy for updating nodes; the default is `null_node_update`. To use the `order_of_key` and `find_by_order` methods, use `tree_order_statistics_node_update`.
- **Allocator**: the allocator type.

34.3 Construction

```

1 __gnu_pbds::tree<std::pair<int, int>, __gnu_pbds::null_type,
2             std::less<std::pair<int, int>>, __gnu_pbds::rb_tree_tag,
3             __gnu_pbds::tree_order_statistics_node_update>
4     trr;

```

34.4 Member Functions

- `insert(x)`: inserts element `x` into the tree, returning `std::pair<point_iterator, bool>`, whose first element is an iterator to the insertion position and whose second element indicates whether the insertion succeeded.
- `erase(x)`: deletes an element/iterator `x` from the tree. If `x` is an iterator, returns an iterator to the one after `x` (if `x` is `end()`, returns `end()`); if `x` is a `Key`, returns whether the deletion succeeded (if the key does not exist, the deletion fails).
- `order_of_key(x)`: returns the number of elements strictly smaller than `x` (with `Cmp_Fn` as the comparison logic), i.e., the zero-based rank.
- `find_by_order(x)`: returns an iterator to the element corresponding to the rank compared by `Cmp_Fn`.
- `lower_bound(x)`: returns an iterator to the first element not smaller than `x` (with `Cmp_Fn` as the comparison logic).
- `upper_bound(x)`: returns an iterator to the first element strictly greater than `x` (with `Cmp_Fn` as the comparison logic).
- `join(x)`: merges the tree `x` into the current tree; `x` is emptied (you must ensure the two trees have the same **comparison function** and **element type**).
- `split(x,b)`: comparing with `Cmp_Fn`, elements $\leq x$ stay in the current tree and the rest go to tree `b`.
- `empty()`: returns whether the tree is empty.
- `size()`: returns the size.

Warning — Note

`join(x)` requires that the key ranges of the merged tree and the merged-into tree be **disjoint** (that is, all values in the merged tree must be greater/less than all values in the current tree); otherwise a `join_error` exception is thrown.

To merge two trees whose key ranges intersect, insert the elements of one tree into the other one by one.

34.5 Example

```

1 // Common Header Simple over C++11
2 #include <iostream>
3 using namespace std;
4 using ll = long long;
5 using ull = unsigned long long;
6 using ld = long double;
7 using pii = pair<int, int>;
8 #include <ext/pb_ds/assoc_container.hpp>
9 #include <ext/pb_ds/tree_policy.hpp>
10 __gnu_pbds::tree<pair<int, int>, __gnu_pbds::null_type, less<pair<int, int>>,
11                 __gnu_pbds::rb_tree_tag,
```

```

12         __gnu_pbds::tree_order_statistics_node_update>
13     trr;
14
15 int main() {
16     int cnt = 0;
17     trr.insert(make_pair(1, cnt++));
18     trr.insert(make_pair(5, cnt++));
19     trr.insert(make_pair(4, cnt++));
20     trr.insert(make_pair(3, cnt++));
21     trr.insert(make_pair(2, cnt++));
22     // elements in the tree {(1,0), (2,4), (3,3), (4,2), (5,1)}
23
24     auto it = trr.lower_bound(make_pair(2, 0));
25     trr.erase(it);
26     // elements in the tree {(1,0), (3,3), (4,2), (5,1)}
27
28     // print the first component of the element of rank 1 among ranks 0 1 2 3
29     auto it2 = trr.find_by_order(1);
30     cout << (*it2).first << endl; // output: 3
31
32     // print its rank
33     int pos = trr.order_of_key(*it2);
34     cout << pos << endl; // output: 1
35
36     // split trr at it2
37     decltype(trr) newtr;
38     trr.split(*it2, newtr);
39     for (auto i = newtr.begin(); i != newtr.end(); ++i) {
40         cout << (*i).first << ' '; // output: 4 5
41     }
42     cout << endl;
43
44     // join the newtr tree into trr; newtr is emptied.
45     trr.join(newtr);
46     for (auto i = trr.begin(); i != trr.end(); ++i) {
47         cout << (*i).first << ' '; // output: 1 3 4 5
48     }
49     cout << endl;
50     cout << newtr.size() << endl; // output: 0
51
52     return 0;
53 }

```

34.6 References

- Tree-Based Containers, https://gcc.gnu.org/onlinedocs/libstdc++/ext/pb_ds/tree_based_containers.html
- Implementation of the `join` function in GCC 14.1.0, https://gcc.gnu.org/onlinedocs/gcc-14.1.0/libstdc++/api/a18391_source.html#l00043
- Implementation of the `erase` function in GCC 14.1.0, https://gcc.gnu.org/onlinedocs/gcc-14.1.0/libstdc++/api/a18211_source.html#l00043

PART VI

Compilation and Optimization

35 Compilation and Optimization (optimizations)

Original title: 编译优化 · source page </lang/optimizations/>

The common programming language of the OI world is C++. Having chosen this language, you are destined to deal with compilers and language standards. As everyone knows, C++ is extremely confusing and evil; this article aims to provide practical compiler knowledge — enough for contests.

35.1 A Brief Introduction to Compiler Optimization

35.1.1 What Is Optimization

According to the as-if rule,⁶⁰ optimizations improve the running speed of a program and the size of its executable, as long as the semantics are unchanged.

35.2 Common Compiler Optimizations

35.2.1 Constant Folding

Constant folding, also called constant propagation: if an expression can be determined to be constant, constant propagation may be performed until its next definition (Definition).

```
1 int x = 1;
2 int y = x; // x = 1, => y = 1
3 x = 3;
4 int z = 2 * y; // z => 2 * y = 2 * 1 = 2
5 int y2 = x * 2; // x = 3, => y2 = 6
```

This code can be transformed at compile time into:

⁶⁰https://en.cppreference.com/w/cpp/language/as_if

```
1 int x = 1;
2 int y = 1;
3 x = 3;
4 int z = 2;
5 int y2 = 6;
```

Example: <https://godbolt.org/z/oEfY35TTd>

35.2.2 Dead-Code Elimination

As the name suggests: a piece of code that is not used gets deleted.

```
1 int test() {
2     int a = 233;
3     int b = a * 2;
4     int c = 234;
5     return c;
6 }
```

will be transformed into

```
1 int test() { return 234; }
```

Note that this code first underwent constant folding, making the return value determinably 234; **a** and **b** are inactive variables and are therefore deleted.

35.2.3 Loop Rotation

A loop is converted from “for” form into “do-while” form, with an extra condition test added in front. This transformation mainly prepares the ground for other transformations.

```
1 for (int i = 0; i < n; ++i) {
2     auto v = *p;
3     use(v);
4 }
```

is transformed into

```
1 if (0 < n) {
2     do {
3         auto v = *p;
4         use(v);
5         ++i;
6     } while (i < n);
7 }
```

35.2.4 Loop-Invariant Code Motion

Based on alias analysis, code inside a loop that is proved invariant (possibly including memory accesses, load/store, hence the reliance on alias analysis) is hoisted out of the loop body, so that the loop body contains less code.

```
1 for (int i = 0; i < n; ++i) {  
2     auto v = *p;  
3     use(v);  
4 }
```

Intuitively this code could be hoisted into:

```
1 auto v = *p;  
2 for (int i = 0; i < n; ++i) {  
3     use(v);  
4 }
```

But in fact, if $n \leq 0$, the loop is never entered, yet we executed one extra instruction (possibly with side effects!). Therefore loops are usually first rotated into do-while form, which makes it easy to insert a “loop guard”. Loop-invariant code motion is then applied.

```
1 if (0 < n) { // loop guard  
2     auto v = *p;  
3     do {  
4         use(v);  
5         ++i;  
6     } while (i < n);  
7 }
```

35.2.5 Loop Unrolling

Loops contain a loop body and all kinds of branches, requiring the modern CPU to do some branch prediction. Unrolling the loop directly trades code size for running time.

```
1 for (int i = 0; i < 3; i++) {  
2     a[i] = i;  
3 }
```

is transformed into

```
1 a[0] = 0;  
2 a[1] = 1;  
3 a[2] = 2;
```

35.2.6 Loop Unswitching

Loop unswitching moves the conditional inside a loop outside of it, and then places one loop under each of the two external conditions; this increases the possibility of loop vectorization and parallelization (simple loops are usually easier to vectorize).

```

1 // clang-format off
2 void before(int x) {
3     for(/* i in some range */) {
4         /* A */;
5         if (/* condition */ x % 2) {
6             /* B */;
7         }
8         /* C */;
9     }
10 }
11
12 void after(int x) {
13     if (/* condition */ x % 2) {
14         for(/* i in some range */) {
15             /* A */;
16             /* B */; // execute B directly, without the loop test
17             /* C */;
18         }
19     } else {
20         for(/* i in some range */) {
21             /* A */;
22             // B is not executed
23             /* C */;
24         }
25     }
26 }

```

35.2.7 Code Layout Optimizations

During execution, a program's paths divide into cold/hot paths. For the CPU, a jump is in the vast majority of cases not as fast as executing sequentially — the latter is called “fallthrough” by compiler writers. Correspondingly, code that is executed often is hot code, and the opposite is cold code. In OI code, if some part is a boundary-condition special case inside a loop, or exception-handling-like logic, that part is cold code.

A basic block is the basic structure of control flow: a procedure consists of several basic blocks forming a directed graph. When generating the executable, the compiler must arrange a layout placing the basic blocks, and how to arrange this layout is the focus of this optimization.

In principle, hot code should be placed together and cold code separated away. The reason is better utilization of the instruction cache: the locality of hot code will be better.

```

1 // clang-format off
2 int hotpath; // <-- hot!
3 if (/* boundary condition */ false) {
4     // <-- cold!
5 }
6 int hotpath_again; // <-- hot!

```


Basic block placement. We use labels to express a kind of “pseudo machine code”. This C++ program has two possible translations:

Note — Layout 1

```

1 // clang-format off
2 hotblock1:
3     Stmts; // <-- hot!
4     if (/* the boundary condition does not hold */ true)
5         goto hotblock2; // happens often! -----+
6 coldblock:                                /* | */
7     Stmt; // <- cold                       |
8     Stmt; // <- cold                       |
9     Stmt; // <- cold                       | spanning many instructions,
10    Stmt; // <- cold                       | which is costly!
11    Stmt; // <- cold                       |
12    Stmt; // <- cold                       |
13    Stmt; // <- cold                       |
14    Stmt; // <- cold                       |
15 hotblock2:                                /* | */
16     Stmts; // <- hot! <-----+

```

Another layout:

Note — Layout 2

```

1 // clang-format off
2 hotblock1:
3     Stmts; // <-- hot!
4     if (/* boundary condition */ false)
5         goto coldblock; // happens rarely
6 hotblock2:                                /* | at low cost! */
7     Stmts; // <- hot! <-----+
8 coldblock:
9     Stmt; // <- cold
10    Stmt; // <- cold
11    Stmt; // <- cold
12    Stmt; // <- cold
13    Stmt; // <- cold

```

We see that in the latter layout, the two hot blocks are put together, and the execution efficiency is better.

To tell the compiler whether a branch is easily executed, use the C++20 `[[likely]]` and `[[unlikely]]`: <https://en.cppreference.com/w/cpp/language/attributes/likely>

If the contest does not use the C++20 standard or later, you can use `__builtin_expect` (GNU extension).

```

1 #define likely(x) __builtin_expect(!(x), 1)
2 #define unlikely(x) __builtin_expect(!(x), 0)
3
4 if (unlikely(/* some boundary-condition checks */ false)) {

```

```

5 // cold code
6 }

```

Hot-cold splitting. A procedure may contain both cold and hot paths, and if the cold code is long, the better approach is to make the cold code a function call instead of blocking the hot path. This also warns us not to be too clever and make every function `inline`: cold code hinders execution speed far more than a function call does.

Note — A bad code layout

```

1 // clang-format off
2 void foo() {
3     // clang-format off
4 hotblock1:
5     Stmts; // <-- hot!
6     if (/* the boundary condition does not hold */ true)
7         goto hotblock2; // happens often! -----+
8 coldblock:                                /* | */
9     Stmt; // <- cold                        |
10    Stmt; // <- cold                        |
11    Stmt; // <- cold                        | spanning many instructions,
12    Stmt; // <- cold                        | which is costly!
13    Stmt; // <- cold                        |
14    Stmt; // <- cold                        |
15    Stmt; // <- cold                        |
16    Stmt; // <- cold                        |
17 hotblock2:                                /* | */
18     Stmts; // <- hot! <-----+
19 }

```

Note — A good code layout

```

1 // clang-format off
2 void foo() {
3 hotblock1:
4     Stmts; // <-- hot!
5     if (/* boundary condition */ false)
6         coldBlock(); // separate out the cold code, making the hot path more
7                       // cache-friendly
8 hotblock2:
9     Stmts; // <- hot!
10 }
11
12 void coldBlock() {
13     Stmt; // <- cold
14     Stmt; // <- cold
15     Stmt; // <- cold
16     Stmt; // <- cold
17     Stmt; // <- cold
18     Stmt; // <- cold

```

```
19  Stmt;  // <- cold
20 }
```

Hot-cold splitting is exactly the reverse operation of function inlining. The existence of this optimization teaches us that inlining does not necessarily make the program faster; if the inlined code is cold code, it can even make the program slower! Some compilers have forced-inlining options, but they are not recommended. Internally the compiler runs a static analysis computing the probability of each basic block and branch, together with a cost model about function calls, to decide whether to inline; deciding inlining by yourself is not necessarily better than the compiler's decision.

In fact, without extra information, the compiler usually assumes the probabilities of branching and not branching are equal, and propagates the hotness of control-flow paths accordingly. Part of PGO (Profile Guided Optimization) is exactly deriving the true branch probabilities of the program in the real environment through several performance tests and experiments; this information can make the code layout even better.

35.2.8 Function Inlining

Function calls usually pass parameters through registers and the stack; caller and callee both need to save some register state — this process is usually called the calling convention. A function call therefore incurs some time cost, and inlining means writing the function's body directly into the caller's procedure, with no real function call.

```
1  int add(int x) { return x + 1; }
2
3  int foo() {
4      int a = 1;
5      a = add(a);
6  }
```

`add()` can be inlined into `foo()`:

```
1  int foo() {
2      int a = 1;
3      a = a + 1;  // <-- the body of add(), inlined without parameter passing
4  }
```

`always_inline`, **`__force_inline`**. <https://clang.llvm.org/docs/AttributeReference.html#always-inline-force-inline>

Some compilers provide a way to inline a function call by hand: add `__attribute__((always_inline))` before the function. Using this is not necessarily faster than a function call; at that moment the compiler trusts the programmer's judgment to be good enough.

35.2.9 Tail Call Optimization

When a function call sits at the tail of the function body, this call is called a tail call. For this special form of call, special optimizations are possible. The vast majority of architectures have a Frame Pointer (FP) and a Stack Pointer (SP) maintaining the call frames of functions; if the call sits at the tail of the function, we need not keep the outer function's call record and can replace it directly with the inner function.

Replacing a function call with a jump instruction. On the vast majority of architectures, a function call must save the current program counter (\$pc) and several caller-saved registers, in order to return to the scene. A tail call does not need this process and is translated directly into a jump instruction, because a tail recursion never returns to where the function was running.

A simple example: <https://godbolt.org/z/e7b1safaW>

```
1 int test(int a);
2
3 int tailCall(int x) { return test(x); }
```

```
1 tailCall(int):                                ; @tailCall(int)
2     jmp     test(int)@PLT                      ; TAILCALL
```

Automatic tail-recursion rewriting. If a function's tail call is to itself, the function is tail-recursive. Broadly speaking, indirect recursion (recursion formed by two or more functions together), where all calls are tail calls, also belongs to tail recursion. Tail recursion can be optimized by the compiler into a non-recursive form, reducing the extra stack cost and function-call cost. Many algorithm-contest contestants are keen on writing non-recursive code; without optimization this can greatly reduce the constant, but with optimization on, the binary produced from recursive code is no different from hand-written code.

```
1 int fac(int n) {
2     if (n < 2) return 1;
3     return /* uses */ n * fac(n - 1); /* uses the variable n, so a direct
4                                     tail-recursion optimization is impossible! */
5 }
```

Note that this function is not tail-recursive, but it can be rewritten as:

```
1 int fac(int acc, int n) {
2     if (n < 2) return acc;
3     return fac(acc * n, n - 1);
4 }
```

The new code is tail-recursive.

Modern compilers can complete this process for you automatically: if your code has a chance of being rewritten into tail recursion, the compiler can recognize the form and complete the rewrite.

Tail-recursion elimination: `-Rpass=tailcallelim`. Since the function is already tail-recursive, the recursion statement can simply be deleted: through some static analysis the function is directly converted into a non-recursive form. We will not dig into how compiler authors do this; from practical experience, for the vast majority of OI code, if both a recursive version and a non-recursive version exist, the code can generally be optimized automatically into the non-recursive version. Some concrete examples for the reader:

- GCD⁶¹

```
1 int gcd(int a, int b) { return b ? gcd(b, a % b) : a; }
```

- Fibonacci sequence⁶²

```
1 // expand the fib(n - 2) term
2 // fib(n - 1) cannot be transformed into a non-recursive form; even after
3 // optimization the code remains exponential
4 int fib(int n) {
5     if (n < 2) return 1;
6     return fib(n - 1) + fib(n - 2);
7 }
```

- Factorial⁶³

```
1 // unrolled into a scalar loop, then auto-vectorized; the generated code is
2 // SIMD
3 unsigned fac(unsigned n) {
4     if (n < 2) return 1;
5     return n * fac(n - 1);
6 }
```

The assembly of these functions after optimization is exactly identical to the non-recursive version: the recursion is directly eliminated. For OI contestants, with O2 on you can confidently write the recursive version of all kinds of algorithms; there will be no difference from the non-recursive version. If the function you wrote cannot be rewritten into a non-recursive form, then the compiler is also powerless.

35.2.10 Strength Reduction

A common compiler optimization. The simplest example is turning `x * 2` into `x << 1`; the second spelling is quite common in OI. The compiler does similar optimizations automatically: with the optimization switch on, `x * 2` and `x << 1` are fully equivalent. Strength reduction converts high-cost instructions into low-cost instructions.

Scalar operator transformations.

⁶¹<https://godbolt.org/z/8Wb6WEnzv>

⁶²<https://godbolt.org/z/4enof6Wcb>

⁶³<https://godbolt.org/z/n64e75xrf>

Shifts instead of multiplication.

```

1 int a;
2 a = x * 2;    // bad!
3 a = x << 1;   // good!

```

Note that signed and unsigned numbers differ markedly in shifting and type promotion. The sign bit is handled specially during shifts, including the two types of arithmetic and logical shifts. This stands out when writing binary search/segment trees with many divide-by-two operations: signed integer division cannot be directly optimized into a single right-shift.

```

1 int l, r;
2 /* codes */
3 int mid = (l + r) / 2; /* if the compiler cannot assume l, r >= 0, it will
4                        // generate worse code */
5                        // cannot be optimized to
6                        // mid = (l + r) >> 1
7                        // counterexample:
8                        // mid = -127
9                        // mid / 2 = -63
10                       // mid >> 1 = -64

```

```

1 int mid = (l + r);
2 int sign = mid >> 31; /* logical shift right, obtaining the sign bit */
3 mid += sign;
4 mid >>= 1; /* arithmetic shift right */

```

Feasible solutions:

- use unsigned `l, r`; subscripts should be unsigned anyway;
- use shifts in the source code.

Multiplication instead of division.

```

1 int x = a / 3;

```

This process can be transformed into `x = a * 0x55555556 >> 32`; see this Zhihu article⁶⁴ or the original paper⁶⁵ for details.

Induction variable strength reduction (IndVars). The compiler automatically recognizes the induction variables in a loop and converts the related high-cost operations into low-cost ones.

```

1 int a = 0;
2 for (int i = 1; i < 10; i++) {
3     a = 3 * i;    // bad!
4     a = a + 3;    // good!
5 }

```

⁶⁴<https://zhuanlan.zhihu.com/p/151038723> (in Chinese)

⁶⁵<https://dl.acm.org/doi/10.1145/773473.178249>

Using `a = 3 * i` directly is very common in OI; the compiler can automatically analyze that the equivalent transformation is `a = a + 3`, replacing multiplication with the cheaper addition. Analyzing the iteration of loop variables is called SCEV (Scalar Evolution).

SCEV can also optimize some loops:

```
1 int test(int n) {
2     int ans = 1;
3     for (int i = 0; i < n; i++) {
4         ans += i * (i + 1);
5     }
6     return ans;
7 }
```

This function is optimized into an $O(1)$ formula summation; see <https://godbolt.org/z/ET8d89vvK>. Currently this behavior appears only in LLVM-based compilers; GCC is more conservative.

```
1 test(int):                                # @test(int)
2     test     edi, edi
3     jle     .LBB0_1
4     lea     eax, [rdi - 1]
5     lea     ecx, [rdi - 2]
6     imul    rcx, rax
7     lea     eax, [rdi - 3]
8     imul    rax, rcx
9     shr     rax
10    imul    eax, eax, 1431655766
11    and     ecx, -2
12    lea     eax, [rax + 2*rcx]
13    lea     eax, [rax + 2*rdi]
14    dec     eax
15    ret
16 .LBB0_1:
17     mov     eax, 1
18     ret
```

35.2.11 Auto-Vectorization

Single instruction, multiple data (SIMD) is a great way to provide single-core parallelism. Using such instructions, you can exploit the CPU's SIMD registers, which are wider than general-purpose registers — e.g., holding 4 integers at once and computing on them. OI contestants need not understand the details of auto-vectorization; generally speaking, the Clang compiler performs more aggressive auto-vectorization than GCC:

```
1 // https://godbolt.org/z/h1hx5sWoE
2 void test(int *a, int *b, int n) {
3     for (int i = 0; i < n; i++) {
4         a[i] += b[i];
5     }
6 }
```

The `__restrict` type specifier (GNU, MSVC). The regions of two arbitrary pointers may overlap, in which case a special check is needed on whether vector code can be used. The figure below shows an example of overlapping pointers:

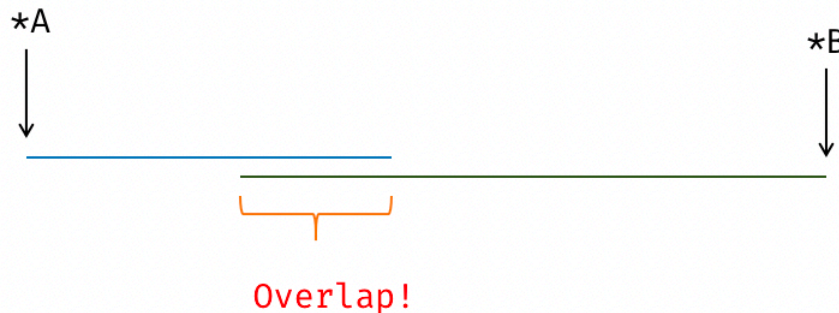


Figure 6: Two pointers whose pointee ranges overlap (from the original page).

`__restrict`, as a convention, makes the compiler assume the memory regions the two pointers point to never overlap.

```
1 void test(int* __restrict a, int* __restrict b, int n) {
2     for (int i = 0; i < n; i++) {
3         a[i] += b[i];
4     }
5 }
```

`__restrict` is not part of the C++ standard, but all major compilers support it. This keyword affects the quality of the code generated by auto-vectorization; it can be used in extreme constant-optimization situations.

35.3 Common Language Misuses Related to Compiler Optimization

35.3.1 `inline` — Inlining

With O2 on, function inlining is usually done automatically by the compiler. The `inline` in a struct definition is completely redundant: if the contest you prepare for enables O2, there is no need at all to declare functions inline. If O2 is off, using `inline` still will not make the compiler truly inline.

The `inline` keyword in modern C++ is treated as a linking / symbol-exporting semantic behavior, not as function inlining.

35.3.2 `register` — A False Register Hint

Modern compilers simply ignore your `register` keyword; your personal idea of register allocation is generally not as smart as letting the compiler run its register-allocation algorithm directly. This keyword was deprecated in C++11 and removed in C++17.⁶⁶

⁶⁶Remove Deprecated Use of the register Keyword (open-std.org), <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/p0001r1.html>

<https://en.cppreference.com/w/cpp/keyword/register>

35.4 Undefined Behavior and Compiler Optimization

The compiler may assume that a C++ program has no undefined behavior (UB);⁶⁷ therefore, when compiling a program with UB, the compiler may produce unexpected results. Meanwhile, the compiler may also optimize more aggressively and freely under the assumption that no UB exists.

Common UB:

1. signed overflow;⁶⁸
2. use of uninitialized variables;
3. out-of-bounds access;
4. null-pointer dereference;
5. infinite loops without side effects.

Other UB and examples can be studied through the extended reading.

35.4.1 Signed Overflow

```
1 int f(int x) { return x * 2 / 2; }
```

The compiler may assume the program has no signed-overflow behavior, and so this function may be optimized into

```
1 int f(int x) { return x; }
```

Examples: <https://godbolt.org/z/WKv3W5hvM>, <https://godbolt.org/z/qQE9nxP1j>.

This assumption can be disabled with the `-fwrapv`⁶⁹ option. Examples: <https://godbolt.org/z/5x3K5KGnr>, <https://godbolt.org/z/4r4a4EzMW>.

35.4.2 Use of Uninitialized Variables

```
1 int f(int x) {
2     int a;
3     if (x) // either x nonzero or UB
4         a = 42;
5     return a;
6 }
```

The compiler may assume the program never uses uninitialized variables, so `a` must have been initialized, and this function may be optimized into

⁶⁷<https://en.cppreference.com/w/cpp/language/ub>

⁶⁸<https://users.cs.utah.edu/~regehr/papers/overflow12.pdf>

⁶⁹<https://gcc.gnu.org/onlinedocs/gcc-13.2.0/gcc/Code-Gen-Options.html#index-fwrapv>

```
1 int f(int) { return 42; }
```

Examples: <https://godbolt.org/z/8WYMYjdG>, <https://godbolt.org/z/qvGd1nv9>.

35.4.3 Out-of-Bounds Access

```
1 int table[4] = {};
2
3 bool exists_in_table(int v) {
4     // return true in one of the first 4 iterations or UB due to out-of-bounds
5     // access
6     for (int i = 0; i <= 4; i++)
7         if (table[i] == v) return true;
8     return false;
9 }
```

The compiler may assume the program never accesses out of bounds, so this function must return before an out-of-bounds access happens, and it may be optimized into

```
1 bool exists_in_table(int) { return true; }
```

Example: <https://godbolt.org/z/xfePeYsE3>.

35.4.4 Null-Pointer Dereference

```
1 int f(int* p) {
2     int x = *p;
3     if (!p)
4         return x; // Either UB above or this branch is never taken
5     else
6         return 0;
7 }
```

The compiler may assume the program never dereferences null pointers, so `!p` is always `false`, and this function may be optimized into

```
1 int f(int*) { return 0; }
```

Examples: <https://godbolt.org/z/GY1jvsrb5>, <https://godbolt.org/z/4ronPsnxf>.

35.4.5 Infinite Loops without Side Effects

Note — Verifying Fermat's Last Theorem

By Fermat's Last Theorem,^a the Diophantine equation $a^3 = b^3 + c^3$ has no positive integer solutions. The program below tries to enumerate the integers in $[1, 1000]$ to check whether the equation holds; if it returns `true`, a set of integer solutions was found in $[1, 1000]$, and Fermat's Last Theorem would be false.

```

1  #include <iostream>
2
3  bool fermat() {
4      const int max_value = 1000;
5
6      // Endless loop with no side effects is UB
7      for (int a = 1, b = 1, c = 1; true;) {
8          if (((a * a * a) == ((b * b * b) + (c * c * c))))
9              return true; // disproved :(
10         a++;
11         if (a > max_value) {
12             a = 1;
13             b++;
14         }
15         if (b > max_value) {
16             b = 1;
17             c++;
18         }
19         if (c > max_value) c = 1;
20     }
21
22     return false; // not disproved
23 }
24
25 int main() {
26     std::cout << "Fermat's Last Theorem ";
27     fermat() ? std::cout << "has been disproved!\n"
28              : std::cout << "has not been disproved.\n";
29 }

```

^ahttps://en.wikipedia.org/wiki/Fermat's_Last_Theorem

The compiler may assume the program has no infinite loops without side effects, and thus that the for loop in `fermat()` must terminate at some moment and return `true`; in the end the program may output:

```
1 Fermat's Last Theorem has been disproved!
```

Examples: <https://godbolt.org/z/d834MK7bz>, <https://godbolt.org/z/Eov9nsKqf>.

35.5 Sanitizers

Sanitizers. They check at runtime whether your program has undefined behavior, out-of-bounds array accesses, null pointers, and so on.

In local debugging mode, it is recommended to enable some sanitizers, which can greatly shorten your debugging time. These sanitizers were developed by Google, and the vast majority work in both GCC and Clang. Sanitizers are more mature in LLVM, so contestants are advised to use the Clang compiler locally for such debugging.

35.5.1 Address Sanitizer: `-fsanitize=address`

<https://clang.llvm.org/docs/AddressSanitizer.html>

Both GCC and Clang support this sanitizer. Its checks include:

- out-of-bounds access;
- use-after-free;
- use-after-return;
- double-free;
- memory leaks;
- use-after-scope.

Applying this checking slows your program by about $2\times$.

35.5.2 Undefined Behavior Sanitizer: `-fsanitize=undefined`

<https://clang.llvm.org/docs/UndefinedBehaviorSanitizer.html>

The Undefined Behavior Sanitizer (UBSan) checks code for undefined behavior. Both GCC and Clang support this sanitizer; it checks your program for undefined behavior automatically. UBSan's checks include:

- shift overflow, e.g. shifting a 32-bit integer left by 72 bits;
- signed integer overflow;
- floating-point-to-integer conversion overflow.

UBSan's checks are optional; see the provided web address for their impact on the program.

35.6 Miscellaneous

35.6.1 Compiler Explorer

Observe the behavior and assembly of various compilers here: <https://godbolt.org>

35.7 Extended Reading

1. The LLVM Project Blog: What Every C Programmer Should Know About Undefined Behavior #1/3, <https://blog.llvm.org/2011/05/what-every-c-programmer-should-know.html>
2. The LLVM Project Blog: What Every C Programmer Should Know About Undefined Behavior #2/3, https://blog.llvm.org/2011/05/what-every-c-programmer-should-know_14.html

3. The LLVM Project Blog: What Every C Programmer Should Know About Undefined Behavior #3/3, https://blog.llvm.org/2011/05/what-every-c-programmer-should-know_21.html

PART VII

Beyond C++

36 Differences Between C++ and Other Common Languages (`cpp-other-langs`)

Original title: C++与其他常用语言的区别 · source page </lang/cpp-other-langs/>

This article introduces the differences between C++ and other commonly used languages, focusing on the important or easily overlooked differences between C and C++. Although C++ is almost a superset of C, and mixing C/C++ code generally causes no problems, understanding the more important differences between C and C++ can spare you some strange bugs. If you are an Oler whose main language is C, this article will also let you pick up C++ more smoothly. For the unique features C++ adds over C, read the tutorial in the Classes and Modern C++ part (§24 and beyond). In addition, this article briefly covers the differences between Python, Java, and C++.

36.1 Differences Between C and C++

36.1.1 Macros and Templates

One purpose of C++ templates at design time was to replace macro definitions. Learning template programming is an important step from C towards C++. Unlike the textual replacement of macros, templates get more comprehensive compiler checks at compile time, making it easier to write more robust code. After C++11, templates support variadic template parameter lists, which can replace C's variadic functions while guaranteeing type safety.

36.1.2 Pointers and References

In C++ you can still use C-style pointers, but for passing variables around, it is more recommended to use the C++ [reference](#) feature to achieve similar functionality. Since the object a reference points to cannot be null, some null-address access problems are avoided. Pointers, however, still have their uses thanks to their flexibility. It is worth mentioning that the C null pointer `NULL` has had a type-safe replacement, `nullptr`, since C++11. References and pointers can be converted into each other via the `*` and `&` operators.

36.1.3 bool

See also the Boolean type (§3).

Unlike C++, the C language originally had no Boolean type.

The C99 standard added the `_Bool` keyword (plus the equivalent `bool` macro) and the two macros `true` and `false`. If you want to use the three macros `bool`, `true`, `false`, you must include the `stdbool.h` header in your program. Using `_Bool` requires no extra header at all.

```
1 bool x = true; // requires including stdbool.h
2 _Bool x = 1;  // no need to include stdbool.h
```

From C23 on, `true`, `false`, and `bool` became keywords of the C language; using them no longer requires including `stdbool.h`, while `_Bool` is kept as an alternative spelling of `bool`.⁷⁰

The table below shows how support for the `bool` type changed across the C standards (with C++ added for comparison):

Language standard	<code>bool</code>	<code>true/false</code>
C89	/	/
C99 until C23	macro, equivalent to <code>_Bool</code> ; needs <code>stdbool.h</code>	macros: <code>true</code> equivalent to <code>1</code> , <code>false</code> equivalent to <code>0</code>
C23 on	keyword	keyword
C++	keyword	keyword

36.1.4 struct

Although both C and C++ have the concept of struct, the things they denote must not be mixed! A struct in C describes a fixed memory organization, whereas a struct in C++ is a class, **whose only difference from a class is that its members and inheritance default to public**, while an ordinary class's members default to private. This is especially fatal when writing mixed C/C++ code.

Also, when declaring a struct, C++ does not need C's ceremony. The C version:

```
1 typedef struct Node_t {
2     struct Node_t *next;
3     int key;
4 } Node;
```

⁷⁰<https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3054.pdf>

The C++ version:

```
1 struct Node {
2     Node *next;
3     int key;
4 };
```

36.1.5 const

In C, `const` only qualifies that a variable cannot be modified; in C++, with the arrival of many new features, `const` has been given more uses. C's `const` is succeeded in C++ by `constexpr`, while the uses of C++'s `const` are described on the [Constants](#) page.

36.1.6 Memory Allocation

C++ added the `new` and `delete` keywords to allocate space on the “free store”, which may be the heap or static storage; they appeared to go with “classes”. `delete[]` can also directly free the memory of a dynamic array, which is very convenient. The `new` and `delete` keywords call the type's constructor and destructor; compared with C's `malloc()`, `realloc()`, and `free()` functions they support types more completely, but they are less efficient than those C functions.

In short: if the objects you need to allocate dynamically are basic types or arrays of them, you can use `malloc()` for more efficient allocation; but if the objects you create are non-basic types, use `new` for the sake of the safety checks. Note that although both `new` and `malloc()` return pointers, a pointer from `new` can **only** be reclaimed with `delete`, and a pointer from `malloc()` only with `free()`; otherwise there is a risk of memory leaks.

36.1.7 Variable Declarations

Before C99, C variable declarations had to be at the head of a statement block; C++ and C99 onward have no such restriction.

36.1.8 Variable-Length Arrays

After C99, C supports VLAs (variable-length arrays); C++ never has.

36.1.9 Struct Initialization

After C99, C supports designated initializers for structs⁷³ (optional in C11); C++ only supports designated initializers with ordering requirements from C++20 on, and the features C supports — out-of-order, nested, array, and mixing with ordinary initializers — are all unsupported in C++.⁷⁴

36.1.10 Comment Syntax

C++-style single-line comments `//` were not supported in C before C99.

⁷³https://en.cppreference.com/w/c/language/struct_initialization

⁷⁴https://en.cppreference.com/w/cpp/language/aggregate_initialization

36.2 Differences Between Python and C++

Python is currently the most used language in machine learning. Compared with C++, Python's advantages are that it is easy to learn and easy to practice. Python has simpler, more direct syntax; for instance, when defining a variable you need not declare its type in advance. But this simplicity has a price: Python sacrifices performance compared with C++. C++ is applicable to almost all platforms, including embedded systems, and runs faster, while Python only runs on certain platforms supporting high-level languages. C++ is closer to the metal, so it can be used to write operating systems.

36.3 Differences Between Java and C++

Java and C++ are both object-oriented languages, both using object-oriented ideas (encapsulation, inheritance, polymorphism). Because object orientation has many excellent properties (inheritance, composition, etc.), both have good reusability. So compared with Python, Java and C++ resemble each other more.

The biggest difference between the two is that Java has the JVM mechanism. JVM stands for Java Virtual Machine. One very important feature of the Java language is platform independence, and the Java virtual machine is the key to realizing it. Ordinary high-level languages must at least be compiled into different object code to run on different platforms. With the Java virtual machine, Java programs need no recompilation across platforms. The Java language uses the JVM to shield platform-specific information, so a compiled Java program only needs to generate object code (bytecode) that runs on the JVM, and it can then run on many platforms unmodified.

Because of this feature, Java is often used to develop programs that must be ported to different platforms. But also because compiling a Java program starts from bytecode, Java's performance is not as good as C++'s.

37 An Emergency Guide: Pascal to C++ (pas-cpp)

Original title: Pascal转C++急救 · source page </lang/pas-cpp/>

37.1 Quick Installation and Environment Setup for C++

The following steps are all performed on Windows.

37.1.1 Using an IDE

Refer to the following pages (OI Wiki → Tools → Editors):

- Dev-C++⁷⁵
- Code::Blocks⁷⁶

⁷⁵<https://oi-wiki.org/tools/editor/devcpp/>

⁷⁶<https://oi-wiki.org/tools/editor/codeblocks/>

37.1.2 Using an Editor + Compiler + Debugger

Refer to the VS Code page (OI Wiki → Tools → Editors → VS Code). The official Visual Studio Code website has documentation explaining how to configure C++. Generally VS Code is more convenient with extensions; see the official VS Code site.⁷⁷

37.2 A Quick Syntax Digest of C++

A C++ program always starts running from the `main` part.

Curly braces mark the start and end of block statements: `{` corresponds to Pascal's `begin`, and `}` to `end`.

Note that, as in Pascal, every C++ statement ends with a semicolon `;`; but the closing brace needs no semicolon after it, and the end of the program is not marked with a period `..`.

For comments: `//` is a line comment, and `/* */` a block comment.

By convention, let us look at Hello World.

37.2.1 Hello World: The First C++ Program

```
1 #include <iostream> // import the iostream library
2
3 int main() // the main part
4 {
5     std::cout << "Hello World!" << std::endl;
6
7     return 0;
8 }
```

Then compile and run it, and look at the result.

A brief explanation. The first line, `#include <iostream>`, means: import the `iostream` library.

Note — Pascal's library files

Pascal actually has library files too; it is just that many students have never used them...

See `main` on the third line? The program starts executing from `main`.

The next most important statement is

```
1 std::cout << "Hello World!" << std::endl;
```

`std::cout` is the output command. You may have seen some C++ programs writing just `cout`.

Note — About the `std::` prefix

For the question of the `std::` prefix, see the note “What is `std`?” under the Input and Output part of §2.

⁷⁷<https://code.visualstudio.com/>

The `<<` in the middle vividly expresses flow; it really does show how the output “flows”. The meaning of this line of code is that `"Hello World!"` is first pushed onto the output stream, and then `std::endl` is pushed onto the output stream.

And `std::endl` is the **output** newline command, similar to Pascal’s `writeln` — though C++ has no `coutln`. The difference between Pascal and C++: `write('Hello World!')` is equivalent to `std::cout << "Hello World!"`, while `writeln('Hello World!')` is equivalent to `std::cout << "Hello World!" << std::endl`.

Here `"Hello World!"` is a string. In Pascal strings always use single quotes `'` and never double quotes, while C++ strings must use double quotes. In C++, characters wrapped in single quotes have a different meaning, mentioned later.

Well, that about covers Hello World.

Some may ask: what does that `return 0` at the end mean? What does `int main()` mean? **Don’t worry about it for now:** at the beginning, just write it as a template (this one was written from the template too). Since beginners do not use `main`’s arguments, there is no need to write `int main(int argc, char const *argv[])`.

Simple exercises. 1. Try outputting a different string. 2. Try learning about escape characters.

37.2.2 A+B Problem: The Second C++ Program

The classic A+B Problem.

```
1 #include <iostream>
2
3 int main() {
4     int a, b, c;
5
6     std::cin >> a >> b;
7
8     c = a + b;
9
10    std::cout << c << std::endl;
11
12    return 0;
13 }
```

Note: the code has many blank lines; if you are not used to them, remove them.

A brief explanation. `std::cin` is for reading (`cin` as in C-in), and `>>` is analogous to the output syntax.

Of the new statements here, the two most important are, first, the variable declaration statement:

```
1 int a, b, c;
```

You may be used to Pascal’s variable declarations:

```
1 var
2 a, b, c: integer;
```

C++ declarations start directly with the data type name; here, starting with `int` (integer type) announces that variables follow.

Next, the most important statement is the assignment statement:

```
1 c = a + b;
```

This is a major difference between Pascal and C++ syntax: Pascal's assignment is `:=`, C++'s is `=`; and C++ tests equality with `==`.

C++ can also initialize variables with values right at declaration.

```
1 int a = 0, b = 0, c = 0;
```

Simple exercises. 1. Rewrite the code, submit it to an OJ, and get AC. 2. For more input/output syntax, see the `scanf` and `printf` part of §2, and try learning C++'s formatted output.

37.2.3 Closing Words and Next Steps

By now you have mastered the most basic things; what remains is finding the correspondences between Pascal and C++ syntax and the differing features.

But before that, it is strongly recommended to first read Variable scope: global variables and local variables (below).

Make good use of `Alt`+`←` and `Alt`+`→` to jump back while reading.

37.3 Syntax

37.3.1 Variables

Basic data types. C++ and Pascal are roughly the same; the common ones are

- `bool`: Boolean type;
- `int`: integer type;
- `float`: single-precision floating-point type;
- `double`: double-precision floating-point type;
- `char`: character type;
- `void`: no type.

C++'s single quotes are reserved for single characters (the character type), like `'a'`, while strings (arrays of characters) must use double quotes.

C++ has many additional data types; see Fundamental types — [cppreference.com](https://zh.cppreference.com/w/cpp/language/types).⁷⁸

Constant declarations.

```
1 const double PI = 3.1415926;
```

If you are unclear about the issues of macro expansion, prefer constants to macro definitions.

37.3.2 Operators

See directly the [Operators and Operations](#) page. The appendix also provides a comparison table of operators and mathematical functions.

37.3.3 Conditionals

The if statement.

```
1 if (a = b) and (a > 0) and (b > 0) then
2   begin
3     b := a;
4   end
5 else
6   begin
7     a := b;
8   end;
```

```
1 if (a == b && a > 0 && b > 0) {
2   b = a;
3 } else {
4   a = b;
5 }
```

Boolean operations and comparisons:

- and -> &&
- or -> ||
- not -> !
- = -> ==
- <> -> !=

Notes:

1. Pascal's **and** and C++'s **&&** differ in precedence; C++ needs no parentheses around the test conditions.
2. In Pascal, equality is **=** and assignment is **:=**; in C++, equality is **==** and assignment is **=**.

⁷⁸<https://zh.cppreference.com/w/cpp/language/types>

3. If you write `a = b` instead of `a == b` inside the `if` parentheses, the program will not error out: it assigns `b` to `a`, and the assignment expression `a = b` as a whole takes the value of `a` after the assignment.
4. C++ spares you deciding whether a semicolon belongs after `end`.
5. In C++ Boolean operations, non-Boolean values convert automatically to Boolean values.

Warning — A common mistake

Pay special attention: **do not write `=` where you mean `==`!**

Because C/C++ syntax is more flexible than Pascal's, if you write `if (a=b)` in a test, the program will happily keep running, since in C++ `a=b` has a return value.

case and switch. Not used much; not elaborated here.

Note: C++ has no `1..n`, and no chained inequalities (e.g., `1 < x < 2`).

37.3.4 Loops

The following three loops, six pieces of code, all accomplish the same thing.

The while loop. `while` is very similar. (The C++ here is not a complete program; some framework boilerplate is omitted, same below.)

```

1 var i: integer;
2
3 begin
4     i := 1;
5     while i <= 10 do
6         begin
7             write(i, ' ');
8             inc(i); // or i := i + 1;
9         end;
10 end.
```

```

1 int i = 1;
2 while (i <= 10) {
3     std::cout << i << " ";
4     i++;
5 }
```

The for loop. C++'s `for` statement is very different.

```

1 var i: integer;
2
3 begin
4     for i:= 1 to 10 do
5         begin
6             write(i, ' ');
```

```

7     end;
8 end.

```

```

1 for (int i = 1; i <= 10; i++) {
2     std::cout << i << " ";
3 }

```

Notes:

1. The line `for (int i = 1; i <= 10; i++){` packs many statements: `for` holds three statements.
2. The first statement, `int i = 1;`, declares a local variable `i` and initializes it. (This design is far more sensible than Pascal's.)
3. The second statement, `i <= 10;`, is the criterion for whether the loop continues.
4. The third statement, `i++`, runs at the end of each iteration; it means roughly Pascal's `inc(i)`, and writing `++i` here is the same. For the difference between `i++` and `++i`, consult other material.

repeat until versus do while. Note that `repeat until` and `do while` are different; compare the following code.

```

1 var i: integer;
2
3 begin
4     i := 1;
5     repeat
6         write(i, ' ');
7         inc(i);
8     until i = 11;
9 end.

```

```

1 int i = 1;
2 do {
3     std::cout << i << " ";
4     i++;
5 } while (i <= 10);

```

Loop control. C++'s `break` does the same as Pascal's: exit the loop.

And `continue` is the same too: skip the current iteration and enter the next one (return to the beginning).

37.3.5 Arrays and Strings

Variable-length arrays: the standard-library type `vector`. See the [Sequence Containers](#) page.

The C++ standard library provides `vector`, the equivalent of a variable-length array; import the library before using it.

```

1  #include <iostream>
2  #include <vector> // import the vector library
3
4  int main() {
5      std::vector<int> a; // declare vector a and define a as an empty vector object
6      int n;
7
8      std::cin >> n;
9      // read into a
10     for (int i = 0; i < n; i++) {
11         int t;
12         std::cin >> t;
13         a.push_back(t); // put the read number t at the end of vector a; this
14                         // operation has complexity O(1)
15         /* You cannot assign via subscript access here, because at declaration
16            time a is still empty; using `a[i] = t;` here is the wrong approach.
17         */
18     }
19
20     // print all numbers read into a
21     for (int i = 0; i < n; i++) {
22         std::cout << a[i] << ", "; // !note that the first number in a is a[0];
23         // if the subscript is out of bounds, it returns an unknown value
24         // (overflow) without raising an error
25     }
26     std::cout << std::endl;
27
28     return 0;
29 }
```

C++'s access to array members resembles Pascal's, with one important difference: the first item of the array is `a[0]`, whereas in Pascal you may choose the starting index yourself.

Strings: the standard-library type `string`. See the [string](#) page.

The C++ standard library provides `string`, whose operations overlap `vector`'s in places; it also requires importing the library.

```

1  #include <iostream>
2  #include <string>
3
4  int main() {
5      std::string s; // declare string s
6
7      std::cin >> s; // read s;
8      // reading skips all leading whitespace (spaces, newlines, tabs), and the
```



```

9  // string read runs until the next whitespace character.
10
11  std::cout << s << std::endl;
12
13  return 0;
14 }

```

C-style arrays. See the [Arrays](#) page.

If you need variable-length arrays, use **vector**, not C-style arrays.

C-style arrays are closely tied to pointers, so they are not elaborated here.

37.4 Important Differences

37.4.1 Variable Scope: Global and Local Variables

C++ lets you declare variables in **almost any place**. See Variable scope in §3.

When writing Pascal procedures/functions, it is easy to forget to declare the local variables **i** or **j**, while the main program usually has loops, so in most cases **i** and **j** are global variables; in that situation, operating on **i** inside a procedure/function is extremely error-prone. Worse, if you forget to declare such a local variable, the compiler compiles without complaint and the program runs. (Many hard-to-find bugs are born exactly this way.)

So when using C++, when declaring variables — e.g., the **i** used in loops — **do not use global variables; use local variables whenever possible**. Do this and you need not worry about name collisions (like **i**) inside functions.

Note — An extra remark

Pascal can avoid this problem to some degree: imitating the C++ way, have the main program only call procedures/functions and never declare collision-prone globals like **i** and **j**; if a loop is needed, write a separate procedure to run it.

37.4.2 C++ Converts Types Automatically

```

1  int i = 2;
2  if (i) { // i = 0 yields false; any other value yields true
3      std::cout << "true";
4  } else {
5      std::cout << "false";
6  }

```

It is not just **int** converting to **bool**; **int** and **float** also convert to each other. In Pascal you may assign an integer to a floating-point type but not the reverse; C++ has no such problem.

```

1  int a;
2  a = 3.2; // now a = 3
3  float b = a; // now b = 3.0

```

Whether `/` is integer or floating-point division is decided by the types of the divisor and the dividend:

```
1 float a = 32 / 10;    // 32/10 gives 3 (integer division); a = 3.0
2 float b = 32.0 / 10;  // 32.0/10 gives 3.2; b = 3.2
```

`pow(a, b)` computes a^b ; the function returns a floating-point type. If you use it directly to compute integer powers, automatic conversion means you need not fear an error:

```
1 int a = pow(2, 3);    // compute 2^3
```

There are also conversions between `char` and `int`:

```
1 char a = 48;          // ASCII 48 is '0'
2 int b = a + 1;         // b = 49
3 std::cout << (a == '0'); // true prints 1
```

In fact `char` and `bool` in C++ are essentially integer types.

For details, see Implicit conversions — [cppreference.com](https://zh.cppreference.com/w/cpp/language/implicit_conversion).⁷⁹

37.4.3 Many C++ Statements Have Return Values: Reading an Unknown Amount of Data as an Example

Sometimes you need to read until the data ends — e.g., summing a group of numbers of unknown quantity (the data may span several lines) until end of file. The implementation:

Note — End of file, EOF

EOF, the end-of-file marker, is typed on the command line as `Ctrl+Z` on Windows (plus `Enter`), and `Ctrl+D` on Unix-like systems.

```
1 #include <iostream>
2
3 int main() {
4     int sum = 0, a = 0;
5
6     while (std::cin >> a) {
7         sum += a;
8     }
9     std::cout << sum << std::endl;
10
11     return 0;
12 }
```

How it works: in `while (std::cin >> a)`, if the input has problems or the file ends, `std::cin >> a` returns `false`, breaking the loop.

⁷⁹https://zh.cppreference.com/w/cpp/language/implicit_conversion

37.4.4 Functions

C++ has only functions, no procedures, but it has **void**; no function-value variables, but it has **return**.

A comparison of a Pascal function and a C++ function:

```

1  function abs(x:integer):integer;
2  begin
3      if x < 0 then
4          begin
5              abs := -x;
6          end
7      else
8          begin
9              abs := x;
10         end;
11 end;
```

```

1  int abs(int x) {
2      if (x < 0) {
3          return -x;
4      } else {
5          return x;
6      }
7  }
```

The C++ function declaration `int abs` defines the function `abs()` with return type `int` (integer); the function's return value is the value given by the `return` statement.

If you do not want a return value (i.e., Pascal's "procedure"), use `void`. `void` means "empty"; nothing is returned.

```

1  var ans: integer;
2
3  procedure printAns(ans:integer);
4  begin
5      writeln(ans);
6  end;
7
8  begin
9      ans := 10;
10     printAns(ans);
11 end.
```

```

1  #include <iostream>
2
3  void printAns(int ans) {
4      std::cout << ans << std::endl;
5
6      return;
7  }
8
```

```

9  int main() {
10     int ans = 10;
11     printAns(ans);
12
13     return 0;
14 }

```

C++'s **return** differs from Pascal's function-value assignment in one very big way. C++'s **return** returns a value and ends the function the moment it executes; Pascal's function-value assignment does not leave the function — execution continues. So Pascal needs an extra command to break out of a function/procedure midway: **exit**. C++ does not: to break at some point you can use **return** directly. For example (since no short-and-useful example came to mind, this will do):

```

1  #include <iostream>
2
3  void printWarning(int x) {
4      if (x >= 0) {
5          return; // this statement here corresponds to `exit;` in Pascal
6      }
7      std::cout << "Warning: input a negative number.";
8  }
9
10 int main() {
11     int a;
12
13     std::cin >> a;
14     printWarning(a);
15
16     return 0;
17 }

```

And in a sense, the earlier **abs** function is only strictly equivalent when written as

```

1  function abs(x:integer):integer;
2  begin
3      if x < 0 then
4          begin
5              abs := -x; exit; // !note this place
6          end
7      else
8          begin
9              abs := x;  exit; // !note this place
10         end;
11 end;

```

```

1  int abs(int x) {
2      if (x < 0) {
3          return -x;
4      } else {
5          return x;
6      }
7  }

```

Note — A special reminder

In C++, `exit` means exiting the program; do not type `exit` out of habit — use `return`!

C++ treats functions and procedures uniformly as functions — not even `main` is spared: writing `int main`, C++ regards `main` as an integer-typed function whose return value is `0`. It is a customary convention; returning `0` means the program exited normally.

You may have guessed already: the arguments in `main(int argc, char const *argv[])` are `int argc` and `char const *argv[]`; for their meaning consult other material.

37.4.5 Passing Arguments into Functions

C++ has no Pascal `var` keyword for changing the passed arguments, but C++ can use references and pointers to achieve the same effect.

```

1  var a, b: integer;
2
3  procedure swap(var x,y:integer);
4  var temp:integer;
5  begin
6      temp := x;
7      x := y;
8      y := temp;
9  end;
10
11 begin
12     a := 10; b:= 20;
13     swap(a, b);
14     writeln(a, ' ', b);
15 end.
```

```

1  // code using pointers
2  #include <iostream>
3
4  void swap(int* x, int* y) {
5      int temp;
6      temp = *x;
7      *x = *y;
8      *y = temp;
9  }
10
11 int main() {
12     int a = 10, b = 20;
13     swap(&a, &b);
14     std::cout << a << " " << b;
15
16     return 0;
17 }
```

Note that this C++ code **involves pointers**. Pointer matters are rather thorny; reading the relevant material is recommended.

```
1 // code using references
2 #include <iostream>
3
4 void swap(int& x, int& y) {
5     int temp;
6     temp = x;
7     x = y;
8     y = temp;
9 }
10
11 int main(int argc, char const* argv[]) {
12     int a = 10, b = 20;
13     swap(a, b);
14     std::cout << a << " " << b;
15
16     return 0;
17 }
```

Note that this C++ code involves **reference-type issues**. When calling some STL libraries or template libraries with references you may encounter problems, at which point you must manually declare alias types. See *C++ Primer*, 5th edition, or online material.

C++ has other ways of passing arguments into functions, one of which is **passing arguments via global variables**; if you cannot use pointers yet, start with this method. But its defect is that there is no stack to hold data, so **there is no way to pass arguments in recursive functions**. (Unless you hand-write a stack — note, hand-writing a stack is also a way to break through the system stack limit.)

37.5 The C++ Standard Library and Reference Material

Never reinvent the wheel (except for practice): before writing a feature yourself, first look for an existing function or data structure.

37.5.1 The C++ Standard Library

The C++ standard library's `<algorithm>` has many useful functions — quicksort, binary search, etc. — callable directly. See the [STL Algorithms](#) page.

There are also STL containers, such as arrays, vectors (resizable arrays), queues, stacks, and more, with many functions attached. See the [Introduction to STL Containers](#) page.

For functions on strings, see

- `std::basic_string` — cppreference.com, https://zh.cppreference.com/w/cpp/string/basic_string
- `<string>` — C++ Reference, <https://www.cplusplus.com/reference/string/>

C/C++ pointers are very flexible things; see the [Pointers](#) page. To thoroughly understand pointers, a book or reference manual read carefully is recommended.

37.5.2 Error Hunting and Techniques

- Common mistakes (OI Wiki → Contest → Common mistakes)⁸⁰
- Common tricks (OI Wiki → Contest → Common tricks)⁸¹

37.5.3 C++ Language Material

- Learning resources (OI Wiki → Contest → Resources)⁸²
- cppreference.com: the most important C/C++ reference, <https://zh.cppreference.com/>
- C++ Tutorial — runoob, <https://www.runoob.com/cplusplus/cpp-tutorial.html>
- C++ Language — C++ Tutorials, <https://www.cplusplus.com/doc/tutorial/>
- Reference — C++ Reference, <https://www.cplusplus.com/reference/>
- C++ Standard Library — Wikipedia, https://en.wikipedia.org/wiki/C++_Standard_Library
- The Ultimate Question of Programming, Refactoring, and Everything, <https://www.gitbook.com/book/alexastva/the-ultimate-question-of-programming-refactoring-details>
- Google C++ Style Guide, <https://google.github.io/styleguide/cppguide.html>

37.6 Afterword

Writing up to here, many readers will feel this is not very “emergency” at all, with many things not mentioned. That is simply how it is.

Although this is meant as first aid, many things — like converting strings to numbers, or searching characters in strings — do not fit a short, punchy first-aid post; writing all of that out would make it a full C++ beginner’s tutorial. So please make full use of this Wiki, the reference manuals, and search engines.

One point should be made: much of the “C++ syntax” above actually comes from the C language; a better title would have been “An Emergency Guide: Pascal to C/C++”.

Pascal was a very popular language in the second half of the last century; it predates C. But as UNIX spread and Microsoft adopted C, Pascal has become history. Pascal’s later development did exist — e.g., the open-source compiler project Free Pascal, adding object-oriented features (the Delphi language). Besides informatics contests, Pascal today has one use no other language has: it compiles on a great many very old machines, e.g., the Gameboy, Nintendo’s last-century game console; and another use is appearing as pseudocode (Pascal-style pseudocode) in all kinds of textbooks.

⁸⁰<https://oi-wiki.org/contest/common-mistakes/>

⁸¹<https://oi-wiki.org/contest/common-tricks/>

⁸²<https://oi-wiki.org/contest/resources/>

Finally: Pascal's circle is really quite small, while the C/C++ circle is huge, with abundant and complete manuals and tutorials — so be sure to master English. There are many, many programming languages in the world, and computer science as a discipline and technology is far more than informatics contests and programming languages.

37.6.1 Pascal Language References for This Article

- Lazarus wiki, <https://wiki.freepascal.org/>
- Free Pascal Reference guide, <https://freepascal.org/docs-html/current/ref/ref.html>

37.7 Appendix: Comparison Table of Pascal and C++ Operators and Mathematical Functions

Only the most commonly used operators and functions are included.

37.7.1 Basic Arithmetic

	Pascal	C++
addition	<code>a + b</code>	<code>a + b</code>
subtraction	<code>a - b</code>	<code>a - b</code>
multiplication	<code>a * b</code>	<code>a * b</code>
integer division	<code>a div b</code>	<code>a / b</code>
floating-point division	<code>a / b</code>	<code>a / b</code>
modulo	<code>a mod b</code>	<code>a % b</code>

37.7.2 Logic

	Pascal	C++
not	<code>not(a)</code>	<code>!a</code>
and	<code>a and b</code>	<code>a && b</code>
or	<code>a or b</code>	<code>a b</code>

37.7.3 Comparison

	Pascal	C++
equal	<code>a = b</code>	<code>a == b</code>
not equal	<code>a <> b</code>	<code>a != b</code>
greater	<code>a > b</code>	<code>a > b</code>
less	<code>a < b</code>	<code>a < b</code>
greater or equal	<code>a >= b</code>	<code>a >= b</code>
less or equal	<code>a <= b</code>	<code>a <= b</code>

37.7.4 Assignment

Pascal	C++
<code>a := b</code>	<code>a = b</code>
<code>a := a + b</code>	<code>a += b</code>
<code>a := a - b</code>	<code>a -= b</code>
<code>a := a * b</code>	<code>a *= b</code>
<code>a := a div b</code> or <code>a := a / b</code>	<code>a /= b</code>
<code>a := a mod b</code>	<code>a %= b</code>

37.7.5 Increment / Decrement

	Pascal	C++
increment	<code>inc(a)</code>	<code>a++</code>
increment	<code>inc(a)</code>	<code>++a</code>
decrement	<code>dec(a)</code>	<code>a--</code>
decrement	<code>dec(a)</code>	<code>--a</code>

37.7.6 Mathematical Functions

Using these requires importing the `<cmath>` library.

	Pascal	C++
absolute value	<code>abs(a)</code>	<code>abs(a)</code> (integers)
absolute value	<code>abs(a)</code>	<code>fabs(a)</code> (floating point)
a^b	<code>N/A</code> ⁸³	<code>pow(a, b)</code>
truncation	<code>trunc(a)</code>	<code>trunc(a)</code>
rounding	<code>round(a)</code>	<code>round(a)</code>

For other functions, see:

- Common mathematical functions — cppreference.com, <https://zh.cppreference.com/w/cpp/numeric/math>

38 A Crash Course in Python (python)

Original title: Python速成 · source page [/lang/python/](#)

38.1 About Python

Python is an interpreted language already in wide use around the world. It provides efficient high-level data structures and supports simple yet effective object-oriented programming; it can also be used in algorithm contests.

38.1.1 Python's Advantages

- Python is an **interpreted** language: Python needs no compilation or linking, which reduces the operational steps to some extent.
- Python is an **interactive** language: the Python interpreter implements interactive operation — you can type and execute instructions directly in a terminal.
- Python is **easy to learn and use**: Python offers a wealth of data structures and also supports developing large programs.
- Python is **highly compatible**: Python supports Windows, macOS, and Unix operating systems alike.
- Python is **practical**: from simple input/output to scientific computing and even large web applications, suitable Python programs can be written for all of them.
- Python **programs are concise and readable**: Python code is usually shorter than code in other languages implementing the same functionality.
- Python **supports extension**: Python is developed in C (i.e., CPython), and the Python interpreter can be linked with applications developed in C, using Python to extend and control the application.

38.1.2 Notes on Learning Python

- The mainly used Python version today is Python 3.7 and above; Python 2 and Python 3 versions up to 3.6 are [no longer supported](#), but they are still used by some old systems and code. This article **introduces newer versions of Python**. If you encounter Python 2 code, you can try the [2to3](#) program to convert Python 2 code into Python 3 code.
- Python's design philosophy and syntactic structure **differ considerably from some other languages**, hiding many low-level details, and thus presenting a practical and elegant style.
- Python is a highly dynamic interpreted language, so **its programs run relatively slowly**, especially when using its built-in **for** loop statement. When using Python, try to use built-in functions such as **filter** and **map**, or use [list comprehension](#) syntax to improve program performance.

38.2 Environment Setup

See Python 3 (OI Wiki → Tools → Compilers → Python 3).⁸⁴ Alternatively:

- Windows: Python can also be obtained quickly and free of charge in the Microsoft Store.
- macOS/Linux: usually most Linux distributions already ship with Python. If you only plan to learn Python syntax and have no other development needs, you need not install Python separately.

⁸⁴<https://oi-wiki.org/tools/compiler/#python-3>

Warning — Note

On systems where Python was installed by default (i.e., via a package manager), such as Unix systems, run `python3` in the terminal to open the Python 3 interpreter.^a

^a<https://docs.python.org/zh-cn/3/tutorial/interpreter.html>

In addition, tools such as `venv`, `conda`, and `Nix` can manage the Python toolchain and Python packages, creating isolated virtual environments and avoiding dependency problems.

As an interpreted language, Python executes differently from C++; this difference often does not show up when programming in an IDE, so the different ways of running programs deserve emphasis here.

When you type `python3` on the command line, or when you have just opened IDLE, you have actually entered an interactive programming environment, also called a “REPL” (“read–evaluate–print” loop). Beginners can type statements here and immediately see results, which makes verifying syntax extremely easy; we will also use this form a great deal below.

But to write a complete program, you had better create a new text file (usually with the `.py` suffix) and then run `python3 filename.py` on the command line; the code will run and show its result.

38.2.1 Python Versions Provided by Some Platforms

System / version	Python version
Noi Linux 2.0	3.8.0, includes requests
Luogu judge	3.11.5, NumPy 1.25.2
Hydro-based OJ	3.8.0+, includes NumPy
Ubuntu 22.04 (built in)	3.10.4
Microsoft Store	latest stable release

Warning — Note

This table was valid at the time of writing (2025/01/15); it is advisable to re-verify on the relevant platforms.

At present, the main mirror caches of the Python **source code** in China are the Beijing Jiaotong University free and open-source software mirror⁸⁵ and the Huawei open-source mirror;⁸⁶ you can try downloading the Python installer there.

⁸⁵<https://mirror.bjtu.edu.cn/python/>

⁸⁶<https://repo.huaweicloud.com/python/>

38.3 Installing Third-Party Libraries with `pip`

Python’s vitality comes largely from its rich third-party libraries: when writing practical programs, “calling libraries” is routine, and `pip` is the preferred program for installing third-party libraries. Since Python 3.4 it has been included by default in the Python binary installer.

The third-party libraries in `pip` are stored mainly on the Python Package Index (PyPI);⁸⁷ users may also specify other hosting platforms for third-party libraries. For usage, consult the usage help pages such as “pypi mirror usage help — Tsinghua University open-source software mirror”⁸⁸. You can find more PyPI mirror sources on MirrorZ.⁸⁹

Info — Installing a package using the Tsinghua open-source mirror

```
1 pip install -i https://mirrors.tuna.tsinghua.edu.cn/pypi/web/simple <some-package>
```

38.4 Basic Syntax

Python’s syntax is concise and easy to understand, and there are many official and third-party documents and tutorials. Here we only introduce the language features rather practical for OIers; you can learn more Python tutorials on the Python documentation⁹⁰ and the Python Wiki.⁹¹

38.4.1 Comments

Adding comments does not affect the running of the code, but comments make code easier to understand and use.

```
1 # lines starting with the # character are single-line comments
2
3 """
4 strings spanning multiple lines are wrapped in triple quotes
5 (three single quotes or three double quotes), but these are also often
6 used as comments
7 """
```

Adding comments does not affect the code. We encourage adding comments to make code easier to understand and use.

38.4.2 Basic Data Types

Everything is an object. In Python you need not declare variable names and their types in advance; assigning directly creates variables of various types:

```
1 >>> x = -3 # no semicolon needed at the end of a statement
2 >>> x
```

⁸⁷<https://pypi.org/>

⁸⁸<https://mirrors.tuna.tsinghua.edu.cn/help/pypi/>

⁸⁹<https://mirrorz.org/list/pypi>

⁹⁰<https://docs.python.org/zh-cn/3/>

⁹¹<https://wiki.python.org/moin/>

```

3 -3
4 >>> f = 3.1415926535897932384626; f # if you really want semicolons, they
5                                     # work; this saves a line
6 3.141592653589793
7 >>> s1 = "0"
8 >>> s1 # in Python, double quotes and single quotes work the same
9 '0'
10 >>> b = 'A' == 65 # 'A' and 65 are not of the same data type, so they are
11                  # unequal
12 >>> b # True and False are both capitalized
13 False
14 >>> True + 1 == 2 and not False != 0 # expressions in Python mostly use
15                                     # words, but symbols are supported too
16 True

```

But this does not mean Python has no concept of types; in fact the interpreter infers variable types automatically from assignments or operations, and you can use the built-in function `type()` to view the types of these variables:

```

1 >>> type(x)
2 <class 'int'>
3 >>> type(f)
4 <class 'float'>
5 >>> type(s1) # note: do not name a string str, otherwise the str object
6              # itself gets clobbered
7 <class 'str'>
8 >>> type(b)
9 <class 'bool'>

```

Note — What are built-in functions?

In C/C++, many common functions are scattered across different header files, but Python's interpreter has many practical and general functions built in; you can use them directly without paying attention to their existence. This brings a small problem: the names of these built-in functions are mostly common words, so you must avoid naming your variables the same, otherwise strange results may occur.

As we have seen, Python has built-in integer, floating-point, string, and Boolean types, comparable to `int`, `float`, `string`, and `bool` in C++. But there are obvious differences: there is no `char` character type, and no `double` type (though `float` in fact corresponds to C's double precision); if you need more precise floating-point arithmetic, use the standard library's `decimal`⁹² module; and if you need complex numbers, Python has a built-in `complex` type (which also means it is best not to name a variable `complex`).

You can see that these types all begin with `class` — and this is exactly the key point where Python differs from C++: all data in a Python program is represented by objects or relations between objects; functions are objects, and types themselves are objects too:

```

1 >>> type(int)
2 <class 'type'>

```

⁹²<https://docs.python.org/zh-cn/3/library/decimal.html>

```

3 >>> type(pow) # the built-in exponentiation function, covered later
4 <class 'builtin_function_or_method'>
5 >>> type(type) # type() is also a built-in function, but a special one;
6                 # interested readers may look it up
7 <class 'type'>

```

You may feel these concepts are hard to understand and useless for the moment, so we will not go deeper for now. In the examples below you may gradually appreciate that Python's objects provide powerful methods; when programming we should prefer operations centered on objects rather than procedures, which makes our code more compact and clear.

Numeric operations. Some say you can treat the Python installed on your system as a multi-purpose calculator — and that is a fact.

In interactive mode, you can type an expression after the `>>>` prompt and use the operators `+`, `-`, `*`, `/`, `%` to compute on numbers, just as in most other languages (like C++); you can also use `()` for associative grouping. Readers may experiment on their own; here we only show the parts that differ greatly from C++:

```

1 >>> 5.0 * 6 # the result of floating-point arithmetic is a float
2 30.0
3 >>> 15 / 3 # unlike C/C++, division always returns the floating-point
4            # float type
5 5.0
6 >>> 5 / 100000 # too many digits, so the result is shown in scientific
7               # notation
8 5e-05
9 >>> 5 // 3 # integer division (floor division) rounds down and outputs an
10           # integer type
11 1
12 >>> -5 // 3 # follows the round-down rule; note this differs from C/C++
13 -2
14 >>> 5 % 3 # modulo
15 2
16 >>> -5 % 3 # the modulo of a negative number is always non-negative, which
17            # also differs from C/C++; both satisfy (a//b)*b+(a%b)==a
18 1
19 >>> x = abs(-1e4) # the built-in absolute-value function
20 >>> x += 1 # there is no increment/decrement operator
21 >>> x # scientific notation defaults to float
22 10001.0

```

In the practice above you can see that division (`/`) always returns a floating-point type (in Python 2 it returned an integer). If you want an integer or a floor result, use integer division (`//`). Likewise, as in C++, you can use modulo (`%`) to compute remainders, and scientific notation has the same form.

In particular, Python performs exponentiation directly with `**`, and through the built-in `pow(a, b, mod)` it provides an efficient implementation of fast exponentiation (OI Wiki → Mathematics → Binary exponentiation).

```

1 >>> 3 ** 4 # exponentiation
2 81
3 >>> 2 ** 512
4 13407807929942597099574024998205846127479365820592393377723561443721764030073546976801874298166903427
5 >>> pow(2, 512, int(1e4)) # a fast implementation of 2**512 % 10000; 1e4 is
6                             # a float, so it must be converted to int
7 4096
8 >>> 2048 ** 2048 # try big integers in IDLE?
9 >>> 0.1 + 0.1 + 0.1 - 0.3 == 0. # as in C/C++, beware: floats cannot be
10                                # compared for equality directly
11 False

```

Type inspection. For a variable, `type(object)` returns the variable's type; e.g., `type(8)` and `type('a')` have values `<class 'int'>` and `<class 'str'>` respectively.

Basic input and output. Input/output in Python is done mainly through the built-in functions `input()` and `print()`; the usage of `print()` is highly intuitive:

```

1 >>> a = [1,2,3]; print(a[-1]) # printing appends a newline by default
2 3
3 >>> print(ans[0], ans[1]) # prints any number of variables, separated by
4                             # spaces by default
5 1 2
6 NDONE
7 1 2>>>
8 NDONE2
9 1, 2
10 >>> print(str(a[0]) + ', ' + str(a[1])) # same output, but manually
11                                         # concatenated into one string

```

The behavior of the `input()` function is close to C++'s `getline()`: it reads an entire line as a string, with no trailing newline.

```

1 >>> s = input('Please enter a list of numbers: '); s # when debugging, you
2             # can pass a string to input() as the prompt
3 Please enter a list of numbers: 1 2 3 4 5 6
4 '1 2 3 4 5 6'

```

Strings. Python 3 provides a powerful [Unicode](#)-based string type, used similarly to C++'s `string`; concepts such as escape characters carry over as well. Besides concatenation with the plus sign and index access, it additionally supports repeating strings with scalar multiplication `*` and the `in` operator.

```

1 >>> s1 = "0" # both single and double quotes can wrap strings, sometimes
2             # saving escape characters
3 >>> s1 += 'I-Wiki' # to stay in sync with C++, double quotes are
4                   # recommended
5 >>> '0I' in s1 # substring testing is very convenient

```

```

6 True
7 >>> len(s1) # like C++'s s.length(), but more general
8 7
9 >>> s2 = """ Thank you for reading
10 ... come contribute!
11 """ # strings with triple quotes can span multiple lines
12 >>> s1 + s2
13 'OI-Wiki Thank you for reading\ncome contribute!'
14 >>> print(s1 + s2) # here the print() function prints the string
15 OI-Wiki Thank you for reading
16 come contribute!
17 >>> s2[2] * 2 + s2[3] + s2[-1] # negative indices count from the right;
18                                # adding len(s) makes it like the residue-class
19                                # ring modulo n
20 'Thank you!'
21 >>> s1[0] = 'o' # str is immutable and cannot be modified in place; even
22                 # += actually creates a new object
23 Traceback (most recent call last):
24   File "<stdin>", line 1, in <module>
25 TypeError: 'str' object does not support item assignment

```

Python supports several composite data types that combine different values together. The most common is **list**: a group of values marked by square brackets and separated by commas. For example, `[1, 2, 3]` and `['a', 'b', 'c']` are both lists.

Besides indexing, strings also support *slicing*, whose design is exquisite; the format is `s[left-closed index:right-open index:step]`:

```

1 >>> s = 'OI-Wiki Thank you for reading\ncome contribute!'
2 >>> s[:8] # omitting the left (closed) index starts from the beginning
3 'OI-Wiki '
4 >>> s[8:14] # the beauty of the left-closed right-open design: the length is
5             # 14-8=6, and it joins the previous string seamlessly
6 'Thank you for reading'
7 >>> s[-4:] # omitting the right (open) index runs to the end
8 'ntribute!'
9 >>> s[8:14:2] # step of 2
10 'Tnyu'
11 >>> s[::-1] # with step -1, you get the reversed string
12 '!etubirtnoc emoc\ngnidaer rof uoy knahT ikiW-IO'
13 >>> s # but the original string is unchanged
14 'OI-Wiki Thank you for reading\ncome contribute!'

```

In the newest Python 3 versions, strings are encoded in Unicode — that is, Python strings support multiple languages.⁹³ In Python, you can use the built-in function `ord()` on a Unicode character to convert it to the corresponding Unicode code point, and the reverse conversion uses the built-in `chr()`. In C/C++ the `char` type can also be interconverted with the corresponding ASCII code.

To convert a number into the corresponding string, use the built-in function `str()`; conversely use `int()` and `float()`. You may compare this with the explicit type conversions of C/C++,

⁹³<https://docs.python.org/zh-cn/3/howto/unicode.html>

except that the parentheses are not attached to the type but are part of the function wrapping the argument.

Python's string type provides many powerful methods, including computing the index and occurrence count of some character, changing case, and so on; we will not list them all. It is strongly recommended to read the [official documentation](#) to become familiar with the common methods, and when doing string operations consider using these methods first rather than reinventing them.

38.5 Creating Arrays

Students coming from C++ may be confused about how to create arrays in Python. Here we introduce the syntax for opening “arrays” in Python. It must be emphasized that what we introduce are really several [sequence types](#), essentially different from C arrays and much closer to C++'s [vector](#).

38.5.1 Using list

The list (`list`) is probably the most commonly used and most powerful sequence type in Python. A list can hold elements of any type, including nested lists, which fits the definition of a “generalized list” in data structures. Be careful not to confuse it with the doubly linked list `list` of the C++ STL, so this article uses “list” (the English word) rather than the code name to avoid misunderstanding.

```

1 >>> [] # create an empty list; note that lists use square brackets
2 []
3 >>> nums = [0, 1, 2, 3, 5, 8, 13]; nums # initialize a list; the whole list
4                                     # can be printed directly
5 [0, 1, 2, 3, 5, 8, 13]
6 >>> nums[0] = 1; nums # supports index access and element modification
7 [1, 1, 2, 3, 5, 8, 13]
8 >>> nums.append(nums[-2]+nums[-1]); nums # append() is like vector's
9     # push_back(); neither has a return value
10 [1, 1, 2, 3, 5, 8, 13, 21]
11 >>> nums.pop() # pops and returns the last element, usable as a stack; a
12               # position may also be specified, defaulting to the end
13 21
14 >>> nums.insert(0, 1); nums # same as vector's insert(position, val)
15 [1, 1, 1, 2, 3, 5, 8, 13]
16 >>> nums.remove(1); nums # removes an element by value (only the first
17     # occurrence); raises an error if it does not exist
18 [1, 1, 2, 3, 5, 8, 13]
19 >>> len(nums) # the list's length, like vector's size(), but len() is a
20               # built-in function
21 7
22 >>> nums.reverse(); nums # in-place reversal
23 [13, 8, 5, 3, 2, 1, 1]
24 >>> sorted(nums) # obtain the sorted list
25 [1, 1, 2, 3, 5, 8, 13]
26 >>> nums # but the original list was not sorted
27 [13, 8, 5, 3, 2, 1, 1]
28 >>> nums.sort(); nums # in-place sort; a key parameter can be given as the
29                       # sort criterion

```

```

30 [1, 1, 2, 3, 5, 8, 13]
31 >>> nums.count(1) # like std::count()
32 2
33 >>> nums.index(1) # returns the index of the first occurrence of the value;
34                  # raises an error if absent
35 0
36 >>> nums.clear(); nums # same as vector's clear()

```

The examples above show the similarity between lists and **vector**: operations common in **vector** generally have corresponding methods on lists. But some methods such as `len()` and `sorted()` appear as built-in functions, while functions of the STL algorithms such as `find()`, `count()`, `max_element()`, `sort()`, and `reverse()` become methods of the object in Python; be careful to distinguish them when using. For more methods see “More on lists” in the official documentation.⁹⁴ Below we show some powerful features of lists as Python’s basic sequence type:

Python supports several composite data types that combine different values together. The most common is **list**: a group of values marked by square brackets and separated by commas. For example, `[1, 2, 3]` and `['a', 'b', 'c']` are both lists.

```

1 >>> lst = [1, '1'] + ["2", 3.0] # adding lists directly creates a new list
2 >>> lst # storing different types here only shows that it can be done; it is
3       # not good practice
4 [1, '1', '2', 3.0]
5 >>> 3 in lst # a practical membership test; strings have it too and also
6             # support substring tests
7 True
8 >>> [1, '1'] in lst # only single membership is tested; ``subsequences``
9                   # are not found
10 False
11 >>> lst[1:3] = [2, 3]; lst # slice assignment; the original list is modified
12 [1, 2, 3, 3.0]
13 >>> lst[::-1] # obtain a new reversed list
14 [3.0, 3, 2, 1]
15 >>> lst *= 2; lst # repetition by scalar multiplication
16 [1, 2, 3, 3.0, 1, 2, 3, 3.0]
17 >>> del lst[4:]; lst # could also write lst[4:] = []; the del statement can
18                   # delete more than sequence elements
19 [1, 2, 3, 3.0]

```

The examples above show some common operations on lists as sequences; you can see that many operations, such as slicing, are shared with strings. But strings are “immutable sequences” while lists are “mutable sequences”, so lists can be modified flexibly via slices. In C/C++ we tend to process char arrays with loops; below we show how to use **list comprehensions** to convert between strings and lists:

```

1 >>> # build an array of integers over [65, 70); range is also a type,
2 >>> # viewable as a left-closed right-open interval; the third argument is an
3 >>> # optional step
4 >>> nums = list(range(65,70)) # remember to wrap range() in list()
5 [65, 66, 67, 68, 69]

```

⁹⁴<https://docs.python.org/zh-cn/3/tutorial/datastructures.html>

```

6 >>> lst = [chr(x) for x in nums] # typical structure of a list comprehension:
7                                     # [exp for var in iterable if cond]
8 >>> lst # the two statements above can be merged into
9         # [chr(x) for x in range(65,70)]
10 ['A', 'B', 'C', 'D', 'E']
11 >>> s = ''.join(lst); s # joins the list's elements with the empty string ''
12                             # into a new string
13 'ABCDE'
14 >> list(s) # generate a char list from the string
15 ['A', 'B', 'C', 'D', 'E']
16 >>> # if you did not know about the s.lower() method, you might write the
17 >>> # old-wine-in-a-new-bottle expression below
18 >>> ''.join([chr(ord(ch) - 65 + 97) for ch in s if ch >= 'A' and ch <= 'Z'])
19 'abcde'

```

Below we demonstrate some scenarios more common in OI, such as two-dimensional “arrays”:

```

1 >>> vis = [[0] * 3] * 3 # open a 3x3 array of all 0s
2 >>> vis
3 [[0, 0, 0], [0, 0, 0], [0, 0, 0]]
4 >>> vis[0][0] = 1; vis # why were the other rows modified too?
5 [[1, 0, 0], [1, 0, 0], [1, 0, 0]]
6 >>> # first look at the assignment of one-dimensional lists
7 >>> a1 = [0, 0, 0]; a2 = a1; a3 = a1[:] # lists can also be assigned directly
8                                         # to new variables
9 >>> a1[0] = 1; a1 # modify list a1, seemingly normal
10 [1, 0, 0]
11 >>> a2 # why was a2 changed too
12 [1, 0, 0]
13 >>> a3 # a3 is unchanged
14 [0, 0, 0]
15 >>> id(a1) == id(a2) and id(a1) != id(a3) # the built-in id() gives an
16     # object's ``identity value'', comparable to an address; equal addresses
17     # mean one and the same object
18 True
19 >>> vis2 = vis[:] # copy the two-dimensional list
20 >>> vis[0][1] = 2; vis # vis is modified in batch
21 >>> [[1, 2, 0], [1, 2, 0], [1, 2, 0]]
22 >>> vis2 # but vis2, a slice copy, was also changed
23 >>> [[1, 2, 0], [1, 2, 0], [1, 2, 0]]
24 >>> id(vis) != id(vis2) # vis and vis2 are not the same object
25 True
26 >>> # although vis2 is not a reference to vis, the corresponding rows all
27 >>> # point to the same objects
28 keep
29 [True, True, True]
30 >>> # look back at the two-dimensional list itself
31 >>> [id(x) for x in vis] # the concrete numbers differ from here, but the
32                         # three values must be equal, showing three identical
33                         # objects
34 [139760373248192, 139760373248192, 139760373248192]

```

There is actually an important fact here: in Python, assignment only passes a reference and

does not create a new value. You can create variables of different types and assign them to new variables; verification shows their identity values are the same. It is just that up to now we had only introduced the list, a mutable type; when assigning new values to immutable types like numbers and strings, a new object is actually created, so the two variables do not interfere with each other. But lists are mutable, so when we modify the elements of one list, another list pointing to the same object is modified too. Creating a two-dimensional array is a similar situation: in the example, creating a 2-D list with multiplication is equivalent to repeating the one-dimensional list `[0]*3` three times, so operations involving one of the lists simultaneously affect the other two. Unfortunately, when assigning a 2-D list to another variable, even copying with a slice is only a “shallow copy”: its elements still point to the same objects. Solving this requires the standard library’s `deepcopy`⁹⁵ or, better, avoiding wholesale assignment of 2-D lists. The good news: when creating 2-D lists, avoiding duplicated lists is fairly easy — just use a list comprehension:

```

1 >>> vis1 = [[0] * 3 for _ in range(3)] # naming an unused loop counter with
2     # the underscore _ is a convention
3 >>> # but in the REPL, _ by default refers to the previous expression's
4 >>> # output, so a double underscore may also be used
5 >>> vis1
6 [[0, 0, 0], [0, 0, 0], [0, 0, 0]]
7 >>> [id(x) for x in vis1] # the concrete numbers differ from here, but the
8                           # three values must differ, showing three distinct
9                           # objects
10 [139685508981248, 139685508981568, 139685508981184]
11 >>> vis1[0][0] = 1
12 [[1, 0, 0], [0, 0, 0], [0, 0, 0]]
13 >>> a2[0][0] = 10 # access and assign a 2-D array

```

We introduced list comprehensions before covering loops, because Python is a highly dynamic interpreted language and its programs carry large extra overheads. In particular, **for loops run extremely slowly in Python**. So to get high performance in Python, use list comprehensions where possible, or built-in functions like `filter` and `map` that operate on whole sequences directly, avoiding loops — though of course this depends on the specific problem.

38.5.2 Using NumPy

Note — What is NumPy

NumPy is a famous Python scientific-computing library providing high-performance numeric and matrix operations. When testing algorithm prototypes, NumPy can save you hand-writing sorting, max/min, and similar algorithms. NumPy’s core data structure is the `ndarray`, the n-dimensional array, stored contiguously in memory with fixed length. Moreover, NumPy’s core is written in C, so its computation efficiency is very high. Note, however, that it is not part of the standard library; you can install it with `pip install numpy`, but there is no guarantee it is available in the OI contest environment (see the Python versions table at the top of this page).

⁹⁵<https://docs.python.org/3/library/copy.html>

The following code introduces how to use NumPy to build multidimensional arrays and access them.

```

1 >>> import numpy as np # search on your own for the meaning and usage of
2                               # import
3 >>> np.empty(3) # open an empty array of capacity 3; note it is not zero-# initialized
4 array([0.00000000e+000, 0.00000000e+000, 2.01191014e+180])
5 >>> np.zeros((3, 3)) # open a 3x3 array, initialized to 0
6 array([[0., 0., 0.],
7        [0., 0., 0.],
8        [0., 0., 0.]])
9 >>> a1 = np.zeros((3, 3), dtype=int) # open a 3x3 integer array
10 >>> a1[0][0] = 1 # access and assignment
11 >>> a1[0, 0] = 1 # friendlier syntax
12 >>> a1.shape # the array's shape
13 (3, 3)
14
15 >>> a1[:2, :2] # the submatrix of the first two rows and columns, no copy
16 array([[1, 0],
17        [0, 0]])
18
19 >>> a1[:, [0, 2]] # get columns 1 and 3, no copy
20 array([[1, 0],
21        [0, 0],
22        [0, 0]])
23 >>> np.max(a1) # get the array's maximum
24 1
25 >>> a1.flatten() # flatten the array
26 array([1, 0, 0, 0, 0, 0, 0, 0, 0, 0])
27
28 >>> np.sort(a1, axis = 1) # sort the array along rows, returning the result
29 array([[0, 0, 1],
30        [0, 0, 0],
31        [0, 0, 0]])
32 >>> a1.sort(axis = 1) # sort the array in place along rows

```

38.5.3 Using array

`array` is an efficient numeric array provided by the Python standard library; it can compactly represent arrays of basic-type values, but does not support nesting arrays, and few people use it — mentioned here only in passing.

Unless otherwise stated, “array” in the text below generally means “list”.

38.6 Input and Output

Input/output in Python is done mainly through the built-in functions `input()` and `print()`, introduced earlier. Below we introduce advanced usage.

38.6.1 Formatted Output

Algorithm contests usually involve only basic numeric and string output; `print()` is basically enough, and formatted string output is needed only when the number of decimal places matters. There are three ways to format: the first and oldest is the `printf()`-style `%` operator; another uses the [format function](#); the third is the [f-string](#) added in Python 3.6, the most concise, but the contest's Python version is not guaranteed to be new enough. For detailed and rich explanation, see [this page](#). Although `format()` is more recommended, to obtain an experience close to C, below we only demonstrate the old method similar to `printf()`:

```
1 >>> pi = 3.1415926; print('%.4f' % pi)    # format: %[flags][width][.precision]type
2 3.1416
3 >>> '%.4f - %8f = %d' % (pi, 0.1416, 3)  # multiple arguments on the right
4     # are wrapped in (); as you will see later, this is really a ``tuple''
5 '3.1416 - 0.141600 = 3'
```

38.6.2 The `split()` Function

The `input()` function behaves like C++'s `getline()`: it reads an entire line as a string, with no trailing newline. But in algorithm contests the common input form is several numbers on one line, so you need the string's `split()` method combined with list comprehensions to obtain a list holding numeric types. Taking “input n numbers and compute the average” as an example, here is the way to read n numbers into an “array”:

```
1 >>> s = input('Please enter a list of numbers: '); s    # when debugging, you
2     # can pass a string to input() as the prompt
3 Please enter a list of numbers: 1 2 3 4 5 6
4 '1 2 3 4 5 6'
5 >>> a = s.split(); a
6 ['1', '2', '3', '4', '5', '6']
7 >>> a = [int(x) for x in a]; a
8 [1, 2, 3, 4, 5, 6]
9 >>> # the input process above can be written in one line:
10 >>> # a = [int(x) for x in input().split()]
11 >>> # sum() is a built-in function
12 >>> sum(a) / len(a)
13 3.5
```

Sometimes a problem inputs a fixed number of numbers per line — e.g., an edge's start, end, and weight. With only the method above you would have to read an array each time and assign by index; here you can use Python's “unpacking” feature to assign several variables at once:

```
1 >>> u, v, w = [int(x) for x in input().split()]
2 1 2 4
3 >>> print(u,v,w)
4 1 2 4
```

Problems often input N lines; we have not yet covered the most basic loop statement, but Python's powerful sequence operations can handle multi-line input without loops. Below, assume we read the start, end, and weight of each edge into three arrays:

```

1 >>> N = 4; mat = [[int(x) for x in input().split()] for i in range(N)]
2 1 3 3
3 1 4 1
4 2 3 4
5 3 4 1
6 >>> mat # first read in the 2-D array row by row
7 [[1, 3, 3], [1, 4, 1], [2, 3, 4], [3, 4, 1]]
8 >>> u, v, w = map(list, zip(*mat))
9 # * unpacks mat into the inner lists
10 # zip() aggregates the corresponding elements of several lists into
11 # tuples, yielding an iterator
12 # map(list, iterable) converts the sequence's elements (tuples here) into
13 # lists
14 >>> print(u, v, w) # unpack the iterator returned by map() directly and
15 # assign to u, v, w respectively
16 [1, 1, 2, 3] [3, 4, 3, 4] [3, 1, 4, 1]

```

The program above is actually equivalent to first reading an $N \times 3$ matrix, then transposing it into a $3 \times N$ matrix — the outer list now nests 3 lists — and finally assigning the 3 lists representing start, end, and weight to `u`, `v`, `w` respectively. The built-in `zip()` stitches the corresponding elements of several equal-length sequences into “tuples”, producing a new sequence. And `map()` is really a functional-programming operation: it applies a given function to the elements of the sequence produced by `zip()` — here, using `list()` to turn tuples into lists. You can practice `*` with `zip()` and `map()` yourself to understand their meaning. Note that in Python 3, `zip()` and `map()` no longer return lists but iterators; we will not explain the differences for now — you may think of an iterator as producing the elements of a list one by one, and wrapping an iterator in `list()` generates the list.

38.7 File Reading and Writing

The Python built-in function `open()` reads and writes files. To prevent a file from being left open when reading/writing goes wrong, here we only introduce the safe approach using the `with` statement:

```

1 a = []
2 with open("in.txt") as f:
3     N = int(f.readline()) # read N from the first line
4     a[len(a) :] = [[int(x) for x in f.readline().split()] for i in range(N)]
5
6 with open("out.txt", "w") as f:
7     f.write("1\n")

```

There are many file reading/writing functions, suited to different scenarios; since OI contests do not yet support Python, we omit them here.

38.8 Control Flow

Although we have learned many features of Python, the Python code shown so far consists of single statements, which masks a major difference between Python and C in code style: first, Python does not use `{ }` but uses indentation to express block structure; if indentation is misaligned it errors out immediately, and mixing tabs and spaces also errors out. Second, the line opening a block — such as `if` and `for` statements — must end with a colon `:`. This helps readability, but you may miss the freedom of C: after all, having to realign indentation by hand after copy-pasting is annoying.

38.8.1 Loop Structure

List comprehensions accomplish batch operations efficiently in one line, but sometimes we have already been excessively deliberate in squeezing lines; in many scenarios loops are still the only option. So again taking multi-line input as an example, here is how loops are written in Python:

```

1 # note that from now on we no longer use the REPL; copy the multi-line
2 # data yourself
3 u, v, w = ([] for i in range(3)) # multiple assignment
4 for i in range(4): # assume the input has 4 lines here
5     _u, _v, _w = [int(x) for x in input().split()]
6     u.append(_u), v.append(_v), w.append(_w)
7     # you cannot do anything like cin >> u[i] >> v[i] >> w[i], since it will
8     # certainly exceed the lists' current length
9     # of course you may initialize all-0 lists of length MAXN instead, but
10    # then you must remember the true length and trim the excess
11 print(u, v, w)

```

Note that Python's `for` loop differs greatly from C/C++: it acts like the “range-based `for`” introduced in C++11 — essentially iterating over the elements of a sequence. For example, writing a loop over an array's subscripts requires iterating `range(len(lst))` rather than truly defining start and end conditions, so it is not as flexible as C/C++.

Next, a while loop shows how to input when the number of lines is unknown:

```

1 u, v, w = [], [], [] # multiple assignment, same as above
2 s = input() # note: in Python an assignment cannot appear in a conditional
3             # expression
4 while s: # you cannot write while(!scanf()) as in C
5     # slice concatenation avoids append(); note the list comprehension has
6     # a list nested inside
7     u[len(u) :], v[len(v) :], w[len(w) :] = [[int(x)] for x in s.split()]
8     s = input()
9 # after Python 3.8 introduced the walrus operator you can save two lines,
10 # but the contest environment may well not support it
11 while s := input():
12     u[len(u) :], v[len(v) :], w[len(w) :] = [[int(x)] for x in s.split()]
13 print(u, v, w)

```


38.8.2 Selection Structure

Much the same as C/C++, with some formal differences shown in the example below. Also note that assignment operators are not allowed in conditional expressions (from Python 3.8 the `:=` is available), and that Python [has no switch statement](#).

```

1 # no parentheses around conditional expressions
2 if 4 >= 3 > 2 and 3 != 5 == 5 != 7:
3     print("relational operators can be chained")
4     x = None or [] or -2
5     print("&& || !", "AND OR NOT", "and or not", sep="\n")
6     print("using and/or well saves lines")
7     if not x:
8         print("negative numbers are also True; this line does not run")
9     elif x & 1:
10        print("use elif, not else if\n" "bitwise operators resemble C's; for
11            even numbers x&1 gives 0, so this line does not run")
12    else:
13        print("there is also a ternary operator") if x else
14        print("note the structure")

```

38.8.3 Exception Handling

Although C++ has [try blocks](#) for exception handling, they are generally never used in contests; in Python the [EAFP](#) style is common, so code may use [try-except](#) statements extensively. They will be used again when we introduce the `dict` structure below. A demonstration:

```

1 s = "OI-wiki"
2 pat = "NOIP"
3 x = s.find(pat) # find() returns -1 when not found
4 try:
5     y = s.index(pat) # index() raises an error when not found
6     print(y) # this line is skipped
7 except ValueError:
8     print("not found")
9     try:
10        print(y) # y is undefined at this point, so this raises an error again
11    except NameError as e:
12        print("cannot print y")
13        print("reason:", e)

```

38.9 Built-in Containers

Python has many powerful built-in container types. Only by using them skillfully and knowing their properties can Python truly have a place in algorithm contests. Besides the `list` introduced in detail above, there are `tuple` (tuple), `dict` (dictionary), and `set` (set).

A tuple can be simply understood as an immutable list, though the meaning of “immutable” needs care: if an element of a tuple is a mutable type like a list, that list’s value can still be modified — the tuple stores a reference to the list, so the tuple itself has not changed. The tuple’s advantages

are low overhead and being “hashable”; the latter is very useful when creating dictionaries and sets.

```

1 tup = tuple([[1, 2], 4]) # obtain a tuple from a list
2 # equivalent to tup = ([1,2], 4)
3 tup[0].append(3)
4 print(tup)
5 a, b = 0, "I-Wiki" # multiple assignment is really tuple unpacking
6 print(id(a), id(b))
7 b, a = a, b
8 print(id(a), id(b)) # you should see that the id values of a and b are
9                     # now swapped
10 # this further shows that in Python variables are more like names, and
11 # assignment only makes them denote objects

```

The dictionary is like the `map` of the C++ STL (mind the distinction from Python’s built-in `map()` function) and stores key-value pairs; its form resembles JSON, but in JSON keys must be strings wrapped in double quotes, while the dictionary is more flexible and powerful: any hashable object can be a dictionary key. Note that the dictionary’s behavior has changed considerably across Python version updates, including the order of its elements; explore on your own.

```

1 dic = {"key": "value"} # basic form
2 dic = {chr(i): i for i in range(65, 91)} # a mapping from uppercase letters
3     # to their ASCII codes; mind the phrasing
4 dic = dict(zip([chr(i) for i in range(65, 91)], range(65, 91))) # same effect
5 dic = {dic[k]: k for k in dic} # invert the key-value pairs; for k in dic
6                               # iterates over its keys
7 dic = {v: k for k, v in dic.items()} # same effect as the line above;
8     # dic.items() holds each key-value pair as a tuple
9 dic = {
10     k: v for k, v in sorted(dic.items(), key=lambda x: -x[1])
11 } # sort the dictionary by value in reverse; uses a lambda expression
12
13 print(dic["A"]) # returns the item of dic keyed by 'A', whose value is 65
14 dic["a"] = 97 # sets d[key] to value; if the key was absent it is simply
15               # inserted
16 if "b" in dic: # the LBYL (Look Before You Leap) style
17     print(dic["b"]) # accessing a missing key errors out, so check first
18 else:
19     dic["b"] = 98
20
21 # a classic scenario: counting occurrences
22 # a new key does not exist in the original dictionary and needs extra
23 # handling
24 try: # the EAFP (Easier to Ask Forgiveness than Permission) style
25     cnter[key] += 1
26 except KeyError:
27     cnter[key] = 1

```

The set is like the `set` of the C++ STL: it never stores duplicate elements and can be viewed as a dictionary holding only keys. Note that both sets and dictionaries are wrapped in `{}`, but using `{}` alone creates an empty dictionary rather than an empty set; no example is given here.

38.10 Writing Functions

Python function definitions need no parameter types or return-value types, which imperceptibly reduces the amount of code for OI contestants:

```

1 def add(a, b):
2     return a + b # the advantage of dynamic typing: a and b may also be
3                   # strings
4
5
6 def add_no_swap(a, b):
7     print("in func #1:", id(a), id(b))
8     a += b
9     b, a = a, b
10    print("in func #2:", id(a), id(b)) # a, b have been swapped
11    return a, b # returning several values is really returning a tuple,
12                # which can be unpacked
13
14
15 lst1 = [1, 2]
16 lst2 = [3, 4]
17 print("outside func #1:", id(lst1), id(lst2))
18 add_no_swap(lst1, lst2)
19 # outside the function, lst1 and lst2 were not swapped
20 print("outside func #2:", id(lst1), id(lst2))
21 # though the values did change
22 print(lst1, lst2)

```

38.10.1 Default Arguments

Python function parameters are very flexible — keyword arguments, variadic arguments, etc. — but in algorithm contests these features are not very useful. Here we only introduce default arguments, since C++ also has them, and default arguments in Python very likely hide pitfalls. Consider the following code:

```

1 def append_to(element, to=[]):
2     to.append(element)
3     return to
4
5
6 lst1 = append_to(12)
7 lst2 = append_to(42)
8 print(lst1, lst2)
9
10 # you might expect the output [12] [42]
11 # but the actual output is [12, 42] [12, 42]

```

The reason for the output above: the default argument's value is assigned only once, at function definition. For mutable objects (lists, dictionaries, sets), all calls share the same object: `lst1` and `lst2` in fact both point to the same default list object in memory. Therefore, after the second call, the shared list's content has been modified to `[12, 42]`. So a function's default argument

values should be immutable objects; using `None` as a placeholder is a best practice:

```

1 def append_to(element, to=None):
2     if to is None:
3         to = []
4     to.append(element)
5     return to
6
7
8 lst1 = append_to(12)
9 lst2 = append_to(42)
10 print(lst1, lst2)
11
12 # the output is [12] [42]
```

38.10.2 Type Annotations

Python is a dynamically type-checked language, handling types in a flexible but implicit way: the Python interpreter checks types only at runtime and allows changing variable types at runtime. As the saying goes, “dynamic typing feels great until the code refactoring crematorium”: some errors in the program may only surface at runtime:

```

1 >>> if False:
2 ...     1 + "two" # This line never runs, so no TypeError is raised
3 ... else:
4 ...     1 + 2
5 ...
6 3
7
8 >>> 1 + "two" # Now this is type checked, and a TypeError is raised
9 TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

After Python 3.5, type annotations were introduced, allowing you to set the types of function parameters and return values — but only as hints, with no actual enforcing power; static checking tools are needed to weed out such errors (e.g., [PyCharm](#) and [Mypy](#)), which makes them somewhat chicken-rib. For OIers it suffices to know about them; function parameters and return values can be annotated like this:

```

1 def headline(
2     text, # type: str
3     width=80, # type: int
4     fill_char="-", # type: str
5 ): # type: (...) -> str
6     return f"{text.title()}.center(width, fill_char)"
7
8
9 print(headline("type comments work", width=40))
```

Besides function parameters, variables can be type-annotated too; you can call `__annotations__` to view all type annotations in a function. Variable type annotations give Python a property of statically typed languages: declaration and assignment separated:

```
1 >>> nothing: str
2 >>> nothing
3 NameError: name 'nothing' is not defined
4
5 >>> __annotations__
6 {'nothing': <class 'str'>}
```

38.11 Decorators

A decorator is a function that takes a function or method as its only argument and returns a new function or method that incorporates the decorated function or method with some extra functionality attached. In short, it adds functionality to a function without modifying the function's code. See the [official documentation](#) for related knowledge.

Some decorators are very practical in contests, such as `lru_cache`, which automatically adds memoization to a function — very useful in recursive algorithms:

```
@lru_cache(maxsize=128, typed=False)
```

- There are 2 parameters: `maxsize` and `typed`; if not passed, `maxsize` defaults to 128 and `typed` defaults to `False`.
- `maxsize` is the capacity of the LRU cache, i.e., the maximum number of cacheable results of the decorated method. A value of 128 means the decorated method can cache at most 128 return results; passing `maxsize=None` means unlimited caching.
- If `typed` is `True`, function arguments of different types are cached separately; e.g., `f(3)` and `f(3.0)` are cached twice.

Here is an example of using `lru_cache` to optimize Fibonacci computation:

```
1 @lru_cache(maxsize=None)
2 def fib(n):
3     if n < 2:
4         return n
5     return fib(n - 1) + fib(n - 2)
```

38.12 Common Built-in Libraries

Here are some built-in libraries that may be useful for writing algorithms; for concrete usage, search on your own or read the [official documentation](#).

Library	Purpose
<code>array</code>	fixed-length arrays
<code>argparse</code>	command-line argument processing
<code>bisect</code>	binary search
<code>collections</code>	ordered dictionaries, deques, and other data structures
<code>fractions</code>	rational numbers
<code>heapq</code>	heap-based priority queue
<code>io</code>	file streams, in-memory streams
<code>itertools</code>	iterators
<code>math</code>	mathematical functions
<code>os.path</code>	system paths, etc.
<code>random</code>	random numbers
<code>re</code>	regular expressions
<code>struct</code>	converting between structs and binary data
<code>sys</code>	system information

38.13 Comparing C++ and Python on an Example Problem

Note — Example problem: Luogu P4779 “Single-source shortest path (standard version)”^a

^a<https://www.luogu.com.cn/problem/P4779>

Given a directed graph with non-negative weights on n ($1 \leq n \leq 10^5$) vertices and m ($1 \leq m \leq 2 \times 10^5$) edges, compute the distance from s to every vertex. The data guarantees every vertex is reachable from s .

38.13.1 Declaring Constants

In C++:

```
1 #include <cstdio>
2 #include <cstring>
3 #include <queue>
4 #include <vector>
5 using namespace std;
6 constexpr int N = 1e5 + 5, M = 2e5 + 5;
```

In Python:

```
1 try: # import the priority-queue module
2     import Queue as pq # python version < 3.0
3 except ImportError:
```

```

4     import queue as pq  # python3.*
5
6     N = int(1e5 + 5)
7     M = int(2e5 + 5)
8     INF = 0x3F3F3F3F

```

38.13.2 Declaring the Forward-Star Structure and Other Variables

In C++:

```

1  struct qxx {
2      int nex, t, v;
3  };
4
5  qxx e[M];
6  int h[N], cnt;
7
8  void add_path(int f, int t, int v) { e[++cnt] = qxx{h[f], t, v}, h[f] = cnt; }
9
10 using pii = pair<int, int>;
11 priority_queue<pii, vector<pii>, greater<pii>> q;
12 int dist[N];

```

In Python:

```

1  class qxx: # the forward-star class (struct)
2      def __init__(self):
3          self.nex = 0
4          self.t = 0
5          self.v = 0
6
7
8  e = [qxx() for i in range(M)] # linked lists
9  h = [0 for i in range(N)]
10 cnt = 0
11
12 dist = [INF for i in range(N)]
13 q = pq.PriorityQueue() # define a priority queue; by default a min-heap on
14                        # the first element
15
16
17 def add_path(f, t, v): # add an edge in the forward star
18     # to modify a global variable you must declare it with global
19     global cnt, e, h
20     # a debug print; several variables are packed in a tuple
21     # print("add_path(%d,%d,%d)" % (f,t,v))
22     cnt += 1
23     e[cnt].nex = h[f]
24     e[cnt].t = t
25     e[cnt].v = v
26     h[f] = cnt

```

38.13.3 Dijkstra's Algorithm

In C++:

```

1 void dijkstra(int s) {
2     memset(dist, 0x3f, sizeof(dist));
3     dist[s] = 0, q.push(make_pair(0, s));
4     while (q.size()) {
5         pii u = q.top();
6         q.pop();
7         if (dist[u.second] < u.first) continue;
8         for (int i = h[u.second]; i; i = e[i].nex) {
9             const int &v = e[i].t, &w = e[i].v;
10            if (dist[v] <= dist[u.second] + w) continue;
11            dist[v] = dist[u.second] + w;
12            q.push(make_pair(dist[v], v));
13        }
14    }
15 }

```

In Python:

```

1 def nextedgeid(u): # a generator, usable inside for loops
2     i = h[u]
3     while i:
4         yield i
5         i = e[i].nex
6
7
8 def dijkstra(s):
9     dist[s] = 0
10    q.put((0, s))
11    while not q.empty():
12        u = q.get() # the get function also removes the element from the heap
13        if dist[u[1]] < u[0]:
14            continue
15        for i in nextedgeid(u[1]):
16            v = e[i].t
17            w = e[i].v
18            if dist[v] <= dist[u[1]] + w:
19                continue
20            dist[v] = dist[u[1]] + w
21            q.put((dist[v], v))

```

38.13.4 The Main Function

In C++:

```

1 int n, m, s;
2
3 int main() {
4     scanf("%d%d%d", &n, &m, &s);
5     for (int i = 1; i <= m; i++) {
6         int u, v, w;

```



```

7     scanf("%d%d%d", &u, &v, &w);
8     add_path(u, v, w);
9 }
10 dijkstra(s);
11 for (int i = 1; i <= n; i++) printf("%d ", dist[i]);
12 return 0;
13 }

```

In Python:

```

1 if __name__ == "__main__":
2     # read several integers on one line; note it reads the whole line
3     n, m, s = map(int, input().split())
4     for i in range(m):
5         u, v, w = map(int, input().split())
6         add_path(u, v, w)
7
8     dijkstra(s)
9
10    for i in range(1, n + 1):
11        print(dist[i], end=" ")
12
13    print()

```

38.13.5 Complete Code

In C++:

```

1 #include <cstdio>
2 #include <cstring>
3 #include <queue>
4 #include <vector>
5 using namespace std;
6 constexpr int N = 1e5 + 5, M = 2e5 + 5;
7
8 struct qxx {
9     int nex, t, v;
10 };
11
12 qxx e[M];
13 int h[N], cnt;
14
15 void add_path(int f, int t, int v) { e[++cnt] = qxx{h[f], t, v}, h[f] = cnt; }
16
17 using pii = pair<int, int>;
18 priority_queue<pii, vector<pii>, greater<pii>> q;
19 int dist[N];
20
21 void dijkstra(int s) {
22     memset(dist, 0x3f, sizeof(dist));
23     dist[s] = 0, q.push(make_pair(0, s));
24     while (q.size()) {
25         pii u = q.top();

```

```

26     q.pop();
27     if (dist[u.second] < u.first) continue;
28     for (int i = h[u.second]; i; i = e[i].nex) {
29         const int &v = e[i].t, &w = e[i].v;
30         if (dist[v] <= dist[u.second] + w) continue;
31         dist[v] = dist[u.second] + w;
32         q.push(make_pair(dist[v], v));
33     }
34 }
35 }
36
37 int n, m, s;
38
39 int main() {
40     scanf("%d%d%d", &n, &m, &s);
41     for (int i = 1; i <= m; i++) {
42         int u, v, w;
43         scanf("%d%d%d", &u, &v, &w);
44         add_path(u, v, w);
45     }
46     dijkstra(s);
47     for (int i = 1; i <= n; i++) printf("%d ", dist[i]);
48     return 0;
49 }

```

In Python:

```

1  try: # import the priority-queue module
2      import Queue as pq # python version < 3.0
3  except ImportError:
4      import queue as pq # python3.*
5
6  N = int(1e5 + 5)
7  M = int(2e5 + 5)
8  INF = 0x3F3F3F3F
9
10
11 class qxx: # the forward-star class (struct)
12     def __init__(self):
13         self.nex = 0
14         self.t = 0
15         self.v = 0
16
17
18 e = [qxx() for i in range(M)] # linked lists
19 h = [0 for i in range(N)]
20 cnt = 0
21
22 dist = [INF for i in range(N)]
23 q = pq.PriorityQueue() # define a priority queue; by default a min-heap on
24                        # the first element
25
26
27 def add_path(f, t, v): # add an edge in the forward star

```

```

28     # to modify a global variable you must declare it with global
29     global cnt, e, h
30     # a debug print; several variables are packed in a tuple
31     # print("add_path(%d,%d,%d)" % (f,t,v))
32     cnt += 1
33     e[cnt].nex = h[f]
34     e[cnt].t = t
35     e[cnt].v = v
36     h[f] = cnt
37
38
39 def nextedgeid(u): # a generator, usable inside for loops
40     i = h[u]
41     while i:
42         yield i
43         i = e[i].nex
44
45
46 def dijkstra(s):
47     dist[s] = 0
48     q.put((0, s))
49     while not q.empty():
50         u = q.get()
51         if dist[u[1]] < u[0]:
52             continue
53         for i in nextedgeid(u[1]):
54             v = e[i].t
55             w = e[i].v
56             if dist[v] <= dist[u[1]] + w:
57                 continue
58             dist[v] = dist[u[1]] + w
59             q.put((dist[v], v))
60
61
62 # if this Python code is run directly (not imported as a module), run the
63 # command
64 if __name__ == "__main__":
65     # read several integers on one line; note it reads the whole line
66     n, m, s = map(int, input().split())
67     for i in range(m):
68         u, v, w = map(int, input().split())
69         add_path(u, v, w)
70
71     dijkstra(s)
72
73     for i in range(1, n + 1):
74         # both output syntaxes work
75         print("{}".format(dist[i]), end=" ")
76         # print("%d" % dist[i],end=' ')
77
78     print() # newline at the end

```

38.14 Reference Documents

1. Python Documentation, <https://www.python.org/doc/>
2. Official Python tutorial (Chinese), <https://docs.python.org/zh-cn/3/tutorial/>
3. Learn Python3 In Y Minutes, <https://learnxinyminutes.com/docs/python3/>
4. Real Python Tutorials, <https://realpython.com/>
5. Liao Xuefeng's Python tutorial (Chinese), <https://www.liaoxuefeng.com/wiki/1016959663602400/>
6. GeeksforGeeks: Python Tutorials, <https://www.geeksforgeeks.org/python-programming-language/>

38.15 References and Notes

- The Python interpreter — Python 3 documentation, <https://docs.python.org/zh-cn/3/tutorial/interpreter.html>
- Unicode HOWTO — Python 3 documentation, <https://docs.python.org/zh-cn/3/howto/unicode.html>

39 A Crash Course in Java (java)

Original title: Java速成 · source page [/lang/java/](#)

39.1 About Java

Java is a widely used computer programming language with **cross-platform**, **object-oriented**, and **generic-programming** features, widely applied in enterprise web application development and mobile application development.

39.2 Environment Installation

See JDK (OI Wiki → Tools → Compilers → JDK).⁹⁶

39.3 Basic Syntax

39.3.1 The Main Function

Java, like C/C++, needs a function (called a *method* in object-oriented terms) as the entry point of program execution.

The format of Java's main function is fixed, as follows:

⁹⁶<https://oi-wiki.org/tools/compiler/#jdk>

```

1 class Test {
2     public static void main(String[] args) {
3         // the program's code
4     }
5 }

```

A packaged Java program (usually named `*.jar`) may contain many similar functions, but when the program runs, only one of them is executed — this is defined in the `Jar's Manifest` file. In OI contests you generally do not need knowledge about it.

39.3.2 Comments

As in C/C++, Java uses `//` and `/* */` to comment single and multiple lines respectively.

39.3.3 Basic Data Types

Type name	Meaning
<code>boolean</code>	Boolean type
<code>byte</code>	byte type
<code>char</code>	character type
<code>double</code>	double-precision float
<code>float</code>	single-precision float
<code>int</code>	integer type
<code>long</code>	long integer
<code>short</code>	short integer
<code>null</code>	null

39.3.4 Declaring Variables

```

1 int a = 12; // set a to integer type and assign 12 to a
2 String str = "Hello, OI-wiki"; // declare the string variable str
3 char ch = 'W';
4 double PI = 3.1415926;

```

39.3.5 The final Keyword

`final` means this is a final, unchangeable result: a variable qualified by `final` can only be assigned once and does not change after assignment.

```

1 final double PI = 3.1415926;

```

39.3.6 Arrays

```
1 // an integer-type array of ten elements
2 // syntax: DataType[] variableName = new DataType[arraySize]
3 int[] ary = new int[10];
```

39.3.7 Strings

- Strings are a built-in class of Java.

```
1 // the simplest way to construct a string variable is as follows
2 String a = "Hello";
3
4 // a string variable can also be constructed from a char array
5 char[] stringArray = { 'H', 'e', 'l', 'l', 'o' };
6 String s = new String(stringArray);
```

39.3.8 Packages and Importing Packages

Classes in Java are placed in packages (**package**). Within one package, same-named classes are not allowed. The first line of a class usually states which package the class belongs to. For example:

```
1 package org.oi-wiki.tutorial;
```

The naming convention for packages is generally: **projectOwnerTLD.projectOwnerSLD.projectName**.

Use the **import** keyword to import classes not under this class's own package. For example the **Scanner** used below:

```
1 import java.util.Scanner;
```

If you want to import all classes under some package, just replace the class name before the final semicolon with *****.

39.3.9 Input

Command-line input can be handled through the **Scanner** class.

```
1 package org.oiwiki.tutorial;
2
3 import java.util.Scanner;
4
5 class Test {
6     public static void main(String[] args) {
7         Scanner scan = new Scanner(System.in); // System.in is the input stream
8         int a = scan.nextInt();
9         double b = scan.nextDouble();
10        String c = scan.nextLine();
11    }
```

```
12 }
```

39.3.10 Output

Variables can be output in formatted fashion.

Symbol	Meaning
%f	floating-point type
%s	string type
%d	integer type
%c	character type

```
1 class Test {
2     public static void main(String[] args) {
3         int a = 12;
4         char b = 'A';
5         double s = 3.14;
6         String str = "Hello world";
7         System.out.printf("%f\n", s);
8         System.out.printf("%d\n", a);
9         System.out.printf("%c\n", b);
10        System.out.printf("%s\n", str);
11    }
12 }
```

39.3.11 Control Statements

Java's flow-control statements are basically the same as C++'s.

Selection. if:

```
1 class Test {
2     public static void main(String[] args) {
3         if ( /* test condition */ ){
4             // the code here runs when the condition holds
5         }
6     }
7 }
```

if...else:

```
1 class Test {
2     public static void main(String[] args) {
3         if ( /* test condition */ ) {
4             // the code here runs when the condition holds
5         } else {
6             // the code here runs when the condition does not hold
7         }
8     }
9 }
```

```

8     }
9 }

```

if...else if...else:

```

1  class Test {
2      public static void main(String[] args) {
3          if ( /* test condition */ ) {
4              // runs when the test condition holds
5          } else if ( /* test condition 2 */ ) {
6              // runs when test condition 2 holds
7          } else {
8              // runs when none of the conditions above hold
9          }
10     }
11 }

```

switch...case:

```

1  class Test {
2      public static void main(String[] args) {
3          switch ( /* expression */ ){
4              case /* value 1 */:
5              // this code runs when the expression's value matches value 1
6                  break; // without a break statement, execution falls through in
7                          // order until a break
8              case /* value 2 */:
9              // this code runs when the expression's value matches value 2
10                 break;
11             default:
12                 // this code runs when the expression matches none of the listed
13                 // values
14             }
15         }
16     }

```

Loops. **for:** the **for** keyword has two usages. The first is the ordinary **for** loop, in the form:

```

1  class Test {
2      public static void main(String[] args) {
3          for ( /* init */; /* loop test condition */; /* step after each loop */ ) {
4              // the loop body runs while the loop condition holds
5          }
6      }
7  }

```

The second is a usage like C++'s **foreach**, looping over the data in an array or collection; it hides the loop variable of the previous form. The form is:

```

1  class Test {
2      public static void main(String[] args) {
3          for ( /* elementType X */ /* elementName Y */ : /* collection Z */ ) {

```



```
4         // in each iteration of this block, Y is one element of Z.
5     }
6 }
7 }
```

while:

```
1 class Test {
2     public static void main(String[] args) {
3         while ( /* test condition */ ) {
4             // the loop body runs while the condition holds
5         }
6     }
7 }
```

do...while:

```
1 class Test {
2     public static void main(String[] args) {
3         do {
4             // the code to be executed
5         } while ( /* loop test condition */ );
6     }
7 }
```

39.4 Points of Caution

39.4.1 Class Name Must Match File Name

Creating a Java source program requires the class name and the file name to match for compilation to succeed; otherwise the compiler reports that it cannot find the class. The file name is usually specified by the concrete OJ.

Example: `Add.java`

```
1 class Add {
2     public static void main(String[] args) {
3         // ...
4     }
5 }
```

In this file the class name must be `Add` for compilation to succeed.

40 Advanced Java (java-pro)

Original title: Java进阶 · source page /lang/java-pro/

Warning — Note

The following content is all written against Java JDK 8; changes in higher versions are not excluded.

40.1 Faster Input and Output

`Scanner` and `System.out.print` work fine at the very beginning, but efficiency drops when handling larger inputs, so we need some methods to raise IO speed.

40.1.1 Using Kattio + StringTokenizer for Input

One of the most common methods is to use `Kattio.java` from Kattis⁹⁷ to raise IO efficiency.⁹⁸ This method wraps `StringTokenizer` and `PrintWriter` in one class for convenience. When actually solving problems (if the contest/organizer permits), you can use this template directly.

Below is the IO template to include in your code. Since Kattis's original Kattio contains some rarely used features, the template below has been adjusted (the original Kattio uses the MIT license).

```

1  class Kattio extends PrintWriter {
2      private BufferedReader r;
3      private StringTokenizer st;
4      // standard IO
5      public Kattio() { this(System.in, System.out); }
6      public Kattio(InputStream i, OutputStream o) {
7          super(o);
8          r = new BufferedReader(new InputStreamReader(i));
9      }
10     // file IO
11     public Kattio(String input, String output) throws IOException {
12         super(output);
13         r = new BufferedReader(new FileReader(input));
14     }
15     // returns null when there is no further input
16     public String next() {
17         try {
18             while (st == null || !st.hasMoreTokens())
19                 st = new StringTokenizer(r.readLine());
20             return st.nextToken();
21         } catch (Exception e) {}
22         return null;
23     }
24     public int nextInt() { return Integer.parseInt(next()); }
25     public double nextDouble() { return Double.parseDouble(next()); }

```

⁹⁷<https://github.com/Kattis/kattio/blob/master/Kattio.java>

⁹⁸Input & Output — USACO Guide, <https://usaco.guide/general/input-output?lang=java#method-3---io-template>

```

26     public long nextLong() { return Long.parseLong(next()); }
27 }

```

The code below briefly shows Kattio in use:

```

1  class Test {
2      public static void main(String[] args) {
3          Kattio io = new Kattio();
4          // string input
5          String str = io.next();
6          // int input
7          int num = io.nextInt();
8          // output
9          io.println("Result");
10         // make sure to close the IO streams so the output is written correctly
11         io.close();
12     }
13 }

```

40.1.2 Using StreamTokenizer for Input

In some situations `StringTokenizer` causes MLE (Memory Limit Exceeded); then we need `StreamTokenizer` for input.

```

1  import java.io.*;
2  public class Main {
3      // IO code
4      public static StreamTokenizer in = new StreamTokenizer(new BufferedReader(new
5      ⇨ InputStreamReader(System.in), 32768));
6      public static PrintWriter out = new PrintWriter(new OutputStreamWriter(System.out))
7      ⇨ ;
8      public static double nextDouble() throws IOException { in.nextToken(); return in.
9      ⇨ nval; }
10     public static float nextFloat() throws IOException { in.nextToken(); return (float)
11     ⇨ in.nval; }
12     public static int nextInt() throws IOException { in.nextToken(); return (int)in.
13     ⇨ nval; }
14     public static String next() throws IOException { in.nextToken(); return in.sval; }
15     public static long nextLong() throws Exception { in.nextToken(); return (long)in.
16     ⇨ nval; }
17
18     // usage example
19     public static void main(String[] args) throws Exception {
20         int n = nextInt();
21         out.println(n);
22         out.close();
23     }
24 }

```

40.1.3 Analysis and Comparison of Kattio + StringTokenizer versus StreamTokenizer

1. `StreamTokenizer` uses less memory than `StringTokenizer`; when a Java model solution MLEs, you can try `StreamTokenizer`. But `StreamTokenizer` loses precision, and problems appear when reading certain data:
 - the `StreamTokenizer` source has a `Type` that decides the type from the input content: if the input is a string like `1230i starting with a digit`, it forcibly assumes the type is `double`, so reading a `String` as `double` throws an exception;
 - `StreamTokenizer` loses precision on numbers of size `1e14` and above.
2. When using `PrintWriter`, note that at the end of the program you must `close()` the output stream, or use `flush()` when output is needed; otherwise the content is not written to the console/file.
3. `Kattio` inherits from the `PrintWriter` class, so its objects have `PrintWriter`'s functionality: you can directly call `PrintWriter`'s functions to output, while modifying `StringTokenizer` as its own member variable. The second `Main` instead makes both `StreamTokenizer` and `PrintWriter` its member variables, so usage differs slightly.

In summary: in most situations, `StringTokenizer` is in the better position; in extreme-MLE situations try `StreamTokenizer`; meanwhile, for data beyond the `int` range, `StreamTokenizer` is powerless.

40.2 BigInteger and Number Theory

`BigInteger` is Java's high-precision computation class, solving high-precision problems conveniently.

40.2.1 Initialization

`BigInteger` is commonly created in the following ways:

```

1  import java.io.PrintWriter;
2  import java.math.BigInteger;
3
4  class Main {
5      static PrintWriter out = new PrintWriter(System.out);
6      public static void main(String[] args) {
7          BigInteger a = new BigInteger("12345678910"); // create a BigInteger
8                  // object from the string, interpreted in decimal
9          out.println(a); // a has value 12345678910 BigInteger b = new BigInteger("1E",
↪ 16); // create a BigInteger object
10                  // from the string in the given radix
11          out.println(b); // b has value 30 out.close();
12      }
13  }
```

40.2.2 Basic Operations

Below, `this` stands for the current `BigInteger`:

Function	Effect
<code>abs()</code>	returns the absolute value of <code>this</code>
<code>negate()</code>	returns the opposite of <code>this</code>
<code>add(BigInteger val)</code>	returns the sum of <code>this</code> and <code>val</code>
<code>subtract(BigInteger val)</code>	returns the difference of <code>this</code> and <code>val</code>
<code>multiply(BigInteger val)</code>	returns the product of <code>this</code> and <code>val</code>
<code>divide(BigInteger val)</code>	returns the quotient of <code>this</code> and <code>val</code>
<code>remainder(BigInteger val)</code>	returns the remainder of <code>this</code> divided by <code>val</code>
<code>mod(BigInteger val)</code>	returns <code>this</code> modulo <code>val</code>
<code>pow(int val)</code>	returns <code>this</code> raised to the power <code>val</code>
<code>and(BigInteger val)</code>	returns the bitwise AND of <code>this</code> and <code>val</code>
<code>or(BigInteger val)</code>	returns the bitwise OR of <code>this</code> and <code>val</code>
<code>not()</code>	returns the bitwise NOT of <code>this</code>
<code>xor(BigInteger val)</code>	returns the bitwise XOR of <code>this</code> and <code>val</code>
<code>shiftLeft(int n)</code>	returns <code>this</code> shifted left by <code>n</code> bits
<code>shiftRight(int n)</code>	returns <code>this</code> shifted right by <code>n</code> bits
<code>max(BigInteger val)</code>	returns the larger of <code>this</code> and <code>val</code>
<code>min(BigInteger val)</code>	returns the smaller of <code>this</code> and <code>val</code>
<code>bitCount()</code>	returns the number of 1s in the binary of <code>this</code> , excluding the sign bit
<code>bitLength()</code>	returns the length of the binary of <code>this</code> , excluding the sign bit
<code>getLowestSetBit()</code>	returns the position of the rightmost set bit in the binary of <code>this</code>
<code>compareTo(BigInteger val)</code>	compares the values of <code>this</code> and <code>val</code>
<code>toString()</code>	returns the decimal string form of <code>this</code>
<code>toString(int radix)</code>	returns the radix-radix string form of <code>this</code>

Usage example:

```

1  import java.io.PrintWriter;
2  import java.math.BigInteger;
3
4  public class Main {
5      static PrintWriter out = new PrintWriter(System.out);
6      static BigInteger a, b;
7
8      static void abs() {
9          out.println("abs:");
10         a = new BigInteger("-123");
11         out.println(a.abs()); // result: 123
12         a = new BigInteger("123");
13         out.println(a.abs()); // result: 123
14     }
15
16     static void negate() {
17         out.println("negate:");

```

```
18     a = new BigInteger("-123");
19     out.println(a.negate()); // result: 123
20     a = new BigInteger("123");
21     out.println(a.negate()); // result: -123
22 }
23
24 static void add() {
25     out.println("add:");
26     a = new BigInteger("123");
27     b = new BigInteger("123");
28     out.println(a.add(b)); // result: 246
29 }
30
31 static void subtract() {
32     out.println("subtract:");
33     a = new BigInteger("123");
34     b = new BigInteger("123");
35     out.println(a.subtract(b)); // result: 0
36 }
37
38 static void multiply() {
39     out.println("multiply:");
40     a = new BigInteger("12");
41     b = new BigInteger("12");
42     out.println(a.multiply(b)); // result: 144
43 }
44
45 static void divide() {
46     out.println("divide:");
47     a = new BigInteger("12");
48     b = new BigInteger("11");
49     out.println(a.divide(b)); // result: 1
50 }
51
52 static void remainder() {
53     out.println("remainder:");
54     a = new BigInteger("12");
55     b = new BigInteger("10");
56     out.println(a.remainder(b)); // result: 2
57     a = new BigInteger("-12");
58     b = new BigInteger("10");
59     out.println(a.remainder(b)); // result: -2
60 }
61
62 static void mod() {
63     out.println("mod:");
64     a = new BigInteger("12");
65     b = new BigInteger("10");
66     out.println(a.mod(b)); // result: 2
67     a = new BigInteger("-12");
68     b = new BigInteger("10");
69     out.println(a.mod(b)); // result: 8
70 }
71
```

```
72 static void pow() {
73     out.println("pow:");
74     a = new BigInteger("2");
75     out.println(a.pow(10)); // result: 1024
76 }
77
78 static void and() {
79     out.println("and:");
80     a = new BigInteger("3"); // 11
81     b = new BigInteger("5"); // 101
82     out.println(a.and(b)); // result: 1
83 }
84
85 static void or() {
86     out.println("or:");
87     a = new BigInteger("2"); // 10
88     b = new BigInteger("5"); // 101
89     out.println(a.or(b)); // result: 7
90 }
91
92 static void not() {
93     out.println("not:");
94     a = new BigInteger("2147483647"); // 01111111 11111111 11111111 11111111
95     out.println(a.not()); // result: -2147483648, whose binary form is
96     // 10000000 00000000 00000000 00000000
97 }
98
99 static void xor() {
100     out.println("xor:");
101     a = new BigInteger("6"); // 110
102     b = new BigInteger("5"); // 101
103     out.println(a.xor(b)); // 011 prints 3
104 }
105
106 static void shiftLeft() {
107     out.println("shiftLeft:");
108     a = new BigInteger("1");
109     out.println(a.shiftLeft(10)); // result: 1024
110 }
111
112 static void shiftRight() {
113     out.println("shiftRight:");
114     a = new BigInteger("1024");
115     out.println(a.shiftRight(8)); // result: 4
116 }
117
118 static void max() {
119     out.println("max:");
120     a = new BigInteger("6");
121     b = new BigInteger("5");
122     out.println(a.max(b)); // result: 6
123 }
124
125 static void min() {
```

```
126     out.println("min:");
127     a = new BigInteger("6");
128     b = new BigInteger("5");
129     out.println(a.min(b)); // result: 5
130 }
131
132 static void bitCount() {
133     out.println("bitCount:");
134     a = new BigInteger("6"); // 110
135     out.println(a.bitCount()); // result: 2
136 }
137
138 static void bitLength() {
139     out.println("bitLength:");
140     a = new BigInteger("6"); // 110
141     out.println(a.bitLength()); // result: 3
142 }
143
144 static void getLowestSetBit() {
145     out.println("getLowestSetBit:");
146     a = new BigInteger("8"); // 1000
147     out.println(a.getLowestSetBit()); // result: 3
148 }
149
150 static void compareTo() {
151     out.println("compareTo:");
152     a = new BigInteger("8");
153     b = new BigInteger("9");
154     out.println(a.compareTo(b)); // result: -1
155     a = new BigInteger("8");
156     b = new BigInteger("8");
157     out.println(a.compareTo(b)); // result: 0
158     a = new BigInteger("8");
159     b = new BigInteger("7");
160     out.println(a.compareTo(b)); // result: 1
161 }
162
163 static void toStringTest() {
164     out.println("toString:");
165     a = new BigInteger("15");
166     out.println(a.toString()); // result: 15
167     out.println(a.toString(16)); // result: f
168 }
169
170 public static void main(String[] args) {
171     abs();
172     negate();
173     add();
174     subtract();
175     multiply();
176     divide();
177     remainder();
178     mod();
179     pow();
```



```

180         and();
181         or();
182         not();
183         xor();
184         shiftLeft();
185         shiftRight();
186         max();
187         min();
188         bitCount();
189         bitLength();
190         getLowestSetBit();
191         compareTo();
192         toStringTest();
193         out.close();
194     }
195 }

```

40.2.3 Mathematical Operations

Below, **this** stands for the current `BigInteger`:

Function	Effect
<code>gcd(BigInteger val)</code>	returns the gcd of the absolute values of this and <code>val</code>
<code>isProbablePrime(int val)</code>	returns a Boolean indicating whether this is prime
<code>nextProbablePrime()</code>	returns the first prime greater than this
<code>modPow(BigInteger b, BigInteger p)</code>	returns this to the power <code>b</code> , modulo <code>p</code>
<code>modInverse(BigInteger p)</code>	returns the multiplicative inverse of this modulo <code>p</code>

Usage example:

```

1  import java.io.PrintWriter;
2  import java.math.BigInteger;
3
4  public class Main {
5      static PrintWriter out = new PrintWriter(System.out);
6      static BigInteger a, b, p;
7
8      static void gcd() { // greatest common divisor a = new BigInteger
9          ↪ ("120032414321432144212100");
10         b = new BigInteger("240231431243123412432140");
11         out.println(String.format("gcd(%s,%s)=%s", a.toString(), b.toString(), a.gcd(b)
12         ↪ .toString())); // gcd(120032414321432144212100,240231431243123412432140)=20
13     }
14
15     static void isPrime() { // Miller-Rabin-based primality test; larger parameters
16         ↪ give higher

```

```

14      // accuracy and higher complexity. Accuracy is (1-1/(val*2)) a = new BigInteger
    ↪ ("1200324143214321442127");
15      out.println("a:" + a.toString());
16      out.println(a.isProbablePrime(10) ? "a is prime" : "a is not prime"); // a is
    ↪ not prime
17  }
18
19  static void nextPrime() { // find the next prime after the number a = new
    ↪ BigInteger("1200324143214321442127");
20      out.println("a:" + a.toString());
21      out.println(String.format("a nextPrime is %s", a.nextProbablePrime().toString()
    ↪ )); // a nextPrime is 1200324143214321442199
22  }
23
24  static void modPow() { // fast exponentiation, faster than the ordinary version,
    ↪ with internal
25      // mathematical optimizations a = new BigInteger("2");
26      b = new BigInteger("10");
27      p = new BigInteger("1000");
28      out.println(String.format("a:%s b:%s p:%s", a, b, p));
29      out.println(String.format("a^b mod p:%s", a.modPow(b, p).toString())); // 24
30  }
31
32  static void modInverse() { // modular inverse a = new BigInteger("10");
33      b = new BigInteger("3");
34      out.println(a.modInverse(b)); // a ^ (p-2) mod p = 1
35  }
36
37  public static void main(String[] args) {
38      gcd();
39      isPrime();
40      nextPrime();
41      modPow();
42      modInverse();
43      out.close();
44  }
45  }

```

For Miller–Rabin background, see Miller–Rabin primality testing (OI Wiki → Mathematics → Number theory → Prime).⁹⁹

40.3 Primitive Types and Wrapper Types

⁹⁹<https://oi-wiki.org/math/number-theory/prime/>

40.3.1 Introduction

Because primitive types lack object-oriented features, so that they can join object-oriented development, Java provides a corresponding wrapper class for each of the eight primitive types: `Byte`, `Double`, `Float`, `Integer`, `Long`, `Short`, `Character`, and `Boolean`. The correspondence:

Primitive type	Wrapper type
<code>byte</code>	<code>Byte</code>
<code>short</code>	<code>Short</code>
<code>boolean</code>	<code>Boolean</code>
<code>char</code>	<code>Character</code>
<code>int</code>	<code>Integer</code>
<code>long</code>	<code>Long</code>
<code>float</code>	<code>Float</code>
<code>double</code>	<code>Double</code>

40.3.2 Differences

Taking `int` and `Integer` as the example:

1. `Integer` is the wrapper class of `int`; `int` is a Java primitive data type.
2. `Integer` must be instantiated before use; `int` need not be.
3. `Integer` actually corresponds to a reference: when you `new` an `Integer`, an object is really generated, while `int` stores the data directly.
4. `Integer`'s default value is `null`, and it accepts both `null` and `int` data; `int`'s default value is 0 and it cannot accept `null`.
5. Using `==` to test whether two `Integer` variables are the same may produce incorrect results; only `equals()` may be used, whereas `int` can use `==` directly.

40.3.3 Boxing and Unboxing

Again taking `int` and `Integer` as the example:

`Integer` is essentially an object; `int` is a primitive type. The two cannot be assigned to each other directly. When conversion is needed, converting the primitive type into the wrapper type is called boxing, and the reverse unboxing.

```
1 // primitive type
2 int value1 = 1;
3 // autoboxing converts to the wrapper type
4 Integer integer = Integer.valueOf(value1);
5 // unboxing converts to the primitive type
6 int value2 = integer.intValue();
```

Java 5 introduced automatic boxing/unboxing:

```
1 Integer integer = 1;
2 int value = integer;
```

Warning — Note

Although the JDK added automatic boxing/unboxing, choose the right type when declaring variables: the wrapper type `Integer` can accept `null` while the primitive `int` cannot. Unboxing a wrapper type holding `null` throws an exception. The code below shows this behavior.

```
1 Integer integer = Integer.valueOf(null);
2 integer.intValue(); // throws a java.lang.NumberFormatException exception
3
4 Integer integer = null;
5 integer.intValue(); // throws a java.lang.NullPointerException exception
```

40.4 Inheritance

Creating a new design based on an existing design is inheritance in object-oriented programming. In inheritance the new class does not arise out of thin air; it is defined based on an already existing class. Through inheritance the new class automatically obtains all members of the base class, including member variables and methods, members of all access attributes, whether **public** or **private**. Clearly, defining a new class through inheritance is far simpler, faster, and more convenient than writing a new class from scratch. Inheritance is one of the important means of supporting code reuse.

In Java, the keyword for inheritance is **extends**; Java supports only single inheritance, but multiple interfaces can be implemented.

In Java, every class is a subclass of **Object**.

When a subclass inherits from a parent class, all the parent class's members, including variables and methods, become members of the subclass — except constructors. Constructors are exclusive to the parent class: since their names are the class's name, the parent's constructors do not exist in the subclass. Other than that, the subclass inherits all the parent class's members.

Each member has a different access attribute. The subclass inherits all the parent's members, but the differing access attributes make the subclass's use of them different: some parent members directly become the subclass's outward-facing interface; others are deeply hidden, inaccessible even to the subclass itself.

The table below lists the access attributes of parent members with different access attributes inside the subclass:

Parent member access	Meaning in the parent class	Meaning in the subclass
<code>public</code>	open to all classes	open to all classes
<code>protected</code>	only other classes in the package, itself, and subclasses	only other classes in the package, itself,
default	only other classes in the package	if the subclass is in the same package as
<code>private</code>	only itself	inaccessible

40.5 Polymorphism

In Java, when an object is assigned to a variable, the object’s type must match the variable’s type. But since Java has inheritance, this can be restated as **a variable may hold objects of its declared type or any subtype of that type**.

If a type implements an interface, it may also be called a subtype of that interface.

Variables holding object types in Java are polymorphic variables. The term “polymorphism” (literally “many forms”) refers to a variable being able to hold objects of different types (its declared type or any subtype).

Polymorphic variables:

1. Java’s object variables are polymorphic: they can hold objects of more than one type.
2. They can hold objects of the declared type, or objects of subclasses of the declared type.
3. When a subclass object is assigned to a parent-class variable, an upcast occurs.

40.6 Generics

Generics mean that when a class is defined, the concrete types of its attributes or method parameters are not set; the types are defined when using (or creating) objects. Generics are essentially parameterized types: the data type being operated on is specified as a parameter.

Generics provide a mechanism of compile-time type-safety checking, allowing illegal types to be detected at compile time.

40.7 Interfaces

40.7.1 Introduction

An interface in Java is an abstract type, a collection of abstract methods, usually declared with **interface**. A class inherits the interface’s abstract methods by implementing the interface.

An interface is not a class; the way one writes an interface resembles a class a great deal, but they are different concepts. A class describes the attributes and methods of objects; an interface contains the methods a class must implement.

Unless the class implementing the interface is abstract, the class must define all methods of the interface.

An interface cannot be instantiated but can be implemented. A class implementing an interface must implement all methods described in the interface, or else be declared abstract. Also, in Java an interface type can declare a variable: it may become a null pointer, or be bound to an object implementing that interface.

40.7.2 Differences from Classes

1. An interface cannot be used to instantiate objects.
2. An interface has no constructors.
3. All methods of an interface must be abstract methods; from Java 8 on, an interface may use non-abstract methods decorated with the `default` keyword.
4. An interface cannot contain member variables, except `static` and `final` ones.
5. An interface is not inherited by a class but implemented by a class.
6. Interfaces support multiple inheritance; classes do not.

40.7.3 Declaration

```
1 [visibility] interface InterfaceName [extends OtherInterfaces] {  
2     // declare variables  
3     // abstract methods  
4 }
```

40.7.4 Implementation

```
1 ...implements InterfaceName[, OtherInterfaceNames, OtherInterfaceNames...,  
2     ...] ...
```

40.8 Lambda Expressions

40.8.1 Introduction

Lambda expressions, also called closures, are the most important new feature of Java 8.

Lambda expressions allow a function to be passed as a parameter of a method (functions passed into methods as arguments).

Using lambda expressions makes code more concise and compact.

40.8.2 Syntax

- Optional type declarations: parameter types need not be declared; the compiler can infer parameter values uniformly.
- Optional parentheses around parameters: one parameter needs no parentheses; multiple parameters do.

- Optional curly braces: if the body contains one statement, no curly braces are needed.
- Optional return keyword: if the body has a single expression returning a value, the compiler returns it automatically; curly braces require specifying that the expression returns a value.

Lambda expressions are declared as follows:

```
1 // 1. no parameters; returns 5
2 () -> 5
3
4 // 2. takes one parameter (a number) and returns twice its value
5 x -> 2 * x
6
7 // 3. takes two parameters (numbers) and returns their difference
8 (x, y) -> x - y
9
10 // 4. takes two int integers and returns their sum
11 (int x, int y) -> x + y
12
13 // 5. takes a String object and prints it to the console, returning nothing
14 // (as if returning void)
15 (String s) -> System.out.print(s)
```

Taking a custom comparator sorting an array of strings by length as an example, lambda expressions can be applied like this:

```
1 import java.util.Arrays;
2
3 public class Main {
4     static PrintWriter out = new PrintWriter(System.out);
5
6     public static void main(String[] args) {
7         String[] plants = {"Mercury", "venus", "Earth", "Mars", "Jupiter", "Saturn", "
↳ Uranus", "Neptune"};
8         Arrays.sort(plants, (String first, String second) -> (first.length() - second.
↳ length()));
9         for (String word : plants) {
10             out.print(word + " ");
11         }
12         out.close();
13     }
14 }
```

You can also use several statements inside a lambda expression, as in the following example:

```
1 import java.io.PrintWriter;
2 import java.util.Arrays;
3
4 public class Main {
5     static PrintWriter out = new PrintWriter(System.out);
6
7     public static void main(String[] args) {
8         String[] plants = {"Mercury", "venus", "Earth", "Mars", "Jupiter", "Saturn", "
↳ Uranus", "Neptune"};
```

```

9      Arrays.sort(plants, (first, second) ->
10      {
11          // formal parameters need no types; they can be inferred from context
12          int result = first.length() - second.length();
13          return result;
14      });
15      for (String word : plants) {
16          out.print(word + " ");
17      }
18      out.close();
19  }
20 }

```

Here $\rightarrow$ is a derivation symbol: the parentheses in front receive the arguments and derive the return value behind (it is really passing a method).

40.8.3 Functional Interfaces

1. It is an interface, conforming to Java's interface definition.
2. It is an interface containing only one abstract method.
3. Because there is only one unimplemented method, a lambda expression can automatically fill it in.

Functional interfaces are used as follows:

Example — Output the strings whose lengths are multiples of 2

```

1  import java.io.PrintWriter;
2
3  public class Main {
4      static PrintWriter out = new PrintWriter(System.out);
5
6      public static void main(String[] args) {
7          String[] plants = {"Mercury", "venus", "Earth", "Mars", "Jupiter", "Saturn"
8          ↪ , "Uranus", "Neptune"};
9          Test test = s -> { // a lambda expression as an instance of the
10                           // functional interface
11              if (s.length() % 2 == 0) {
12                  return true;
13              }
14              return false;
15          };
16          for (String word : plants) {
17              if (test.check(word)) {
18                  out.print(word + " ");
19              }
20          }
21          out.close();
22      }
23 }

```



```
24 interface Test {  
25     public boolean check(String s);  
26 }
```

Example — Implement the four arithmetic operations

```
1  import java.io.PrintWriter;  
2  
3  public class Main {  
4      static PrintWriter out = new PrintWriter(System.out);  
5  
6      public static double calc(double a, double b, Calculator util) {  
7          return util.operation(a, b);  
8      }  
9  
10     public static void main(String[] args) {  
11         Calculator util[] = new Calculator[4]; // an array of functional  
12                                                // interfaces  
13         util[0] = (a, b) -> a + b;  
14         util[1] = (a, b) -> a - b;  
15         util[2] = (a, b) -> a * b;  
16         util[3] = (a, b) -> a / b;  
17         double a = 20, b = 15;  
18         for (Calculator c : util) {  
19             System.out.println(calc(a, b, c));  
20         }  
21         out.close();  
22     }  
23 }  
24  
25 interface Calculator {  
26     public double operation(double a, double b);  
27 }
```

40.9 Collection

Collection is an interface in Java implemented by several generic container interfaces. Here, **Collection** refers to data structures that store object types.

The element types of a **Collection** in Java must be objects when defined; primitive data types are not allowed.

The usage below all relies on Java's polymorphism: everything appears in the form of implementing interfaces.

The common interfaces include **List**, **Queue**, **Set**, and **Map**.

40.9.1 Container Definitions

When defining a generic container class, the data type must be specified at definition. If no data type is specified and data is added freely as **Object**, compilation succeeds in Java 8 but with many warning risks.

For example, the following definition is safe: the container accepts only **Integer**.

```
1 List<Integer> list1 = new LinkedList<>();
```

Whereas the following definition produces warnings.

```
1 List list = new ArrayList<>();
2 list.add(1);
3 list.add(true);
4 list.add(1.01);
5 list.add(1L);
6 list.add("I am String");
```

So unless there are special needs, the second style is not recommended: the compiler cannot help check whether the stored data is safe. When fetching with `list.get(index)` the data's type is unclear (everything fetched is **Object**), and you must manually cast back to the original type; a moment of carelessness can produce a mis-cast exception.

If the type is explicit, like `List<Integer>`, the compiler checks the type of inserted data: only integers may be inserted. When declaring collection variables you can only use wrapper types like `List<Integer>` or your own `Class`, not primitive types like `List<int>`.

40.9.2 List

ArrayList. `ArrayList` is an array that supports growing dynamically on demand; the default initial length is 10. Exceeding the current length grows it by $\frac{3}{2}$.

Initialization.

```
1 import java.io.PrintWriter;
2 import java.util.ArrayList;
3 import java.util.List;
4
5 public class Main {
6     static PrintWriter out = new PrintWriter(System.out);
7
8     public static void main(String[] args) {
9         List<Integer> list1 = new ArrayList<>(); // create a growable array
10            // named list1 with the default initial length (10)
11         List<Integer> list2 = new ArrayList<>(30); // create a growable array
12            // named list2 with initial length 30
13         List<Integer> list3 = new ArrayList<>(list2); // create a growable
14            // array named list3, using list2's elements and size as its
15            // initial values
16     }
17 }
```

LinkedList. `LinkedList` is a doubly linked list; the bottom layer is a doubly linked list structure.

Initialization.

```

1  import java.io.PrintWriter;
2  import java.util.LinkedList;
3  import java.util.List;
4
5  public class Main {
6      static PrintWriter out = new PrintWriter(System.out);
7
8      public static void main(String[] args) {
9          List<Integer> list1 = new LinkedList<>(); // create a doubly linked list named
↪ list1 List<Integer> list2 = new LinkedList<>(list1); // create a doubly linked
↪ list named list2, adding all elements of
10         // list1 }
11     }

```

Common methods.

```

1  import java.io.PrintWriter;
2  import java.util.ArrayList;
3  import java.util.LinkedList;
4  import java.util.List;
5
6  public class Main {
7      static PrintWriter out = new PrintWriter(System.out);
8      static List<Integer> array = new ArrayList<>();
9      static List<Integer> linked = new LinkedList<>();
10
11     static void add() {
12         array.add(1); // time complexity O(1) linked.add(1); // time complexity O(1)
↪ }
13
14     static void get() {
15         array.get(10); // time complexity O(1) linked.get(10); // time complexity O
↪ (11) }
16
17     static void addIdx() {
18         array.add(0, 2); // worst-case time complexity O(n)
19         linked.add(0, 2); // worst-case time complexity O(n)
20     }
21
22     static void size() {
23         array.size(); // time complexity O(1)
24         linked.size(); // time complexity O(1)
25     }
26
27     static void set() { // this method returns the value previously at that
28         // position
29         array.set(0, 1); // time complexity O(1)
30         linked.set(0, 1); // worst-case time complexity O(n)
31     }
32
33 }

```

Traversal.

```

1  import java.io.PrintWriter;
2  import java.util.ArrayList;
3  import java.util.Iterator;
4  import java.util.LinkedList;
5  import java.util.List;
6
7  public class Main {
8      static PrintWriter out = new PrintWriter(System.out);
9      static List<Integer> array = new ArrayList<>();
10     static List<Integer> linked = new LinkedList<>();
11
12     static void function1() { // naive traversal
13         for (int i = 0; i < array.size(); i++) {
14             out.println(array.get(i)); // traversing the growable array costs O(n)
15         }
16         for (int i = 0; i < linked.size(); i++) {
17             out.println(linked.get(i)); // traversing the doubly linked list
18             // costs O(n^2), because LinkedList's get(i) costs O(i)
19         }
20     }
21
22     static void function2() { // enhanced for-loop traversal for (int e : array) {
23         out.println(e);
24     }
25     for (int e : linked) {
26         out.println(e); // both are O(n) }
27     }
28
29     static void function3() { // iterator traversal Iterator<Integer> iterator1 =
    ↪ array.iterator();
30         Iterator<Integer> iterator2 = linked.iterator();
31         while (iterator1.hasNext()) {
32             out.println(iterator1.next());
33         }
34         while (iterator2.hasNext()) {
35             out.println(iterator2.next());
36         } // both are O(n) }
37
38     }

```

Warning — Note

Do not remove elements from a `List` while traversing it with `for` or `foreach`; doing so throws an exception.

The reason is simple: `list.size()` has changed, but the number of iterations already performed in the loop has not changed accordingly. The data originally expected at the next **index** has become the data at the current **index** due to the deletion; the next loop iteration operates on what was originally expected at the index after next, and in the end the operated

data does not match expectations.

40.9.3 Queue

LinkedList. A plain queue can be implemented with `LinkedList`; the bottom layer simulates the queue with a linked list.

Initialization.

```
1 Queue<Integer> q = new LinkedList<>();
```

`LinkedList` implements both the `List` interface and the `Deque` interface at the bottom, and the `Deque` interface inherits from the `Queue` interface, so `LinkedList` can act as both `List` and `Queue`.

ArrayDeque. A plain queue can also be implemented with `ArrayDeque`; the bottom layer simulates the queue with an array.

Initialization.

```
1 Queue<Integer> q = new ArrayDeque<>();
```

`ArrayDeque` implements the `Deque` interface at the bottom, and the `Deque` interface inherits from the `Queue` interface, so `ArrayDeque` can act as a `Queue`.

The difference between `LinkedList` and `ArrayDeque` in implementing the `Queue` interface.

1. Data structure: structurally, both `ArrayDeque` and `LinkedList` implement Java's `Deque` double-ended queue interface. But `ArrayDeque` does not implement Java's `List` interface, so it has no index-position-based operations.
2. Thread safety: neither `ArrayDeque` nor `LinkedList` considers thread synchronization; neither guarantees thread safety.
3. Bottom-layer implementation: `ArrayDeque` is based on a dynamic array, while `LinkedList` is based on a doubly linked list.
4. Traversal speed: `ArrayDeque` is one contiguous memory region and, by locality, hits CPU cache lines better; `LinkedList`'s discrete memory is cache-unfriendly.
5. Operation speed: the stack and queue behaviors of both `ArrayDeque` and `LinkedList` are $O(1)$ time complexity. `ArrayDeque`'s push/enqueue may trigger resizing, but amortized analysis still gives $O(1)$.
6. Extra memory consumption: `ArrayDeque` has idle space outside the array's head and tail pointers, while `LinkedList` adds predecessor and successor pointers on each node.

PriorityQueue. `PriorityQueue` is the priority queue; the default is a min-heap.

Initialization.

```

1 Queue<Integer> q1 = new PriorityQueue<>(); // a min-heap
2 Queue<Integer> q2 = new PriorityQueue<>((x, y) -> {return y - x;}); // a
3 // max-heap

```

Common methods. In the following table the queue is defined as `Queue<Integer>`.

Function	Effect
<code>size()</code>	returns the current queue length
<code>add(Integer val)</code>	inserts <code>val</code> into the queue; violating the queue's capacity restriction throws an exception
<code>offer(Integer val)</code>	inserts <code>val</code> into the queue; violating the capacity restriction fails silently, without an exception
<code>isEmpty()</code>	tests whether the queue is empty; returns <code>true</code> if empty
<code>peek()</code>	returns the head element; returns <code>null</code> if the queue is empty
<code>poll()</code>	returns and deletes the head element; returns <code>null</code> if the queue is empty

Usage example and comparison of the differences:

```

1 import java.io.PrintWriter;
2 import java.util.LinkedList;
3 import java.util.PriorityQueue;
4 import java.util.Queue;
5
6 public class Main {
7     static PrintWriter out = new PrintWriter(System.out);
8     static Queue<Integer> q1 = new LinkedList<>();
9     static Queue<Integer> q2 = new PriorityQueue<>();
10
11     static void add() { // add and offer are functionally identical; the difference is
12         ↪ whether
13         // they throw an exception q1.add(1); // time complexity O(1) q2.add(1); //
14         ↪ time complexity O(logn) }
15
16     static void isEmpty() {
17         q1.isEmpty(); // time complexity O(1) q2.isEmpty(); // space complexity O(1)
18         ↪ }
19
20     static void size() {
21         q1.size(); // time complexity O(1) q2.size(); // returns the length of q2 }
22
23     static void peek() {
24         q1.peek(); // time complexity O(1) q2.peek(); // time complexity O(logn) }
25
26     static void poll() {
27         q1.poll(); // time complexity O(1) q2.poll(); // time complexity O(logn) }
28 }

```

Traversal.

```

1 import java.io.PrintWriter;
2 import java.util.LinkedList;
3 import java.util.PriorityQueue;
4 import java.util.Queue;
5
6 public class Main {
7     static PrintWriter out = new PrintWriter(System.out);
8     static Queue<Integer> q1 = new LinkedList<>();
9     static Queue<Integer> q2 = new PriorityQueue<>();
10
11     static void test() {
12         while (!q1.isEmpty()) { // complexity O(n) out.println(q1.poll());
13         }
14         while (!q2.isEmpty()) { // complexity O(nlogn) out.println(q2.poll());
15         }
16     }
17 }
18 }

```

40.9.4 Deque

Deque is Java's double-ended queue, which we usually use for queue operations and stack operations.

Main functions. In the following table the deque is defined as `Deque<Integer>`.

Function	Effect
<code>addFirst(Integer val)</code>	inserts <code>val</code> at the head; violating the capacity restriction throws an exception
<code>offerFirst(Integer val)</code>	inserts <code>val</code> at the head; violating the capacity restriction fails silently
<code>removeFirst()</code>	returns and deletes the head; throws an exception if the deque is empty
<code>pollFirst()</code>	returns and deletes the head; returns <code>null</code> if the deque is empty
<code>peekFirst()</code>	returns the head; returns <code>null</code> if the deque is empty
<code>push(Integer val)</code>	inserts <code>val</code> at the head; equivalent to <code>addFirst</code>
<code>pop()</code>	returns and deletes the head; equivalent to <code>removeFirst</code>
<code>remove()</code>	deletes the head; equivalent to <code>removeFirst</code>
<code>poll()</code>	deletes the head; equivalent to <code>pollFirst</code>
<code>addLast(Integer val)</code>	inserts <code>val</code> at the tail; violating the capacity restriction throws an exception
<code>offerLast(Integer val)</code>	inserts <code>val</code> at the tail; violating the capacity restriction fails silently
<code>removeLast()</code>	returns and deletes the tail; throws an exception if the deque is empty
<code>pollLast()</code>	returns and deletes the tail; returns <code>null</code> if the deque is empty
<code>peekLast()</code>	returns the tail; returns <code>null</code> if the deque is empty
<code>add(Integer val)</code>	inserts <code>val</code> at the tail; equivalent to <code>addLast</code>
<code>offer(Integer val)</code>	inserts <code>val</code> at the tail; equivalent to <code>offerLast</code>

Stack operations.

```

1 import java.util.ArrayDeque;

```

```

2 import java.util.Deque;
3
4 public class Main {
5     static Deque<Integer> stack = new ArrayDeque<>();
6     static int[] a = {1, 2, 3, 4, 5};
7
8     public static void main(String[] args) {
9         for (int v : a) {
10             stack.push(v);
11         }
12         while (!stack.isEmpty()) { // prints 5 4 3 2 1
13             System.out.println(stack.pop());
14         }
15     }
16 }

```

Deque operations.

```

1 import java.util.ArrayDeque;
2 import java.util.Deque;
3
4 public class Main {
5     static Deque<Integer> deque = new ArrayDeque<>();
6
7     static void insert() {
8         deque.addFirst(1);
9         deque.addFirst(2);
10        deque.addLast(3);
11        deque.addLast(4);
12    }
13
14    public static void main(String[] args) {
15        insert();
16        while (!deque.isEmpty()) { // prints 2 1 3 4
17            System.out.println(deque.poll());
18        }
19        insert();
20        while (!deque.isEmpty()) { // prints 4 3 1 2
21            System.out.println(deque.pollLast());
22        }
23    }
24 }

```

40.9.5 Set

Set is a data structure that keeps the elements of the container unique.

HashSet. A Set inserted at random positions.

Initialization.

```

1 Set<Integer> s1 = new HashSet<>();

```


LinkedHashSet. A `Set` that preserves insertion order.

Initialization.

```
1 Set<Integer> s2 = new LinkedHashSet<>();
```

TreeSet. A `Set` that keeps the container's elements ordered, ascending by default.

Initialization.

```
1 Set<Integer> s3 = new TreeSet<>();
2 Set<Integer> s4 = new TreeSet<>((x, y) -> {return y - x;}); // descending order
```

More uses of TreeSet. These methods are newly created and implemented by `TreeSet`; we cannot call the following methods through the `Set` interface, so we create it as follows:

```
1 TreeSet<Integer> s3 = new TreeSet<>();
2 TreeSet<Integer> s4 = new TreeSet<>((x, y) -> {return y - x;}); // descending
3 // order
```

In the following table, `this` stands for the current `TreeSet<Integer>`.

Function	Effect
<code>first()</code>	returns the first element of <code>this</code> , or <code>null</code> if none
<code>last()</code>	returns the last element of <code>this</code> , or <code>null</code> if none
<code>floor(Integer val)</code>	returns the first element of <code>this</code> that is $\leq$ <code>val</code> , or <code>null</code> if none
<code>ceiling(Integer val)</code>	returns the first element of <code>this</code> that is $\geq$ <code>val</code> , or <code>null</code> if none
<code>higher(Integer val)</code>	returns the first element of <code>this</code> that is $>$ <code>val</code> , or <code>null</code> if none
<code>lower(Integer val)</code>	returns the first element of <code>this</code> that is $<$ <code>val</code> , or <code>null</code> if none
<code>pollFirst()</code>	returns and deletes the first element of <code>this</code> , or <code>null</code> if none
<code>pollLast()</code>	returns and deletes the last element of <code>this</code> , or <code>null</code> if none

Code example:

```
1 import java.util.TreeSet;
2
3 public class Main {
4     static int[] a = {4,7,1,2,3,6};
5
6     public static void main(String[] args) {
7         TreeSet<Integer> set = new TreeSet<>();
8         for(int v:a) {
9             set.add(v);
10        }
11        Integer a2 = set.first();
12        System.out.println(a2); // returns 1
13        Integer a3 = set.last();
14        System.out.println(a3); // returns 7
```

```

15     Integer a4 = set.floor(5);
16     System.out.println(a4); // returns 4
17     Integer a5 = set.ceiling(6);
18     System.out.println(a5); // returns 6
19     Integer a6 = set.higher(7);
20     System.out.println(a6); // returns null
21     Integer a7 = set.lower(2);
22     System.out.println(a7); // returns 1
23     Integer a8 = set.pollFirst();
24     System.out.println(a8); // returns 1
25     Integer a9 = set.pollLast();
26     System.out.println(a9); // returns 7
27 }
28 }

```

Common Set methods.

Function	Effect
<code>size()</code>	returns the current set's size
<code>add(Integer val)</code>	inserts <code>val</code> into the set
<code>contains(Integer val)</code>	tests whether the set contains the element <code>val</code>
<code>addAll(Collection e)</code>	adds all elements of container <code>e</code> into the current set
<code>retainAll(Collection e)</code>	deletes the current set's elements not appearing in container <code>e</code> , i.e., intersects <code>e</code>
<code>removeAll(Collection e)</code>	deletes the current set's elements appearing in container <code>e</code> , i.e., subtracts <code>e</code> from the current set

```

1  import java.io.PrintWriter;
2  import java.util.HashSet;
3  import java.util.LinkedHashSet;
4  import java.util.Set;
5
6  public class Main {
7      static PrintWriter out = new PrintWriter(System.out);
8      static Set<Integer> s1 = new HashSet<>();
9      static Set<Integer> s2 = new LinkedHashSet<>();
10
11     static void add() {
12         s1.add(1);
13     }
14
15     static void contains() { // tests whether set contains an element of value 2: true
16         ⇨ if yes, false
17         // otherwise s1.contains(2);
18     }
19
20     static void test1() { // the union of s1 and s2
21         ⇨ Set<Integer> res = new HashSet<>();
22         res.addAll(s1);
23         res.addAll(s2);
24     }
25 }

```

```

22     }
23
24     static void test2() { // the intersection of s1 and s2 Set<Integer> res = new
    ↪ HashSet<>();
25         res.addAll(s1);
26         res.retainAll(s2);
27     }
28
29     static void test3() { // difference set: s1 - s2 Set<Integer> res = new HashSet
    ↪ <>();
30         res.addAll(s1);
31         res.removeAll(s2);
32     }
33 }

```

Traversal.

```

1 import java.io.PrintWriter;
2 import java.util.HashSet;
3 import java.util.LinkedHashSet;
4 import java.util.Set;
5
6 public class Main {
7     static PrintWriter out = new PrintWriter(System.out);
8     static Set<Integer> s1 = new HashSet<>();
9     static Set<Integer> s2 = new LinkedHashSet<>();
10
11     static void test() {
12         for (int key : s1) {
13             out.println(key);
14         }
15         out.close();
16     }
17 }

```

40.9.6 Map

Map is a data structure maintaining key-value pairs `<Key, Value>`, where the Key is unique.

HashMap. A Map inserted at random positions.

Initialization.

```

1 Map<Integer, Integer> map1 = new HashMap<>();

```

LinkedHashMap. A Map that preserves insertion order.

Initialization.

```

1 Map<Integer, Integer> map2 = new LinkedHashMap<>();

```

TreeMap. A Map that keeps keys ordered, ascending by default.

Initialization.

```

1 Map<Integer, Integer> map3 = new TreeMap<>();
2 Map<Integer, Integer> map4 = new TreeMap<>((x, y) -> {return y - x;}); // in
3 // descending order

```

Common methods. Below, `this` stands for the current `Map<Integer, Integer>`:

Function	Effect
<code>put(Integer key, Integer value)</code>	inserts <code><key, value></code> into <code>this</code>
<code>size()</code>	returns the size of <code>this</code>
<code>containsKey(Integer key)</code>	tests whether some element of <code>this</code> has key <code>key</code>
<code>get(Integer key)</code>	returns the value of the element of <code>this</code> with key <code>key</code>
<code>keySet()</code>	returns the keys of all elements of <code>this</code> as a set

Usage example:

```

1 import java.io.PrintWriter;
2 import java.util.HashMap;
3 import java.util.LinkedHashMap;
4 import java.util.Map;
5 import java.util.TreeMap;
6
7 public class Main {
8     static PrintWriter out = new PrintWriter(System.out);
9
10    static Map<Integer, Integer> map1 = new HashMap<>();
11    static Map<Integer, Integer> map2 = new LinkedHashMap<>();
12    static Map<Integer, Integer> map3 = new TreeMap<>();
13    static Map<Integer, Integer> map4 = new TreeMap<>((x,y)->{return y-x;});
14
15    static void put(){ // inserts the element with key 1 and value 1
16        map1.put(1, 1);
17    }
18    static void get(){ // returns the value of key 1
19        map1.get(1);
20    }
21    static void containsKey(){ // tests whether a key-value pair with key 1
22                               // exists
23        map1.containsKey(1);
24    }
25    static void KeySet(){
26        map1.keySet();
27    }
28 }

```

Traversal.

```

1 import java.io.PrintWriter;
2 import java.util.HashMap;
3 import java.util.Map;

```

```

4
5 public class Main {
6     static PrintWriter out = new PrintWriter(System.out);
7
8     static Map<Integer, Integer> map1 = new HashMap<>();
9
10    static void print() {
11        for (int key : map1.keySet()) {
12            out.println(key + " " + map1.get(key));
13        }
14    }
15 }

```

Of course, the key-value types may be changed. For example a **Map** may also be defined as:

```

1 Map<String, Set<Integer>> map = new HashMap<>();

```

40.10 Arrays

Arrays is a utility class in `java.util` for operating on arrays. Its methods are all static and can be called with the class name directly.

40.10.1 Arrays.sort()

Arrays.sort() sorts an array; the main overloads are:

```

1 import java.util.Arrays;
2 import java.util.Comparator;
3
4 public class Main {
5     static int[] a = new int[10];
6     static Integer[] b = new Integer[10];
7     static int firstIdx, lastIdx;
8
9     public static void main(String[] args) {
10         Arrays.sort(a); // 1
11         Arrays.sort(a, firstIdx, lastIdx); // 2
12         Arrays.sort(b, new Comparator<Integer>() { // 3
13             @Override
14             public int compare(Integer o1, Integer o2) {
15                 return o2 - o1;
16             }
17         });
18         Arrays.sort(b, firstIdx, lastIdx, new Comparator<Integer>() { // 4
19             @Override
20             public int compare(Integer o1, Integer o2) {
21                 return o2 - o1;
22             }
23         });
24         // since Java 8 has lambda expressions, the third and fourth overloads can
25         // also be written as Arrays.sort(b, (x, y) -> { // 5
26             return y - x;
27         });

```

```

28     Arrays.sort(b, (x, y) -> { // 6
29         return y - x;
30     });
31 }
32 }

```

The meanings of the numbered overloads:

1. sorts the array `a`, ascending by default.
2. sorts the specified positions of array `a`, ascending by default; the sort range is half-open `[firstIdx, lastIdx)`.
3. sorts array `a` in a custom way: parameter two — parameter one gives descending order, parameter one — parameter two gives ascending order; with a custom comparator the array's element type must be an object type.
4. sorts the specified positions of array `a` in a custom way; the range is half-open `[firstIdx, lastIdx)`; with a custom comparator the element type must be an object type.
5. same as 3, with the code shortened using a lambda expression.
6. same as 4, with the code shortened using a lambda expression.

Note — The underlying functions of `Arrays.sort()`

1. When the element type of the array passed to `Arrays.sort` is a primitive data type (`byte`, `short`, `char`, `int`, `long`, `double`, `float`), the default is `DualPivotQuicksort` (dual-pivot quicksort), whose worst-case complexity can reach $O(n^2)$.
2. When the element type is not primitive, the default is `legacyMergeSort` and `TimSort` (merge sort), with complexity $O(n \log n)$.

This can be verified with the following code:

Example — Codeforces 1646B - Quality vs Quantity^a

^a<https://codeforces.com/problemset/problem/1646/B>

You are given n integers, and you must split them into two groups: does there exist a grouping where one group's length is smaller than the other's while its sum is greater?

Note — Example code

```

1  import java.io.BufferedReader;
2  import java.io.IOException;
3  import java.io.InputStreamReader;
4  import java.io.PrintWriter;
5  import java.util.Arrays;
6  import java.util.StringTokenizer;
7
8  public class Main {

```

```
9      static class FastReader {
10          StringTokenizer st;
11          BufferedReader br;
12
13          public FastReader() {
14              br = new BufferedReader(new InputStreamReader(System.in));
15          }
16
17          String next() {
18              while (st == null || !st.hasMoreElements()) {
19                  try {
20                      st = new StringTokenizer(br.readLine());
21                  } catch (IOException e) {
22                      e.printStackTrace();
23                  }
24              }
25              return st.nextToken();
26          }
27
28          int nextInt() {
29              return Integer.parseInt(next());
30          }
31
32          long nextLong() {
33              return Long.parseLong(next());
34          }
35
36          double nextDouble() {
37              return Double.parseDouble(next());
38          }
39
40          String nextLine() {
41              String str = "";
42              try {
43                  str = br.readLine();
44              } catch (IOException e) {
45                  e.printStackTrace();
46              }
47              return str;
48          }
49      }
50
51      static PrintWriter out = new PrintWriter(System.out);
52      static FastReader in = new FastReader();
53
54      static void solve() {
55          int n = in.nextInt();
56          // changing the array type here from Integer to int would cause TLE
57          Integer[] a = new Integer[n + 10];
58          for (int i = 1; i <= n; i++) {
59              a[i] = in.nextInt();
60          }
61          Arrays.sort(a, 1, n + 1);
```

```

62     long left = a[1];
63     long right = 0;
64     int x = n;
65     for (int i = 2; i < x; i++, x--) {
66         left = left + a[i];
67         right = right + a[x];
68         if (right > left) {
69             out.println("YES");
70             return;
71         }
72     }
73     out.println("NO");
74 }
75
76 public static void main(String[] args) {
77     int t = in.nextInt();
78     while (t-- > 0) {
79         solve();
80     }
81     out.close();
82 }
83 }

```

40.10.2 Arrays.binarySearch()

`Arrays.binarySearch()` binary-searches a contiguous range of an array; the array must be sorted, and the time complexity is $O(\log n)$. The main overloads:

```

1  import java.util.Arrays;
2
3  public class Main {
4      static int[] a = new int[10];
5      static Integer[] b = new Integer[10];
6      static int firstIdx, lastIdx;
7      static int key;
8
9      public static void main(String[] args) {
10         Arrays.binarySearch(a, key); // 1
11         Arrays.binarySearch(a, firstIdx, lastIdx, key); // 2
12     }
13 }

```

The source code:

```

1  private static int binarySearch0(int[] a, int fromIndex, int toIndex, int key) {
2      int low = fromIndex;
3      int high = toIndex - 1;
4
5      while (low <= high) {
6          int mid = (low + high) >>> 1;
7          int midVal = a[mid];
8

```



```

9         if (midVal < key)
10             low = mid + 1;
11         else if (midVal > key)
12             high = mid - 1;
13         else
14             return mid; // key found
15     }
16     return -(low + 1); // key not found.
17 }

```

The meanings of the numbered overloads:

1. binary-searches array **a** for **key**; if present, returns its subscript; if absent, returns a negative number.
2. binary-searches array **a** for **key**; if present, returns its subscript; the search range is half-open [**firstIdx**,**lastIdx**); if absent, returns a negative number.

40.10.3 Arrays.fill()

The **Arrays.fill()** method assigns the elements at consecutive positions of an array to one uniform value. Its parameters are the array, **fromIndex**, **toIndex**, and the value to fill. After the method runs, all elements in the half-open range [**firstIdx**,**lastIdx**) of the array have the filled value.

40.11 Collections

Collections is a utility class in **java.util** for operating on collections. Its methods are all static and can be called with the class name directly.

40.11.1 Collections.sort()

The underlying principle of **Collections.sort()** is to convert all elements into an array, call **Arrays.sort()**, and after sorting assign back into the original collection. Since the element types of a Java **Collection** are all object types, merge sort is always what handles it.

This method cannot sort a specified range of the collection.

The underlying source code:

```

1 default void sort(Comparator<? super E> c) {
2     Object[] a = this.toArray();
3     Arrays.sort(a, (Comparator) c);
4     ListIterator<E> i = this.listIterator();
5     for (Object e : a) {
6         i.next();
7         i.set((E) e);
8     }
9 }

```

40.11.2 Collections.binarySearch()

`Collections.binarySearch()` binary-searches a specified range of a collection; its function is the same as `Arrays.binarySearch()`.

```
1 Collections.binarySearch(list, key);
```

This method cannot search a specified range.

40.11.3 Collections.swap()

The function of `Collections.swap()` is to swap the elements at two specified positions of a collection.

```
1 Collections.swap(list, i, j);
```

40.12 Miscellaneous

40.12.1 Numeric Comparison Issues

In Java, for pure numeric types, $-0.0 = 0.0$; for object types, $-0.0 \neq 0.0$. If you try to count slopes with a `Set`, this issue brings trouble. The provided fix is to add 0.0 to every slope before inserting it into the `Set`.

```
1 import java.io.PrintWriter;
2
3 public class Main {
4     static PrintWriter out = new PrintWriter(System.out);
5
6     static void A() {
7         Double a = 0.0;
8         Double b = -0.0;
9         out.println(a.equals(b)); // false
10    }
11
12    static void B() {
13        Double a = 0.0;
14        Double b = -0.0 + 0.0;
15        out.println(a.equals(b)); // true
16    }
17
18    static void C() {
19        double a = 0.0;
20        double b = -0.0;
21        out.println(a == b); // true
22    }
23
24
25    public static void main(String[] args) {
26        A();
27        B();
28        C();
```

```
29     out.close();
30 }
31 }
```

40.13 References

- Input & Output — USACO Guide, <https://usaco.guide/general/input-output?lang=java>